

Teoria da Informação e da Codificação



Leonardo C. Araújo

Teoria da Informação e Codificação

Leonardo Araújo

23 de junho de 2025

Copyright © 2025 Leonardo Araújo

Este livro está licenciado sob os termos da *Creative Commons -
Atribuição-Não Comercial-Sem Derivações 4.0 Internacional (CC
BY-NC-ND 4.0)*.

Você não pode usar esta obra para fins comerciais.

Você não pode distribuir material modificado.

Para mais informações, visite

<http://creativecommons.org/licenses/by-nc-nd/4.0/>



Conteúdo

Introdução 4

Aleatoriedade 7

Princípios Fundamentais 15

Propriedade da Equipartição Assintótica 70

Processos Estocásticos 95

Codificação de Fonte 104

Canais de Comunicação 140

Códigos de Correção de Erro 164

Introdução

O avanço das tecnologias de comunicação trouxe a necessidade de buscar a transmissão de informação de forma eficiente. Informação, um conceito abstrato, que, no contexto da comunicação e da tecnologia, advém da organização e interpretação de dados, fornecendo uma visão sobre tendências ou padrões, possibilitando a interpretação e emergência de significado. Dados em si, são apenas um conjunto de símbolos, geralmente organizados como uma sequência. Assim, determinada informação pode ser representada na forma de dados de maneiras distintas. A representação na forma de dados pode ser mais ou menos propícia a um determinado meio, sob o qual a informação será transmitida ou armazenada. Uma mesma informação pode ainda ser representada de forma mais concisa ou prolixa.

Se voltarmos na história, em meados do século XIX, Samuel Morse e Alfred Vail propuseram o código Morse, como uma forma econômica de se comunicar através das redes telegráficas. A proposta consistia em usar um código de pontos (tom curto), traços (tom longo) e espaços de separação entre eles (silêncio) para tornar a comunicação de uma mensagem mais eficiente, ou seja, utilizando menos pulsos e, assim, reduzindo o tempo necessário para enviar uma mensagem. O código Morse, criado originalmente para o inglês, representa os símbolos mais frequentes através de sequências curtas e os símbolos infrequentes através de sequências longas, o que proporciona a redução do comprimento esperado da mensagem transmitida.

Também no século XIX, foi criada a escrita noturna por Charles Barbier. Esta forma de escrita foi posteriormente adaptada por Louis Braille para criar um sistema mais simples e acessível para deficientes visuais. Cada célula de 6 pontos é capaz de representar letras (ou sequências de letras) na forma de combinações binárias. O código Braille, inicialmente proposto para o francês, foi mais tarde adaptado para outras línguas, bem como para matemática e música, dentre outras áreas. A adaptação da forma de escrita e leitura ao meio é fundamental e evidente na forma escrita tátil, sendo preponderante para garantir a eficácia da comunicação. É essencial que o leitor seja capaz de decodificar facilmente a informação ali representada. Para tanto, a

distinção de diferentes símbolos é facilitada pela clareza e simplicidade do sistema. A eficiência e a economicidade da representação são evidentes na capacidade de transmitir informações de forma compacta, reduzindo o espaço necessário.

Usualmente, quando falamos da representação de informação, pensamos na escrita como uma forma de expressá-la. Entretanto, devemos nos atentar ao fato de que certas informações podem utilizar outros formatos representacionais. Por exemplo, a cotação de uma moeda é representada por uma sequência de números; um sinal eletrocardiográfico é medido pela diferença de potencial; uma imagem digital é constituída por uma matriz de pixels; e informações sobre produtos, endereços ou dados de pagamento podem ser armazenados em códigos de barras e códigos QR. Embora a natureza representacional dessas informações, ao serem geradas, não seja expressa por meio de um alfabeto convencional, elas podem ser transcodificadas para serem representadas através de um alfabeto padrão, como o Base64¹, que é utilizado para codificar, por exemplo, anexos de e-mails.

A ideia de representação binária é muito antiga, possivelmente anterior aos estudos de Thomas Harriot e Gottfried Leibniz, nos séculos XVI e XVII (Shirley 1951).² No entanto, Hartley (1928) foi um dos primeiros a quantificar a informação introduzindo o conceito de ‘bit’ como a unidade básica. Os trabalhos de Hartley ocorreram no contexto de rápida expansão das tecnologias de comunicação, como o telégrafo e o rádio, onde havia uma crescente necessidade de medir e otimizar a transmissão de dados.

O trabalho de Shannon (1948), que começou a ser desenvolvido durante a Segunda Guerra Mundial, permaneceu sigiloso devido à sua aplicação em comunicações militares. Embora suas ideias tenham sido formuladas na década de 1940, o artigo seminal *A Mathematical Theory of Communication* foi publicado apenas em 1948. Motivado pela necessidade de entender como a informação poderia ser codificada e transmitida de forma eficiente, Shannon desenvolveu uma teoria matemática que abordava questões práticas da comunicação. Ele introduziu conceitos fundamentais, como a entropia, que mede a incerteza ou a quantidade de informação em uma mensagem, e a capacidade do canal, que determina a quantidade máxima de informação que pode ser transmitida sem erro. O trabalho de Shannon estabeleceu um novo campo de estudo que influenciou profundamente a computação, a teoria da comunicação e outras disciplinas. Seu trabalho é considerado o marco de surgimento da teoria da informação.

Na década de 1960, a teoria da codificação passou por avanços significativos que moldaram a forma como entendemos e aplicamos a comunicação digital. O trabalho de Hamming (1950) estabeleceu as bases para a detecção e correção de erros, permitindo que sistemas de

¹ Base64 é um esquema de codificação que transforma dados binários em uma representação textual utilizando um conjunto de 64 caracteres, que inclui letras maiúsculas (A-Z), letras minúsculas (a-z), dígitos (0-9) e os símbolos ‘+’ e ‘/’. Cada grupo de três bytes de dados binários é convertido em quatro caracteres ASCII.

² Thomas Harriot (c. 1560–1621) investigou formas alternativas de notação matemática, incluindo representações próximas ao binário, embora seus escritos tenham permanecido inéditos em vida. Gottfried Wilhelm Leibniz (1646–1716), por sua vez, publicou em 1703 o ensaio *Explication de l’Arithmétique Binaire*, no qual formalizou o sistema binário como base aritmética, relacionando-o também ao simbolismo do I Ching chinês (Shirley 1951; Zandonade 2024).

comunicação se tornassem mais robustos. Paralelamente, os códigos de Golay (1949) emergiram como uma solução eficaz para a correção de múltiplos erros, ampliando as possibilidades de transmissão confiável. A aplicação do teorema de Shannon sobre a capacidade do canal continuou a ser explorada, fornecendo uma compreensão crítica dos limites da comunicação eficiente. Além disso, os códigos de convolução começaram a ganhar destaque, oferecendo novas abordagens para melhorar a confiabilidade na transmissão de dados. O trabalho inovador de Gallager (1962), com os códigos de paridade de baixa densidade, introduziu uma nova classe de códigos que se mostraram extremamente eficazes na correção de erros, influenciando profundamente o desenvolvimento da teoria da codificação. Esses avanços não apenas solidificaram a teoria da codificação como um campo essencial da teoria da informação, mas também tiveram um impacto duradouro em diversas aplicações práticas na comunicação moderna.

A teoria da informação tornou-se central nas comunicações digitais, servindo como a base para o desenvolvimento de tecnologias que transformaram a forma como nos comunicamos. Com a ascensão da internet e das redes de comunicação, os princípios estabelecidos por Claude Shannon, como a quantificação da informação e a capacidade dos canais, tornaram-se indispensáveis para otimizar a transmissão de dados e garantir a integridade das comunicações. Além de seu papel fundamental nas telecomunicações, a teoria da informação é amplamente aplicada em diversas outras áreas. Na ecologia, por exemplo, é utilizada para analisar a diversidade de espécies e a complexidade dos ecossistemas, ajudando a entender as interações entre organismos. Na criptografia, os conceitos de entropia e codificação são essenciais para garantir a segurança das informações transmitidas. Na linguística, a teoria da informação auxilia na análise da estrutura e da semântica das línguas, permitindo a modelagem de padrões de comunicação. Outros campos, como a biologia, onde a informação genética é estudada, e a psicologia, que investiga a percepção e a cognição, também se beneficiam dos princípios da teoria da informação, demonstrando sua relevância e aplicabilidade em diferentes áreas.

Aleatoriedade

A *aleatoriedade* é um conceito fundamental em diversas disciplinas, sendo essencial entender o que ela realmente é. Há muitos anos a humanidade faz uso da aleatoriedade, quer seja em jogos de azar, na determinação do destino de pessoas ou em sua seleção para exercer alguma função ou cargo. Como método de obter aleatoriedade, há milênios se utiliza dados ou moedas (Aruz, Benzel e Evans 2008). No domínio computacional, a aleatoriedade é vital em diversas aplicações, como criptografia, simulações, jogos, arte e teoria da informação.

Quando lidamos com computadores, a produção de números verdadeiramente aleatórios é um desafio intrínseco. Em geral, os números gerados por algoritmos computacionais são *pseudoaleatórios*, ou seja, seguem sequências determinísticas que apenas aparentam não possuir uma ordem subjacente. Esses números são o resultado de funções matemáticas bem definidas, como geradores lineares congruentes ou algoritmos mais aprimorados, como o *Mersenne Twister*, que, apesar de produzirem sequências com propriedades estatísticas desejáveis, carecem da imprevisibilidade essencial à aleatoriedade genuína.

Para tarefas críticas, onde imprevisibilidade e falta de padrões são realmente essenciais, adota-se a geração de aleatoriedade baseada em hardware. Exemplos incluem o uso da captação de imagem e áudio em celulares (Krhovják, Švenda, Matyáš et al. 2007; Chen 2013), atividade de teclado, mouse e disco (Guterman, Pinkas e Reinman 2006) e até mesmo *Lava Lamps* (Cloudflare 2025).

Stephen Wolfram, renomado por suas contribuições à complexidade computacional e à teoria dos sistemas complexos, propõe três fontes distintas de aleatoriedade que explicam como ela pode emergir em sistemas: a aleatoriedade do ambiente, a aleatoriedade nas condições iniciais e a geração intrínseca de aleatoriedade. A *aleatoriedade do ambiente* surge de influências externas imprevisíveis, como variações naturais em sistemas climáticos, que injetam incerteza contínua em um sistema. A *aleatoriedade nas condições iniciais* reflete a sensibilidade de sistemas caóticos a pequenas perturbações iniciais, exemplificada pelo “efeito borboleta”, onde mudanças mínimas podem levar a resultados drasticamente distintos. Já a *geração intrínseca de*

aleatoriedade ocorre nas dinâmicas internas de sistemas determinísticos complexos, que, por meio de interações não-lineares, produzem comportamentos aparentemente aleatórios, como o ruído quântico em circuitos eletrônicos. Wolfram argumenta que essas fontes frequentemente coexistem, e, em suas palavras, até “programas simples podem produzir comportamento aparentemente aleatório mesmo sem entrada aleatória alguma”, sugerindo que os mecanismos subjacentes à aleatoriedade na natureza e em simulações computacionais compartilham uma essência comum (Wolfram 2002).

Fontes de aleatoriedade genuína, por outro lado, frequentemente recorrem ao mundo físico, indo além dos métodos algorítmicos determinísticos. Exemplos clássicos incluem o ruído térmico em circuitos eletrônicos, a decadência radioativa ou fenômenos atmosféricos captados por sensores, usados em geradores de números aleatórios baseados em hardware. Em sistemas computacionais modernos, busca-se capturar essa aleatoriedade verdadeira por meio de eventos imprevisíveis do ambiente operacional, como os pressionamentos de teclas no teclado, os movimentos do mouse, as variações no tráfego de rede, a velocidade das ventoinhas do processador, o ruído de acesso a discos rígidos ou até mesmo as flutuações nos tempos de execução de processos do sistema. Esses dados, coletados em tempo real, são usados como sementes ou entradas diretas para aumentar a imprevisibilidade dos números gerados. Contudo, essas abordagens enfrentam limitações práticas, pois dependem de sistemas computacionais determinísticos que podem comprometer a coleta e o uso desses dados brutos ao processá-los de maneira previsível.

Um exemplo prático disso é encontrado no sistema operacional Linux, que utiliza duas fontes principais de números aleatórios: `/dev/random` e `/dev/urandom`. O dispositivo `/dev/random` gera números aleatórios de alta qualidade a partir de uma “piscina de entropia” alimentada por ruído ambiente, como os eventos mencionados anteriormente, mas pode bloquear a leitura se a entropia disponível for insuficiente, tornando-o ideal para operações criptográficas que priorizam segurança sobre velocidade. Já o `/dev/urandom`, embora também utilize a mesma piscina de entropia, não bloqueia a leitura, gerando dados adicionais mesmo em baixa entropia (Bansal, Subramanyan e Nandakumar 2023; Guterman, Pinkas e Reinman 2006); isso o torna mais adequado para aplicações como simulações, que demandam grandes quantidades de números aleatórios rapidamente.

O Gerador Lehmer

Um dos primeiros métodos propostos para gerar números aleatórios é o *método de Lehmer*, proposto por Derrick Henry Lehmer na década de 1940. O método se baseia em uma relação de recorrência

linear para produzir sequências de valores aparentemente aleatórios. Esse gerador, frequentemente classificado como um *Gerador Linear Congruencial* (veja a definição na página 12), utiliza a multiplicação e o módulo como peças fundamentais, definindo cada novo número na sequência por meio de:

$$X_{k+1} = g \cdot X_k \mod n, \quad (1)$$

onde X_k é o estado atual, g é um multiplicador (idealmente uma raiz primitiva módulo n), n é o módulo (geralmente um número primo ou potência de um primo), e X_0 é a semente inicial, que deve ser coprima com n .

Apesar de sua simplicidade, o Gerador Lehmer revela questões importantes sobre a percepção de aleatoriedade, tema explorado por Stephen Wolfram em seus estudos sobre sistemas complexos. Wolfram demonstrou que a aparente aleatoriedade de uma sequência pode variar drasticamente dependendo de como ela é visualizada. Por exemplo, ao representar os números gerados por um gerador linear congruencial como uma sequência unidimensional de valores, a saída pode parecer caótica e desprovida de padrões. No entanto, ao organizá-los em uma matriz bidimensional — onde cada valor é plotado como um ponto em linhas e colunas — ou mesmo em um cubo tridimensional, estruturas subjacentes, como repetições ou diagonais, podem emergir, revelando a natureza determinística do processo. A ?? ilustra um exemplo utilizando $g = 3$ e $n = 2^{32}$. Esse fenômeno também ocorre

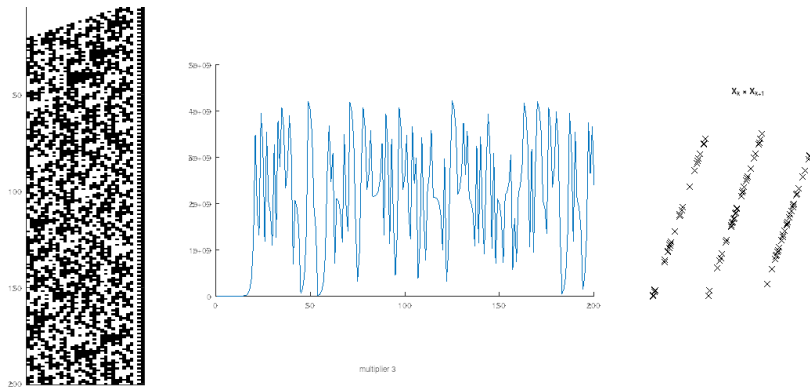


Figura 1: Representação da sequência de 200 números gerados pelo gerador Lehmer. À esquerda, a representação binária dos números, no meio um gráfico da sequência e, por fim, a representação de $X_k \times X_{k+1}$.

na Regra 30 proposta por Wolfram, onde um simples processo iterativo gera saídas que parecem aleatórias em uma dimensão, mas exibem ordem em representações visuais alternativas.

A relação entre aleatoriedade e visualização no contexto do Gerador Lehmer destaca suas limitações práticas. Se os parâmetros g e n forem mal escolhidos, a sequência pode apresentar ciclos curtos ou padrões visíveis, comprometendo sua utilidade em aplicações que exigem alta

imprevisibilidade. Por exemplo, em testes gráficos como aquele apresentado na ?? , geradores lineares simples frequentemente revelam retas ou grades, evidenciando correlações indesejadas. Mesmo com parâmetros otimizados, a natureza determinística do método implica que ele nunca produz aleatoriedade genuína. No entanto, sua simplicidade o torna adequado para simulações básicas e aplicações que não exigem números pseudoaleatórios de alta qualidade.

O Gerador Mersenne Twister

O *Mersenne Twister* é um algoritmo de geração de números pseudoaleatórios desenvolvido por Matsumoto e Nishimura (1998), amplamente reconhecido por sua capacidade de produzir sequências com propriedades estatísticas de alta qualidade e um período excepcionalmente longo. O nome “Mersenne Twister” reflete sua fundamentação matemática: o algoritmo utiliza números primos de Mersenne — números primos da forma $2^p - 1$, onde p é também um primo — e emprega uma técnica chamada *twisting* para garantir a geração de números com boa distribuição estatística e independência aparente. O algoritmo é projetado para prover uma distribuição uniforme e baixa correlação entre valores consecutivos.

O Mersenne Twister também oferece um equilíbrio entre qualidade estatística e eficiência computacional. Embora não seja o gerador mais rápido disponível — métodos mais simples, como geradores lineares congruenciais, podem ser mais rápidos em termos de tempo de execução —, ele compensa com sua robustez e confiabilidade, tornando-o adequado para uma ampla gama de aplicações.

O Mersenne Twister também possui uma boa eficiência computacional, sendo por isso amplamente adotado em linguagens de programação, como na função `rand` do Octave e do MATLAB, que utiliza a variante MT19937 com período $2^{19937} - 1$ (seu período extremamente longo), bem como em bibliotecas padrão de linguagens como Python (`random`) e C++ (`std::mt19937`). Apesar de suas qualidades, o Mersenne Twister é inadequado para aplicações criptográficas.

MT19937

O Mersenne Twister (Matsumoto e Nishimura 1998) é um dos geradores de números pseudoaleatórios (RNGs) mais amplamente utilizados, sendo um GFSR (*Generalized Feedback Shift Register*) com *twist*.

De acordo com Tan (2016), o algoritmo Mersenne Twister pode ser descrito pelas seguintes etapas:

1. Inicialização: Gere r números de w bits, representados por $(x_i, i = 1, 2, \dots, r)$, que serão os valores iniciais. Esses números são obtidos

a partir de uma semente (*seed*) fornecida.

2. Definição da Matriz de Recorrência: Construa a matriz A , dada por:

$$A = \begin{pmatrix} 0 & I_{w-1} \\ a_w & a_{w-1} \cdots a_1 \end{pmatrix},$$

onde I_{w-1} é a matriz identidade de tamanho $(w-1) \times (w-1)$; a_i , para $i = 1, \dots, w$, são valores fixos iguais a 0 ou 1, definidos previamente para garantir boas propriedades estatísticas.

3. Recorrência: Para cada $i \geq 0$, calcule o próximo número da sequência com a fórmula:

$$x_{i+r} = (x_{i+s} \oplus (x_i^{(w:(l+1))} \mid x_{i+1}^{(l:1)})A),$$

onde s é um inteiro predefinido que determina o deslocamento na sequência de estados, geralmente fixado como $s = r - n$ (onde n é um parâmetro do algoritmo, como o grau de recorrência); $x_i^{(w:(l+1))}$ são os $w - l$ bits mais significativos (parte superior) de x_i ; $x_{i+1}^{(l:1)}$ são os l bits menos significativos (parte inferior) de x_{i+1} ; $\mid$ representa a concatenação desses bits; $\oplus$ é a operação *bitwise XOR*.

4. Temperamento (*Tempering*): Para cada x_{i+r} , aplique as seguintes operações em sequência para melhorar a qualidade estatística do número gerado:

$$\begin{aligned} z &= x_{i+r} \oplus (x_{i+r} \gg t_1), \\ z &= z \oplus ((z \ll t_2) \wedge m_1), \\ z &= z \oplus ((z \ll t_3) \wedge m_2), \\ z &= z \oplus (z \gg t_4), \\ u_{i+r} &= z / (2^w - 1), \end{aligned}$$

onde t_1, t_2, t_3, t_4 são inteiros que definem o número de posições dos deslocamentos de bits; $\gg$ e $\ll$ são operações de deslocamento lógico à direita e à esquerda, respectivamente; m_1, m_2 são máscaras de bits predefinidas; $\wedge$ é a operação *bitwise AND*; u_{i+r} é o número final gerado, que estará no intervalo $[0, 1]$. A sequência u_{i+r} , para $i = 1, 2, \dots$, representa os números pseudoaleatórios gerados.

Com os parâmetros corretamente ajustados, como o valor de r , o Mersenne Twister pode gerar sequências com um período de até 2^{19937} , além de apresentar excelentes propriedades estatísticas (Matsmoto e Nishimura 1998).

Abordagens para Geração de Números Pseudoaleatórios

Existem diversas abordagens para gerar números pseudoaleatórios (Knuth 1997; Matsumoto e Nishimura 1998; Wolfram 2002; Kocarev e Lian 2011; Kneusel 2018). Abaixo listamos algumas delas:

- Geradores Lineares Congruenciais (*linear congruential generator*, LCGs):

Baseiam-se em uma relação de recorrência linear da forma

$$X_{n+1} = (a \cdot X_n + c) \mod m, \quad (2)$$

como o exemplo do Gerador Lehmer visto anteriormente. Este tipo de gerador é simples, mas com período curto e baixa qualidade estatística.

- Geradores de Fibonacci com Atraso (*Lagged Fibonacci generator*, LFGs):

Geram números combinando valores anteriores da sequência usando uma operação (como adição ou XOR) com atrasos:

$$X_n = (X_{n-k} \oplus X_{n-l}) \mod 2^w, \quad (3)$$

oferecendo períodos longos, mas exigindo estados iniciais grandes.

- Autômatos Celulares:

Aplicam regras como a Regra 30 vista anteriormente,

$$c_i(t+1) = c_{i-1}(t) \oplus (c_i(t) \vee c_{i+1}(t)), \quad (4)$$

gerando padrões caóticos, porém computacionalmente custosos.

- Funções de Hash e Criptográficas:

Utilizam funções como SHA-256 ($R_i = H(S||C)$)³ ou cifras de fluxo como ChaCha ($R_i = \text{ChaCha}(K, C)$), sendo seguras para criptografia, porém mais lentas.

³ Aqui, $S||C$ denota a concatenação das sequências S e C . Apesar da similaridade visual, é distinta da notação $P || Q$ para divergência de Kullback-Leibler, apresentada em capítulos futuros.

- Geradores Baseados em Caos:

Empregam sistemas caóticos, como o mapa logístico

$$X_{n+1} = r \cdot X_n \cdot (1 - X_n), \quad (5)$$

com comportamento caótico, mas sensíveis a precisão numérica.

Geradores de Números Pseudoaleatórios Criptograficamente Seguros (CSPRNGs)

Os geradores de números pseudoaleatórios (PRNGs), como aqueles abordados anteriormente, são destinados a aplicações onde a qualidade estatística da sequência é suficiente, como em simulações ou modelagem. Geradores de números aleatórios são fundamentais para a criptografia e a segurança de algoritmos criptográficos depende diretamente da qualidade desses números. Neste aspecto, referimos à imprevisibilidade e impossibilidade de se encontrar padrões nos números gerados.

Desta forma, os geradores mencionados anteriormente não são rigorosos suficiente para serem utilizados em contextos criptográficos. Um Gerador de Números Pseudoaleatórios Criptograficamente Seguro (CSPRNG⁴) é um PRNG projetado para garantir não apenas boas propriedades estatísticas, mas também imprevisibilidade e resistência a ataques, mesmo contra adversários com recursos computacionais significativos.

⁴ Cryptographically secure pseudorandom number generator.

Sistemas criptográficos requerem números aleatórios em diversas situações, por exemplo: geração de chaves criptográficas (as chaves devem ser imprevisíveis para evitar ataques de força bruta ou inferência); vetores de inicialização (muitos métodos requerem dados de inicialização para garantir imprevisibilidade e vazamento de informação); nonces (números utilizados uma única vez em diversos protocolos necessitam de valores únicos e inesperados para evitar ataques de repetição); salts (sequências aleatórias adicionadas a senhas dificultam ataques de dicionário ou tabelas arco-íris); tokens (são utilizados em sistemas de autenticação para garantir unicidade e segurança).

Além dos requisitos típicos de PRNGs, como distribuição uniforme e período longo, os CSPRNGs devem atender a critérios adicionais:

- Imprevisibilidade: Dado um trecho da sequência gerada, não deve ser possível prever os próximos bits com probabilidade significativamente maior que a aleatória.
- Resistência a ataques: Mesmo com acesso a grandes quantidades de saída ou controle parcial sobre as entradas, um adversário não deve ser capaz de reconstruir o estado interno ou comprometer a segurança.
- Entropia adequada: A fonte de inicialização (*seed*) deve ter entropia suficiente, frequentemente coletada de fontes físicas, como ruído térmico ou eventos de hardware.

Esses requisitos tornam os CSPRNGs mais lentos que PRNGs comuns, mas sua robustez é essencial para aplicações sensíveis à segurança.

Exemplo 1 (CSPRNG Baseado em AES no Modo CTR). Um exemplo ilustrativo de CSPRNG é o uso do algoritmo AES (*Advanced Encryption Standard*) no modo *Counter* (CTR). Nesse esquema, uma chave secreta K e um contador inicial C_0 são usados para gerar uma sequência pseudoaleatória. A cada iteração, o contador C_i é incrementado, e a saída é obtida cifrando C_i com K :

$$\text{Saída}_i = \text{AES}_K(C_i)$$

A sequência resultante é concatenada para formar a saída do CSPRNG. A segurança depende da força do AES e da entropia da chave K e do contador inicial C_0 .

Require: Chave K , contador inicial C_0 , tamanho da saída n

Ensure: Sequência pseudoaleatória S de tamanho n

$S \leftarrow \emptyset$

$C \leftarrow C_0$

while $|S| < n$ **do**

$S \leftarrow S || \text{AES}_K(C)$

$C \leftarrow C + 1$

return $S[0 : n]$

Algorithm 1: Geração de Sequência
Pseudoaleatória com AES-CTR

Princípios Fundamentais

A *comunicação* envolve alguns componentes fundamentais: uma fonte, que gera a mensagem; um transmissor, que envia a mensagem através de algum meio; um canal de comunicação, que é o meio pelo qual a comunicação se estabelece, sendo suscetível a ruídos e interferências; um receptor, que recebe a mensagem; e, por fim, o destinatário da mensagem (veja o diagrama na ?? , adaptado de “*A Mathematical Theory of Communication*” Shannon 1948). No âmago do processo de comunicação reside um problema fundamental: reconstruir no receptor a exata mensagem pretendida pelo emissor. Independentemente de estarmos lidando com a comunicação falada, escrita, ou por meio de sinais digitais, o objetivo é o mesmo. Embora cada sistema guarde suas nuances, a teoria da informação proposta por Shannon (1948) busca lidar com essa problemática sob uma perspectiva unificada. Para tanto, ele introduziu conceitos fundamentais como *entropia* e *capacidade de canal*. Neste livro, procuraremos apresentar uma visão geral da teoria, seus fundamentos e aspectos práticos.

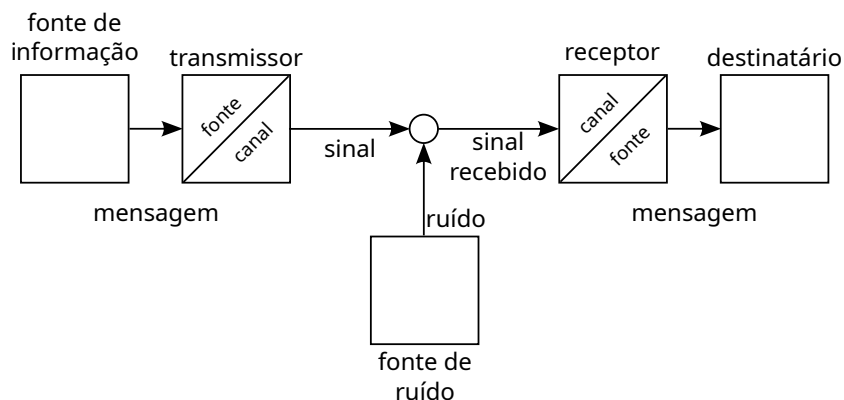


Figura 2: Diagrama genérico de um sistema de comunicações.

O diagrama ilustrado na ?? apresenta a sistematização proposta por Shannon (1948), incluindo conceitos e aspectos da comunicação já utilizados à época, mas trazendo uma abordagem sistemática para o conceito de comunicação. Além disso, ele divide a tarefa do codificador e decodificador em duas partes distintas: a codificação/decodificação

de fonte e a codificação/decodificação de canal. Essa separação permite abordar cada uma dessas partes do sistema de comunicação de forma independente, possibilitando, assim, projetar e otimizar cada uma delas separadamente.

Para lidar matematicamente com a comunicação, um processo que envolve incerteza e variabilidade, e trazer então uma perspectiva probabilística na abordagem do problema, inicialmente, é essencial entender o conceito de *variável aleatória* (v.a.). Uma variável aleatória é uma variável que pode assumir diferentes valores dependendo de fatores aleatórios, ou seja, há um mecanismo não determinístico que torna impossível prever seu valor. Ela representa a incerteza inerente a eventos ou processos aleatórios e pode ser utilizada para modelar diversos experimentos, como o lançamento de um dado, a previsão do tempo ou a sequência de caracteres em um texto. Na notação aqui adotada, as variáveis aleatórias serão representadas por letras maiúsculas, como, por exemplo, X . Uma *variável aleatória discreta* é aquela que pode assumir valores em um conjunto enumerável. As realizações de uma v.a. se dão de acordo com uma distribuição subjacente p , e assim dizemos que $X \sim p$. Para representar o conjunto de valores que a variável aleatória pode assumir, utilizaremos a letra em forma caligráfica $\mathcal{X}$, enquanto um valor específico que a variável assume será denotado por uma letra minúscula x . Assim, podemos descrever que a variável aleatória X assume o valor x através da realização do evento $\{X = x\}$.

Considere uma v.a. X , em um alfabeto $\mathcal{X}$ de cardinalidade $N = |\mathcal{X}|$, onde $\mathcal{X} = \{a_1, \dots, a_N\}$ e a_i , $i = 1, \dots, N$, representa cada um dos possíveis valores que a v.a. pode assumir com probabilidade p_i , ou seja, $p_i = \Pr(X = a_i)$. A distribuição subjacente que rege a v.a. X é $p = \mathcal{P}_X = \{p_1, \dots, p_N\}$, tal que $p_i \geq 0$ e $\sum_{i=1}^N p_i = 1$ ou, de forma equivalente, $\sum_{x \in \mathcal{X}} \Pr(X = x) = 1$.

Hartley (1928) introduziu a ideia de que a quantidade de informação pode ser medida em termos de possibilidades. Hartley propôs que a informação é diretamente proporcional ao logaritmo do número de estados possíveis de um sistema, estabelecendo assim uma base para a quantificação da informação. A medida de informação associada a uma variável aleatória X , com alfabeto de tamanho N , é expressa por

$$I(X) = \log_b L, \quad (6)$$

onde b é a base utilizada para medir tal informação. Para $b = 2$, a informação será medida em ‘bits’ (nome sugerido por J.W. Tukey).

O conceito proposto por Hartley sobre a quantificação da informação é consonante com a interpretação da capacidade representacional de uma unidade de memória. Uma unidade de memória com n bits é capaz de representar 2^n sequências binárias distintas, o que implica

que essa unidade possui n bits de informação. Quando consideramos duas unidades de memória, cada uma com n bits, a capacidade total se torna $2n$ bits, permitindo a representação de 2^{2n} sequências binárias distintas. Essa relação demonstra que, conforme a quantidade de bits de memória aumenta, apesar do número de sequências possíveis crescer exponencialmente, a capacidade de representação de informações cresce proporcionalmente ao crescimento da memória, alinhando-se perfeitamente com a proposta de Hartley de que a informação é medida em função do logaritmo do número de estados possíveis.

Um conceito fundamental que emerge na discussão sobre a teoria da informação é a *redundância*, ou ainda, a previsibilidade. A redundância refere-se à parte da informação que é repetitiva ou desnecessária para a compreensão, podendo ser eliminada sem perda de significado. Enquanto, sob uma primeira vista, pode ser vista como um desperdício de recursos. A redundância também desempenha um papel crucial na detecção e correção de erros, permitindo que a informação seja transmitida de forma mais confiável em ambientes ruidosos. Exemplo disso é a redundância presente na comunicação escrita. Veja como o trecho a seguir pode ser facilmente compreendido, mesmo com lacunas: “Nda se mudria, o regim, si era posívl, ms tmbém s muda d roupa sm trcar d pele.”⁵ A capacidade de compreender o trecho, apesar das palavras truncadas e da ausência de letras, demonstra que a linguagem é intrinsecamente projetada para lidar com a incerteza e a imperfeição no processo de transmissão. Se não houvesse redundância, qualquer perda de informação tornaria a decodificação da mensagem inviável, resultando em confusão e mal-entendidos.

Shannon, por outro lado, incorporou o conceito de redundância em sua análise da informação. Ao considerar eventos com probabilidades distintas, é importante notar que a incerteza associada a esses eventos não pode ser a mesma. Assim, a incerteza não depende apenas do tamanho do alfabeto, mas também das probabilidades associadas a cada evento. Para um evento E_k com probabilidade p_k , a quantidade de informação (medida em bits) associada a esse evento é expressa por

$$I(E_k) = \log_2(1/p_k) = -\log_2(p_k). \quad (7)$$

Essa relação indica que eventos menos prováveis, ou seja, aqueles com probabilidades mais baixas, geram uma maior quantidade de informação, pois sua ocorrência é inesperada. Em contrapartida, eventos com alta probabilidade resultam em uma quantidade menor de informação, refletindo a redundância presente na comunicação. Essa compreensão da relação entre probabilidade e informação é essencial para o desenvolvimento do conceito de entropia, que serve como uma medida da incerteza, aleatoriedade e quantidade de informação associada a uma variável aleatória.

⁵ Trecho original: “Nada se mudaria; o regime, sim, era possível, mas também se muda de roupa sem trocar de pele. Comércio é preciso. Os bancos são indispensáveis. No sábado, ou quando muito na segunda-feira, tudo voltaria ao que era na véspera, menos a constituição.” (cap. LXIV de Esaú e Jacó, Machado de Assis).

Entropia

Shannon (1948) propôs a entropia com forma de quantificar a incerteza associada a uma v.a., fornecendo assim uma medida da aleatoriedade. A entropia nos permite entender como a distribuição de probabilidades de uma variável aleatória influencia na informação associada a ela. A entropia de uma v.a. é então expressa como o valor esperado da informação própria associada aos eventos no alfabeto desta v.a., ou seja, $E_p[I(E_k)]$ e será expressa por $H(X)$.

Definição 1 (Entropia).

$$H(X) \triangleq E_p[-\log(p_k)] \quad (8a)$$

$$= - \sum_{k=1}^{|\mathcal{X}|} p_k \log p_k \quad (8b)$$

$$= - \sum_{x \in \mathcal{X}} p(x) \log p(x), \quad (8c)$$

Passamos a adotar a convenção $\log = \log_2$, uma vez que estaremos sempre medindo a informação em bits. Quando necessário utilizar uma base diferente, expressaremos explicitamente (por exemplo, $\ln = \log_e$ para base e e $\log_{10}$ para base decimal).

Para encontrar a formulação dada na Equação (8), Shannon (1948) propôs uma função de entropia H , que depende da distribuição de massa p . Dado um alfabeto de tamanho $N = |\mathcal{X}|$, e distribuição $p = \{p_1, \dots, p_N\}$, Shannon estabeleceu três propriedades fundamentais que a função de entropia deveria satisfazer. A primeira propriedade exige que H seja contínua em relação às probabilidades p_i , o que significa que pequenas mudanças nas probabilidades não devem causar grandes saltos na medida de informação. A segunda propriedade afirma que, se todos os p_i são iguais, a entropia deve aumentar monotonicamente com o tamanho do alfabeto N . Isso reflete a intuição de que mais opções disponíveis para escolha devem resultar em maior incerteza e, portanto, maior entropia. Por fim, a terceira propriedade, conhecida como a propriedade da aditividade, estabelece que se uma escolha pode ser decomposta em uma sequência de escolhas independentes, a entropia total deve ser a soma ponderada das entropias individuais de cada escolha. Essa propriedade é crucial para a análise de sistemas complexos, onde a informação pode ser tratada em partes menores e combinadas para obter um resultado global.

A formulação matemática da entropia de Shannon, dada na Equação (8), emerge naturalmente ao considerar essas propriedades. A função logarítmica utilizada possui a sua capacidade de transformar multiplicações em somas, o que é essencial para garantir a aditividade da medida de informação.

Interlúdio

Este interlúdio fornece uma pausa para apresentar a demonstração contida no Anexo 2 do artigo de Shannon (1948). O leitor, se desejar, pode ir direto à página 20, onde finaliza-se este entreato.

Define-se a $A(N)$ como o caso específico de H para uma distribuição uniforme com alfabeto de tamanho N :

$$A(N) = H\left(\frac{1}{N}, \frac{1}{N}, \dots, \frac{1}{N}\right). \quad (9)$$

Deseja-se que uma escolha dentre s^M opções igualmente prováveis possa ser decomposta como uma sequência de M escolhas que se subdividem em s possibilidades igualmente prováveis.

Teremos então que

$$A(s^M) = MA(s). \quad (10)$$

Da mesma forma, para t e N , teremos $A(t^N) = NA(t)$. Podemos tomar N arbitrariamente grande e encontrar M que satisfaça

$$s^M \leq t^N \leq s^{(M+1)}. \quad (11)$$

Tomando o logaritmo⁶ da expressão acima e dividindo por $N \log s$ ⁷, teremos

$$\frac{M}{N} \leq \frac{\log t}{\log s} \leq \frac{M}{N} + \frac{1}{N}, \quad (12)$$

o que é equivalente a

$$\left| \frac{M}{N} - \frac{\log t}{\log s} \right| < \epsilon, \quad (13)$$

onde ϵ é arbitrariamente pequeno, para N arbitrariamente grande.

Usando agora a propriedade desejada de monotonicidade de $A(N)$, teremos

$$A(s^M) \leq A(t^N) \leq A(s^{(M+1)}) \quad (14a)$$

$$MA(s) \leq NA(t) \leq (M+1)A(s) \quad (14b)$$

Dividindo a expressão acima por $NA(s)$, teremos

$$\frac{M}{N} \leq \frac{A(t)}{A(s)} \leq \frac{M}{N} + \frac{1}{N}, \quad (15)$$

ou, de forma equivalente,

$$\left| \frac{M}{N} - \frac{A(t)}{A(s)} \right| < \epsilon, \quad (16)$$

e assim, como as duas frações ($\log t / \log s$ e $A(t)/A(s)$) estão ϵ próximas de M/N , podemos concluir que, para N grande suficiente,

$$\left| \frac{A(t)}{A(s)} - \frac{\log t}{\log s} \right| < 2\epsilon. \quad (17)$$

Como ϵ é arbitrariamente pequeno, no limite teremos

$$\frac{A(t)}{A(s)} = \frac{\log t}{\log s} \quad (18a)$$

$$A(t) = \frac{A(s)}{\log s} \log t = K \log t, \quad (18b)$$

onde K deve ser positivo, de forma que $A(N)$ seja monótona crescente.

⁶ Logaritmo é uma função monótona crescente.

⁷ $N \log s$ é positivo para $N \geq 0$ e $s \geq 1$.

Suponha uma escolha com N possibilidades em que as probabilidades são comensuráveis, $p_i = N_i / \sum N_i$, onde N_i são inteiros. De forma equivalente, uma escolha entre $\sum N_i$ opções pode ser expressa como uma escolha dentre N opções com probabilidades $p_1, \dots, p_N$, e para uma i -ésima dada escolha, realizar uma nova escolha dentre N_i opções igualmente prováveis. Teremos então:

$$\overbrace{K \log \left(\sum N_i \right)}^{A(\sum N_i)} = H(p_1, \dots, p_N) + \overbrace{K \log N_i}^{A(N_i)} \quad (19a)$$

$$K \underbrace{\left(\sum_{i=1} p_i \right)}_{=1} \log \left(\sum N_i \right) = H(p_1, \dots, p_N) + K \underbrace{\left(\sum_{i=1} p_i \right)}_{=1} \log N_i. \quad (19b)$$

E assim,

$$H(p_1, \dots, p_N) = K \left[\left(\sum p_i \right) \log \left(\sum N_i \right) - \left(\sum p_i \right) \log N_i \right] \quad (20a)$$

$$= -K \sum p_i \log \frac{N_i}{\sum N_i} = -K \sum p_i \log p_i. \quad (20b)$$

□

Exemplo 2 (Entropia de uma v.a. binária). Considere uma v.a. binária $X \in \mathcal{X} = \{0, 1\}$. Para simplificar, iremos denotar arbitrariamente por $\theta = Pr(X = 1)$ e $1 - \theta = Pr(X = 0)$. Usando agora a definição dada na Equação (8), teremos

$$H(X) = -\theta \log \theta - (1 - \theta) \log(1 - \theta) \quad (21a)$$

$$= h(\theta) \quad (21b)$$

onde, para simplificar, denotamos esta grandeza por $h(\theta)$. Sempre que utilizarmos esta notação, como por exemplo em $h(1/4)$, $h(1/3)$ ou $h(\pi)$, estaremos nos referindo à *entropia binária* definida na Equação (21). Em alguns casos, pode aparecer a notação $H(\theta)$ para representar a entropia binária, sem comprometer o entendimento, uma vez que ficará claro pelo contexto se tratar de uma entropia binária.

A Figura 3 apresenta o gráfico da entropia binária dada pela Equação (21). Note como $H = 0$ quando $\theta = 1$ ou $\theta = 0$, ou seja, nos casos em que um dos eventos ocorre com probabilidade 1, não existindo assim incerteza. Observe ainda como o máximo é alcançado em $\theta = 1/2$, quando ambos eventos são equiprováveis (incerteza máxima). A função observada é côncava, o que, mais à frente, mostraremos ser válido para toda função de entropia. A concavidade garante ainda que ao misturarmos (ponderarmos) distribuições diferentes não é possível obter uma entropia menor que a ponderação das entropias das partes.

Exemplo 3 (Entropia de uma v.a. uniforme). Vamos considerar agora o exemplo de uma v.a. com distribuição uniforme $X \sim u$.

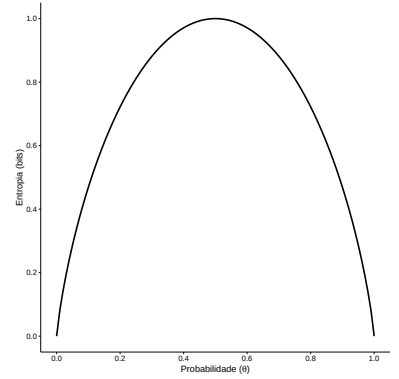


Figura 3: Entropia binária

Usando a definição na Equação (8), teremos que a entropia de X será dada por

$$H(X) = - \sum_{k=1}^{|\mathcal{X}|} \frac{1}{|\mathcal{X}|} \log \frac{1}{|\mathcal{X}|} \quad (22a)$$

$$= - \log \frac{1}{|\mathcal{X}|} = \log |\mathcal{X}|. \quad (22b)$$

Como esperávamos, a entropia de uma distribuição uniforme é monotonicamente crescente com o tamanho do alfabeto. Intuitivamente, como já mencionado, a entropia é máxima para uma distribuição uniforme, uma vez que é a situação de maior incerteza para um dado alfabeto.

Ao analisarmos a Equação (8), podemos nos questionar o que ocorre quando algum símbolo tem probabilidade zero, uma vez que aparecerá um termo $0 \log 0$ no somatório. Como este termo é indefinido, iremos, neste caso, usar o valor limite $\lim_{x \rightarrow 0} x \log x = 0$. Para demonstrar (a menos de uma constate pela mudança de base do logaritmo), usaremos para tanto a regra de L'Hospital:

$$\lim_{x \rightarrow 0} x \ln x = \lim_{x \rightarrow 0} \frac{\ln x}{1/x} \quad (23a)$$

$$= \lim_{x \rightarrow 0} \frac{1/x}{-1/x^2} \quad (23b)$$

$$= \lim_{x \rightarrow 0} -x = 0. \quad (23c)$$

Devemos então nos atentar a isto, sobretudo quando realizarmos cálculos numéricos (a função para cálculo da entropia deve tratar estes casos).

Exemplo 4 (Entropia de uma distribuição de Zipf). Neste exemplo, vamos buscar aproximar a entropia de textos escritos. Sabemos que a distribuição das palavras em textos segue uma lei de potência conhecida como Lei de Zipf (Zipf 1935; Zipf 1949; Ferrer i Cancho e Solé 2001; Araújo, Cristófar-Silva e Yehia 2013). A Lei de Zipf é uma observação empírica que também é observada em diversos outros fenômenos naturais. Ao analisar um corpus de texto, Zipf constatou que a frequência de uma palavra é inversamente proporcional à sua posição em um ranking de frequência. Essa relação pode ser expressa matematicamente como

$$p_k(s, N) = C k^{-s}, \quad (24)$$

onde p_k é a probabilidade de ocorrência da k -ésima palavra mais frequente, C é uma constante de normalização ($C^{-1} = \sum_{n=1}^N n^{-s}$), k é o *rank*, s a constante que caracteriza a distribuição e N o tamanho do léxico. A Exemplo 4 apresenta o gráfico de Zipf (frequência/probabilidade vs. *rank*) para a obra completa de William Shakespeare, dividida em 44 textos, conforme organizados no Projeto

Gutenberg⁸. Cada uma das série de pontos na Exemplo 4 representa um destes 44 textos. Note como o comportamento observado entre eles é bem similar.

⁸ <https://www.gutenberg.org/>

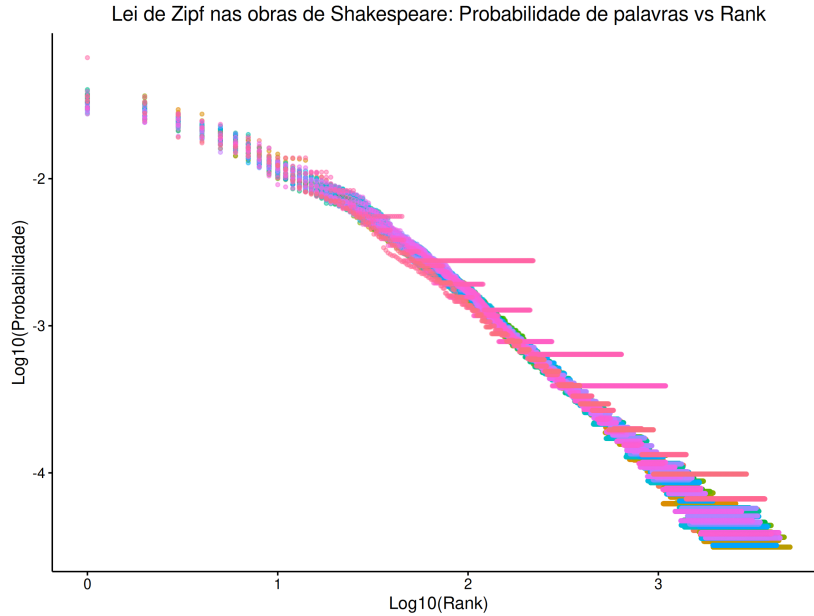


Figura 4: Gráfico de Zipf para 44 textos de William Shakespeare no Projeto Gutenberg.

Usando a Equação (24) em conjunto com a Equação (8), obtemos a formulação da entropia para uma distribuição de Zipf:

$$H(X) = sC \sum_{k=1}^N \frac{\log k}{k^s} - \log C. \quad (25)$$

Após estimar a entropia de cada um dos textos de Shakespeare, podemos observar no histograma da Figura 5 como a concentração dos valores encontrados está por acima de 9 bits. Como observaremos mais à frente, conhecer o valor da entropia é importante para sabermos qual é o limite representacional para uma fonte (qual é o limite da compressão). Além disso, linguistas utilizam a entropia para analisar a complexidade de uma língua, redundância e eficiência do sistema de escrita.

Propriedades da Entropia

A entropia possui várias propriedades fundamentais que nos ajudam a entender seu comportamento e medidas relacionadas, como também serão úteis em diversas demonstrações. Nesta seção, exploraremos algumas das principais propriedades da entropia, incluindo a não negatividade, a concavidade, valor máximo, a continuidade, e a mudança de base.

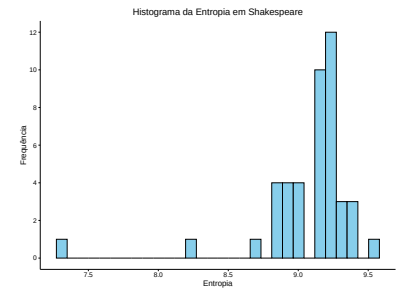


Figura 5: Histograma da entropia nos 44 textos de William Shakespeare.

Usualmente a unidade escolhida para medir a informação é bit, e portanto utilizamos a base 2. Entretanto, a entropia pode ser dada também em outras bases: nats (base e), trits (base 3), hartley (base 10). Para obtermos a entropia em outras bases, basta utilizar a regra de mudança de base do logaritmo: $\log_a x = \log_b x / \log_b a$. Teremos assim $H_b(X) = (\log_b a) H_a(X)$. A transformação para a base e costuma ser útil nas demonstrações para evitar o fardo de se carregar uma constante durante todo o seu percurso.

Uma propriedade muito importante é a não negatividade, ou seja, $H(X) \geq 0$. A incerteza nunca⁹ será negativa, o que faz sentido intuitivamente. Além disso, a não negatividade é uma propriedade importante para encontrarmos outras relações. Para verificá-la, basta notar que a entropia é a soma de termos da forma $-p \log p$. Como $0 \leq p \leq 1$, teremos que cada termo da soma é maior ou igual a zero. Por conseguinte, a entropia será não negativa.

Para verificar que a entropia é uma função contínua em p , devemos primeiramente definir $l(x)$ como a extensão contínua de $x \log x$, da seguinte forma:

$$l(x) = \begin{cases} x \log x & \text{se } x > 0, \\ 0 & \text{se } x = 0, \end{cases} \quad (26)$$

e assim, $H(X) = H(p)$ é definido como

$$H(p) = - \sum_{x \in \mathcal{X}} l(p_x). \quad (27)$$

Sendo a entropia definida como um somatório finito de funções contínuas, fica evidente que ela também será uma função contínua.

Antes de abordarmos uma próxima propriedade fundamental da entropia, vamos estabelecer uma desigualdade simples e importante em diversas demonstrações futuras. Ela é apresentada graficamente na Figura 6 e demonstrada formalmente no Lema 1.

Lema 1 (Desigualdade Fundamental) *Para qualquer $z > 0$,*

$$\ln z \leq z - 1, \quad (28)$$

com igualdade se e somente se $z = 1$.

Demonstração. Sabemos que para $z = 1$ é verdadeiro, pois $0 = \ln 1 \leq 1 - 1 = 0$. Vamos demonstrar que $\ln z \leq z - 1$, para $z > 1$ por contradição. Suponha que existe $b > 1$ tal que $\ln b > b - 1$, para $z = b$. Vamos definir $f(z) = \ln z - z + 1$, logo $f(1) = 0$ (conforme visto acima) e $f(b) > 0$ (por hipótese). Pelo Teorema do Valor Médio $\exists c$, $1 < c < b$, tal que

$$f'(c) = \frac{f(b) - f(1)}{b - 1} = \underbrace{\frac{f(b)}{b - 1}}_{>0} > 0. \quad (29)$$

⁹ Importante ressaltar que estamos aqui tratando de variáveis discretas. No contexto contínuo a entropia pode ser negativa e terá outra interpretação.

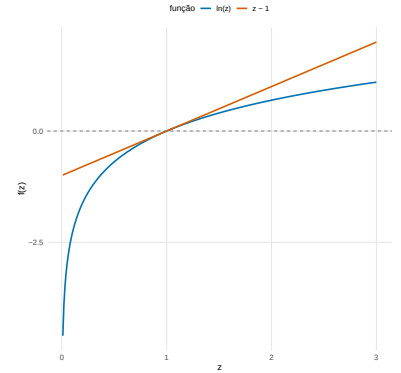


Figura 6: Demonstração gráfica da desigualdade $\ln z \leq z - 1$ (embora não seja uma demonstração formalmente válida, serve como intuição).

Mas, $f'(z) = 1/z - 1$, e assim $f'(z) < 0$ para $z > 1$. Logo há uma contradição e nossa hipótese é falsa. Teremos assim $\ln z \leq z - 1$, para $z \geq 1$.

Para $z \in (0, 1)$, basta seguir os mesmos passos, escolhendo um ponto $z = b$, tal que $0 < b < 1$. Vamos encontrar um ponto c tal que $0 < b < c < 1$. E usar o Teorema do Valor Médio para mostrar uma contradição na hipótese. $\square$

Outra propriedade importante que mencionamos é o limite máximo da entropia, sendo este a entropia da distribuição uniforme.

Teorema 2 (Limite superior da entropia) *A entropia é limitada superiormente pela entropia da distribuição uniforme:*

$$H(X) \leq \log N. \quad (30)$$

Demonstração. Note que, mostrar que $H(X) \leq \log N$ é equivalente a mostrar que $H(X) - \log N \leq 0$. Para tanto, vejamos:

$$H(X) - \log N = - \sum_x p(x) \log p(x) - \log N \overbrace{\sum_x p(x)}^{=1} \quad (31a)$$

$$= - \sum_x p(x) \log p(x) - \sum_x p(x) \log N \quad (31b)$$

$$= - \sum_x p(x) \log (p(x) N) \quad (31c)$$

$$= \log_2 e \sum_x p(x) \ln \frac{1}{p(x)N} \quad (31d)$$

$$\leq \log_2 e \sum_x p(x) \left[\frac{1}{p(x)N} - 1 \right] \quad (31e)$$

$$= \log_2 e \left[\underbrace{\sum_{x \in \mathcal{X}} \frac{1}{N}}_{=N \cdot \frac{1}{N}=1} - \underbrace{\sum_x p(x)}_{=1} \right] = 0. \quad (31f)$$

Em 31e, utilizamos a Equação (28), com $z = 1/p(x)N$. A igualdade $\ln z = z - 1$ se dará no ponto $z = 1$, isto é, quando $1/p(x)N = 1$, ou seja, quando $p(x) = 1/N$, e teremos assim uma distribuição uniforme. $\square$

A concavidade da entropia é uma propriedade importante. Garante que a entropia terá um único máximo global. A maximização da entropia é um princípio fundamental em várias áreas como estatística, para a inferência de distribuições de probabilidade, e aprendizado de máquina, para garantir que os algoritmos de otimização converjam e

para garantir a estabilidade dos modelos. Esta propriedade será demonstrada mais à frente no Teorema 11, pois antes precisaremos fazer mais algumas definições e demonstrações.

Entropia Conjunta e Entropia Condicional

Quando lidamos com incerteza em situações que envolvem múltiplas variáveis aleatórias, *entropia conjunta* e a *entropia condicional* são conceitos que surgem da extensão das definições vistas anteriormente. Um par de variáveis aleatórias (X, Y) pode ser visto como uma variável aleatória vetorial.

A entropia conjunta de duas variáveis aleatórias X e Y , denotada como $H(X, Y)$, mede a incerteza total associada ao par de variáveis. É definida como:

Definição 2 (Entropia Conjunta).

$$H(X, Y) \triangleq - \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y) \log p(x, y) \quad (32a)$$

$$= \mathbb{E}_{p(x, y)} \log \frac{1}{p(X, Y)} \quad (32b)$$

$$= \mathbb{E} \log \frac{1}{p(X, Y)}, \quad (32c)$$

onde, em 32c, para simplificar¹⁰, omitimos a distribuição em respeito a qual o valor esperado é tomado.

Generalizando para um vetor de variáveis aleatórias $X_{1:N} = (X_1, X_2, \dots, X_N)$:

$$H(X_{1:N}) = H(X_1, X_2, \dots, X_N) \quad (33a)$$

$$= \sum_{x_1, x_2, \dots, x_N} p(x_1, \dots, x_N) \log \frac{1}{p(x_1, \dots, x_N)} \quad (33b)$$

$$= \mathbb{E} \log \frac{1}{p(X_1, \dots, X_N)}. \quad (33c)$$

Dada a definição, vamos abordar um exemplo prático para analisar a interdependência entre caracteres consecutivos em uma língua natural. Para calcular a entropia conjunta de dois caracteres em sequência, devemos primeiramente calcular a probabilidade conjunta. A ?? apresenta o resultado encontrado para a obra completa de Shakespeare. Note como os maiores valores correspondem às sequências ‘th’, ‘he’, ‘er’, ‘an’, dentre outras (devido às palavras de alta frequência de ocorrência na língua, como “the”, “in”, “her”, “there”, etc.

Ao calcular a entropia conjunta encontramos $H(X, Y) = 7.8482$ bits, o que é menor do que o dobro da entropia de um único caractere, $2 \times H(X) = 8.3798$ bits. O fato da entropia conjunta de dois caracteres consecutivos ser menor do que o dobro da entropia de um único

¹⁰ Tais simplificações podem aparecer ao longo do texto. Porém as adotaremos apenas em casos que não gerem ambiguidades.

caractere indica que os caracteres não são independentes: a ocorrência de um caractere influencia a probabilidade do próximo, reduzindo a incerteza total. Em uma língua natural, essa interdependência é esperada devido a padrões linguísticos.

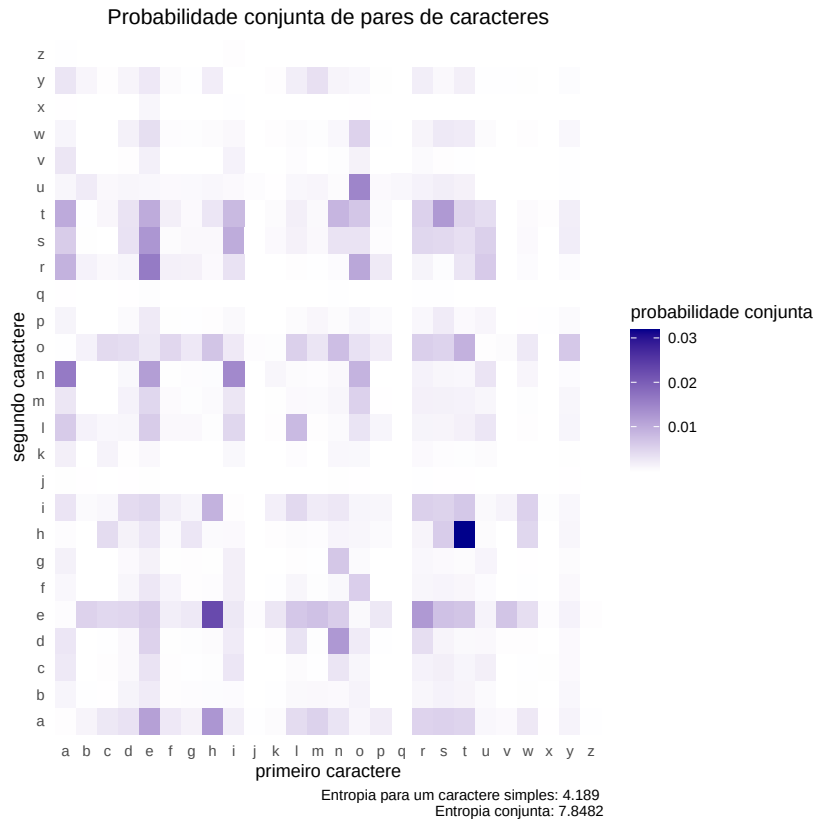


Figura 7: Probabilidade conjunta do primeiro e segundo caractere analisando os 44 textos de William Shakespeare no Projeto Gutenberg. As entropias simples e conjunta são apresentadas, evidenciando que a entropia conjunta é maior. Porém, analisando a entropia por caractere, a entropia conjunta abarca dependências entre caracteres consecutivos, reduzindo o efeito de incerteza por caractere.

Essa interdependência sugere que a incerteza associada ao segundo caractere (Y), dado o conhecimento do primeiro caractere (X), é menor do que a incerteza do segundo (Y) quando considerado isoladamente. Em outras palavras, o conhecimento de X fornece informação que reduz a incerteza sobre Y , como ocorre em pares comuns como 'th', onde 't' aumenta a probabilidade de 'h'. Essa incerteza reduzida é formalmente chamada de entropia condicional, denotada por $H(Y|X)$, e é definida como

Definição 3 (Entropia Condicional).

$$H(Y|X) \triangleq \sum_x p(x) H(Y|X = x) \quad (34a)$$

$$= - \sum_x p(x) \sum_y p(y|x) \log p(y|x) \quad (34b)$$

$$= - \sum_{x,y} p(x, y) \log p(y|x) \quad (34c)$$

$$= E_{p(x,y)} \log \frac{1}{p(Y|X)}. \quad (34d)$$

A definição dada pela Equação (34a) nos mostra como a entropia condicional de Y , dado X , é o valor esperado da entropia de Y condicionada aos eventos x . Observamos anteriormente (e será demonstrado posteriormente) que $H(Y|X) \leq H(Y)$, pois conhecer X reduz a incerteza sobre Y . Entretanto, devemos ressaltar que nada podemos dizer com relação a $H(Y|X = x)$, podendo este ser menor, igual ou maior que $H(Y)$. Para um caso específico, condicionar pode aumentar a incerteza, embora, na média, a entropia condicional seja sempre menor ou igual a entropia sem ser condicionada. Um exemplo interessante, adaptado de Cover e Thomas (2006), é o contexto de um julgamento: uma única evidência pode aumentar a incerteza sobre um determinado caso, o que pode levar a um julgamento enviesado; entretanto, conhecer um conjunto de evidência nos leva esperar uma incerteza menor, eliminando o viés no julgamento.

Note que, condicionando $H(Y|X)$ a Z , usando a formulação da Equação (34a) podemos escrever

$$H(Y|X, Z) = \sum_z p(z) H(Y|X, Z = z), \quad (35)$$

onde

$$H(Y|X, Z = z) = - \sum_{x,y} p(x, y|z) \log p(y|x, z). \quad (36)$$

Observando as definições de entropia conjunta e entropia condicional, surge, naturalmente, a constatação de que a entropia de um par de variáveis aleatória é a entropia de uma somada à entropia condicional da outra. Esta observação é formalizada pelo Teorema da Regra da Cadeia.

Teorema 3 (Regra da Cadeia)

$$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y) \quad (37)$$

Demonstração. Como $p(x, y) = p(x)p(y|x)$ (Teorema de Bayes), e como o logaritmo de um produto é a soma dos logaritmos dos fatores, teremos

$$-\log p(x, y) = -\log p(x) - \log p(y|x) \quad (38)$$

onde multiplicamos por (-1) ambos os lados. Calculando o valor esperado de ambos os lados, obtemos o resultado desejado. $\square$

Exemplo 5. Considere duas variáveis aleatórias X e Y , com $\mathcal{X} = \{x_1, x_2, x_3\}$ e $Y = \{y_1, y_2, y_3\}$. A distribuição conjunta $p(x, y)$ é dada:

$p(x, y)$	y_1	y_2	y_3
x_1	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{16}$
x_2	$\frac{1}{8}$	$\frac{1}{16}$	$\frac{1}{32}$
x_3	$\frac{1}{16}$	$\frac{1}{32}$	$\frac{1}{4}$

A entropia conjunta é dada por

$$\begin{aligned} H(X, Y) &= -2\frac{1}{4} \log \frac{1}{4} - 2\frac{1}{8} \log \frac{1}{8} - 3\frac{1}{16} \log \frac{1}{16} - 2\frac{1}{32} \log \frac{1}{32} \\ &= 1 + \frac{6}{8} + \frac{3}{4} + \frac{10}{32} = 2.8125 \text{ bits.} \end{aligned}$$

Para calcular $H(X)$ devemos primeiramente calcular a marginal

$$p(x) = \sum_y p(x, y) = \left\{ \frac{7}{16}, \frac{7}{32}, \frac{11}{32} \right\}.$$

e assim podemos calcular a entropia de X

$$\begin{aligned} H(X) &= -\frac{7}{16} \log \frac{7}{16} - \frac{7}{32} \log \frac{7}{32} - \frac{11}{32} \log \frac{11}{32} \\ &= \frac{7}{16} (4 - \log 7) + \frac{7}{32} (5 - \log 7) + \frac{11}{32} (5 - \log 11) \\ &= \frac{146}{32} - \frac{21}{32} \log 7 + \frac{55}{32} - \frac{11}{32} \log 11 = 1.5310 \text{ bits.} \end{aligned}$$

Para calcular a entropia condicional $H(Y|X)$, devemos calcular a entropia condicional $p(y|x)$:

$p(y x)$	y_1	y_2	y_3
x_1	$\frac{4}{7}$	$\frac{4}{7}$	$\frac{2}{11}$
x_2	$\frac{2}{7}$	$\frac{2}{7}$	$\frac{1}{11}$
x_3	$\frac{1}{7}$	$\frac{1}{7}$	$\frac{8}{11}$

Dada as duas distribuições $p(x, y)$ e $p(y|x)$, podemos agora utilizar a Equação (34c) para calcular $H(Y|X)$,

$$\begin{aligned} H(Y|X) &= -\frac{1}{4} \log \frac{4}{7} - \frac{1}{8} \log \frac{4}{7} - \frac{1}{16} \log \frac{2}{11} - \frac{1}{8} \log \frac{2}{7} - \frac{1}{16} \log \frac{2}{7} - \frac{1}{32} \log \frac{1}{11} - \frac{1}{16} \log \frac{1}{7} - \frac{1}{32} \log \frac{1}{7} - \frac{1}{4} \log \frac{8}{11} \\ &= -\frac{3}{8} \log \frac{4}{7} - \frac{3}{16} \log \frac{2}{7} - \frac{3}{32} \log \frac{1}{7} - \frac{1}{16} \log \frac{2}{11} - \frac{1}{32} \log \frac{1}{11} - \frac{1}{4} \log \frac{8}{11} \\ &= \frac{3}{8} (\log 7 - 2) + \frac{3}{16} (\log 7 - 1) + \frac{3}{32} (\log 7) + \frac{1}{16} (\log 11 - 1) + \frac{1}{32} (\log 11) + \frac{1}{4} (\log 11 - 3) \\ &= \frac{21}{32} \log 7 + \frac{11}{32} \log 11 - \frac{56}{32} \\ &= 1.2815 \text{ bits.} \end{aligned}$$

Com os resultados anteriores podemos verificar a regra da cadeia:

$$H(X, Y) = H(X) + H(Y|X) \Rightarrow 2.8125 = 1.5308 + 1.2817 \text{ bits.}$$

Mesmo para um exemplo simples como este, é mais prático calcular numericamente, como apresentado no código Octave a seguir:

```
p_xy = [1/4, 1/8, 1/16; 1/8, 1/16, 1/32; 1/16, 1/32, 1/4];
H_XY = -sum(sum(p_xy .* log2(p_xy)));
p_y = sum(p_xy, 1);
p_y_given_x = p_xy ./ p_y; % p(y|x);
H_Y_given_X = -sum(sum(p_xy .* log2(p_y_given_x)));
```

Informação Mútua e Entropia Relativa

Definimos anteriormente a entropia como uma medida da incerteza sobre uma variável aleatória, e a entropia condicional como a incerteza condicionada ao conhecimento de outra variável aleatória. Estas definições nos levam a questionar o quanto uma variável aleatória revela sobre a outra, ou, equivalentemente, o quanto a incerteza sobre uma é reduzida ao se conhecer a outra. Esta medida é dada pela *informação mútua*, definida como

Definição 4 (Informação Mútua).

$$I(X; Y) \triangleq H(X) - H(X|Y) = H(Y) - H(Y|X), \quad (39)$$

onde as duas igualdades são válidas, por simetria. A primeira reflete a redução da incerteza sobre X quando Y é conhecido, e, de forma similar, a segunda reflete a redução da incerteza sobre Y quando X é conhecido.

Note que a notação $I(X; Y)$ usa ponto e vírgula em vez de vírgula para enfatizar que a informação mútua mede a relação ou dependência entre as variáveis X e Y , distinguindo-se de uma função conjunta como $H(X, Y)$, onde a vírgula separa argumentos de uma distribuição.

A partir da definição dada na Equação (39), podemos reescrevê-la como a seguir:

$$I(X; Y) = H(X) - H(X|Y) \quad (40a)$$

$$= E_{p(x)} \log \frac{1}{p(x)} - E_{p(x,y)} \log \frac{1}{p(x|y)} \quad (40b)$$

$$= E_{p(x,y)} \log \frac{p(x|y)}{p(x)} \quad (40c)$$

$$= E_{p(x,y)} \log \frac{p(x|y)p(y)}{p(x)p(y)} \quad (40d)$$

$$= \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}. \quad (40e)$$

A informação mútua, dada pela Equação (40e), pode ser vista como a *divergência de Kullback-Leibler* entre a distribuição conjunta e o produto das marginais, ou seja,

$$I(X; Y) = D(p(x, y) \parallel p(x)p(y)). \quad (41)$$

Vejamos então a definição de divergência de Kullback-Leibler.

Definição 5 (Divergência de Kullback-Leibler). A divergência de Kullback-Leibler (KL) entre duas distribuições p e q , em um alfabeto comum $\mathcal{X}$, é definida como

$$D(p \parallel q) \triangleq \sum_x p(x) \log \frac{p(x)}{q(x)} \quad (42a)$$

$$= \mathbb{E}_p \log \frac{p(X)}{q(X)}. \quad (42b)$$

Note que, em geral, a divergência de KL não é simétrica, ou seja, $D(p \parallel q) \neq D(q \parallel p)$.

Na definição de acima, adotamos a convenção de que o somatório se dá sobre o suporte¹¹ da distribuição p , ou seja, S_p .

A divergência de KL é conhecida também na literatura como entropia relativa. Por não ser simétrica, não pode ser considerada como uma métrica ou distância, e ainda não satisfaz a desigualdade triangular.

Como observado anteriormente, seja $\mu_1(x, y) = p(x, y)$ (distribuição conjunta) e $\mu_2(x, y) = p(x)p(y)$ (produto das marginais), com $p(x) = \sum_y p(x, y)$ e $p(y) = \sum_x p(x, y)$, então

$$D(\mu_1 \parallel \mu_2) = \sum_{x, y} \mu_1(x, y) \log \frac{\mu_1(x, y)}{\mu_2(x, y)} \quad (43a)$$

$$= \sum_{x, y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} = I(X; Y). \quad (43b)$$

A informação mútua é a “distância” entre a distribuição conjunta em X e Y e o produto das distribuições marginais em X e Y . Se as variáveis aleatórias são independentes, teremos $p(x, y) = p(x)p(y)$ e por conseguinte a divergência será nula, a informação mútua entre X e Y será zero. A informação mútua é o erro em se assumir independência entre as variáveis aleatórias.

O produto das distribuições marginais $p(x)p(y)$ é uma projeção da distribuição conjunta $p(x, y)$ sobre o conjunto dos produtos de distribuições independentes, minimizando a divergência entre a distribuição conjunta e o produto das marginais. Ou seja,

$$p(x)p(y) = \operatorname{argmin}_{p'(x, y) \setminus p'(x, y) = p'(x)p'(y)} D(p(x, y) \parallel p'(x, y)). \quad (44)$$

¹¹ O suporte de uma distribuição de probabilidade ou de uma função é o conjunto de valores para os quais a função (ou distribuição) assume valores não nulos. Formalmente, para uma função f , o suporte é definido como $S_f = \{x \in \mathbb{R} \mid f(x) \neq 0\}$. No contexto de distribuições, é o menor conjunto fechado onde a medida de probabilidade é positiva.

Podemos agora considerar uma aplicação prática em problemas de estimação paramétrica: minimizar a divergência KL entre uma distribuição subjacente p_θ e uma distribuição empírica $\hat{p}$ revela-se equivalente a maximizar o logaritmo da verossimilhança dos dados sob o modelo $\hat{p}$. A divergência KL quantifica o custo ao usar $\hat{p}$ para aproximar p_θ , e otimizar esse custo nos leva diretamente a ajustar os parâmetros θ para melhor descrever os dados observados.

Seja $x_{1:N} = x_1, \dots, x_N \in \mathcal{X}$, N observações i.i.d.¹² de uma variável aleatória X . A distribuição empírica será dada por

$$\hat{p}(x) = \frac{1}{N} \sum_{n=1}^N \delta(x - x_n), \quad (45)$$

onde δ é a função de Dirac. De forma equivalente, podemos expressar a distribuição empírica por

$$\hat{p}(x) = \frac{n(x|x_{1:N})}{N}, \quad (46)$$

onde $n(x|x_{1:N})$ representa o número de ocorrência de x na amostra $x_{1:N}$.

Seja p_θ uma distribuição em $\mathcal{X}$ parametrizada por θ . Maximizar a verossimilhança de $p_\theta(x)$ é equivalente a minimizar a divergência $D_{\text{KL}}(\hat{p} \parallel p_\theta)$.

$$D_{\text{KL}}(\hat{p} \parallel p_\theta) = \sum_{x \in \mathcal{X}} \hat{p}(x) \log \frac{\hat{p}(x)}{p_\theta(x)} \quad (47a)$$

$$= -H(\hat{p}) - \sum_{x \in \mathcal{X}} \hat{p}(x) \log p_\theta(x) \quad (47b)$$

$$= -H(\hat{p}) - \frac{1}{N} \sum_{x \in \mathcal{X}} \sum_{n=1}^N \delta(x - x_n) \log p_\theta(x) \quad (47c)$$

$$= -H(\hat{p}) - \frac{1}{N} \sum_{n=1}^N \log p_\theta(x_n) \quad (47d)$$

$$= -H(\hat{p}) - \ell(p_\theta(x)), \quad (47e)$$

onde $\ell(\cdot)$ é a função log-verossimilhança.

A estimativa de máxima verossimilhança de θ a partir das N observações é dada por

$$\hat{\theta}_n = \operatorname{argmax}_{\theta \in \Theta} \prod_{n=1}^N p_\theta(x_n) \quad (48a)$$

$$= \operatorname{argmax}_{\theta \in \Theta} \sum_{n=1}^N \log p_\theta(x_n) \quad (48b)$$

$$= \operatorname{argmin}_{\theta \in \Theta} \frac{1}{N} \sum_{n=1}^N -\log p_\theta(x_n). \quad (48c)$$

¹² Uma coleção de variáveis aleatórias é independente e identicamente distribuída (i.i.d.) se todas possuírem uma mesma distribuição e forem mutuamente independentes.

Desta forma, podemos constatar que a distribuição que minimiza a divergência de KL para a distribuição empírica é aquela que maximiza a verossimilhança (ou logaritmo desta). Pela lei dos grandes números¹³, $\frac{1}{N} \sum_{i=1}^N \log p_\theta(x_i) \xrightarrow{q.c.} E[\log p_\theta(X)]$.

Ao ajustar os parâmetros θ para minimizar divergência KL, estamos buscando o modelo $\hat{p}$ que melhor representa p . A não negatividade da divergência, garante que o custo informacional nunca será negativo e que a aproximação ótima ocorre quando as distribuições coincidem. Graças à convexidade da função logarítmica, sabemos que esse ótimo é bem definido e único.

Vejamos então uma propriedade essencial da divergência: a não negatividade.

Teorema 4 (Não negatividade da Divergência) *Para duas distribuições p e q em um alfabeto comum $\mathcal{X}$,*

$$D(p \parallel q) \geq 0 \quad (49)$$

com igualdade se e somente se $p = q$.

Demonstração. Se $q(x) = 0$ para algum $x \in S_p$, então $D(p \parallel q) = \infty$ e o teorema é verdadeiro (caso trivial). Vamos assumir então que $q(x) > 0, \forall x \in S_p$. Teremos então

$$D(p \parallel q) = \log e \sum_{x \in S_p} p(x) \ln \frac{p(x)}{q(x)} \quad (50a)$$

$$\geq \log e \sum_{x \in S_p} p(x) \left(1 - \frac{q(x)}{p(x)} \right) \quad (50b)$$

$$= \log e \left[\sum_{x \in S_p} p(x) - \sum_{x \in S_p} q(x) \right] \quad (50c)$$

$$\geq \log e (1 - 1) \quad (50d)$$

$$= 0, \quad (50e)$$

onde em 50b utilizamos a Equação (28) substituindo z por $1/z$ (o que fornece $\ln z \geq 1 - 1/z$) e para obter 50d utilizamos o fato de que

$$\sum_{x \in S_p} q(x) \leq 1, \quad (51)$$

uma vez que o somatório é tomado no suporte de p , podendo assim excluir alguns pontos não nulos de q .

□

O fato da divergência ser não negativa, nos leva como consequência a não negatividade da informação mútua, uma vez que esta é um caso específico de divergência.

¹³ A Lei Forte dos Grandes Números afirma que, dada um sequência infinita $X_1, X_2, X_3, \dots$ de variáveis aleatórias independentes e identicamente distribuídas (i.i.d.) com média finita $E[X_i] = \mu$, a média amostral $\bar{X}_n = 1/n \sum_{i=1}^n X_i$ converge quase certamente para μ , ou seja, $\bar{X}_n \xrightarrow{q.c.} \mu$.

Lema 5 (Não negatividade da Informação Mútua) *A informação mútua entre duas variáveis aleatórias X e Y é tal que*

$$I(X; Y) \geq 0, \quad (52)$$

com igualdade se e somente se $X \perp Y$, ou seja, X e Y são independentes.

Demonstração. A informação mútua é a divergência entre a distribuição conjunta e o produto das marginais, logo, aplicando o Teorema 4, obtemos

$$I(X; Y) = D(p(x, y) \parallel p(x)p(y)) \geq 0. \quad (53)$$

Quando $X \perp Y$, teremos $p(x, y) = p(x)p(y)$ e assim a divergência será nula e também a informação mútua. □

Exemplo 6 (Divergência de Kullback-Leibler em Textos e Dados Aleatórios). Neste exemplo, usamos a divergência de KL para comparar a distribuição de caracteres em um texto natural com a de dados aleatórios, usando a distribuição uniforme como referência.

Consideramos um alfabeto reduzido de 26 letras minúsculas $\mathcal{A} = \{a, b, \dots, z\}$. Para o texto natural, usamos um trecho de *Ulysses*, de James Joyce, filtrando para manter apenas letras minúsculas (convertendo maiúsculas em minúsculas e removendo pontuação, espaços, etc.). Para os dados aleatórios, usamos bytes gerados por `/dev/urandom`, mapeados para o alfabeto $[a-z]$ via `byte mod 26`, onde cada valor é associado a uma letra (e.g., $0 \rightarrow a$, $1 \rightarrow b$, ..., $25 \rightarrow z$).

Para cada fonte de dados (texto e `/dev/urandom`), extraímos amostras de tamanhos $n = 100, 1000, 10000, 1000000$. Estimamos a distribuição empírica $p(x)$ usando a máxima verossimilhança com suavização de Laplace. A distribuição uniforme é $q(x) = \frac{1}{26}$ para todo $x \in \mathcal{A}$.

Calculamos $D(p \parallel q)$ para cada amostra e plotamos histogramas das distribuições empíricas para visualizar como elas diferem da uniforme. Esperamos que, para `/dev/urandom`, $D(p \parallel q)$ seja próximo de zero, refletindo a aleatoriedade dos dados, enquanto para o texto, $D(p \parallel q)$ permaneça maior que zero devido à estrutura linguística.

Os histogramas das distribuições são mostrados na Exemplo 6. Para *Ulysses*, observamos que certas letras (e.g., ‘e’, ‘t’) são mais frequentes, refletindo as propriedades do idioma inglês. Para `/dev/urandom`, as frequências são aproximadamente uniformes.

A Figura 9 mostra a divergência de KL em função de n . Para `/dev/urandom`, $D(p \parallel q)$ é pequeno e diminui com n , indicando que a distribuição empírica se aproxima da uniforme. Para *Ulysses*, $D(p \parallel q)$ estabiliza em um valor positivo (aproximadamente 0.5), refletindo a estrutura não uniforme do texto. Observe que, para um alfabeto de

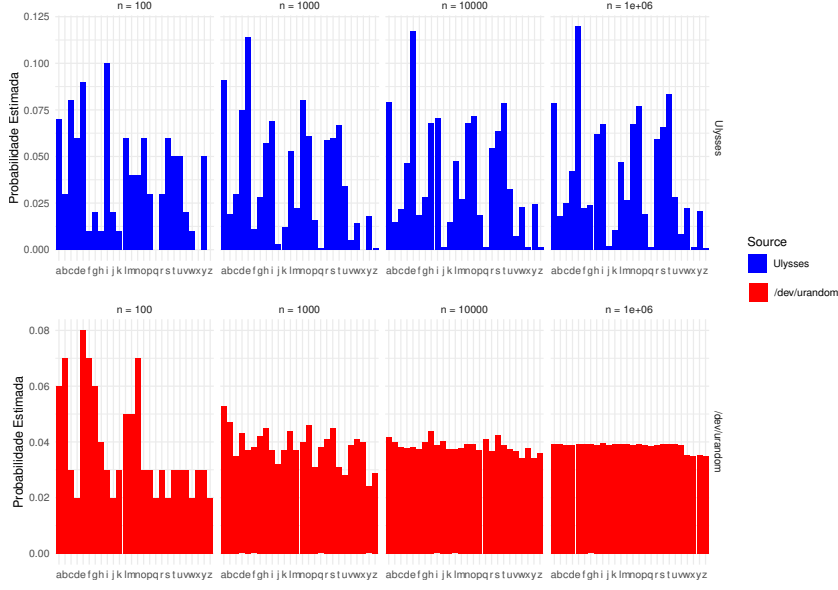


Figura 8: Histogramas da distribuição empírica de letras em Ulysses e em /dev/urandom para $n = 100, 1000, 10000, 1000000$.

26 caracteres, a entropia máxima é $\log(26) = 4.7$ bits e, calculando o valor da entropia para Ulysses obtemos aproximadamente 4.2. A diferença é justamente o valor para o qual a divergência se estabiliza na Figura 9.

A divergência também pode ser utilizada para demonstrar o limite máximo da entropia, de forma alternativa àquela apresentada na Equação (31).

Teorema 6 (Limite Máximo da Entropia – revisitado) $H(X) \leq \log |\mathcal{X}|$, onde $|\mathcal{X}|$ denota a cardinalidade do alfabeto $\mathcal{X}$, com igualdade se e somente se X possuir distribuição uniforme.

Demonstração. Seja $u(x) = 1/|\mathcal{X}|$ a função probabilidade de massa uniforme em $\mathcal{X}$, e seja $p(x)$ a função probabilidade de massa para X . Então

$$\begin{aligned}
 D(p \parallel u) &= \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{u(x)} \\
 &= \sum_{x \in \mathcal{X}} p(x) \log p(x) + \sum_{x \in \mathcal{X}} p(x) \log \frac{1}{u(x)} \\
 &= -H(X) + \log |\mathcal{X}| \sum_{x \in \mathcal{X}} p(x) \\
 &= \log |\mathcal{X}| - H(X).
 \end{aligned} \tag{54}$$

Como a entropia relativa é não negativa, $D(p \parallel u) \geq 0$, teremos

$$D(p \parallel u) = \log |\mathcal{X}| - H(X) \geq 0, \tag{55}$$

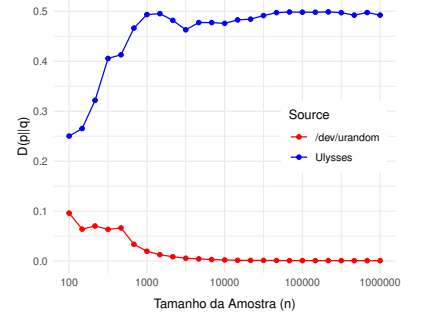


Figura 9: Divergência de KL entre a distribuição empírica e a uniforme para Ulysses e /dev/urandom.

e assim

$$H(X) \leq \log |\mathcal{X}| \quad (56)$$

□

Observação 1. Para uma variável aleatória em um alfabeto D -ário, ou seja, $|\mathcal{X}| = D$, a entropia na base D será

$$H_D(X) \leq 1. \quad (57)$$

Observação 2. O Teorema 6 estabelece um valor máximo para a entropia desde que o alfabeto seja finito. Para o caso de alfabeto de tamanho infinito, a entropia pode ser finita ou não.

Vejamos dois exemplos:

Exemplo 7. Seja X uma variável aleatória com distribuição dada por

$$Pr(X = i) = 2^{-i}, \quad i = 1, 2, \dots \quad (58)$$

Neste caso, teremos¹⁴

$$H(X) = \sum_{i=1}^{\infty} i 2^{-i} = 2. \quad (59)$$

Exemplo 8 (Yeung 2002). Considere uma variável aleatória X com valores em pares de inteiros

$$\left\{ (i, j) : 1 \leq i \leq \infty \text{ e } 1 \leq j \leq \frac{2^i}{2^i} \right\} \quad (60)$$

tal que

$$Pr(X = (i, j)) = 2^{-2^i}, \quad (61)$$

para todo i e j . A entropia será dada por

$$H(X) = - \sum_{i=1}^{\infty} \sum_{j=1}^{2^{2^i}/2^i} 2^{-2^i} \log 2^{-2^i} = \sum_{i=1}^{\infty} 1 \rightarrow \infty, \quad (62)$$

ou seja, não converge.

Vejamos ainda algumas outras observações que podemos fazer sobre a informação mútua. Primeiramente, pela Definição 4, é fácil notar que a informação mútua $I(X; Y)$ é simétrica em X e Y , ou seja, $I(X; Y) = I(Y; X)$.

Em seguida, vamos analisar o que ocorre quando tomamos a informação mútua entre uma variável aleatória X e ela mesma, ou seja, $I(X; X)$.

¹⁴ A série $\sum_{i=1}^{\infty} i 2^{-i}$, conhecida como “Gabriel’s Staircase”, é um caso especial da série aritmético-geométrica $\sum_{i=1}^{\infty} i x^i$, cuja soma, para $|x| < 1$, é dada por $\frac{x}{(1-x)^2}$. Essa fórmula é derivada a partir da série geométrica $\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$ por diferenciação e multiplicação por x . Para $x = \frac{1}{2}$, obtém-se $\sum_{i=1}^{\infty} i 2^{-i} = 2$. Nicole Oresme (c. 1323–1382) forneceu uma prova geométrica para essa convergência no século XIV, um feito notável para a época (Mazet 2003; Wikipédia 2025; Edgar 2018). Para mais detalhes, consulte a entrada sobre séries geométricas na Wikipédia ou o trabalho de Oresme em suas *Questiones super Geometriam Euclidis*.

Proposição 1 (A informação mútua de uma variável aleatória e ela mesma é a entropia) *A informação mútua de uma variável aleatória X com ela mesma, a auto-informação de X , é igual à entropia de X .*

Demonstração.

$$I(X; X) = E \log \frac{p(X)}{p(X)^2} \quad (63a)$$

$$= -E \log p(X) \quad (63b)$$

$$= H(X). \quad (63c)$$

□

Podemos descrever uma nova relação, ao observar a Definição 4 ($I(X; Y) = H(X) - H(X|Y)$) e o Teorema 3 ($H(X|Y) = H(X, Y) - H(Y)$)

Proposição 2

$$I(X; Y) = H(X) + H(Y) - H(X, Y). \quad (64)$$

Todas essas relações observadas entre entropia, entropia conjunta, entropia condicional e informação mútua podem ser facilmente representadas através do diagrama de informação, um tipo de digrama de Venn utilizado em teoria da informação para ilustrar tais relações (veja a Figura 10). Esta correspondência entre o diagrama de Venn e as medidas de informação de Shannon não são mera coincidência (veja mais detalhes na seção Medida de Informação, página 57).

De forma semelhante ao que vimos anteriormente, quando foi introduzido o conceito de entropia condicional, podemos agora estender o conceito de informação mútua entre duas variáveis aleatórias X e Y dado o conhecimento de uma terceira variável Z , definindo assim a informação mútua condicional.

Definição 6 (Informação Mútua Condicional). Dadas as variáveis aleatórias X , Y e Z , a informação mútua entre X e Y , condicionada ao conhecimento de Z é dada por

$$I(X; Y|Z) \triangleq \sum_z p(z) I(X; Y|Z = z) \quad (65a)$$

$$= \sum_z p(z) E_{p(x,y|z)} \log \frac{p(x, y|Z = z)}{p(x|Z = z)p(y|Z = z)} \quad (65b)$$

$$= \sum_{x,y,z} p(x, y, z) \log \frac{p(x, y|z)}{p(x|z)p(y|z)} \quad (65c)$$

$$= \sum_{x,y,z} p(x, y, z) \log \frac{p(x|y, z)p(y|z)}{p(x|z)p(y|z)} \quad (65d)$$

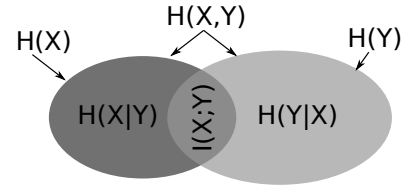


Figura 10: Diagrama de informação ilustrando as relações entre entropias, entropias condicionais e informação mútua, facilitando a visualização das grandezas informacionais.

$$= E_{p(x,y,z)} \left[\log \frac{1}{p(x|z)} - \log \frac{1}{p(x|y,z)} \right] \quad (65e)$$

$$= H(X|Z) - H(X|Y, Z). \quad (65f)$$

Generalização da Regra da Cadeia

A generalização é fundamental para analisar sistemas com múltiplas variáveis ou séries temporais. Uma sequência de variáveis aleatórias $X_1, X_2, \dots, X_N = X_{1:N}$ pode representar estados ou observações ao longo do tempo. A regra da cadeia, vista anteriormente no Teorema 3, pode ser generalizada para uma sequência de N variáveis aleatórias $X_{1:N}$.

Proposição 3 (Regra da Cadeia Generalizada da Entropia) *Dada uma sequência de N variáveis aleatórias $X_{1:N}$, a entropia conjunta pode ser dada por*

$$H(X_1, X_2, \dots, X_N) = \sum_{i=1}^N H(X_i | X_1, \dots, X_{i-1}). \quad (66)$$

Demonstração. A demonstração é feita por indução. Sabemos que para $N = 2$ é verdadeiro, como visto no Teorema 3. Vamos supor que para $N = M$ é verdadeiro e mostrar que para $N = M + 1$ é verdadeiro.

$$H(X_1, X_2, \dots, X_M, X_{M+1}) = H(X_1, X_2, \dots, X_M) + H(X_{M+1} | X_1, X_2, \dots, X_M) \quad (67a)$$

$$= \sum_{i=1}^M H(X_i | X_1, \dots, X_{i-1}) + H(X_{M+1} | X_1, X_2, \dots, X_M) \quad (67b)$$

$$= \sum_{i=1}^{M+1} H(X_i | X_1, \dots, X_{i-1}), \quad (67c)$$

onde em 67a utilizamos o Teorema 3, fazendo $X = (X_1, X_2, \dots, X_M)$ e $Y = X_{M+1}$, e em 67b utilizamos a premissa de que a relação 66 é válida para $N = M$. Isto prova então que a relação 66 é válida para $N = M + 1$ e, como é sabidamente válida para $N = 2$, será válida para todo N . □

A regra da cadeia vista no Teorema 3 pode ser estendida para o caso do condicionamento à uma terceira variável aleatória.

Proposição 4 (Regra da Cadeia Condicional) *Para duas variáveis aleatórias X e Y condicionadas a uma terceira, Z , a entropia conjunta condicional pode ser decomposta como:*

$$H(X, Y | Z) = H(X | Z) + H(Y | X, Z). \quad (68)$$

A incerteza total sobre (X, Y) , dado Z , pode ser quebrada em duas partes: primeiro, a incerteza sobre X dado Z , e depois a incerteza adicional de Y dado tanto X quanto Z .

Demonstração. A demonstração segue os mesmos passos da demonstração do Teorema 3, bastando condicionar a Z para obtermos a adaptação para a regra da cadeia condicional. $\square$

Fazemos agora a generalização para uma sequência de variáveis aleatórias $X_{1:N}$ condicionada a uma variável aleatória Y .

Proposição 5 (Regra da Cadeia Condicional Generalizada)

$$H(X_1, X_2, \dots, X_N | Y) = \sum_{i=1}^N H(X_i | X_1, \dots, X_{i-1}, Y). \quad (69)$$

Demonstração.

$$H(X_1, X_2, \dots, X_N | Y) = \sum_y H(X_1, X_2, \dots, X_N | Y = y) \quad (70a)$$

$$= \sum_y p(y) \sum_{i=1}^N H(X_i | X_1, \dots, X_{i-1}, Y = y) \quad (70b)$$

$$= \sum_{i=1}^N \sum_y p(y) H(X_i | X_1, \dots, X_{i-1}, Y = y) \quad (70c)$$

$$= \sum_{i=1}^N H(X_i | X_1, \dots, X_{i-1}, Y), \quad (70d)$$

onde utilizamos 34a em 70a, 35 em 70d, e 70b segue de 66. $\square$

Por fim, podemos aplicar as mesmas ideias para obter a regra da cadeia para informação mútua, que descreve como a informação compartilhada entre um conjunto de variáveis e outra variável pode ser decomposta em contribuições condicionais.

Proposição 6 (Regra da Cadeia da Informação Mútua)

$$I(X_1, X_2, \dots, X_N; Y) = \sum_{i=1}^N I(X_i; Y | X_1, \dots, X_{i-1}). \quad (71)$$

Demonstração.

$$I(X_1, X_2, \dots, X_N; Y) = H(X_1, X_2, \dots, X_N) - H(X_1, X_2, \dots, X_N | Y) \quad (72a)$$

$$= \sum_{i=1}^N [H(X_i | X_1, \dots, X_{i-1}) - H(X_i | X_1, \dots, X_{i-1}, Y)] \quad (72b)$$

$$= \sum_{i=1}^N I(X_i; Y | X_1, \dots, X_{i-1}), \quad (72c)$$

onde utilizamos as Proposições 3 e 5.

□

Desigualdade de Jensen

Nesta seção, apresentamos a *desigualdade de Jensen*, uma relação observada para funções convexas entre o valor esperado da função de uma variável aleatória e a função do valor esperado desta variável aleatória. Mais precisamente, $Ef(X) \geq f(EX)$, para f uma função convexa e X uma variável aleatória.

A desigualdade de Jensen é uma ferramenta fundamental para demonstrar diversas propriedades essenciais. Entretanto, primeiramente devemos rever o teste da segunda derivada para convexidade.

Definição 7 (Função Convexa). Dizemos que f é convexa em (a, b) se para todo $x_1, x_2 \in (a, b)$, $0 \leq \lambda \leq 1$,

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2) \quad (73)$$

A Figura 11 ilustra uma função convexa no intervalo, evidenciando como a imagem de um ponto intermediário $\lambda x_1 + (1 - \lambda)x_2$, entre x_1 e x_2 , é menor ou igual ao correspondente na corda que passa por $(x_1, f(x_1))$ e $(x_2, f(x_2))$.

Alguns exemplos de funções convexas são: $f(x) = x^2$; $f(x) = x^4$; $f(x) = e^x$; e $x \log x$, $x \geq 0$. Teremos que f é estritamente convexa se a igualdade for verdadeira apenas para $\lambda = 0$ ou $\lambda = 1$. Uma função f é dita côncava se $-f$ (f multiplicada por -1) for uma função convexa, ou seja, côncavo é o oposto de convexo.

Para determinar se uma função é convexa, podemos realizar o simples teste da segunda derivada.

Proposição 7 (Teste da Segunda Derivada para Convexidade) *Se uma função f possui derivada segunda não-negativa (positiva) em um intervalo, a função é convexa (estritamente convexa) no intervalo.*

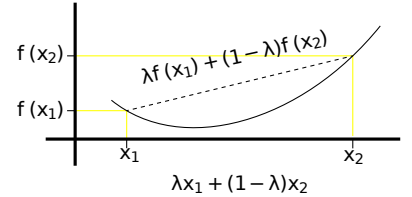


Figura 11: Ilustração de uma função convexa no intervalo.

Demonstração. A expansão de Taylor de uma função f em torno do ponto x_0 é dada por

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x^*)}{2}(x - x_0)^2 \quad (74)$$

onde $x^* \in (x_0, x)$. Por hipótese, $f''(x^*) \geq 0$, e desta forma, o último termo é não-negativo.

Seja $x_0 = \lambda x_1 + (1 - \lambda)x_2$. Analisando em $x = x_1$, teremos

$$f(x_1) \geq f(x_0) + f'(x_0)(x_1 - \lambda x_1 - (1 - \lambda)x_2) \quad (75a)$$

$$= f(x_0) + f'(x_0)((1 - \lambda)(x_1 - x_2)) \quad (75b)$$

Da mesma forma, em $x = x_2$, teremos

$$f(x_2) \geq f(x_0) + f'(x_0)(x_2 - \lambda x_1 - (1 - \lambda)x_2) \quad (76a)$$

$$= f(x_0) + f'(x_0)(\lambda(x_2 - x_1)) \quad (76b)$$

Somando λ 75b com $(1 - \lambda)$ 76b, obtemos

$$\begin{aligned} \lambda f(x_1) + (1 - \lambda)f(x_2) &\geq \lambda f(x_0) + \lambda f'(x_0)((1 - \lambda)(x_1 - x_2)) + \\ &\quad (1 - \lambda)f(x_0) + (1 - \lambda)f'(x_0)(\lambda(x_2 - x_1)) \end{aligned} \quad (77a)$$

$$\geq f(x_0) = f(\lambda x_1 + (1 - \lambda)x_2), \quad (77b)$$

sendo assim a função a função convexa no intervalo. □

Podemos agora retornar à desigualdade de Jensen. Vemos na Figura 12 uma ilustração do que tal propriedade representar.

Teorema 7 (Desigualdade de Jensen) *Seja f uma função convexa e X uma variável aleatória, então*

$$Ef(X) = \sum_x p(x)f(x) \geq f(EX) = f\left(\sum_x p(x)x\right) \quad (78)$$

Demonstração. Para uma distribuição de massa com apenas dois pontos

$$E[f(X)] = p_1 f(x_1) + p_2 f(x_2) \geq f(p_1 x_1 + p_2 x_2) = f(EX) \quad (79)$$

já que f é convexa e $p_1 + p_2 = 1$.

Para uma distribuição com mais de dois pontos, iremos fazer uma demonstração por indução.

Suponha que o teorema seja verdadeiro para uma distribuição com $k - 1$ pontos de massa. Para uma distribuição com k pontos de massa podemos escrever $p'_i = p_i/(1 - p_k)$, para $i = 1, 2, \dots, k - 1$.

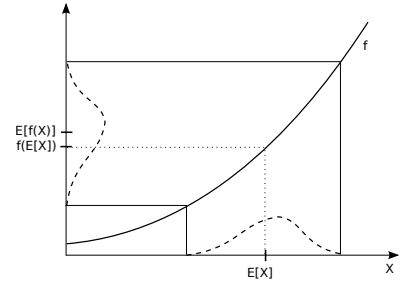


Figura 12: Ilustração da desigualdade de Jensen, fornecendo uma intuição para ela.

Desta forma, teremos

$$\mathbb{E}[f(X)] = \sum_{i=1}^k p_i f(x_i) \quad (80a)$$

$$= \sum_{i=1}^{k-1} (1 - p_k) p'_i f(x_i) + p_k f(x_k) \quad (80b)$$

$$= (1 - p_k) \sum_{i=1}^{k-1} p'_i f(x_i) + p_k f(x_k) \quad (80c)$$

$$\geq (1 - p_k) f \left(\sum_{i=1}^{k-1} p'_i x_i \right) + p_k f(x_k) \quad (80d)$$

$$\geq f \left((1 - p_k) \sum_{i=1}^{k-1} p'_i x_i + p_k x_k \right) \quad (80e)$$

$$= f \left(\sum_{i=1}^k p_i x_i \right) \quad (80f)$$

onde em 80c utilizamos a hipótese de indução, já que

$$\sum_{i=1}^{k-1} p'_i = \sum_{i=1}^{k-1} \frac{p_i}{1 - p_k} = \frac{1 - p_k}{1 - p_k} = 1. \quad (81)$$

e em 80d utilizamos a definição de convexidade.

Desta forma, sendo o teorema válido para uma distribuição de massa com $k - 1$ pontos, também será verdadeiro para uma distribuição de massa com k pontos. Como mostramos que para $k = 2$ é verdadeiro, logo o teorema é verdadeiro para qualquer k .

□

A desigualdade de Jensen pode ser utilizada para demonstrar a não negatividade da divergência de KL.

Lema 8 (Não negatividade da Divergência de Kullback–Leibler)

$$D(p \parallel q) \geq 0 \text{ com igualdade se e somente se } p(x) = q(x), \forall x. \quad (82)$$

Demonstração. De forma equivalente, iremos mostrar que $-D(p \parallel q) \leq 0$.

Seja $S_p = \{x : p(x) > 0\}$, o suporte de p , então

$$-D(p \parallel q) = -\sum_x p(x) \log \frac{p(x)}{q(x)} = -\sum_{x \in S_p} p(x) \log \frac{p(x)}{q(x)} \quad (83a)$$

$$= \sum_{x \in S_p} p(x) \log \frac{q(x)}{p(x)} = E \log \frac{q(X)}{p(X)} \quad (83b)$$

$$\leq \log \left(E \frac{q(X)}{p(X)} \right) = \log \left(\sum_{x \in S_p} p(x) \frac{q(x)}{p(x)} \right) \quad (83c)$$

$$= \log \left(\sum_{x \in S_p} q(x) \right) \leq \log \left(\sum_x q(x) \right) = \log 1 = 0 \quad (83d)$$

onde em 83c utilizamos a desigualdade de Jensen, Equação (78). $\square$

A *desigualdade da soma dos logaritmos* é derivada da desigualdade de Jensen e da convexidade da função logarítmica. Esta desigualdade também aparece em algumas demonstrações importantes.

Proposição 8 (Desigualdade da Soma de Logaritmos) *Dados $(a_1, \dots, a_n)$ e $(b_1, \dots, b_n)$, com $a_i \geq 0$ e $b_i \geq 0$, temos*

$$\sum_{i=1}^n a_i \log \frac{a_i}{b_i} \geq \left(\sum_{i=1}^n a_i \right) \log \frac{\sum_{i=1}^n a_i}{\sum_{i=1}^n b_i} \quad (84)$$

e teremos igualdade se e somente se $a_i/b_i = c$, onde c é uma constante.

Demonstração. Considere $f(t) = t \log t = t(\ln t)(\log e)$, que é estritamente convexa, pois $f''(t) = 1/t \log e > 0$, $\forall t > 0$.

Dada f convexa, a desigualdade de Jensen diz que

$$\sum_i \alpha_i f(t_i) \geq f \left(\sum_i \alpha_i t_i \right) \text{ com } \alpha_i \geq 0 \text{ e } \sum_i \alpha_i = 1. \quad (85)$$

$f(x) = x \log x$ é estritamente convexa para $x > 0$, já que $f''(x) = \frac{1}{x} \log e > 0$ para $x > 0$.

Vamos fazer $\alpha_i = b_i / \sum_{j=1}^n b_j$ e $t_i = a_i / b_i$, então obteremos

$$\sum_i \alpha_i f(t_i) \geq f\left(\sum_i \alpha_i t_i\right) \quad (86a)$$

$$\sum_i \left(\frac{b_i}{\sum_j b_j} f\left(\frac{a_i}{b_i}\right) \right) \geq f\left(\sum_i \frac{b_i}{\sum_j b_j} \frac{a_i}{b_i}\right) \quad (86b)$$

$$\frac{1}{\sum_j b_j} \sum_i \left(b_i \frac{a_i}{b_i} \log \frac{a_i}{b_i} \right) \geq \left(\sum_i \frac{a_i}{\sum_j b_j} \right) \log \sum_i \frac{a_i}{\sum_j b_j} \quad (86c)$$

$$\sum_i a_i \log \frac{a_i}{b_i} \geq \left(\sum_i a_i \right) \log \sum_i \frac{a_i}{\sum_j b_j} \quad (86d)$$

$$\sum_i a_i \log \frac{a_i}{b_i} \geq \sum_i a_i \log \frac{\sum_i a_i}{\sum_j b_j}. \quad (86e)$$

□

A desigualdade da soma de logaritmos pode ser utilizada para mostrar que $D(p \parallel q) \geq 0$.

Proposição 9 (Não negatividade da Divergência – revisitado) $D(p \parallel q) \geq 0$

Demonstração.

$$D(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \quad (87a)$$

$$\geq \left(\sum_x p(x) \right) \log \frac{\sum_x p(x)}{\sum_x q(x)} \quad (87b)$$

$$= 1 \log \frac{1}{1} = 0. \quad (87c)$$

□

A entropia relativa, ou divergência de Kullback-Leibler, possui uma propriedade importante: ela é convexa no par de distribuições. A convexidade no par considera a interpolação simultânea de ambas as distribuições e garante que a divergência KL se comporte de forma previsível ao combinar distribuições.

Teorema 9 (A Entropia Relativa é Convexa no Par) *Para dois pares de distribuições (p_1, q_1) e (p_2, q_2) ,*

$$D(\lambda p_1 + (1-\lambda)p_2 \parallel \lambda q_1 + (1-\lambda)q_2) \leq \lambda D(p_1 \parallel q_1) + (1-\lambda)D(p_2 \parallel q_2), \quad (88)$$

para todo $0 \leq \lambda \leq 1$.

Demonstração. Usando a Definição 5, definição de divergência de KL, temos

$$\begin{aligned}
 D(\lambda p_1 + (1 - \lambda)p_2 \parallel \lambda q_1 + (1 - \lambda)q_2) &= \sum_x (\lambda p_1(x) + (1 - \lambda)p_2(x)) \log \frac{\lambda p_1(x) + (1 - \lambda)p_2(x)}{\lambda q_1(x) + (1 - \lambda)q_2(x)} \\
 &\quad (89a) \\
 &\leq \sum_x \left(\lambda p_1(x) \log \frac{\lambda p_1(x)}{\lambda q_1(x)} + (1 - \lambda)p_2(x) \log \frac{(1 - \lambda)p_2(x)}{(1 - \lambda)q_2(x)} \right) \\
 &\quad (89b) \\
 &= \lambda D(p_1 \parallel q_1) + (1 - \lambda)D(p_2 \parallel q_2) \\
 &\quad (89c)
 \end{aligned}$$

onde, em 89a, podemos considerar que, dentro do somatório em x , temos um somatório com dois termos: $(a_1 + a_2) \log ((a_1 + a_2)/(b_1 + b_2))$. Utilizamos então a desigualdade da soma de logaritmos (veja a Equação (84) da Proposição 8) para o caso com apenas dois termos:

$$\left(\sum_{i=1}^2 a_i \right) \log \frac{\sum_{i=1}^2 a_i}{\sum_{i=1}^2 b_i} \leq \sum_{i=1}^2 a_i \log \frac{a_i}{b_i} = a_1 \log \frac{a_1}{b_1} + a_2 \log \frac{a_2}{b_2}, \quad (90)$$

o que nos leva ao resultado em 89b. E usamos novamente a definição de divergência (Definição 5) para obter 89c.

□

Teorema 10 (A Entropia Relativa é Convexa no Primeiro Argumento) *A divergência de Kullback-Leibler, $D(p \parallel q)$, é convexa em relação a p , fixando-se q . Ou seja, dadas p_1 e p_2 , duas distribuições de probabilidade, e fixando-se q , a distribuição de referência ou distribuição base. Temos então, para $\lambda \in [0, 1]$,*

$$D(\lambda p_1 + (1 - \lambda)p_2 \parallel q) \leq D(p_1 \parallel q) + (1 - \lambda)D(p_2 \parallel q). \quad (91)$$

Demonstração.

$$\begin{aligned}
 D(\lambda p_1 + (1 - \lambda)p_2 \parallel q) &= \sum_x (\lambda p_1 + (1 - \lambda)p_2) \log \frac{\lambda p_1 + (1 - \lambda)p_2}{q} \\
 &\quad (92a) \\
 &\leq \lambda \sum_x p_1 \log \frac{p_1}{q} + (1 - \lambda) \sum_x p_2 \log \frac{p_2}{q} \quad (92b) \\
 &= \lambda D(p_1 \parallel q) + (1 - \lambda)D(p_2 \parallel q), \quad (92c)
 \end{aligned}$$

onde em 92a utilizamos que a função $f(p) = p \log p$ é convexa em p (lembrando que q é fixo e assim $f''(p) = 1/p > 0$, para $p > 0$) e utilizando a desigualdade de Jensen (Teorema 7). Em 92b apenas utilizamos a definição de divergência (Definição 5).

□

A entropia relativa, entretanto, não é convexa no segundo argumento. Será entretanto côncava em q para p fixo. A demonstração deixamos a cargo do leitor.

O Teorema 10 pode ser utilizado para demonstrar a concavidade da entropia.

Teorema 11 (A Entropia é Côncava) $H(p)$ é uma função concava de p .

Demonstração.

$$H(p) = - \sum_i p_i \log p_i = - \sum_i p_i \log p_i + \log |\mathcal{X}| - \log |\mathcal{X}| \quad (93a)$$

$$= \log |\mathcal{X}| - \sum_i p_i \log p_i - \log |\mathcal{X}| \underbrace{\sum_i p_i}_{=1} \quad (93b)$$

$$= \log |\mathcal{X}| - \sum_i (p_i \log p_i + p_i \log |\mathcal{X}|) \quad (93c)$$

$$= \log |\mathcal{X}| - \sum_i p_i (\log p_i - \log 1/|\mathcal{X}|) \quad (93d)$$

$$= \underbrace{\log |\mathcal{X}|}_{\text{constante}} - \underbrace{D(p \parallel u)}_{\text{convexo}} \quad (93e)$$

□

A partir da relação em 93e, podemos ver a entropia como a similaridade com a distribuição uniforme. Quanto maior a entropia (menor a divergência $D(p \parallel u)$), mais próximo estaremos da distribuição uniforme.

Analisaremos agora a convexidade ou concavidade da informação mútua $I(X; Y)$, quando fixamos uma das distribuições condicionais ou marginais. Essa análise possui implicação direta na determinação da capacidade de canal em teoria da comunicação, onde a maximização da informação mútua, sobre a distribuição de entrada, será um problema bem comportado com um único máximo global. A concavidade da função objetivo implica que o máximo é atingível e pode ser encontrado usando técnicas de otimização convexa.

Teorema 12 (Concavidade/Convexidade da Informação Mútua) *Seja $(X, Y) \sim p(x, y) = p(x)p(y|x)$, a informação mútua $I(X; Y)$ é uma função côncava de $p(x)$ para $p(y|x)$ fixo e uma função convexa de $p(y|x)$ para $p(x)$ fixo.*

Demonstração. Vejamos primeiro o motivo de $I(X; Y)$ ser uma função côncava de $p(x)$ para $p(y|x)$ fixo. Podemos escrever a informação

mútua como uma função de $p(x)$:

$$I(X; Y) = D(p(x, y) || p(x)p(y)) \quad (94a)$$

$$= \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} \quad (94b)$$

$$= \sum_{x,y} p(x)p(y|x) \log \frac{p(x)p(y|x)}{p(x) \sum_x p(x)p(y|x)}, \quad (94c)$$

onde em 94a utilizamos a definição de informação mútua, em 94b utilizamos a definição de divergência e aplicamos o Teorema de Bayes em 94c. Se $p(y|x)$ é constante, então a informação mútua é função de $p(x)$: $I_{p(x)}(X; Y)$. Utilizando a propriedade da convexidade da divergência de Kullback-Leibler,

$$I_{\lambda p_1(x) + (1-\lambda)p_2(x)}(X; Y) \geq \lambda I_{p_1(x)}(X; Y) + (1-\lambda) I_{p_2(x)}(X; Y). \quad (95)$$

Então a informação mútua é uma função concava de $p(x)$ para $p(y|x)$ fixo.

Agora iremos demonstrar que $I(X; Y)$ é uma função convexa de $p(y|x)$ para $p(x)$ fixo. Aplicamos a mesma ideia, porém agora consideraremos $p(x)$ fixo. Escrevemos então a informação mútua como uma função de $p(y|x)$:

$$I_{p(y|x)}(X; Y) = \sum_{x,y} p(x)p(y|x) \log \frac{p(x)p(y|x)}{p(x) \sum_x p(x)p(y|x)}. \quad (96)$$

Utilizando a propriedade da convexidade da divergência de Kullback-Leibler, obtemos

$$I_{\lambda p_1(y|x) + (1-\lambda)p_2(y|x)}(X; Y) \leq \lambda I_{p_1(y|x)}(X; Y) + (1-\lambda) I_{p_2(y|x)}(X; Y). \quad (97)$$

□

Desigualdade de Processamento de Dados

Quando falamos em processamento de dados, podemos pensar no seguinte modelo: $X \rightarrow Y \rightarrow Z$, onde a variável aleatória X representa os dados originais, a serem transmitidos; Y representa os dados recebidos no outro lado de um canal de comunicação; e Z o resultado de um processamento adicional aplicado a Y . O processamento de dados é modelado como uma cadeia de Markov. Nessa estrutura, Z depende de X apenas através de Y , ou, de outra forma, Z e X são condicionalmente independentes, dado Y .

Definição 8 (Cadeia de Markov). As variáveis aleatórias X , Y e Z formam uma *cadeia de Markov* nesta ordem (denotado $X \rightarrow Y \rightarrow Z$)

se a distribuição condicional de Z depende apenas de Y e é condicionalmente independente de X . Especificamente, X , Y e Z formam uma cadeia de Markov $X \rightarrow Y \rightarrow Z$ se a função massa de probabilidade conjunta pode ser escrita como

$$p(x, y, z) = p(x)p(y|x)p(z|y) \quad (98)$$

Temos uma cadeia de Markov $X \rightarrow Y \rightarrow Z$ se, e somente se, X e Z são condicionalmente independentes dado Y ($X \perp\!\!\!\perp Z|Y$). Isto é,

$$p(x, z|y) = \frac{p(x, y, z)}{p(y)} = \frac{p(x)p(y|x)p(z|y)}{p(y)} = p(x|y)p(z|y) \quad \forall x, y, z. \quad (99)$$

Proposição 10 (Cadeia de Markov Inversa) *Se X , Y e Z formam uma cadeia de Markov nesta ordem ($X \rightarrow Y \rightarrow Z$), então também formam uma cadeia de Markov na ordem inversa ($Z \rightarrow Y \rightarrow X$).*

Demonstração.

$$p(x, y, z) = p(x)p(y|x)p(z|y) = p(x, y)p(z|y) \quad (100a)$$

$$= \frac{p(x, y)p(z|y)p(y)}{p(y)} = p(x|y)p(y, z) \quad (100b)$$

$$= p(x|y)p(y|z)p(z) \quad (100c)$$

□

A *Desigualdade do Processamento de Dados* estabelece que a informação mútua entre variáveis aleatórias não aumenta sob o processamento de dados. Em outras palavras, a informação relevante para uma variável não pode ser aumentada ao processar outra variável relacionada em uma cadeia de Markov.

Teorema 13 (Desigualdade do Processamento de Dados) *Se $X \rightarrow Y \rightarrow Z$ então*

$$I(X; Y) \geq I(X; Z) \quad (101)$$

Demonstração. Utilizando a regra da cadeia da informação mútua (Proposição 6), teremos

$$I(X; Y, Z) = I(X; Z) + I(X; Y|Z) \quad (102a)$$

$$= I(X; Y) + I(X; Z|Y) \quad (102b)$$

Como $X \perp\!\!\!\perp Z|Y$ (X e Z são condicionalmente independentes, dado Y), temos que $I(X; Z|Y) = 0$. Como $I(X; Y|Z) \geq 0$, usando eqs. (102a) e (102b), teremos

$$I(X; Z) + \underbrace{I(X; Y|Z)}_{\geq 0} = I(X; Y) + \overbrace{I(X; Z|Y)} \rightarrow 0 \quad (103)$$

Podemos concluir então que

$$I(X; Z) \leq I(X; Y). \quad (104)$$

□

Note que teremos igualdade na Equação (101) se, e somente se, $I(X; Y|Z) = 0$ (i.e. $X \rightarrow Z \rightarrow Y$). De forma similar, também podemos mostrar que $I(Y; Z) \geq I(X; Z)$.

Corolário 13.1 *Se $Z = g(Y)$, então $I(X; Y) \geq I(X; g(Y))$.*

Demonstração. $X \rightarrow Y \rightarrow g(Y)$ forma uma cadeia de Markov.

□

Corolário 13.2 *Se $X \rightarrow Y \rightarrow Z$ então $I(X; Y|Z) \leq I(X; Y)$.*

Demonstração.

$$\underbrace{I(X; Z) + I(X; Y|Z)}_{\geq 0} = I(X; Y) + \overbrace{I(X; Z|Y)} \rightarrow 0 \quad (105)$$

então

$$I(X; Y|Z) \leq I(X; Y). \quad (106)$$

□

Note também que

$$I(X; Y|Z) = H(X|Z) - H(X|Y, Z) \quad (107a)$$

$$\leq H(X) - H(X|Y) \quad (107b)$$

$$= I(X; Y). \quad (107c)$$

O processamento digital de imagem para remoção de ruído, por exemplo, é uma aplicação prática para a desigualdade de processamento de dados. Considere X como a imagem original, Y como a imagem ruidosa recebida em uma comunicação, e Z a imagem após o processamento para remoção de ruído. Podemos considerar que temos aqui uma cadeia de Markov $X \rightarrow Y \rightarrow Z$ e assim podemos aplicar a desigualdade de processamento de dados, que nos diz: $I(X; Z) \leq I(X; Y)$. O processo de remoção de ruído não pode aumentar a informação mútua entre a imagem original X e a imagem processada Z além do que já existe entre X e Y , indicando uma perda inevitável de informação devido ao ruído. No entanto, a remoção de ruído é possível porque o algoritmo pode explorar regularidades estatísticas ou estruturais na imagem.

Algumas Relações Notáveis

Proposição 11 (Determinismo e a incerteza nula) $H(X) = 0$ se, e somente se, X for determinístico.

Demonstração. Se X é determinístico, então existe $x^* \in \mathcal{X}$ tal que $p(x^*) = 1$ e $p(x) = 0, \forall x \neq x^*$. Neste caso, teremos $H(X) = 0$. Por outro lado, se X não for determinístico, então existe $x^* \in \mathcal{X}$ tal que $0 < p(x^*) < 1$. Desta forma, $H(X) > -p(x^*) \log p(x^*) > 0$. Podemos concluir assim que $H(X) = 0$ se e somente se X for determinístico. $\square$

Proposição 12 (Quando Y é uma função de X) $H(Y|X) = 0$ se, e somente se, Y for um função de X , ou seja, $Y = g(X)$.

Demonstração. Observando a Equação (34a), podemos concluir que $H(Y|X) = 0$ se e somente se todos os termos do somatório forem nulos, ou seja, $H(Y|X = x) = 0$ para cada $x \in S_X$. A partir da Proposição 11, sabemos que isto ocorre se, e somente se, Y é determinístico para cada x dado. Em outras palavras, Y é uma função de X . $\square$

Demonstração. Uma outra forma de demonstração é por contradição. Assuma que existe x , digamos x_0 , e dois valores diferentes de y , digamos y_1 e y_2 , tal que $p(x_0, y_1) > 0$ e $p(x_0, y_2) > 0$. Então a marginal é $p(x_0) \geq p(x_0, y_1) + p(x_0, y_2) > 0$. Temos também

$$p(y_1|x_0) = \frac{p(x_0, y_1)}{p(x_0)} \text{ e } p(y_2|x_0) = \frac{p(x_0, y_2)}{p(x_0)} \quad (108)$$

então ambos $p(y_1|x_0)$ e $p(y_2|x_0)$ não são iguais a 0 (zero) ou 1 (um).

$$H(Y|X) = E[H(Y|X)] \quad (109a)$$

$$= - \sum_x p(x) H(Y|X = x) \quad (109b)$$

$$= - \sum_x p(x) \sum_y p(y|x) \log p(y|x) \quad (109c)$$

$$\geq -p(x_0) \sum_y p(y|x_0) \log p(y|x_0) \quad (109d)$$

$$\geq - \underbrace{p(x_0)}_{>0} \left[\underbrace{p(y_1|x_0)}_{>0} \underbrace{\log p(y_1|x_0)}_{<0} + \underbrace{p(y_2|x_0)}_{>0} \underbrace{\log p(y_2|x_0)}_{<0} \right] \quad (109e)$$

$$> 0 \quad (109f)$$

Então, a entropia condicional $H(Y|X)$ é nula se e somente se Y for uma função de X . Se Y for uma função de X , teremos $p(y_i|x_0) = 0$ ou 1, ou seja, a probabilidade $p(y_i|x_0)$ será igual a 1 apenas para um y_i e zero para os demais. $\square$

Teorema 14 (Informação mútua condicional é não nula) *Para variáveis aleatórias X , Y e Z ,*

$$I(X; Y|Z) \geq 0, \quad (110)$$

com igualdade se, e somente se, X e Y forem independentes quando condicionados a Z , ou seja, $X \perp\!\!\!\perp Y|Z$.

Demonstração. Observe que

$$I(X; Y|Z) = \sum_{x,y,z} p(x, y, z) \log \frac{p(x, y|z)}{p(x|z)p(y|z)} \quad (111a)$$

$$= \sum_z p(z) \sum_{x,y} p(x, y|z) \log \frac{p(x, y|z)}{p(x|z)p(y|z)} \quad (111b)$$

$$= \sum_z p(z) D(p_{XY|z} \parallel p_{X|z} p_{Y|z}), \quad (111c)$$

onde $p_{XY|z}$ representa $\{p(x, y|z), (x, y) \in \mathcal{X} \times \mathcal{Y}\}$, $p_{X|z}$ representa $\{p(x|z), x \in \mathcal{X}\}$, e $p_{Y|z}$ representa $\{p(y|z), y \in \mathcal{Y}\}$. Dado z , ambos $p_{XY|z}$ e $p_{X|z}p_{Y|z}$ são distribuições conjuntas em $\mathcal{X} \times \mathcal{Y}$. Teremos então

$$D(p_{XY|z} \parallel p_{X|z} p_{Y|z}) \geq 0. \quad (112)$$

Concluimos assim que $I(X; Y|Z) \geq 0$. A partir da Definição 5, observamos que $I(X; Y|Z) = 0$ se, e somente se, para todo $z \in S_z$,

$$p(x, y|z) = p(x|z)p(y|z) \quad (113a)$$

ou

$$p(x, y, z) = p(x, z)p(y|z) \quad (113b)$$

para todo x e y . Então X e Y são independentes quando condicionados a Z . □

Proposição 13 (Independência e informação mútua nula) $I(X; Y) = 0$ se, e somente se, $X \perp\!\!\!\perp Y$.

Demonstração. Este caso equivale ao Teorema 14 com a variável Z degenerada. □

Teorema 15 (Condicionar não aumenta a entropia)

$$H(Y|X) \leq H(Y) \quad (114)$$

com igualdade se, e somente se, $X \perp\!\!\!\perp Y$.

Demonstração. Basta considerar

$$H(Y|X) = H(Y) - \underbrace{I(X; Y)}_{\geq 0} \leq H(Y). \quad (115)$$

A igualdade ocorrerá se, e somente se, $I(X; Y) = 0$, o que equivale a $X \perp\!\!\!\perp Y$, conforme a Proposição 13. □

A próxima desigualdade estabelece um teto para a entropia conjunta de um conjunto de variáveis aleatórias, assumindo que elas são independentes.

Teorema 16 (Limite de independência para entropia) *A entropia conjunta $H(X_1, X_2, \dots, X_n)$ de um conjunto de variáveis aleatórias $X_1, X_2, \dots, X_n$ é sempre menor ou igual à soma das entropias individuais, ou seja,*

$$H(X_1, X_2, \dots, X_n) \leq \sum_{i=1}^n H(X_i), \quad (116)$$

com igualdade se, e somente se, X_i , $i = 1, 2, \dots, n$, forem mutuamente independentes.

Demonstração.

$$H(X_1, X_2, \dots, X_n) = \sum_{i=1}^n H(X_i | X_1, \dots, X_{i-1}) \quad (117a)$$

$$\leq \sum_{i=1}^n H(X_i), \quad (117b)$$

onde em 117a utilizamos a regra da cadeia e em 117b utilizamos que condicionar não aumenta a entropia. A desigualdade é justa se e somente se for justa para cada i em 117b, ou seja,

$$H(X_i | X_1, \dots, X_{i-1}) = H(X_i), \quad (118)$$

para $i = 1, \dots, n$. Isto equivale a ter $X_i \perp\!\!\!\perp X_j$, $\forall i \neq j$. □

Teorema 17 (Monotonicidade da informação mútua)

$$I(X; Y, Z) \geq I(X; Y), \quad (119)$$

com igualdade se, e somente se, tivermos uma cadeia de Markov $X \rightarrow Y \rightarrow Z$.

Demonstração. Pela regra da cadeia da informação mútua obtemos

$$I(X; Y, Z) = I(X; Y) + I(X; Z | Y) \geq I(X; Y), \quad (120)$$

onde, para obter a desigualdade, consideramos o fato de que qualquer informação mútua é não negativa. Teremos $I(X; Z | Y) = 0$ se $X \rightarrow Y \rightarrow Z$ formar uma cadeia de Markov. □

Note que o Teorema 17 não contradiz o Corolário 13.2. Quando existir uma cadeia de Markov $X \rightarrow Y \rightarrow Z$, teremos simplesmente $I(X; Y, Z) = I(X; Y)$, mostrando que Z não adiciona informação sobre X além do que Y já fornece, uma propriedade que reflete a estrutura de dependência da cadeia de Markov.

Veremos agora uma relação importante que conecta a entropia à probabilidade de colisão entre amostras independentes, sendo útil em aplicações como compressão de dados e análise de colisões em funções hash, além de reforçar a propriedade da distribuição uniforme como maximizadora da entropia.

Proposição 14 (Limite inferior da probabilidade de colisão) *Considere duas variáveis aleatórias i.i.d. X e X' com entropia $H(X)$. Teremos o seguinte limite inferior para a probabilidade de colisão ($X = X'$):*

$$\Pr(X = X') \geq 2^{-H(X)} \quad (121)$$

com igualdade se e somente se X possuir distribuição uniforme.

Demonstração. Podemos calcular explicitamente o valor probabilidade de coincidência para duas variáveis aleatórias:

$$\Pr(X = X') = \Pr(X = x_1 | X' = x_1) \Pr(X' = x_1) + \dots + \Pr(X = x_n | X' = x_n) \Pr(X' = x_n) \quad (122a)$$

$$= \Pr(X = x_1) \Pr(X' = x_1) + \dots + \Pr(X = x_n) \Pr(X' = x_n) \quad (122b)$$

$$= p^2(x_1) + \dots + p^2(x_n) = \sum_x p^2(x). \quad (122c)$$

Em muitas aplicações práticas, a distribuição exata p pode não ser conhecida, mas a entropia pode ser estimada. Torna-se também intuitivo buscar interpretar a probabilidade de coincidência em termos de incerteza. Para tanto, suponha que $X \sim p(x)$. Usando a desigualdade de Jensen temos

$$2^{E[\log p(X)]} \leq E[2^{\log p(X)}], \quad (123)$$

pois 2^x é convexa. Logo,

$$2^{-H(X)} = 2^{\sum_x p(x) \log p(x)} = 2^{E[\log p(X)]} \quad (124a)$$

$$\leq E[2^{\log p(X)}] \quad (124b)$$

$$= \sum_x p(x) 2^{\log p(x)} = \sum_x p(x) p(x) \quad (124c)$$

$$= \sum_x p^2(x) = \Pr(X = X') \quad (124d)$$

□

Note que, para maximizar a probabilidade $\Pr(X = X')$, devemos minimizar a entropia. No limite, quando $H(X) = 0$, teremos $\Pr(X = X') \geq 1$, logo será igual a 1 e assim $X = X'$ sem dúvida. Por outro lado, maximizar a entropia garante que este limite inferior será mínimo. Isso apenas nos garante que há margem para minimizar

a probabilidade de colisão até determinado ponto, não restringe superiormente esta probabilidade, o que seria desejável de se encontrar.

A Equação (122c) fornece a probabilidade de colisão.

Definição 9 (Entropia de Colisão). A chamada *entropia de colisão*, ou *entropia de Rényi* de segunda ordem ($n = 2$) é definida com

$$H_2(X) = -\log(\Pr(X = X')). \quad (125)$$

$H_2(X)$ mede a incerteza de que duas realizações de dois experimentos aleatórios independentes X e X' tenham o mesmo resultado. Muitas das propriedades básicas da entropia de Shannon não podem ser aplicadas à entropia de Rényi. Para mais informações sobre este tópico, recomendamos a leitura de Delfs e Knebl (2015).

Vamos analisar agora a probabilidade de colisão quando as duas variáveis aleatórias possuem distribuições distintas.

Corolário 17.1 (Limite inferior da probabilidade de colisão com distribuições distintas) *Seja X, X' independentes com $X \sim p(x)$ e $X' \sim q(x)$, $x, x' \in \mathcal{X}$, então*

$$\Pr(X = X') \geq 2^{-H(p) - D(p||q)} \quad (126a)$$

$$\Pr(X = X') \geq 2^{-H(q) - D(q||p)} \quad (126b)$$

ou seja,

$$\Pr(X = X') \geq \max\left(2^{-H(p) - D(p||q)}, 2^{-H(q) - D(q||p)}\right) \quad (127)$$

Demonstração.

$$2^{-H(p) - D(p||q)} = 2^{\sum_x p(x) \log p(x) + \sum_x p(x) \log \frac{q(x)}{p(x)}} \quad (128a)$$

$$= 2^{\sum_x p(x) \log q(x)} \quad (128b)$$

$$= 2^{E_p[\log q(X)]} \quad (128c)$$

$$\leq \sum_x p(x) 2^{\log q(x)} \quad (128d)$$

$$= \sum_x p(x) q(x) \quad (128e)$$

$$= \Pr(X = X'), \quad (128f)$$

onde em 128d aplicamos a desigualdade de Jensen. □

Comunicação em um canal ruidoso

A comunicação em um *canal ruidoso* é um cenário clássico em teoria da informação, onde uma mensagem X é transmitida através de um canal sujeito a ruído, resultando em uma versão distorcida Y . O receptor, tenta estimar X a partir de Y , gerando uma estimativa $\hat{X} = g(Y)$.

Esse processo forma uma cadeia de Markov: $X \rightarrow Y \rightarrow \hat{X}$. O processo de estimação está inevitavelmente sujeito a erros. Para quantificar esse erro e estabelecer limites sobre a probabilidade de erro na estimação de X , introduziremos a desigualdade de Fano (Fano 1961).

Primeiramente, definiremos o erro na estimação como o evento em que $X \neq \hat{X}$, e a *probabilidade de erro* será denotada por $P_e = p(X \neq \hat{X})$. Em muitas situações, não é possível calcular diretamente P_e ; no entanto, veremos que a incerteza sobre X dado Y , representada por $H(X|Y)$, pode ser usada para estabelecer um limite inferior para essa probabilidade de erro. Para derivar esse limite, será suficiente conhecer as características do canal, ou seja, a distribuição condicional $p(y|x)$, que descreve a relação entre a entrada X e a saída Y .

Teorema 18 (Desigualdade de Fano) *Para qualquer estimador $\hat{X}$ tal que $X \rightarrow Y \rightarrow \hat{X}$, com $P_e = Pr(X \neq \hat{X})$, temos*

$$H(P_e) + P_e \log(|\mathcal{X}| - 1) \geq H(X|\hat{X}) \geq H(X|Y). \quad (129)$$

Esta desigualdade pode ser simplificada¹⁵ na forma

$$P_e \geq \frac{H(X|Y) - 1}{\log(|\mathcal{X}| - 1)}, \quad (131)$$

onde utilizamos $H(P_e) \leq 1$.

Demonstração. Vamos primeiramente definir uma variável aleatórias associada ao erro na comunicação:

$$E = \begin{cases} 1 & , \text{ se } \hat{X} \neq X (\text{erro}) \\ 0 & , \text{ se } \hat{X} = X (\text{sem erro}). \end{cases} \quad (132)$$

Utilizando a regra da cadeia obtemos:

$$H(E, X|\hat{X}) = H(X|\hat{X}) + \underbrace{H(E|X, \hat{X})}_{=0} \quad (133a)$$

$$= \underbrace{H(E|\hat{X})}_{\leq H(E)=H(P_e)} + \underbrace{H(X|E, \hat{X})}_{\leq P_e \log(|\mathcal{X}|-1)}. \quad (133b)$$

O termo $H(E|X, \hat{X})$ é nulo pois o erro é uma função determinística de X e $\hat{X}$, então, sabendo X e $\hat{X}$, determinamos E . Como condicionar não aumenta a entropia, usamos $H(E|\hat{X}) \leq H(E) = H(P_e)$. E por fim, note que

$$H(X|\hat{X}, E) = p(E=0) \underbrace{H(X|\hat{X}, E=0)}_{=0} + \quad (134a)$$

$$p(E=1)H(X|\hat{X}, E=1)$$

$$= (1 - P_e)0 + P_e H(X|\hat{X}, E=1) \quad (134b)$$

$$\leq P_e \log(|\mathcal{X}| - 1), \quad (134c)$$

¹⁵ Para o caso de um alfabeto binário ($|\mathcal{X}| = 2$), a desigualdade de Fano na forma da Equação (131) não poderá ser aplicada. Devemos então utilizar a forma mais relaxada:

$$P_e \geq \frac{H(X|Y) - 1}{\log(|\mathcal{X}| - 1)} > \frac{H(X|Y) - 1}{\log |\mathcal{X}|}. \quad (130)$$

onde usamos que, se não há erro ($E = 0$) e conhecemos $\hat{X}$, então determinamos X . Não existe entropia residual em X quando é dado $\hat{X}$ e $E = 0$. Logo $H(X|\hat{X}, E = 0) = 0$. Usamos ainda que, se conhecemos $\hat{X}$ e existe um erro ($E = 1$), então sabemos que $X \neq \hat{X}$, logo isto nos deixa com $(|\mathcal{X}| - 1)$ alternativas. Assim, teremos $H(X|\hat{X}, E = 1) \leq \log(|\mathcal{X}| - 1)$. Aplicando eq. (134c) na Equação (133), obtemos agora

$$H(X|\hat{X}) = H(E|\hat{X}) + H(X|E, \hat{X}) \quad (135a)$$

$$\leq H(P_e) + P_e \log(|\mathcal{X}| - 1). \quad (135b)$$

Como $X \rightarrow Y \rightarrow \hat{X}$ é uma cadeia de Markov, podemos utilizar a desigualdade de processamento de dados:

$$I(X; Y) \geq I(X; \hat{X}) \quad (136a)$$

$$H(X) - H(X|Y) \geq H(X) - H(X|\hat{X}) \quad (136b)$$

$$H(X|\hat{X}) \geq H(X|Y) \quad (136c)$$

Então, utilizando as Equações 135 e 136, obtemos

$$H(P_e) + P_e \log(|\mathcal{X}| - 1) \geq H(X|\hat{X}) \geq H(X|Y). \quad (137)$$

□

Exemplo 9. Considere uma variável aleatória discreta $X \in \mathcal{X} = \{1, 2, \dots, 5\}$ com função massa de probabilidade $p(x) = (0.35, 0.35, 0.1, 0.1, 0.1)$. Seja $Y \in \mathcal{Y} = \{1, 2\}$, de forma que, se $x \leq 2$ teremos $y = x$ com probabilidade $6/7$ e, se $x > 2$, teremos $y = 1$ ou 2 com igual probabilidade. A melhor estratégia é utilizar o estimador $\hat{x} = y$. Calcule a probabilidade de erro e o limite dado pela desigualdade de Fano.

A distribuição condicional $p(y|x)$ é

X \ Y	Y	
	1	2
1	6/7	1/7
2	1/7	6/7
3	1/2	1/2
4	1/2	1/2
5	1/2	1/2

A efetiva probabilidade de erro é dada por

$$P_e = 1 - P_a \quad (138a)$$

$$= 1 - \sum_{i=1}^5 P(x_i = y_i) \quad (138b)$$

$$= 1 - (p(y = 1|x = 1)p(x = 1) + p(y = 2|x = 2)p(x = 2)) \quad (138c)$$

$$= 1 - \left(\frac{6}{7} 0.35 + \frac{6}{7} 0.35 \right) = 0.4 = \frac{2}{5} \quad (138d)$$

onde denotamos por P_a a probabilidade de acerto.

A desigualdade de Fano fornece um limite inferior pra a probabilidade de erro (predição incorreta do valor de X baseado em Y). Este limite inferior é determinado pela incerteza remanescente $H(X|Y)$ sobre X quando Y é conhecido.

Pelo Teorema da Desigualdade de Fano temos que

$$P_e \geq \frac{H(X|Y) - 1}{\log(|\mathcal{X}| - 1)} \quad (139)$$

$$H(X|Y) = - \sum_{x,y} p(x,y) \log p(x|y) \quad (140a)$$

$$= - \sum_{x,y} p(y|x)p(x) \log p(x|y) \quad (140b)$$

onde $p(y|x)$ e $p(x)$ são dados do problema e ainda será necessário calcular $p(x|y)$ para encontrar $H(X|Y)$.

$$P(X|Y=1) = \frac{P(X, Y=1)}{P(Y=1)} = \frac{P(Y=1|X)P(X)}{P(Y=1)} \quad (141a)$$

$$= \frac{(\frac{6}{7}, \frac{1}{7}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}) \cdot (0.35, 0.35, 0.1, 0.1, 0.1)}{\sum ((\frac{6}{7}, \frac{1}{7}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}) \cdot (0.35, 0.35, 0.1, 0.1, 0.1))} \quad (141b)$$

$$= \frac{(0.3, 0.05, 0.05, 0.05, 0.05)}{1/2} \quad (141c)$$

$$= (0.6, 0.1, 0.1, 0.1, 0.1) \quad (141d)$$

$$P(X|Y=2) = \frac{(\frac{1}{7}, \frac{6}{7}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}) \cdot (0.35, 0.35, 0.1, 0.1, 0.1)}{\sum ((\frac{1}{7}, \frac{6}{7}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}) \cdot (0.35, 0.35, 0.1, 0.1, 0.1))} \quad (142a)$$

$$= (0.1, 0.6, 0.1, 0.1, 0.1) \quad (142b)$$

Desta forma teremos

$$H(X|Y) = H(X|Y=1)P(Y=1) + H(X|Y=2)P(Y=2) \quad (143a)$$

$$= -\frac{1}{2} (0.6 \log 0.6 + 4 \times 0.1 \log 0.1) - \quad (143b)$$

$$\frac{1}{2} (4 \times 0.1 \log 0.1 + 0.6 \log 0.6) \\ = 1.771 \text{ bits.} \quad (143c)$$

Utilizando a desigualdade de Fano

$$P_e \geq \frac{H(X|Y) - 1}{\log(|\mathcal{X}| - 1)} = \frac{1.771 - 1}{\log(5 - 1)} = 0.3855. \quad (144)$$

De fato, temos $P_e = 0.4 \geq 0.3855$.

Medida de Informação

Exploramos conceitos como entropia, informação mútua e divergência de Kullback-Leibler sob a perspectiva de variáveis aleatórias e suas distribuições, mas essas grandezas podem ser entendidas de forma mais profunda através de um paralelo com a teoria dos conjuntos e a teoria da medida. Desta forma, buscaremos uma interpretação geométrica e estrutural para relações como a regra da cadeia e a desigualdade do processamento de dados, permitindo uma compreensão mais ampla de como a informação é estruturada e quantificada em espaços probabilísticos.

Dado um conjunto de variáveis aleatórias: $X_1, X_2, \dots, X_n$, a cada uma delas associamos um conjunto $\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_n$.

Definição 10 (Campo). Um campo $\mathcal{F}_n$ gerado pelos conjuntos $\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_n$ é a coleção de conjuntos que podem ser obtidos através de qualquer sequência de operações usuais de conjuntos (união, interseção, complemento, e diferença) sobre os conjuntos $\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_n$.

Definição 11 (Átomo). Os átomos de $\mathcal{F}_n$ são os conjuntos da forma $\cap_{i=1}^n Y_i$, onde Y_i é $\tilde{X}_i$ ou $\tilde{X}_i^c$, o complemento de $\tilde{X}_i$.

Exemplo 10. Vamos considerar o seguinte exemplo com $n = 2$. Neste caso, teremos os conjuntos $\tilde{X}_1, \tilde{X}_2$ e seus complementos, respectivamente, $\tilde{X}_1^c, \tilde{X}_2^c$. Existirão 4 átomos: $\tilde{X}_1 \cap \tilde{X}_2$, $\tilde{X}_1 \cap \tilde{X}_2^c$, $\tilde{X}_1^c \cap \tilde{X}_2$, e $\tilde{X}_1^c \cap \tilde{X}_2^c$, que estão representados na Figura 13.

Exemplo 11. Considerando agora o caso com $n = 3$. Teremos os conjuntos $\tilde{X}_1, \tilde{X}_2, \tilde{X}_3$ e seus complementos, respectivamente, $\tilde{X}_1^c, \tilde{X}_2^c, \tilde{X}_3^c$. Existirão 8 átomos: $\tilde{X}_1 \cap \tilde{X}_2 \cap \tilde{X}_3$, $\tilde{X}_1 \cap \tilde{X}_2 \cap \tilde{X}_3^c$, $\tilde{X}_1 \cap \tilde{X}_2^c \cap \tilde{X}_3$, $\tilde{X}_1 \cap \tilde{X}_2^c \cap \tilde{X}_3^c$, $\tilde{X}_1^c \cap \tilde{X}_2 \cap \tilde{X}_3$, $\tilde{X}_1^c \cap \tilde{X}_2 \cap \tilde{X}_3^c$, $\tilde{X}_1^c \cap \tilde{X}_2^c \cap \tilde{X}_3$, e $\tilde{X}_1^c \cap \tilde{X}_2^c \cap \tilde{X}_3^c$, como representados na Figura 14.

Consideremos n variáveis aleatórias discretas $X_1, X_2, \dots, X_n$, onde cada X_i tem um suporte finito S_{X_i} , com $|S_{X_i}| = k_i$, e os suportes S_{X_i} podem ser distintos. Por exemplo, $S_{X_1} = \{0, 1\}$, $S_{X_2} = \{a, b, c\}$, e assim por diante. O espaço amostral é o produto cartesiano $\Omega = S_{X_1} \times S_{X_2} \times \dots \times S_{X_n}$, com $|\Omega| = k_1 \cdot k_2 \cdot \dots \cdot k_n$, e o campo $F_n = 2^\Omega$ contém todos os subconjuntos de Ω . Nesse contexto, os átomos são os eventos $\{(X_1, \dots, X_n) = (x_1, \dots, x_n)\}$, e grandezas como a entropia $H(X_1, \dots, X_n)$ podem ser interpretadas como medidas sobre F_n . Para ilustrar propriedades específicas, consideremos o caso particular em que $S_{X_i} = \{0, 1\}, \forall i$, ou seja, $\Omega = \{0, 1\}^n$, com $|\Omega| = 2^n$ e $F_n = 2^\Omega$. Nesse cenário, teremos as seguintes propriedades:

1. há exatamente 2^n átomos;
2. $|F_n| = 2^\Omega = 2^{2^n}$;
3. Todos os átomos em F_n são disjuntos;

	$\tilde{X}_2$	$\tilde{X}_2^c$
$\tilde{X}_1$	$\tilde{X}_1 \cap \tilde{X}_2$	$\tilde{X}_1 \cap \tilde{X}_2^c$
$\tilde{X}_1^c$	$\tilde{X}_1^c \cap \tilde{X}_2$	$\tilde{X}_1^c \cap \tilde{X}_2^c$

Figura 13: Exemplo de átomos para $n = 2$.

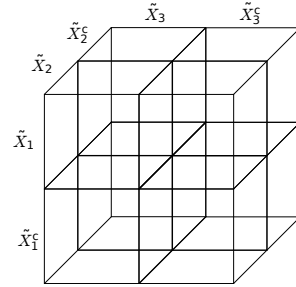


Figura 14: Exemplo de átomos para $n = 3$.

4. Todo conjunto $A \in \mathcal{F}_n$ pode ser expresso de forma única como uma união de um subconjunto dos átomos.

Em análise matemática, uma medida em um conjunto S é uma forma sistemática de atribuir números a todo subconjunto de S , sendo intuitivamente interpretada como o seu tamanho. Medida com sinal é uma generalização do conceito de medida permitindo que esta assuma valores negativos.

Definição 12 (Medida com sinal). Uma função real μ definida em $\mathcal{F}_n$ é chamada medida com sinal se for aditiva no conjunto, i.e., para A e B disjuntos em $\mathcal{F}_n$,

$$\mu(A \cup B) = \mu(A) + \mu(B). \quad (145)$$

Para uma medida com sinal μ teremos $\mu(\emptyset) = 0$, já que $\mu(A) = \mu(A \cup \emptyset) = \mu(A) + \mu(\emptyset)$.

Uma medida com sinal μ em $\mathcal{F}_n$ é completamente especificada por seus valores nos átomos de $\mathcal{F}_n$. Os valores de μ em outros conjuntos de $\mathcal{F}_n$ podem ser obtidos pela aditividade de conjuntos, já que qualquer $\tilde{X} \in \mathcal{F}_n$ pode ser representado como $\tilde{X} = \cup_{i=1} Y_i$, onde Y_i são átomos escolhidos apropriadamente.

Exemplo 12 ($n = 2$). Uma medida com sinal μ em $\mathcal{F}_2$ é completamente especificada pelos valores $\mu(\tilde{X}_1 \cap \tilde{X}_2)$, $\mu(\tilde{X}_1 \cap \tilde{X}_2^c)$, $\mu(\tilde{X}_1^c \cap \tilde{X}_2)$, e $\mu(\tilde{X}_1^c \cap \tilde{X}_2^c)$.

O valor de μ em $\tilde{X}_1$ pode ser obtido da seguinte forma

$$\mu(\tilde{X}_1) = \mu((\tilde{X}_1 \cap \tilde{X}_2) \cup (\tilde{X}_1 \cap \tilde{X}_2^c)) \quad (146a)$$

$$= \mu(\tilde{X}_1 \cap \tilde{X}_2) + \mu(\tilde{X}_1 \cap \tilde{X}_2^c). \quad (146b)$$

O valor de μ em $\tilde{X}_1 \setminus \tilde{X}_2$ é dado por

$$\mu(\tilde{X}_1 \setminus \tilde{X}_2) = \mu(\tilde{X}_1 \cap \tilde{X}_2^c). \quad (147)$$

O valor de μ em $\tilde{X}_1 \cup \tilde{X}_2$ pode ser obtido através de

$$\mu(\tilde{X}_1 \cup \tilde{X}_2) = \mu((\tilde{X}_1 \cap \tilde{X}_2) \cup (\tilde{X}_1 \cap \tilde{X}_2^c) \cup (\tilde{X}_1^c \cap \tilde{X}_2)) \quad (148a)$$

$$= \mu(\tilde{X}_1 \cap \tilde{X}_2) + \mu(\tilde{X}_1 \cap \tilde{X}_2^c) + \mu(\tilde{X}_1^c \cap \tilde{X}_2) \quad (148b)$$

Os conjuntos $\tilde{X}_1$ e $\tilde{X}_2$ estão associados às variáveis aleatórias X_1 e X_2 . O campo $\mathcal{F}_2$ é gerado por $\tilde{X}_1$ e $\tilde{X}_2$, através dos átomos $(\tilde{X}_1 \cap \tilde{X}_2)$, $(\tilde{X}_1 \cap \tilde{X}_2^c)$, $(\tilde{X}_1^c \cap \tilde{X}_2)$, e $(\tilde{X}_1^c \cap \tilde{X}_2^c)$. O diagrama de informação é apresentado na Figura 15.

O conjunto universo será considerada como sendo $\Omega = \tilde{X}_1 \cup \tilde{X}_2$. Desta forma, o átomo $\tilde{X}_1^c \cap \tilde{X}_2^c$ se degenera ao conjunto vazio,

$$\tilde{X}_1^c \cap \tilde{X}_2^c = (\tilde{X}_1 \cup \tilde{X}_2)^c = \Omega^c = \emptyset. \quad (149)$$

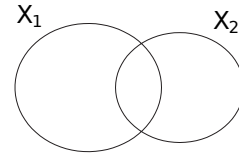


Figura 15: Diagrama de informação para X_1 e X_2 .

Para as v.a.s X_1 e X_2 , as medidas de informação de Shannon são

$$H(X_1), H(X_2), H(X_1|X_2), H(X_2|X_1), H(X_1, X_2), I(X_1; X_2). \quad (150)$$

Utilizando a notação $A \cap B^c \equiv A \setminus B$, definimos uma medida com sinal μ^*

$$\mu^*(\tilde{X}_1 \setminus \tilde{X}_2) = H(X_1|X_2), \quad (151a)$$

$$\mu^*(\tilde{X}_2 \setminus \tilde{X}_1) = H(X_2|X_1), \quad (151b)$$

$$\mu^*(\tilde{X}_1 \cap \tilde{X}_2) = I(X_1; X_2). \quad (151c)$$

Estes são os valores de μ^* nos átomos não vazios de $\mathcal{F}_2$. Os valores de μ^* nos demais conjuntos de $\mathcal{F}_2$ podem ser obtidos por adição de conjuntos. Em particular, temos as relações

$$\mu^*(\tilde{X}_1 \cup \tilde{X}_2) = H(X_1, X_2), \quad (152a)$$

$$\mu^*(\tilde{X}_1) = H(X_1), \quad (152b)$$

$$\mu^*(\tilde{X}_2) = H(X_2). \quad (152c)$$

Por exemplo, a Equação 152a pode ser verificada

$$\begin{aligned} \mu^*(\tilde{X}_1 \cup \tilde{X}_2) &= \mu^*((\tilde{X}_1 \setminus \tilde{X}_2) \cup (\tilde{X}_2 \setminus \tilde{X}_1) \cup (\tilde{X}_1 \cap \tilde{X}_2)) \\ &= \mu^*(\tilde{X}_1 \setminus \tilde{X}_2) + \mu^*(\tilde{X}_2 \setminus \tilde{X}_1) + \mu^*(\tilde{X}_1 \cap \tilde{X}_2) \\ &= H(X_1|X_2) + H(X_2|X_1) + I(X_1; X_2) \\ &= H(X_1, X_2). \end{aligned} \quad (153a)$$

A Equação 152b também pode ser facilmente verificada

$$\begin{aligned} \mu^*(\tilde{X}_1) &= \mu^*((\tilde{X}_1 \cap \tilde{X}_2) \cup (\tilde{X}_1 \cap \tilde{X}_2^c)) \\ &= \mu^*(\tilde{X}_1 \cap \tilde{X}_2) + \mu^*(\tilde{X}_1 \cap \tilde{X}_2^c) \\ &= I(X_1; X_2) + H(X_1|X_2) = H(X_1). \end{aligned} \quad (154a)$$

É possível então verificar a seguinte correspondência com as medidas de informação de Shannon

$$H/I \leftrightarrow \mu^* \quad (155a)$$

$$, \leftrightarrow \cup \quad (155b)$$

$$; \leftrightarrow \cap \quad (155c)$$

$$| \leftrightarrow \setminus \quad (155d)$$

Com a notação de medida, não existe distinção entre H e I , podemos escrever $H(X; Y) = I(X; Y)$, utilizando a notação do ponto-e-vírgula.

Estimação da Entropia

Vimos anteriormente conceitos como a entropia de Shannon, entropia conjunta e informação mútua, assumindo que a distribuição de probabilidade da fonte é conhecida. Em aplicações práticas, como compressão de dados, criptografia ou modelagem estatística, raramente temos acesso à distribuição exata da fonte. Em vez disso, dispomos de dados — por exemplo, uma sequência de bits — e precisamos estimar a distribuição e, consequentemente, a entropia. Este processo de estimativa envolve suposições sobre a fonte e limitações impostas pela quantidade finita de dados.

A entropia de Shannon assume uma fonte sem memória e ignora dependências ou padrões temporais. Ao utilizá-la como uma medida de incerteza de uma fonte, devemos ter em mente tais limitações. Sequências previsíveis, como 010101... ou 000...111..., podem ter a mesma entropia estimada que sequências imprevisíveis (e.g., uma sequência verdadeiramente aleatória) se apresentarem a mesma distribuição de símbolos (e.g., $p(0) = p(1) = 1/2$).

Ainda, a distribuição estimada supõe a adoção de algum modelo e está sujeita a erros de estimação desta distribuição. Por exemplo, podemos assumir que cada bit é um símbolo produzido pela fonte, ou cada sequência de 2 bits é um símbolo, ou sequências de 3 bits, etc. Além disso, devemos lidar com a imprecisão da estimativa da distribuição subjacente, uma vez que temos tão somente uma amostra finita em mãos.

Estimar a entropia requer primeiro estimar a distribuição da fonte. Um método muito utilizado é a *estimativa de máxima verossimilhança* (MLE), que se relaciona diretamente com a minimização da divergência de Kullback-Leibler.

Suponha uma variável aleatória $\mathbf{X} = (X_1, \dots, X_N)$ com distribuição subjacente p , que depende de um parâmetro θ , podemos a chamar também de p_θ . Dadas N observações i.i.d. $x_1, \dots, x_N$, queremos estimar θ via uma estatística $\hat{\theta} = T(x_1, \dots, x_N)$. A *verossimilhança* $L(\theta)$ é a probabilidade conjunta dos dados sob o modelo:

$$L(\theta) = \Pr(X_1 = x_1, \dots, X_N = x_N | \theta) = \prod_{n=1}^N p(x_n | \theta). \quad (156)$$

Como o logaritmo é monótono crescente, maximizar $L(\theta)$ equivale a maximizar o logaritmo da verossimilhança:

$$\ell(\theta) = \log L(\theta) = \sum_{n=1}^N \log p(x_n | \theta). \quad (157)$$

O estimador de máxima verossimilhança é:

$$\hat{\theta}_{\text{MLE}} = \underset{\theta \in \Theta}{\operatorname{argmax}} \ell(\theta; x_1, \dots, x_N). \quad (158)$$

A distribuição empírica $\hat{p}$, baseada nas observações, é definida como uma função massa de probabilidade:

$$\hat{p}(x) = \frac{1}{N} \sum_{n=1}^N \delta(x_n = x) = \frac{N(x|x_1, \dots, x_N)}{N} = \frac{N_x}{N}, \quad (159)$$

onde $\delta(x_n = x)$ é a delta de Kronecker, que vale 1 se $x_n = x$ e 0 caso contrário, e $N_x = N(x|x_1, \dots, x_N)$ expressa o número de ocorrências do símbolo x na amostra $x_1, \dots, x_N$. Seja p_θ uma distribuição parametrizada por θ , maximizar a verossimilhança de $p_\theta(x)$ equivale a minimizar a divergência de KL entre $\hat{p}$ e p_θ :

$$\begin{aligned} D_{\text{KL}}(\hat{p}||p_\theta) &= \sum_{x \in \mathcal{X}} \hat{p}(x) \log \frac{\hat{p}(x)}{p_\theta(x)} \\ &= -H(\hat{p}) - \sum_{x \in \mathcal{X}} \hat{p}(x) \log p_\theta(x) \end{aligned} \quad (160a)$$

$$\begin{aligned} &= -H(\hat{p}) - \frac{1}{N} \sum_{x \in \mathcal{X}} \sum_{n=1}^N \delta(x_n = x) \log p_\theta(x) \\ &= -H(\hat{p}) - \frac{1}{N} \sum_{n=1}^N \log p_\theta(x_n) \end{aligned} \quad (160b)$$

$$= -H(\hat{p}) - \frac{1}{N} \ell(\theta). \quad (160c)$$

Como $H(\hat{p})$ é constante para um conjunto fixo de dados, minimizar $D_{\text{KL}}(\hat{p}||p_\theta)$ equivale a maximizar $\ell(\theta)$. Portanto, o estimador de máxima verossimilhança é:

$$\hat{\theta}_{\text{MLE}} = \underset{\theta \in \Theta}{\operatorname{argmin}} D_{\text{KL}}(\hat{p}||p_\theta). \quad (161)$$

Essa equivalência mostra que escolher o modelo p_θ que melhor se ajusta aos dados (via MLE) minimiza a diferença entre a distribuição empírica e o modelo.

Suavização: Regra de Laplace e Dirichlet

Quando realizamos a estimativa de probabilidades a partir de amostras finitas, devemos nos atentar ao seguinte problema: símbolos raros podem não ser observados devido ao tamanho limitado da amostra, levando a estimativas imprecisas. Devemos considerar que é um mero acaso da amostra observamos determinado símbolo uma vez ou nenhuma vez. Em uma sequência binária de N bits, se nenhum 0 for observado, o estimador atribuirá $\hat{p}(0) = 0$, sugerindo que 0 é impossível, o que pode ser um artefato da amostra. Isso distorce a estimativa da distribuição, especialmente em fontes com baixa probabilidade para certos símbolos. Para abordar essa questão, técnicas de *suavização* (em inglês, *smoothing*) ajustam as probabilidades empíricas, atribuindo

valores não nulos a símbolos não observados. A Regra de Sucessão de Laplace e sua generalização via Dirichlet são exemplos simples de técnicas de suavização utilizadas.

Regra de Laplace e o Problema do Nascer do Sol

A *Regra de Sucessão de Laplace*, proposta no século XVIII por Pierre-Simon Laplace, é uma abordagem bayesiana para estimar probabilidades em experimentos binários, como o famoso “problema do nascer do sol”: qual é a probabilidade de que o sol nasça amanhã, dado que nasceu todos os dias nos últimos n dias? Essa regra suaviza as estimativas ao assumir uma distribuição *a priori* uniforme sobre a probabilidade p_0 no intervalo $[0, 1]$. Isso reflete a suposição de ignorância total sobre a distribuição, antes de observar os dados.

Considere uma fonte binária com alfabeto $\mathcal{X} = \{0, 1\}$, onde $X_1, \dots, X_N$ são variáveis aleatórias i.i.d. com $p(0) = p_0$ e $p(1) = p_1 = 1 - p_0$. Dadas N observações $x_1, \dots, x_N$, o número de ocorrências de 0 e 1 é:

$$N_0 = \sum_{n=1}^N \delta(x_n = 0), \quad N_1 = \sum_{n=1}^N \delta(x_n = 1), \quad N = N_0 + N_1. \quad (162)$$

A verossimilhança dos dados é:

$$p(x_1, \dots, x_N | p_0) = p_0^{N_0} p_1^{N_1}. \quad (163)$$

Assumimos uma distribuição *a priori* uniforme para p_0 , ou seja, $\Pr(p_0) = 1$ para $p_0 \in [0, 1]$.

$$\Pr(p_0 | x_1, \dots, x_N) = \frac{p(x_1, \dots, x_N | p_0) \Pr(p_0)}{\Pr(x_1, \dots, x_N)} \quad (164a)$$

$$= \frac{p_0^{N_0} p_1^{N_1}}{\Pr(x_1, \dots, x_N)}. \quad (164b)$$

O denominador da Equação (164), a probabilidade marginal dos dados, é calculado usando a função Beta¹⁶:

$$\Pr(x_1, \dots, x_N) = \int_0^1 \Pr(x_{1:N}, p_0) dp_0 \quad (168a)$$

$$= \int_0^1 p_0^{N_0} p_1^{N_1} \Pr(p_0) dp_0 \quad (168b)$$

$$= \int_0^1 p_0^{N_0} p_1^{N_1} dp_0 \quad (168c)$$

$$= B(N_0 + 1, N_1 + 1) \quad (168d)$$

$$= \frac{\Gamma(N_0 + 1) \Gamma(N_1 + 1)}{\Gamma(N_0 + N_1 + 2)} \quad (168e)$$

$$= \frac{N_0! N_1!}{(N_0 + N_1 + 1)!}. \quad (168f)$$

¹⁶ A função Beta é definida como:

$$B(a, b) = \int_0^1 x^{a-1} (1-x)^{b-1} dx. \quad (165)$$

A função Beta pode ser expressa em termos da função Gamma:

$$B(a, b) = \frac{\Gamma(a) \Gamma(b)}{\Gamma(a+b)}. \quad (166)$$

A função Gamma para inteiros pode ser expressa como:

$$\Gamma(n) = (n-1)!, \quad (167)$$

onde n é inteiro.

Em 168a utilizamos que a probabilidade marginal $\Pr(x_1, \dots, x_N)$ é obtida integrando a probabilidade conjunta $\Pr(x_{1:N}, p_0)$ sobre p_0 . Em 168c utilizamos que a distribuição *a priori* é uniforme, $\Pr(p_0) = 1$. A integral remanescente em 168c é característica da função Beta. Em 168e, avaliando a função gamma nos inteiros positivos n , temos $\Gamma(n) = (n-1)!$. Para prever a probabilidade do próximo símbolo ser 0, calculamos:

$$\Pr(X_{N+1} = 0 | x_1, \dots, x_N) = \int_0^1 p_0 \Pr(p_0 | x_1, \dots, x_N) dp_0 \quad (169a)$$

$$= \int_0^1 p_0 \frac{p_0^{N_0} (1-p_0)^{N_1}}{\Pr(x_1, \dots, x_N)} dp_0 \quad (169b)$$

$$= \frac{\int_0^1 p_0^{N_0+1} (1-p_0)^{N_1} dp_0}{\int_0^1 p_0^{N_0} (1-p_0)^{N_1} dp_0} \quad (169c)$$

$$= \frac{B(N_0 + 2, N_1 + 1)}{B(N_0 + 1, N_1 + 1)} \quad (169d)$$

$$= \frac{\frac{(N_0+1)! N_1!}{(N_0+N_1+2)!}}{\frac{N_0! N_1!}{(N_0+N_1+1)!}} \quad (169e)$$

$$= \frac{N_0 + 1}{N_0 + N_1 + 2}. \quad (169f)$$

Essa é a Regra de Laplace, que estima:

$$\hat{p}(0) = \frac{N_0 + 1}{N + 2}, \quad \hat{p}(1) = \frac{N_1 + 1}{N + 2}. \quad (170)$$

A adição de 1 a cada contagem (chamada de ‘contagem pseudo’) garante que nenhum símbolo tenha probabilidade nula, suavizando a estimativa. No contexto do ‘problema do nascer do sol’, se o sol nasceu em $n = 1.826.251$ dias, a probabilidade de nascer amanhã é $\frac{1.826.251+1}{1.826.251+2} \approx 0.99999945$, refletindo alta confiança, mas evitando certeza absoluta.

Generalização: Abordagem de Dirichlet

A Regra de Laplace assume uma distribuição *a priori* uniforme, equivalente a uma distribuição Beta com parâmetros $\alpha_0 = \alpha_1 = 1$. A abordagem de Dirichlet generaliza isso, permitindo distribuições *a priori* mais flexíveis. Para uma fonte binária, a distribuição *a priori* é uma distribuição Beta com parâmetros $\alpha_0, \alpha_1 > 0$, e a probabilidade do próximo símbolo é:

$$\Pr(X_{N+1} = 0 | x_1, \dots, x_N) = \frac{N_0 + \alpha_0}{N_0 + N_1 + \alpha_0 + \alpha_1}. \quad (171)$$

Escolher $\alpha_0 = \alpha_1 = 1$ simplifica para a Regra de Laplace, enquanto outros valores ajustam o peso da distribuição *a priori*, útil em contextos onde há conhecimento prévio sobre a fonte. Para alfabetos maiores, a regra de Dirichlet estende a suavização a múltiplos símbolos.

O Desafio da Estimativa de Entropia

A entropia de Shannon para uma fonte binária X com símbolos em $\mathcal{X} = \{0, 1\}$ é dada por:

$$H(X) = - \sum_{x \in \{0,1\}} p(x) \log_2 p(x), \quad (172)$$

onde $p(x)$ é a probabilidade de cada símbolo. Quando $p(0)$ e $p(1)$ são desconhecidos, devemos estimá-los a partir de uma sequência observada, como 010110... A entropia estimada, $\hat{H}(X)$, é então calculada usando essas probabilidades empíricas. No entanto, há vários fatores que devemos considerar: Os dados são finitos, sendo assim, uma sequência curta pode não representar a verdadeira distribuição da fonte, levando a estimativas imprecisas; Podem existir dependências desconhecidas na sequência de símbolos produzida pela fonte (a entropia de Shannon assume uma fonte sem memória); A granularidade com que os dados são tomados afeta todo problema (por exemplo, devemos decidir se os símbolos são bits ou sequências, blocos, de bits).

A entropia de Shannon, definida como $H(X) = - \sum_{x \in \mathcal{X}} p(x) \log p(x)$, quantifica a incerteza de uma fonte, mas sua determinação exige o conhecimento da distribuição subjacente $p(x)$. Em cenários reais, essa distribuição é geralmente desconhecida, sendo necessário estimá-la a partir de uma amostra finita. Este processo de estimativa apresenta limitações, devido à finitude/escassez dos dados, dependências entre símbolos e a sensibilidade da entropia a eventos raros.

Estimador Plug-In

O método mais comum para estimar a entropia é o *estimador plug-in*, que utiliza a estimativa de máxima verossimilhança (MLE) para as probabilidades dos símbolos. Para uma fonte com alfabeto $\mathcal{X} = \{a_1, \dots, a_M\}$ e uma amostra de N observações de símbolos produzidos por esta fonte $x_1, \dots, x_N$, a probabilidade do k -ésimo símbolo é estimada como:

$$\hat{p}_k = \frac{n_k}{M}, \quad \text{onde} \quad n_k = \sum_{n=1}^N \delta(x_n = a_k) \quad (173)$$

é o número de ocorrências de a_k na amostra. A entropia estimada é então:

$$\hat{H}(X) = - \sum_{k=1}^M \hat{p}_k \log \hat{p}_k. \quad (174)$$

Esse estimador assume que os símbolos são independentes e identicamente distribuídos (i.i.d.), ou seja, não há dependências entre símbolos consecutivos. Em fontes reais, como textos, onde palavras ou bits subsequentes exibem correlações, essa suposição é irrealista, levando a

estimativas imprecisas. Por exemplo, em uma sequência binária como 010101..., a dependência entre bits reduz a entropia efetiva, mas o estimador *plug-in* pode superestimá-la.

Sensibilidade a Eventos Raros

A entropia de Shannon é particularmente sensível às probabilidades de eventos raros, que contribuem significativamente para o valor final devido à função logarítmica. A informação associada a um evento de probabilidade p_k é $-\log_2 p_k$, que cresce inversamente com p_k . Pequenas perturbações nas estimativas de probabilidades de cauda (eventos raros) podem causar grandes variações em $\hat{H}(X)$. Em amostras finitas, eventos raros são frequentemente subestimados ou não observados, resultando em $\hat{p}_k = 0$ para alguns símbolos, o que subestima a entropia. Por exemplo, em uma fonte binária com $N = 10$ e apenas zeros observados ($n_0 = 10, n_1 = 0$), o estimador *plug-in* dá $\hat{H}(X) = 0$, sugerindo certeza nula, o que é irrealista, uma vez que observação tomada como base é muito pequena (o fato de não termos observado nenhuma ocorrência do símbolo 1 nesta amostra pode ser tão somente um acaso da amostra tomada).

Técnicas de Suavização

Para mitigar o problema de eventos raros, técnicas de *suavização* atribuem probabilidades não nulas a símbolos não observados. A Regra de Sucessão de Laplace (Laplace 1840), vista anteriormente, é uma das primeiras abordagens. Para o exemplo de uma fonte binária, a Laplace estima $\hat{p}(0) = \frac{n_0+1}{M+2}$, garantindo que $\hat{p}(1) > 0$ mesmo se $n_1 = 0$. No século XX, Alan Turing e Irving John Good (Good 1953) sistematizaram métodos de suavização.¹⁷ Hoje, tais técnicas são essenciais no Processamento de Linguagem Natural. Essas técnicas melhoram a robustez da estimativa de entropia, mas não eliminam o problema de dependências em sequências com padrões.

Propriedades Estatísticas do Estimador Plug-In

O estimador *plug-in* é não paramétrico, intuitivo e apresenta convergência rápida entre os estimadores de entropia (Z. Zhang e X. Zhang 2012). Antos e Kontoyiannis (2001) mostraram que, para distribuições com entropia finita, a variância de $\hat{H}(X)$ tem um limite superior que decai com o tamanho da amostra N a uma taxa $\mathcal{O}\left(\frac{\ln^2 N}{N}\right)$, válido até para alfabetos infinitos contáveis. Para alfabetos finitos de tamanho conhecido $|\mathcal{X}| = M$, a variância decai a uma taxa $\mathcal{O}\left(\frac{1}{N}\right)$, que é ótima, pois o estimador *plug-in* coincide com a MLE quando M é conhecido (Z. Zhang e X. Zhang 2012).

No entanto, o estimador é enviesado, tendendo a subestimar a entropia verdadeira, especialmente devido à subestimação de eventos

¹⁷ As técnicas de suavização foram cruciais durante a Segunda Guerra Mundial no esforço dos criptoanalistas em Bletchley Park, Reino Unido. O estimador de Good-Turing, desenvolvido por Turing e formalizado por Good, foi usado para estimar probabilidades de eventos raros em mensagens cifradas pela máquina Enigma, permitindo prever a frequência de letras ou padrões não observados em amostras limitadas e contribuindo significativamente para a decifração de comunicações alemãs.

raros. Harris (1975) derivou uma aproximação para o viés esperado:

$$\mathbb{E}[\hat{H} - H] = -\frac{M-1}{2N} + \frac{1}{12N^2} \left(1 - \sum_{k=1}^M p_k^{-1} \right) + \mathcal{O}(N^{-3}), \quad (175)$$

onde o termo dominante $-\frac{M-1}{2N}$ indica que o viés é mais pronunciado para alfabetos grandes em relação ao tamanho da amostra. A variância do estimador é dada por:

$$\text{Var}(\hat{H}) = \frac{1}{N} \left(\sum_{k=1}^M p_k (\log p_k)^2 - H^2 \right) + \frac{M-1}{2N^2} + \mathcal{O}(N^{-3}). \quad (176)$$

Essas expressões, originalmente em nats (Harris 1975), devem ser convertidas para bits dividindo por $\ln 2$ quando necessário. Quando M é desconhecido, como em fontes com alfabetos grandes, a estimativa do próprio M adiciona complexidade, ampliando o viés.

Outros Estimadores de Entropia

Embora o estimador *plug-in* seja amplamente utilizado devido à sua simplicidade e convergência rápida, sua tendência a subestimar a entropia, especialmente em amostras finitas com eventos raros, motivou o desenvolvimento de estimadores alternativos.

O estimador de Miller 1955 ajusta o viés do estimador *plug-in* adicionando um termo de correção baseado no número de símbolos observados. Dada a entropia *plug-in*, $\hat{H}$ (definida na Equação (174)), o estimador de Miller é definido como:

$$\hat{H}_{\text{Miller}} = \hat{H} + \frac{\hat{M} - 1}{2N}, \quad (177)$$

onde $\hat{M}$ é o número de símbolos distintos observados na amostra (i.e., com $n_k > 0$) e N é o tamanho da amostra. O termo $(\hat{M} - 1)/2N$ corresponde à primeira ordem do viés esperado (Equação (175)), compensando a subestimação causada por eventos raros. Embora simples, o estimador de Miller é eficaz para alfabetos moderados, mas pode ser insuficiente em amostras muito pequenas ou quando $\hat{M}$ subestima o tamanho real do alfabeto.

O estimador Jackknife (Zahl 1977), baseado na técnica de reamostragem de Quenouille (1956), reduz o viés ao combinar estimativas de entropia calculadas em subamostras. Para uma amostra $x_1, \dots, x_N$, o estimador é:

$$\hat{H}_{\text{Jackknife}} = N\hat{H} - \frac{N-1}{N} \sum_{i=1}^N \hat{H}_{-i}, \quad (178)$$

onde $\hat{H}_{-i}$ é a entropia *plug-in* calculada na subamostra de tamanho $N - 1$ que exclui o i -ésimo símbolo, com probabilidades:

$$\hat{p}_{k,-i} = \frac{n_k - \delta(x_i = a_k)}{N - 1}. \quad (179)$$

O Jackknife mitiga o viés ao explorar a variabilidade entre subamostras, sendo mais robusto que o estimador de Miller, especialmente para distribuições complexas. No entanto, sua computação é intensiva, e a variância pode ser alta em amostras pequenas.

Esses estimadores complementam técnicas de suavização ao abordar o viés diretamente, sem abordar o problema da interdependência de símbolos. Aqui, apresentamos apenas alguns desses métodos. Outros como Combed Jackknife, Grassberger, Chao–Shen podem ser vistos na literatura (Chao e Shen 2003; Grassberger 2008; Pinchas, Ben-Gal e Painsky 2024).

Métrica	Plug-in	Laplace	Miller	Laplace-Miller
Entropia por caractere (bits/char)	4.2467	4.2467	4.2467	4.2468
Entropia por palavra (bits/word)	10.6757	11.2962	10.7332	11.3537
Entropia por caractere por palavra (bits/char)	2.3687	2.5064	2.3815	2.5192
Comprimento esperado de palavra (chars)	4.5069	4.5069	4.5069	4.5069

Tabela 1: Estimativas de Entropia do Inglês com o Texto de *Ulysses*

Entropia Cruzada

A *entropia cruzada* é uma medida frequentemente utilizada para comparar duas distribuições de probabilidade. Ela desempenha um papel importante em aprendizado de máquina. Podemos interpretar a entropia cruzada como a “surpreza” da previsão feita por um modelo em relação à verdadeira distribuição de probabilidades.

O comprimento ótimo de um código para uma variável aleatória X com distribuição p é dada pela entropia $H(X)$, ou seja, $H(p)$. Entretanto, se X for codificada com um código otimizado para uma distribuição q , o comprimento esperado do código será dado pela entropia cruzada de p e q :

$$E_p[\ell] = -E_p[\log q(x)] \quad (180a)$$

$$= - \sum_{x \in \mathcal{X}} p(x) \log q(x). \quad (180b)$$

Definição 13 (Entropia Cruzada). Seja $p = \{p(x)\}_{x \in \mathcal{X}}$ uma distribuição de probabilidade verdadeira e $q = \{q(x)\}_{x \in \mathcal{X}}$ uma distribuição aproximada, onde $\mathcal{X}$ é o alfabeto. A entropia cruzada entre p e q é definida como:

$$H(p, q) = - \sum_{x \in \mathcal{X}} p(x) \log q(x). \quad (181)$$

A entropia cruzada pode ser interpretada como o número esperado de bits necessários para codificar eventos gerados pela distribuição

verdadeira p , utilizando um código otimizado para a distribuição aproximada q . Intuitivamente, ela quantifica o “custo” de usar uma distribuição errada q para modelar p .

Proposição 15 *A entropia cruzada está relacionada à entropia de p e à divergência de Kullback-Leibler (KL) entre p e q . Especificamente:*

$$H(p, q) = H(p) + D_{KL}(p||q), \quad (182)$$

onde $H(p) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$ é a entropia de p e $D_{KL}(p||q) = \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)}$ é a divergência KL entre p e q .

Demonstração. Escrevendo a entropia cruzada:

$$H(p, q) = -\sum_{x \in \mathcal{X}} p(x) \log q(x) \quad (183a)$$

$$= \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{p(x)q(x)} \quad (183b)$$

$$= \sum_{x \in \mathcal{X}} p(x) \log \frac{1}{p(x)} + \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)} \quad (183c)$$

$$= H(p) + D(p||q). \quad (183d)$$

□

A Equação (183) revela que a entropia cruzada é composta pela incerteza intrínseca de p (i.e. $H(p)$) e pela penalidade devido à diferença entre p e q (i.e. $D(p||q)$). Como $D(p||q) \geq 0$, segue que

$$H(p, q) \geq H(p), \quad (184)$$

com igualdade se, e somente se, $p = q$. Nesse caso, quando o modelo q é perfeito, a entropia cruzada reduz-se à entropia de p , pois $D(p||q) = 0$. Assim, usar um modelo imperfeito nunca reduz a surpresa média em relação à distribuição verdadeira.

Outra característica da entropia cruzada é a assimetria: $H(p, q) \neq H(q, p)$. Isso reflete o fato de que os papéis de p (distribuição verdadeira) e q (modelo) não são intercambiáveis. Trocar p e q altera a interpretação, pois estaríamos codificando eventos de q com um código otimizado para p , o que geralmente leva a resultados distintos.

Aprendizado de Máquina

Em aprendizado de máquina, a entropia cruzada é utilizada como uma função de perda em problemas de classificação. Considere um problema de classificação com K classes, onde a distribuição verdadeira p representa a probabilidade real de cada classe (geralmente, uma distribuição degrau, com $p(y_i) = 1$ para a classe correta y_i e 0 para as

demais). Um modelo preditivo, como uma rede neural, produz uma distribuição aproximada $q = \{q(y_i)\}_{i=1}^K$, onde $q(y_i)$ é a probabilidade prevista para a classe y_i .

A função de perda de entropia cruzada, para uma única amostra com classe verdadeira y_i , é dada por:

$$L = -\log q(y_i). \quad (185)$$

Para um conjunto de N amostras, a perda média é:

$$L = -\frac{1}{N} \sum_{i=1}^N \log q(y_i). \quad (186)$$

Essa perda é minimizada quando $q(y_i) \rightarrow 1$ para a classe correta, ou seja, quando o modelo prediz corretamente a distribuição verdadeira. A minimização da entropia cruzada equivale a minimizar a divergência KL entre p e q , já que $H(p)$ é constante para um conjunto de dados fixo.

Exemplo 13. Em uma tarefa de classificação com três classes (“A”, “B”, “C”), suponha que a classe verdadeira seja “A”. A distribuição verdadeira é $P = (1, 0, 0)$. Se o modelo prediz $Q = (0.7, 0.2, 0.1)$, a perda de entropia cruzada é:

$$L = -\log 0.7 \approx 0.5146.$$

Se o modelo melhorar sua predição para $Q = (0.9, 0.05, 0.05)$, a perda diminui para:

$$L = -\log 0.9 \approx 0.1520.$$

A entropia cruzada é particularmente eficaz em aprendizado de máquina porque fornece um gradiente bem definido para otimização, permitindo que algoritmos como gradiente descendente ajustem os parâmetros do modelo de forma eficiente. Além disso, sua interpretação em termos de teoria da informação conecta diretamente o treinamento de modelos à ideia de redução de incerteza sobre a distribuição verdadeira.

Propriedade da Equipartição Assintótica

A *Propriedade da Equipartição Assintótica* (AEP) visa analisar o comportamento de sequências no limite, quando estas sequências tornam-se muito grandes. A AEP mostra que para uma sequência longa de variáveis aleatórias independentes e identicamente distribuídas (i.i.d.), a probabilidade de observar uma *sequência típica* é aproximadamente $2^{-nH(X)}$, onde $H(X)$ é a entropia da fonte e n o comprimento da sequência.

Primeiramente, vamos rever o conceito de estatística de uma amostra.

Definição 14 (Estatística amostral). Seja $x_1, x_2, \dots, x_n$ uma sequência de comprimento n de valores observados de uma amostra de tamanho n , obtidos a partir da realização de variáveis aleatórias $X_1, X_2, \dots, X_n$, uma *estatística* de uma amostra é qualquer função calculada a partir de uma amostra de dados, $T(x_1, x_2, \dots, x_n)$.

Exemplos comuns incluem a média amostral

$$\bar{x} = T(x_1, \dots, x_n) = \frac{1}{n} \sum_{i=1}^n x_i, \quad (187)$$

a variância amostral

$$s^2 = T(x_1, \dots, x_n) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad (188)$$

a mediana amostral, a primeira amostra x_1 , a última amostra x_n , ou até estatísticas como o máximo ou mínimo da amostra, que capturam aspectos específicos dos dados observados. Quando avaliamos T nas variáveis aleatórias $X_1, X_2, \dots, X_n$, o resultado $T(X_1, X_2, \dots, X_n)$ é uma variável aleatória, com sua própria distribuição.

Exemplo 14 (Ensaio de Bernoulli). Considere o experimento de Bernoulli com $X_1, \dots, X_n$ i.i.d., com $X_i \in \{0, 1\}$ e parâmetro $\theta = Pr(X_i = 1)$.

Uma dada sequência qualquer $x_1, \dots, x_n$ terá então probabilidade dada por

$$p(x_1, x_2, \dots, x_n) = \prod_{i=1}^n \theta^{x_i} (1 - \theta)^{1-x_i} = \theta^{\sum_i x_i} (1 - \theta)^{n - \sum_i x_i} \quad (189)$$

Considere agora a seguinte estatística

$$T(x_1, \dots, x_n) = \sum_{i=1}^n x_i, \quad (190)$$

o somatório da amostra, no caso do ensaio de Bernoulli, a quantidade de valores iguais a 1 que apareceu em uma realização. Uma vez que sabemos tal estatísticas, a probabilidade de uma sequência pode ser expressa sem fazer referência à θ (parâmetro que caracteriza a distribuição).

$$p(x_1, \dots, x_n | T(x_1, \dots, x_n), \theta) = p(x_1, \dots, x_n | T(x_1, \dots, x_n)) \quad (191a)$$

$$= \begin{cases} \frac{1}{\binom{n}{k}} & , \sum_i x_i = k \\ 0 & , \text{caso contrário.} \end{cases} \quad (191b)$$

Em outras palavras, $X_{1:N} \perp\!\!\!\perp \theta | T(X_{1:N})$. Isto implica na *cadeia de Markov*¹⁸ $\theta \rightarrow T(X_{1:N}) \rightarrow X_{1:N}$. Aplicando a desigualdade de processamento de dados, obtemos

$$I(\theta; T(X_{1:N})) \geq I(\theta; X_{1:N}). \quad (192)$$

Por outro lado, sabemos que $T(X_{1:N})$ é uma função de $X_{1:N}$. Desta forma, também temos a seguinte cadeia de Markov: $\theta \rightarrow X_{1:N} \rightarrow T(X_{1:N})$. Novamente, aplicando a desigualdade de processamento de dados, obtemos

$$I(\theta; X_{1:N}) \geq I(\theta; T(X_{1:N})). \quad (193)$$

Então, para que 192 e 193 sejam satisfeitos, devemos ter

$$I(\theta; X_{1:N}) = I(\theta; T(X_{1:N})), \quad (194)$$

e nenhuma informação é perdida sobre θ indo de $X_{1:N}$ para $T(X_{1:N})$.

Definição 15 (Estatística Suficiente). Uma função $T(\cdot)$ é dita ser uma *estatística suficiente* em relação à família $\{f_\theta(x)\}$ se X é independente de θ dado $T(X)$ para qualquer distribuição em θ (i.e. $\theta \rightarrow T(X) \rightarrow X$ forma uma cadeia de Markov). Então

$$I(\theta; X) = I(\theta; T(X)), \quad \forall \theta \quad (195)$$

Uma estatística suficiente preserva a informação mútua e reciprocamente

$$X \perp\!\!\!\perp \theta | T(X). \quad (196)$$

Podemos verificar que, no Exemplo 14, a estatísticas utilizada é uma estatísticas suficiente (veja a Equação (191)).

Um critério prático para identificar estatísticas suficientes é dado pelo Teorema da Fatoração de Fisher-Neyman.

¹⁸ Uma cadeia de Markov é um modelo estocástico que descreve uma sequência de eventos em que a probabilidade de cada evento depende apenas do estado imediatamente anterior.

Teorema 19 (Teorema da Fatoração de Fisher-Neyman) *Uma estatística $T(X_1, \dots, X_N)$ é suficiente para θ se, e somente se, a função de verossimilhança da amostra pode ser escrita como:*

$$\mathcal{L}(\theta; x_1, \dots, x_n) = g(T(x_1, \dots, x_n), \theta) \cdot h(x_1, \dots, x_n), \quad (197)$$

onde g é uma função que depende de θ e da estatística T , e h é uma função que não depende de θ .

A demonstração e mais informações sobre o tema podem ser vistos em Berger e Casella (2002).

O histograma empírico da amostra é uma estatística que descreve a distribuição de frequências relativas dos valores observados em uma amostra.

Definição 16 (Histograma Empírico). Para uma amostra $x_1, x_2, \dots, x_n$, obtida a partir de variáveis aleatórias $X_1, X_2, \dots, X_n$, com suporte finito $\mathcal{X} = \{a_1, a_2, \dots, a_D\}$, o *histograma empírico* pode ser definido como a função que associa cada símbolo $a \in \mathcal{X}$ à sua frequência relativa:

$$\hat{p}_n(a) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{x_i=a\}}, \quad (198)$$

onde $\mathbf{1}_{\{x_i=a\}}$ é a função indicadora

$$\mathbf{1}_{\{x_i=a\}} = \begin{cases} 1, & x = a, \\ 0, & x \neq a. \end{cases} \quad (199)$$

Desta forma, $\hat{p}_n(a)$ representa a proporção de vezes que o valor a aparece na amostra.

Equivalentemente, o histograma empírico pode ser representado como a ênupla que contém as frequências relativas de todos os símbolos de $\mathcal{X}$.

$$P_{x_{1:n}} \triangleq \left(\frac{N(a_1|x_{1:n})}{n}, \frac{N(a_2|x_{1:n})}{n}, \dots, \frac{N(a_D|x_{1:n})}{n} \right), \quad (200)$$

onde $N(a_i|x_{1:n})$ é a contagem do número de ocorrências do símbolo a_i na amostra $x_{1:n}$. O histograma é uma estatística, já que é uma função da amostra. Podemos mostrar ainda que o histograma empírico é uma estatística suficiente:

$$p(x_{1:n}|P_{x_{1:n}}, \theta) = \begin{cases} \frac{1}{\binom{n}{n_1, n_2, \dots, n_D}} & , n_i = nP_{x_{1:n}}(a_i), \forall i \\ 0 & , \text{caso contrário} \end{cases} \quad (201a)$$

$$= p(x_{1:n}|P_{x_{1:n}}) \quad (201b)$$

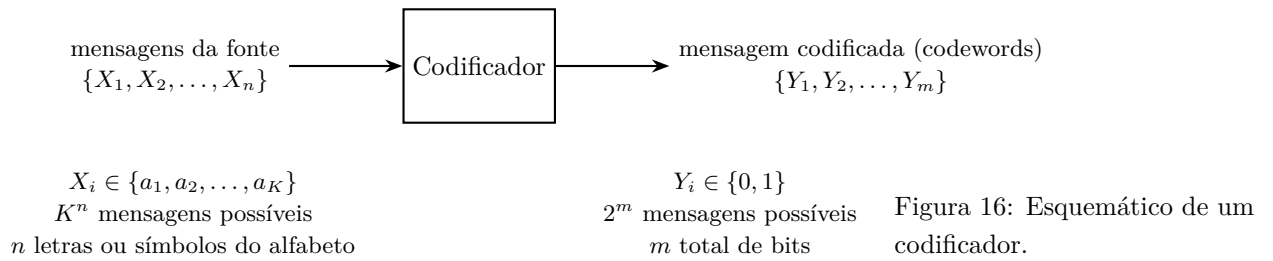
onde temos o coeficiente multinomial¹⁹ $\binom{n}{k_1, k_2, \dots, k_m} = \frac{n!}{k_1! k_2! \dots k_m!}$. Podemos observar que $p(x_{1:n}|P_{x_{1:n}}, \theta) = p(x_{1:n}|P_{x_{1:n}})$, ou seja, a distribuição é independente do modelo, dado o histograma empírico. Então $X_{1:n} \perp\!\!\!\perp \theta | P_{x_{1:n}}$, então $P_{x_{1:n}}$ é uma estatística suficiente.

¹⁹ Teorema Multinomial

$$(x_1 + x_2 + \dots + x_m)^n = \sum_{k_1 + \dots + k_m = n} \binom{n}{k_1, \dots, k_m} \prod_{1 \leq t \leq m} x_t^{k_t} \quad (202)$$

Codificação

O *codificador* é responsável por associar cada sequência $x_1, x_2, \dots, x_n$ produzida pela fonte, ou seja, realizações de variáveis aleatórias $X_1, X_2, \dots, X_n$, a uma sequência de bits de comprimento variável ou fixo, de modo que a sequência original possa ser recuperada perfeitamente por um decodificador. Para simplificar, vamos supor que as mensagens codificadas possuem, todas elas, um mesmo comprimento m . O alfabeto da fonte é $\mathcal{X} = \{a_1, a_2, \dots, a_K\}$, possuindo assim cardinalidade $K = |\mathcal{X}|$. O alfabeto de saída do codificador é $\mathcal{Y} = \{0, 1\}$, ou seja, codificação binária ($|\mathcal{Y}| = 2$). A ?? representa este esquema de codificação.



Para que seja possível termos uma palavra de código para cada mensagem possível, devemos satisfazer a seguinte condição:

$$2^m \geq K^n, \quad (203)$$

ou seja,

$$m \geq (\log K)n \quad (204)$$

A taxa de codificação mede a eficiência de um esquema de codificação ao representar uma sequência de símbolos em forma binária.

Definição 17 (Taxa de Codificação). Para uma sequência de entrada de comprimento n , $X_1, X_2, \dots, X_n$, codificada em uma mensagem binária de comprimento m , $Y_1, Y_2, \dots, Y_m$, a *taxa de codificação* é definida como

$$R = \frac{m}{n}. \quad (205)$$

R representa então o número de bits por símbolo utilizados na tarefa de codificação.

Considerando que a fonte produz uma sequência de símbolos i.i.d., a probabilidade de uma sequência qualquer de comprimento n pode ser expressa por

$$p(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) = \prod_{i=1}^n p(X_i = x_i) \quad (206)$$

A informação sobre um evento é dada por $-\log p(x) = I(x)$, então a informação associada à mensagem $x_1, x_2, \dots, x_n$ é

$$I(x_1, x_2, \dots, x_n) = -\log p(x_1, x_2, \dots, x_n) = -\log \prod_{i=1}^n p(x_i) \quad (207a)$$

$$= \sum_{i=1}^n -\log p(x_i) = \sum_{i=1}^n I(x_i), \quad (207b)$$

onde notamos que eventos independentes são aditivos em relação a esta função de informação. Observe que $EI(X) = H(X)$. A lei fraca dos grandes números²⁰ diz que $\frac{1}{n}S_n \xrightarrow{p} \mu$, onde S_n é a soma de v.a.s i.i.d. com média $\mu = EX_i$. Temos que $I(X_i)$ também é uma v.a. com média $H(X)$. Obteremos assim

$$\frac{1}{n} \sum_{i=1}^n I(X_i) \xrightarrow[n \rightarrow \infty]{p} H(X). \quad (209)$$

Quando n é grande suficiente, podemos escrever

$$\frac{1}{n} \sum_{i=1}^n I(x_i) \approx H(X), \forall i, x_i \sim p(x) \quad (210a)$$

$$-\frac{1}{n} \sum_{i=1}^n \log p(x_i) \approx H(X) \quad (210b)$$

$$-\log \prod_{i=1}^n p(x_i) \approx nH(X) \quad (210c)$$

$$-\log p(x_1, x_2, \dots, x_n) \approx nH(X) \quad (210d)$$

$$p(x_1, x_2, \dots, x_n) \approx 2^{-nH(X)}, \quad (210e)$$

onde em 210b utilizamos a definição de informação sobre um evento; em 210c apenas multiplicamos por n e utilizamos a propriedade do logaritmo; em 210d utilizamos que eventos são i.i.d.; e em 210e, multiplicamos por -1 e tomamos a exponencial. Esta probabilidade final, observada em 210e, não depende da sequência em si. Depende apenas do comprimento n e da entropia da fonte. Quando n fica grande, podemos dizer que todas as sequências terão a mesma probabilidade: 2^{-nH} . Estas sequências que possuem esta probabilidade (praticamente todas as sequências) são chamadas de *sequências típicas*, e são representadas pelo conjunto $A_\epsilon^{(n)}$.

Se todas as sequência de comprimento n possuem aproximadamente a mesma probabilidade $2^{-nH(X)}$, então existe no máximo 2^{nH} sequências de comprimento n . Pode ser que $2^{nH} \ll K^n$, ou seja, o conjunto das sequências típicas é muito menor do que o conjunto de todas as sequências possíveis de comprimento n . Isto implica que, efetivamente, poderemos nos preocupar apenas com a codificação do conjunto menor, o conjunto das sequências que efetivamente ocorrem,

²⁰ Lei dos Grandes Números: Se um evento de probabilidade p é observado repetidamente em ocasiões independentes, a proporção da frequência observada deste evento em relação ao total número de repetições converge em direção a p à medida que o número de repetições se torna arbitrariamente grande.

Sejam $X_1, X_2, \dots, X_n$ v.a.s i.i.d. com $EX_i = \mu$ e $\text{Var}X_i = \sigma^2 < \infty$, para $i = 1, \dots, n$. Seja a média amostral definida por $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$, então, para $\epsilon > 0$, a *Lei Fraca dos Grandes Números* diz que $\bar{X}_n$ converge em probabilidade para μ , ou seja,

$$\lim_{n \rightarrow \infty} \Pr(|\bar{X}_n - \mu| < \epsilon) = 1. \quad (208)$$

as sequências típicas. Desta forma, para representar (ou codificar) as sequências típicas, precisaremos de $nH(X)$ bits. Teremos então

$$m = nH(X), \quad (211)$$

no modelo do codificador. Desta forma, a taxa será $H(X)$.

Teorema 20 (Propriedade da Equipartição Assintótica) *Se $X_1, X_2, \dots, X_n$ são i.i.d. e $X_i \sim p(x)$ para todo i , então*

$$-\frac{1}{n} \log p(X_1, X_2, \dots, X_n) \xrightarrow{p} H(X). \quad (212)$$

Demonstração.

$$-\frac{1}{n} \log p(X_1, X_2, \dots, X_n) = -\frac{1}{n} \log \prod_{i=1}^n p(X_i) \quad (213a)$$

$$= -\frac{1}{n} \sum_i \log p(X_i) \xrightarrow{p} E \log p(X) \quad (213b)$$

$$= H(X), \quad (213c)$$

onde utilizamos a lei fraca dos números grandes em 213b. □

Definição 18 (Conjunto Típico). Um *conjunto típico* $A_\epsilon^{(n)}$ em relação a $p(x)$ é o conjunto de sequências $(x_1, x_2, \dots, x_n) \in \mathcal{X}^n$ com propriedade

$$2^{-n(H(X)+\epsilon)} \leq p(x_1, x_2, \dots, x_n) \leq 2^{-n(H(X)-\epsilon)}. \quad (214)$$

De forma equivalente, podemos escrever

$$A_\epsilon^{(n)} = \left\{ (x_1, x_2, \dots, x_n) : \left| -\frac{1}{n} \log p(x_1, \dots, x_n) - H \right| < \epsilon \right\}. \quad (215)$$

Teorema 21 (Propriedades do Conjunto Típico $A_\epsilon^{(n)}$)

1. Se $(x_1, x_2, \dots, x_n) \in A_\epsilon^{(n)}$, então

$$H(X) - \epsilon \leq -\frac{1}{n} \log p(x_1, x_2, \dots, x_n) \leq H(X) + \epsilon. \quad (216)$$

2. $p(A_\epsilon^{(n)}) = p\left(\left\{x : x \in A_\epsilon^{(n)}\right\}\right) > 1 - \epsilon$ para n grande suficiente, para todo $\epsilon > 0$.

3. Limite superior: $|A_\epsilon^{(n)}| \leq 2^{n(H(X)+\epsilon)}$, onde $|A_\epsilon^{(n)}|$ é o número de elementos no conjunto $A_\epsilon^{(n)}$.

4. Limite inferior: $|A_\epsilon^{(n)}| \geq (1 - \epsilon)2^{n(H(X)-\epsilon)}$ para n grande suficiente.

Demonstração.

1. O primeiro apenas é uma reformulação da definição de AEP.
2. Vamos utilizar a definição expandida de convergência em probabilidade, dada na equação 212.

$$p(A_\epsilon^{(n)}) = p\left(\left| -\frac{1}{n} \sum_i \log p(x_i) - H \right| < \epsilon\right) > 1 - \delta, \quad (217)$$

para n grande suficiente. Podemos escolher qualquer δ . Escolhemos então $\delta = \epsilon$, resultando em

$$p(A_\epsilon^{(n)}) > 1 - \epsilon, \quad \text{para } n \text{ grande suficiente, } \forall \epsilon. \quad (218)$$

3. Limite superior de $A_\epsilon^{(n)}$:

$$1 = \sum_x p(x) \geq \sum_{x \in A_\epsilon^{(n)}} p(x) \geq \sum_{x \in A_\epsilon^{(n)}} 2^{-n(H(X)+\epsilon)} \quad (219a)$$

$$= |A_\epsilon^{(n)}| 2^{-n(H(X)+\epsilon)}, \quad (219b)$$

resultando em $|A_\epsilon^{(n)}| \leq 2^{n(H+\epsilon)}$.

4. Limite inferior do tamanho de $A_\epsilon^{(n)}$. Para n grande suficiente:

$$1 - \epsilon < p(A_\epsilon^{(n)}) \leq \sum_{x \in A_\epsilon^{(n)}} 2^{-n(H(X)-\epsilon)} \quad (220a)$$

$$= 2^{-n(H(X)-\epsilon)} |A_\epsilon^{(n)}|, \quad (220b)$$

resultando em $|A_\epsilon^{(n)}| \geq (1 - \epsilon) 2^{n(H(X)-\epsilon)}$.

□

A AEP e o tamanho do conjunto típico têm implicações diretas na codificação de sequências, como no exemplo de imagens digitais. Considere uma imagem full HD de 1080×720 pixels, com 16 milhões de cores (24 bits por pixel, ou seja, 2^{24} cores possíveis). O número total de pixels é $1080 \times 720 = 777.600$, e cada pixel é representado por 24 bits, totalizando $K = 1080 \times 720 \times 24 = 18.662.400 \approx 10^7$ bits por imagem. O número total de imagens possíveis é $2^K = 2^{18.662.400} \approx 10^{5.617 \times 10^6}$, um valor extremamente grande, muito maior que o número de átomos no universo observável ($\approx 10^{81}$). Pela AEP, se os pixels fossem i.i.d. com entropia $H(X)$ (em bits por pixel), o número de imagens típicas seria aproximadamente $2^{nH(X)}$, onde $n = 777.600$. Mesmo que $H(X)$ fosse pequeno (por exemplo, 1 bit por pixel devido a redundâncias), o número de imagens típicas seria $2^{777.600} \approx 10^{234.000}$, ainda muito maior que 10^{81} . Isso ilustra que, embora o número de imagens típicas seja uma fração minúscula do total, ele ainda é astronomicamente

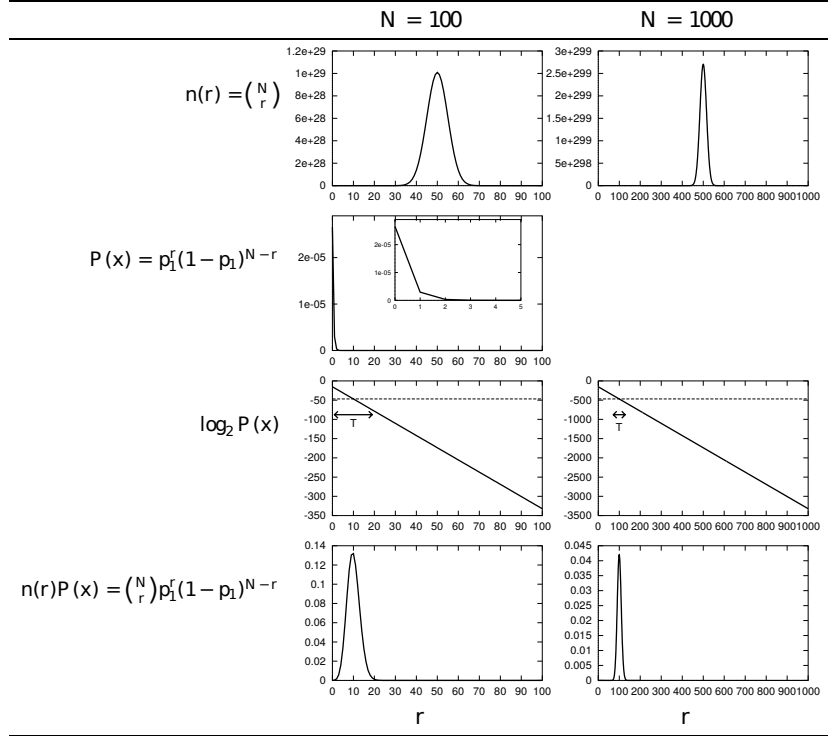


Figura 18: Considerando $p = 0,1$, $n = 100$ e $n = 1000$ os gráficos ilustram $n(r)$, o número de strings contendo r 1's; a probabilidade $Pr(x_{1:n})$ para uma sequência contendo r 1's; a mesma probabilidade em escala logarítmica; e a probabilidade total $n(r)Pr(x_{1:n})$ de todas as sequências contendo r 1s (MacKay 2003).

1's. Verificamos ainda que este pico torna-se mais acentuado com o crescimento de n . No limite, quando n for muito grande, só ocorrerão sequências com 10% de 1's.

Codificação para o Conjunto Típico

Após definirmos o que são sequências típicas e formalizarmos a propriedade da equipartição assintótica, vamos agora abordar o problema de codificação considerando a codificação de sequências típicas. Como vimos que existe um limite para o tamanho do conjunto típicos, iremos utilizar o número de bits mínimos necessários para codificar as sequências que pertençam a este conjunto. As sequências remanescentes, contidas no conjunto não típico, podem ser codificadas utilizando mais bits, sem que isto cause impacto significativo ao código, uma vez que a probabilidade do conjunto típico é aproximadamente 1 (conforme item 2 do Teorema 21).

A ideia consiste em particionar o conjunto de sequências em dois blocos: conjunto típico $A_\epsilon^{(n)}$, e o seu complemento, o conjunto não típico $A_\epsilon^{(n)c} \triangleq \mathcal{X}^n \setminus A_\epsilon^{(n)}$. Tais partições satisfazem $A_\epsilon^{(n)} \cap A_\epsilon^{(n)c} = \emptyset$, e $A_\epsilon^{(n)} \cup A_\epsilon^{(n)c} = \mathcal{X}^n$. A Figura 19 ilustra o particionamento proposto.

Vamos então indexar os elementos de cada conjunto (conjunto tí-

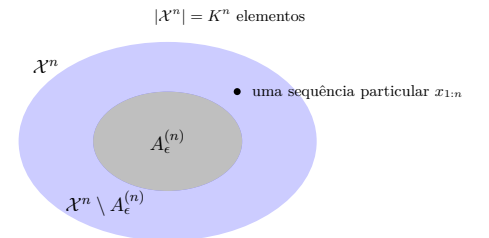


Figura 19: Particionamento de $\mathcal{X}^n$ em dois conjuntos: $A_\epsilon^{(n)}$ e $A_\epsilon^{(n)c}$.

pico e não-típico) separadamente. O número de elementos do conjunto típico é $|A_\epsilon^{(n)}| \leq 2^{n(H+\epsilon)}$, desta forma serão necessários

$$\lceil n(H + \epsilon) \rceil \leq n(H + \epsilon) + 1 \text{bits} \quad (221)$$

para indexar os elementos deste conjunto. Ainda, utilizaremos um bit extra para indicar se o elemento está no conjunto típico ou não, i.e., vamos utilizar uma sequência $(b_0, b_1, b_2, \dots, b_{\lceil n(H+\epsilon) \rceil})$ onde o primeiro bit indica se temos um elemento do conjunto típico ($b_0 = 0$) ou não ($b_0 = 1$), e os demais indexam o elemento do conjunto. O número total de bits necessário para uma sequência típica será então $\leq n(H + \epsilon) + 2$.

Para os elementos do conjunto não típico, vamos utilizar

$$\lceil \log |\mathcal{X}|^n \rceil \leq n \log K + 1 \text{bits}. \quad (222)$$

Teremos então um vetor binário da forma $(b_0, b_1, b_2, \dots, b_{\lceil \log |\mathcal{X}|^n \rceil})$ onde $b_0 = 1$, indicando a atipicidade. Assim, o número total de bits para indexar uma sequência atípica será $\leq n \log K + 2$.

Este código proposto é 1-pra-1, sendo fácil codificar e decodificar, dado o *codebook*. Para o conjunto não-típico $A_\epsilon^{(n)c}$, estamos utilizando mais bits do que o necessário, já que $|A_\epsilon^{(n)c}| = |\mathcal{X}^n| - |A_\epsilon^{(n)}| = K^n - |A_\epsilon^{(n)}| \leq K^n$. Mas isto não importará, como veremos adiante. O importante desta proposta de código é que as sequências típicas possuem um comprimento descritivo curto, aproximadamente nH .

Definição 19 (Comprimento da palavra associada a uma sequência). O *comprimento da palavra (codeword)* associada à sequência $x_{1:n}$ é denominado $l(x_{1:n})$.

Assim, $l(X_{1:n})$ é uma variável aleatória, já que $X_{1:n}$ é uma variável aleatória. Então $El(X_{1:n}) = \sum_{x_{1:n}} p(x_{1:n})l(x_{1:n})$ é o valor esperado do comprimento do código. Queremos que ele seja o menor possível.

Suponha que n seja grande suficiente de forma que $p(A_\epsilon^{(n)}) > 1 - \epsilon$, então

$$El(X_{1:n}) = \sum_{x_{1:n}} p(x_{1:n})l(x_{1:n}) \quad (223a)$$

$$= \sum_{x_{1:n} \in A_\epsilon^{(n)}} p(x_{1:n})l(x_{1:n}) + \sum_{x_{1:n} \in A_\epsilon^{(n)c}} p(x_{1:n})l(x_{1:n}) \quad (223b)$$

$$\leq \sum_{x_{1:n} \in A_\epsilon^{(n)}} p(x_{1:n})[n(H + \epsilon) + 2] + \sum_{x_{1:n} \in A_\epsilon^{(n)c}} p(x_{1:n})[n \log K + 2] \quad (223c)$$

$$= \underbrace{p(A_\epsilon^{(n)})}_{\leq 1} [n(H + \epsilon) + 2] + \underbrace{p(A_\epsilon^{(n)c)})}_{< \epsilon} [n \log K + 2] \quad (223d)$$

$$\leq n(H + \epsilon) + 2 + \epsilon n \log K + \epsilon 2 \quad (223e)$$

$$= n[H + \underbrace{\epsilon + \epsilon \log K + \frac{2}{n} + \frac{2\epsilon}{n}}_{\epsilon'}] = n(H + \epsilon'), \quad (223f)$$

onde definimos ϵ' como

$$\epsilon' = \epsilon + \epsilon \log K + \frac{2}{n} + \frac{2\epsilon}{n}. \quad (224)$$

Podemos fazer ϵ' tão pequeno quanto queremos, fazendo ϵ pequeno e n grande. Desta forma, podemos fazer $n(H + \epsilon')$ tão próximo quanto quisermos de nH .

Teorema 22 (Primeiro Teorema de Shannon (Teorema da Codificação de Fonte)) *Seja $X_{1:n}$ i.i.d. $\sim p(x)$, $\epsilon > 0$, então $\exists$ um código $f_n : \mathcal{X}^n \rightarrow$ string binária e um inteiro n_ϵ , tal que o mapeamento seja um-para-um (desta forma inversível sem erro), e*

$$E\left[\frac{1}{n}l(X_{1:n})\right] \leq H(X) + \epsilon \quad (225)$$

para todo $\epsilon > 0$ e todo $n \geq n_\epsilon$.

Demonstração. A demonstração do Primeiro Teorema de Shannon usa a codificação de sequências típicas de maneira semelhante ao exemplo anterior. □

A demonstração do Primeiro Teorema de Shannon utiliza sequências típicas para garantir a existência de códigos cujo comprimento médio se aproxima, no limite, da entropia da fonte. Contudo, a codificação de sequências típicas requer sequências arbitrariamente longas, o que, na prática, torna a implementação inviável. Assim, embora o teorema não indique um método para construir códigos práticos, ele assegura a possibilidade teórica de atingir o limite definido pela entropia.

É interessante notar que a sequência mais provável não pertence ao conjunto típico $A_\epsilon^{(n)}$. Isso ocorre porque o conjunto típico contém sequências cujo histograma empírico é próximo da distribuição subjacente. A tipicidade não está relacionada a uma única sequência ter a maior probabilidade.

Dado que o conjunto típico é tal que $Pr(A_\epsilon^{(n)}) \rightarrow 1$ à medida que $n \rightarrow \infty$, ou seja, $A_\epsilon^{(n)}$ captura toda probabilidade, podemos nos questionar se existe um conjunto ainda menor que também contenha essencialmente ‘toda’ a probabilidade.

Definição 20 (Sequências assintoticamente iguais até a primeira ordem do expoente). A notação $a_n \doteq b_n$ indica que a_n e b_n são iguais até a primeira ordem do expoente, ou seja, suas taxas de crescimento exponencial são as mesmas à medida que $n \rightarrow \infty$. Formalmente, isso

significa que $\lim_{n \rightarrow \infty} \frac{1}{n} |\log a_n - \log b_n| = 0$, de modo que $\frac{1}{n} \log a_n \rightarrow \alpha$ e $\frac{1}{n} \log b_n \rightarrow \alpha$ para o mesmo α . Para n grande, a_n e b_n possuem aproximadamente o mesmo comportamento.

Teorema 23 *Seja $X_{1:n}$ uma sequência i.i.d. $\sim p(x)$. Para $\delta < 1/2$ e qualquer $\delta' > 0$, se $P(B_\delta^{(n)}) > 1 - \delta$, então*

$$\frac{1}{n} \log |B_\delta^{(n)}| > H - \delta', \quad (226)$$

se n é grande suficiente. Teremos assim

$$|B_\delta^{(n)}| > 2^{n(H-\delta')} \approx 2^{nH} \quad (227)$$

Usando a Definição 20, podemos então reescrever o teorema anterior da seguinte forma:

Se $\delta_n \rightarrow 0$ e $\epsilon_n \rightarrow 0$, então teremos

$$|B_{\delta_n}^{(n)}| \doteq |A_{\epsilon_n}^{(n)}| \doteq 2^{nH}. \quad (228)$$

Demonstração. Seja $X_1, X_2, \dots, X_n$ i.i.d. $\sim p(x)$. Seja $B_\delta^{(n)} \subset \mathcal{X}^n$ tal que $\Pr(B_\delta^{(n)}) > 1 - \delta$. Fixe $\epsilon < 1/2$. Dados dois subconjuntos quaisquer A e B tais que $\Pr(A) > 1 - \epsilon_1$ e $\Pr(B) > 1 - \epsilon_2$. Seja A^c , o complemento de A , e B^c , o complemento de B , então

$$\Pr(A^c \cup B^c) \leq \Pr(A^c) + \Pr(B^c). \quad (229)$$

Como $\Pr(A) > 1 - \epsilon_1$, teremos $\Pr(A^c) \leq \epsilon_1$. De forma similar, $\Pr(B^c) \leq \epsilon_2$. Poderemos assim escrever

$$\Pr(A \cap B) = 1 - \Pr(A^c \cup B^c) \quad (230a)$$

$$\geq 1 - \Pr(A^c) - \Pr(B^c) \quad (230b)$$

$$\geq 1 - \epsilon_1 - \epsilon_2. \quad (230c)$$

Assim, a desigualdade anterior poderá ser reescrita como:

$$\Pr(A_\epsilon^{(n)} \cap B_\delta^{(n)}) \geq 1 - \epsilon - \delta. \quad (231)$$

A probabilidade de um conjunto é dada pela soma das probabilidades de todos os elementos (sequências) neste conjunto, logo teremos

$$\Pr(A_\epsilon^{(n)} \cap B_\delta^{(n)}) = \sum_{x^n \in A_\epsilon^{(n)} \cap B_\delta^{(n)}} p(x^n) \quad (232)$$

A probabilidade dos elementos no conjunto típico é limitada por $2^{-n(H-\epsilon)}$.

Desta forma teremos

$$1 - \epsilon - \delta \leq \Pr(A_\epsilon^{(n)} \cap B_\delta^{(n)}) \quad (233a)$$

$$= \sum_{x^n \in A_\epsilon^{(n)} \cap B_\delta^{(n)}} p(x^n) \quad (233b)$$

$$\leq \sum_{x^n \in A_\epsilon^{(n)} \cap B_\delta^{(n)}} 2^{-n(H-\epsilon)} \quad (233c)$$

$$= |A_\epsilon^{(n)} \cap B_\delta^{(n)}| 2^{-n(H-\epsilon)} \quad (233d)$$

$$\leq |B_\delta^{(n)}| 2^{-n(H-\epsilon)}, \quad (233e)$$

onde utilizamos $A_\epsilon^{(n)} \cap B_\delta^{(n)} \subseteq B_\delta^{(n)}$.

A desigualdade anterior poderá ser reescrita então seguinte forma:

$$|B_\delta^{(n)}| > 2^{n(H-\epsilon)}, \quad (234)$$

onde $\epsilon > 0$.

□

Método de Tipos

O *método de tipos* refina a abordagem das sequências típicas, oferecendo uma análise mais estruturada das propriedades assintóticas. O método de tipos considera todas as possíveis distribuições empíricas (ou ‘tipos’) que sequências $x_{1:n}$ podem ter, agrupando-as em conjuntos de sequências que compartilham um mesmo histograma empírico $\hat{p}_n(x_{1:n})$, ou $P_{x_{1:n}}$. O conjunto de sequências de comprimento n pode ser particionado em subconjuntos de sequências de tipos distintos. Na AEP, o método de tipos confirma que o número de sequências típicas é $\doteq 2^{nH(X)}$. Vamos então estabelecer algumas definições.

Definição 21 (Tipo). Seja $x_1, x_2, \dots, x_n \equiv x_{1:n}$ uma amostra de comprimento n de uma variável aleatória discreta D -ária. Então $x_i \in \mathcal{X}$ e o tamanho do alfabeto é $D = |\mathcal{X}|$, sendo o alfabeto $\mathcal{X} = \{a_1, a_2, \dots, a_D\}$. Definimos a seguinte estatística, o *histograma empírico* da amostras, também chamado *tipo* da amostra:

$$P_{x_{1:n}} \triangleq \left(\frac{n(a_1|x_{1:n})}{n}, \frac{n(a_2|x_{1:n})}{n}, \dots, \frac{n(a_D|x_{1:n})}{n} \right) \quad (235)$$

onde $n(a_i|x_{1:n})$ representa o número de ocorrências do símbolo a_i na amostra $x_{1:n}$.

Note que $P_{x_{1:n}}$ pode ser considerado como uma função massa probabilística. Um tipo $\hat{p}$ é uma função de massa de probabilidade sobre $\mathcal{X}$, definido como o histograma empírico $\hat{p}(a) = k_a/n$, onde $k_a = \sum_{i=1}^n \mathbf{1}_{\{x_i=a\}}$ é o número de ocorrências do símbolo $a \in \mathcal{X}$ na sequência, e $\sum_{a \in \mathcal{X}} k_a = n$.

Definição 22 (Conjunto de Tipos). O conjunto de tipos $\mathcal{P}_n$, ou $\mathcal{P}_n(\mathcal{X})$, para sequências de comprimento n sobre um alfabeto finito $\mathcal{X}$ é o conjunto de todas as possíveis distribuições empíricas (ou tipos) que podem ser formadas por sequências $(x_1, x_2, \dots, x_n) \in \mathcal{X}^n$.

Exemplo 15 (Ensaio de Bernoulli). Para sequências de comprimento n geradas a partir de um ensaio de Bernoulli com alfabeto $\mathcal{X} = \{0, 1\}$, teremos o seguinte conjunto de tipos:

$$\mathcal{P}_n(\mathcal{X}) = \left\{ \left(\frac{0}{n}, \frac{n}{n} \right), \left(\frac{1}{n}, \frac{n-1}{n} \right), \dots, \left(\frac{n}{n}, \frac{0}{n} \right) \right\} \quad (236)$$

onde existem $n + 1$ tipos (histogramas empíricos). O primeiro tipo, na Equação (236), representa a sequência sem nenhuma ocorrência de zeros e n ocorrências de uns; o segundo tipo representa as sequências com apenas uma ocorrência de zero e $n - 1$ ocorrência de uns; e o último tipo representa a sequência com n ocorrência de zeros e nenhuma ocorrência de uns.

Definição 23 (Classe de Tipo). A classe de tipo $T(P)$, ou $T(\hat{p})$, associada a um tipo $\hat{p} \in \mathcal{P}_n(\mathcal{X})$, para sequências de comprimento n sobre um alfabeto finito $\mathcal{X}$, é o conjunto de todas as sequências $(x_1, x_2, \dots, x_n) \in \mathcal{X}^n$ que compartilham um mesmo histograma empírico $\hat{p}$, ou P .

$$T(P) \triangleq \{x_{1:n} \in \mathcal{X}^n : P_{x_{1:n}} = P\}. \quad (237)$$

Exemplo 16 (Ensaio de Bernoulli). Para o caso do ensaio de Bernoulli, vejamos alguns exemplos de classe de tipo.

Para o tipo $P = \left(\frac{0}{n}, \frac{n}{n} \right)$, a classe de tipo $T(P)$ possui apenas uma sequência:

$$T(P) = \{111 \dots 11\}. \quad (238)$$

Para o tipo $P = \left(\frac{1}{n}, \frac{n-1}{n} \right)$, a classe de tipo $T(P)$ possui n sequências, todas com apenas uma ocorrência de zero:

$$T(P) = \{011 \dots 11, 101 \dots 11, \dots, 111 \dots 01, 111 \dots 10\}. \quad (239)$$

Para o tipo $P = \left(\frac{n}{n}, \frac{0}{n} \right)$, a classe de tipo $T(P)$ possui apenas uma sequência:

$$T(P) = \{000 \dots 00\}. \quad (240)$$

Como mencionado anteriormente, o conjunto de todas as sequências $\mathcal{X}^n$ pode ser particionado em diferentes conjuntos $T(P_i)$, com $P_i \in \mathcal{P}_n$, o conjunto de todos os tipos, ou seja, $\mathcal{P}_n = \{P_1, P_2, \dots, P_{|\mathcal{P}_n|}\}$. As partições são disjuntas,

$$T(P_i) \cap T(P_j) = \emptyset, \forall i, j \in \{1, \dots, |\mathcal{P}_n|\} \text{ e } i \neq j, \quad (241)$$

e o particionamento é exaustivo

$$\bigcup_{P \in \mathcal{P}_n} T(P) = \mathcal{X}^n. \quad (242)$$

Este particionamento é ilustrado na Figura 20. O conjunto de todas as seqüências de comprimento n , $\mathcal{X}^n$, é particionado em $|\mathcal{P}_n|$ classes de tipo, para cada um dos tipos existentes. Em particular, a Figura 20, apenas a título de ilustração, evidencia 3 classes de tipo: $T(P_1)$, $T(P_2)$ e $T(P_3)$, e uma seqüência qualquer $x_{1:n}$ em uma determinada classe de tipo.

No exemplo do ensaio de Bernoulli é fácil constatar quantos tipos distintos existem. O Teorema 25 a seguir usa o método estrela traço da análise combinatória para determinar o número de tipos existentes. Vejamos então primeiramente o teorema da análise combinatória.

Teorema 24 (Método Bola-Traço(*stars and bars*)) *Suponha n estrelas que devam ser organizadas em k recipientes, podendo inclusive ter recipiente vazio. Neste caso, o número de maneiras de fazer tal organização é dado por*

$$\binom{n+k-1}{k-1}. \quad (243)$$

Demonstração. Este problema é o mesmo que incluir $k-1$ traços para separar uma seqüência de n estrelas. Os traços podem ser inseridos em qualquer posição, inclusive sem existir estrelas entre eles, como por exemplo, com $n=7$ e $k=4$:

$$★★★★ \mid \mid \star \mid \star\star$$

o que seria equivalente a termos o primeiro recipiente com 4 estrelas, o segundo vazio, o terceiro com apenas uma estrela, e o último com duas estrelas. Note que, neste problema, temos $n+k-1$ símbolos (estrelas e traços) para serem incluídos, dos quais $k-1$ são escolhidos para serem traços. Desta forma existem $\binom{n+k-1}{k-1}$ formas de dispor estes símbolos.

□

Teorema 25 (Número de tipos existentes) *Para seqüências de comprimento n , em um alfabeto $\mathcal{X}$, o número de tipos é dado por*

$$|\mathcal{P}_n| = \binom{n+|\mathcal{X}|-1}{|\mathcal{X}|-1}. \quad (244)$$

Demonstração. Um histograma empírico pode ser visto como uma variação do problema estrela-traço. Se temos um alfabeto com $|\mathcal{X}| = D$ símbolos, o Conjunto de Tipos $\mathcal{P}_n$ contém todas as possíveis distribuições empíricas $\hat{p}$ para seqüências de comprimento n sobre o alfabeto $\mathcal{X}$. Um tipo $\hat{p}$ é uma função de massa de probabilidade tal que $\hat{p}(a) = k_a/n$, onde k_a é o número de ocorrências do símbolo $a \in \mathcal{X}$, e $\sum_{a \in \mathcal{X}} k_a = n$. Assim, $k_a \in \{0, 1, \dots, n\}$, e a restrição é que a soma de todos k_a 's deve ser igual a n , o comprimento da seqüência. Cada tipo $\hat{p}$ é definido pela ênupla $(k_{a_1}, k_{a_2}, \dots, k_{a_D})$, e cada uma delas corresponde a uma forma de organização no problema de estrelas e traços.

□

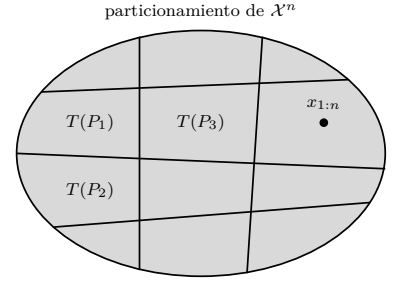


Figura 20: Particionamento do espaço de seqüências de comprimento n .

Embora tenhamos uma fórmula fechada para o número de tipos, dada pela Equação (244), iremos encontrar um limite superior para $|\mathcal{P}_n|$, que, embora seja um limite relativamente largo, é mais fácil de manipular e suficiente para muitas demonstrações teóricas.

Teorema 26 (Limite no número de tipos) *O número de tipos para sequências de comprimento n em um alfabeto $\mathcal{X}$ é limitado por*

$$|\mathcal{P}_n| \leq (n+1)^{|\mathcal{X}|}. \quad (245)$$

Demonstração. O numerador de cada entrada em um tipo pode assumir $(n+1)$ valores distintos (de 0 a n). Existem $|\mathcal{X}|$ entradas em um tipo, e portanto a mesma quantidade de numeradores. Os valores dos numeradores interagem entre si (a soma de todos deve ser igual a n), mas podemos achar um limite superior desconsiderando esta interação.

$$|\mathcal{P}_n| \leq \underbrace{(n+1) \times (n+1) \times \dots \times (n+1)}_{|\mathcal{X}| \text{ vezes}} = (n+1)^{|\mathcal{X}|}. \quad (246)$$

□

É importante notar na Equação (245) que existe no máximo um número polinomial em n de tipos de sequências de comprimento n . Entretanto, sabemos que o número de sequências cresce com n , $|\mathcal{X}|^n$. Com n grande suficiente, eventualmente, teremos que apenas um único tipo (o tipo das sequências típicas) conterà todas as sequências, como será demonstrado no Teorema 33.

Para sequências geradas por uma fonte i.i.d., a probabilidade de uma sequência $(x_1, x_2, \dots, x_n)$ depende apenas do seu tipo e não da sequência específica. Isso significa que todas as sequências em uma mesma classe de tipo, ou seja, sequências que compartilham um mesmo histograma empírico, têm a mesma probabilidade.

Teorema 27 (Probabilidade depende do tipo) *Seja $X_1, X_2, \dots, X_n$ i.i.d. $\sim Q(x)$, com Q arbitrário, e extensão $Q^n(x_{1:n}) = \prod_i Q(x_i)$, a probabilidade da sequência depende apenas do tipo, ou seja, a probabilidade é ‘independente’ da sequência, dado o tipo e Q , isto é,*

$$Q^n(x_{1:n}) = 2^{-n[H(P_{x_{1:n}}) + D(P_{x_{1:n}} || Q)]}. \quad (247)$$

Demonstração.

$$Q^n(x_{1:n}) = \prod_{i=1}^n Q(x_i) = \prod_{a \in \mathcal{X}} Q(a)^{n(a|x_{1:n})} \quad (248a)$$

$$= \prod_{a \in \mathcal{X}} Q(a)^{nP_{x_{1:n}}(a)} = \prod_{a \in \mathcal{X}} 2^{\{nP_{x_{1:n}}(a) \log Q(a)\}} \quad (248b)$$

$$= \prod_{a \in \mathcal{X}} 2^{\left\{ n \left(P_{x_{1:n}}(a) \log Q(a) - \underbrace{P_{x_{1:n}}(a) \log P_{x_{1:n}}(a) + P_{x_{1:n}}(a) \log P_{x_{1:n}}(a)}_{=0} \right) \right\}} \quad (248c)$$

$$= 2^{n \sum_{a \in \mathcal{X}} \left(-P_{x_{1:n}}(a) \log \frac{P_{x_{1:n}}(a)}{Q(a)} + P_{x_{1:n}}(a) \log P_{x_{1:n}}(a) \right)} \quad (248d)$$

$$= 2^{-n(D(P_{x_{1:n}} \| Q) + H(P_{x_{1:n}}))}. \quad (248e)$$

□

Se Q é uma distribuição racional (i.e., um tipo possível) e se $x_{1:n} \in T(Q)$, então

$$Q^n(x_{1:n}) = 2^{-nH(Q)}. \quad (249)$$

Se Q for irracional, podemos fazer $D(P_{x_{1:n}} \| Q)$ tão pequeno quando desejável, fazendo n grande suficiente.

No método de tipos, uma questão natural é identificar qual classe de tipo tem a maior probabilidade total quando as sequências são geradas por uma determinada distribuição. Intuitivamente, esperamos que a classe de tipo mais provável seja aquela cujo histograma empírico esteja mais próximo da distribuição geradora.

Lema 28 (Classe de tipo com maior probabilidade) *Dada a distribuição geradora $Q = P \in \mathcal{P}_n$, teremos que $T(P)$ possui a maior probabilidade. Isto é*

$$P^n(T(P)) \geq P^n(T(\hat{P})), \quad \forall \hat{P} \in \mathcal{P}_n. \quad (250)$$

Demonstração.

$$\frac{P^n(T(P))}{P^n(T(\hat{P}))} = \frac{|T(P)| \prod_{a \in \mathcal{X}} P(a)^{nP(a)}}{|T(\hat{P})| \prod_{a \in \mathcal{X}} P(a)^{n\hat{P}(a)}} \quad (251a)$$

$$= \frac{\binom{n}{nP(a_1) \ nP(a_2) \ \dots \ nP(a_D)}}{\binom{n}{n\hat{P}(a_1) \ n\hat{P}(a_2) \ \dots \ n\hat{P}(a_D)}} \prod_{a \in \mathcal{X}} P(a)^{nP(a)} \quad (251b)$$

$$= \prod_{a \in \mathcal{X}} \frac{[n\hat{P}(a)]!}{[nP(a)]!} P(a)^{n(P(a) - \hat{P}(a))} \quad (251c)$$

$$\geq \prod_{a \in \mathcal{X}} (nP(a))^{n(\hat{P}(a) - P(a))} P(a)^{n(P(a) - \hat{P}(a))} \quad (251d)$$

$$= \prod_{a \in \mathcal{X}} n^{n(\hat{P}(a) - P(a))} \quad (251e)$$

$$\geq \prod_{a \in \mathcal{X}} n^{n(\hat{P}(a) - P(a))} \quad (251f)$$

$$= n^{n[\sum_{a \in \mathcal{X}} \hat{P}(a) - \sum_{a \in \mathcal{X}} P(a)]} \quad (251g)$$

$$= n^{n(1-1)} = 1, \quad (251h)$$

onde em 251d utilizamos que $\frac{m!}{n!} \geq n^{m-n}$, para m e n inteiros não negativos²¹. Por fim, temos que $P^n(T(P)) \geq P^n(T(\hat{P}))$. □

O tamanho de uma classe de tipo $T(P)$ pode ser determinado pelos coeficientes multinomiais. Cada sequência $x_{1:n} \in T(P)$ possui exatamente $nP(a)$ ocorrências de cada símbolo $a \in \mathcal{X}$. O número de sequências corresponde ao número de maneiras de permutar os símbolos de acordo com o número de ocorrência de cada símbolo.

Lema 29 (Tamanho da classe de tipo) *Seja $P \in \mathcal{P}_n$, um tipo para sequências de comprimento n sobre um alfabeto finito $\mathcal{X}$, de tamanho $D = |\mathcal{X}|$, e seja $T(P)$ a classe de tipo associada, ou seja, o conjunto de todas as sequências $x_{1:n} \in \mathcal{X}^n$ com histograma empírico P . Então o tamanho de $T(P)$ é dado por*

$$|T(P)| = \binom{n}{nP(a_1) \ nP(a_2) \ \dots \ nP(a_D)} = \frac{n!}{\prod_{a \in \mathcal{X}} (nP(a))!}. \quad (252)$$

Demonstração. Cada sequência em $T(P)$ tem exatamente $nP(a)$ ocorrências de cada símbolo $a \in \mathcal{X}$, com $\sum_{a \in \mathcal{X}} (nP(a)) = n$. O número de sequências distintas é o número de maneiras de permutar os n símbolos, considerando que as permutações dentro de cada símbolo não geram sequências distintas. Assim, o número de permutações distintas é o coeficiente multinomial, como dado na Equação (252). □

Novamente, apesar de termos o resultado exato, dado pela Equação (252), em muitas situações é mais prático utilizarmos limites que

²¹ Sejam m e n inteiros não negativos, então $\frac{m!}{n!} \geq n^{m-n}$. Se $m > n$, então $\frac{m!}{n!} = m(m-1) \dots (n+1) \geq n^{m-n}$. Se $m < n$, então $\frac{m!}{n!} = \frac{1}{n(n-1) \dots (m+1)} \geq \frac{1}{n^{n-m}}$. Se $m = n$, $\frac{m!}{n!} = 1 = n^0$.

nos forneçam uma notação mais intuitiva e prática para utilização em outras demonstrações. Veremos então a seguir os limites superior e inferior para o tamanho da classe de tipo.

Teorema 30 (Limites no tamanho da classe de tipo) *Dado um tipo $P \in \mathcal{P}_n$, temos*

$$\frac{1}{(n+1)^{|\mathcal{X}|}} 2^{nH(P)} \leq |T(P)| \leq 2^{nH(P)}. \quad (253)$$

Demonstração (limite superior).

$$1 \geq P^n(T(P)) \quad (254a)$$

$$= \sum_{x_{1:n} \in T(P)} P^n(x_{1:n}) \quad (254b)$$

$$= \sum_{x_{1:n} \in T(P)} 2^{-nH(P)} \quad (254c)$$

$$= |T(P)| 2^{-nH(P)}, \quad (254d)$$

onde em 254a utilizamos que a probabilidade de uma classe de tipo em particular é menor ou igual ao todo; e em 254c utilizamos que

$$P^n(x_{1:n}) = \prod_{i=1}^n P(x_i) \quad (255a)$$

$$= \prod_{x \in \mathcal{X}} P(x)^{nP(x)} \quad (255b)$$

$$= 2^{\sum_{x \in \mathcal{X}} nP(x) \log P(x)} \quad (255c)$$

$$= 2^{-n \sum_{x \in \mathcal{X}} P(x) \log \frac{1}{P(x)}} \quad (255d)$$

$$= 2^{-nH(P)}. \quad (255e)$$

□

Demonstração (limite inferior).

$$1 = \sum_{Q \in \mathcal{P}_n} P^n(T(Q)) \leq \sum_{Q \in \mathcal{P}_n} \max_{R \in \mathcal{P}_n} P^n(T(R)) \quad (256a)$$

$$= \sum_{Q \in \mathcal{P}_n} P^n(T(P)) \leq (n+1)^{|\mathcal{X}|} P^n(T(P)) \quad (256b)$$

$$= (n+1)^{|\mathcal{X}|} \sum_{x_{1:n} \in T(P)} P^n(x_{1:n}) \quad (256c)$$

$$= (n+1)^{|\mathcal{X}|} \sum_{x_{1:n} \in T(P)} 2^{-nH(P)} \quad (256d)$$

$$\leq (n+1)^{|\mathcal{X}|} \sum_{x_{1:n} \in T(P)} 2^{-nH(P)} \quad (256e)$$

$$= (n+1)^{|\mathcal{X}|} |T(P)| 2^{-nH(P)}, \quad (256f)$$

onde em 256b utilizamos

$$P = \operatorname{argmax}_{R \in \mathcal{P}_n} P^n(T(R)). \quad (257)$$

Ao final, obtemos

$$|T(P)| \geq \frac{1}{(n+1)^{|\mathcal{X}|}} 2^{nH(P)}. \quad (258)$$

□

Para o caso binário, $\mathcal{X} = \{0, 1\}$, o Teorema 30 nos fornece os seguintes limites para o tamanho da classe de tipo:

$$\frac{1}{(n+1)^2} 2^{nH(\frac{k}{n})} \leq \binom{n}{k} \leq 2^{nH(\frac{k}{n})}, \quad (259)$$

entretanto, o limite inferior pode ser ainda mais restrito, fornecendo assim

$$\frac{1}{(n+1)} 2^{nH(\frac{k}{n})} \leq \binom{n}{k} \leq 2^{nH(\frac{k}{n})}. \quad (260)$$

Os limites superior e inferior para o tamanho de uma classe de tipo $T(P)$, dados no Teorema 30, podem ser utilizados para derivar limites correspondentes na probabilidade total da classe de tipo $Q^n(T(P))$, onde Q é a distribuição subjacente da fonte i.i.d. que gera as sequências. Um resultado importante é que qualquer outro tipo que seja menos próximo de Q (em termos de divergência de Kullback-Leibler) terá sua probabilidade decrescendo exponencialmente com n , decrescendo assim mais rapidamente que o tipo mais provável.

Teorema 31 (Limites da probabilidade da classe de tipo) *Para qualquer $P \in \mathcal{P}_n$ e seja Q a distribuição subjacente da fonte i.i.d., a probabilidade da classe de tipo $T(P)$ sob Q^n é tal que $Q^n(T(P)) \doteq 2^{-nD(P||Q)}$. Especificamente, temos os limites*

$$\frac{1}{(n+1)^{|\mathcal{X}|}} 2^{-nD(P||Q)} \leq Q^n(T(P)) \leq 2^{-nD(P||Q)}. \quad (261)$$

Demonstração.

$$Q^n(T(P)) = \sum_{x_{1:n} \in T(P)} Q^n(x_{1:n}) = \sum_{x_{1:n} \in T(P)} 2^{-n(D(P||Q) + H(P))} \quad (262a)$$

$$= |T(P)| 2^{-n(D(P||Q) + H(P))} \quad (262b)$$

para completar a demonstração, os limites do tamanho da classe de tipo dados pela Equação (253).

□

Iremos agora rever a definição de conjunto típico, utilizando a formalização do métodos de tipos.

Definição 24 (Conjunto Típico). Na Definição 18 vimos a definição de conjunto típico, com relação à sua probabilidade. Definimos agora com relação à divergência entre o histograma empírico e a distribuição subjacente. Seja $X_1, X_2, \dots, X_n$ i.i.d. $\forall i, X_i \sim Q(x)$. Então o conjunto típico é definido como

$$T_Q^\epsilon = \{x_{1:n} : D(P_{x_{1:n}} \parallel Q) \leq \epsilon\}, \quad (263)$$

onde $P_{x_{1:n}}$ é o tipo empírico (distribuição de frequência relativa) da sequência $x_{1:n}$, e $\epsilon > 0$ é um limiar que controla a tolerância na definição de tipicidade.

Podemos agora analisar a probabilidade do conjunto típico.

Teorema 32 (Probabilidade do Conjunto Típico) *Sejam $X_1, X_2, \dots, X_n$ i.i.d. $\forall i, X_i \sim Q(x)$. A probabilidade do complemento do conjunto típico $\bar{T}_Q^\epsilon$ é dada por*

$$Q(\bar{T}_Q^\epsilon) = Q(\{x_{1:n} : D(P_{x_{1:n}} \parallel Q) > \epsilon\}) \leq 2^{-n(\epsilon - |\mathcal{X}| \frac{\log(n+1)}{n})}. \quad (264)$$

Dessa forma,

$$D(P_{x_{1:n}} \parallel Q) \xrightarrow{p} 0, \text{ quando } n \rightarrow \infty, \quad (265)$$

ou seja, a divergência entre $P_{x_{1:n}}$ e Q converge em probabilidade para zero quando n é grande suficiente.

Demonstração.

$$Q(\bar{T}_Q^\epsilon) = \sum_{P \in \mathcal{P}_n : D(P \parallel Q) > \epsilon} Q^n(T(P)) \quad (266a)$$

$$\leq \sum_{P \in \mathcal{P}_n : D(P \parallel Q) > \epsilon} 2^{-nD(P \parallel Q)} \quad (266b)$$

$$\leq \sum_{P \in \mathcal{P}_n : D(P \parallel Q) > \epsilon} 2^{-n\epsilon} \quad (266c)$$

$$\leq (n+1)^{|\mathcal{X}|} 2^{-n\epsilon} = 2^{-n(\epsilon - |\mathcal{X}| \frac{\log(n+1)}{n})} \quad (266d)$$

onde em 266a usamos a definição do complemento do conjunto típico, em 266b utilizamos o Teorema 31, e em 266d utilizamos o Teorema 26. Então, como podemos observar em 266d, a probabilidade vai para zero quando $n \rightarrow \infty$, e desta forma a probabilidade do conjunto típico vai para 1 quando $n \rightarrow \infty$. □

Os tipos que divergem (KL) mais do que ϵ da distribuição subjacente Q terão probabilidade decrescente. Para n grande, o conjunto típico acaba sendo a única coisa que ocorre com uma probabilidade não evanescente.

Dizemos que uma codificação de fonte é universal quando ela não depende da distribuição fonte. Seria possível criar um código universal

que atinja o limite da entropia, ou seja, uma taxa $R > H(Q)$ (em bits por símbolo), onde $H(Q)$ é a entropia da distribuição subjacente Q ? O método de tipos oferece uma abordagem para formalizar o teorema de Shannon, permitindo uma análise mais refinada das sequências com base em seus histogramas empíricos. Existem no máximo $2^{nH(P)}$ sequências do tipo P . Podemos utilizar $nH(P)$ bits para representar tais sequências. Se $R > H(P)$, podemos utilizar nR bits para representar estas sequências. Quando n cresce, apenas os tipos P ‘próximos’ de Q irão ocorrer.

A Figura 21 ilustra um codificador de blocos que associa cada uma das M sequências de n de símbolos da fonte a uma sequência binária de comprimento m . Neste codificador de blocos, cada sequência de n símbolos é codificada conjuntamente, associando a ela uma palavra de código (*codeword*) de comprimento m . Em outras palavras, o codificador faz o mapeamento de sequências de tamanho n produzidas pela fonte em sequências de m bits. M é o número de possíveis mensagens e também o número de palavras do *codebook* do codificador.

Definição 25 (Código de bloco com taxa fixa R). Seja $X_1, X_2, \dots, X_n \sim Q$, i.i.d. mas Q desconhecido. A função do codificador e decodificador são definidas a seguir:

$$\text{codificador: } f_n : \mathcal{X}^n \rightarrow \{1, 2, \dots, 2^{nR}\} \quad (267a)$$

$$\text{decodificador: } \phi_n : \{1, 2, \dots, 2^{nR}\} \rightarrow \mathcal{X}^n \quad (267b)$$

e a probabilidade de erro

$$P_e^{(n)} = Q^n(\{x_{1:n} : \phi(f_n(x_{1:n})) \neq x_{1:n}\}). \quad (268)$$

Definição 26 (Código de bloco universal de taxa R). Um código de bloco de taxa R para uma fonte é dito universal se a função f_n e ϕ_n não dependem da distribuição Q e se

$$P_e^{(n)} \rightarrow 0, \text{ quando } n \rightarrow \infty, \text{ sempre que } H(Q) < R. \quad (269)$$

Veremos que, se $R > H(Q)$, então existe uma sequência (em n) de códigos com erro evanescente. Por outro lado, se $R < H(Q)$ a probabilidade de erro vai pra 1.

Na demonstração do Teorema 33 utilizaremos ainda a definição de simplex probabilístico.

Definição 27 (Simplex Probabilístico). O *simplex probabilístico*²² em $\mathbb{R}^m$ é o conjunto de pontos $x_{1:m} = (x_1, x_2, \dots, x_m) \in \mathbb{R}^m$ tal que $x_i \geq 0$, $\sum_{i=1}^m x_i = 1$.

Exemplo 17 (Simplex probabilístico com $m = 2$). O simplex probabilístico será o conjunto de pontos

$$\{(x_1, x_2) : x_1 \geq 0, x_2 \geq 0, x_1 + x_2 = 1\}, \quad (270)$$

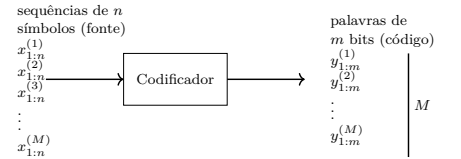


Figura 21: Codificador (M, n) , onde M representa o número de mensagens e n o comprimento das sequências.

²² Note que o simplex probabilístico (ou tipo) de uma sequência coincide com o histograma empírico normalizado.

representado na Figura 22.

Exemplo 18 (Simplex probabilístico com $m = 3$). O simplex probabilístico será o conjunto de pontos

$$\{(x_1, x_2, x_3) : x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, x_1 + x_2 + x_3 = 1\} \quad (271)$$

representado na Figura 23 e com alguns pontos em destaque na Figura 24.

O Teorema da Codificação de Fonte de Shannon é um dos resultados fundamentais da teoria da informação, estabelecendo as condições sob as quais é possível representar as informações produzidas por uma fonte i.i.d. com distribuição subjacente Q sem que haja perdas, utilizando, para tanto uma taxa R . O teorema mostra que, se $R > H(Q)$, então existe um código de bloco universal para representar a informação produzida pela fonte com probabilidade de erro tendendo a zero à medida que o comprimento n das sequências aumenta. Por outro lado, se $R < H(Q)$, não é possível representar a informação da fonte com probabilidade de erro evanescente. No contexto do método de tipos, podemos formalizar esse teorema explorando a estrutura das classes de tipo: como o número de tipos é polinomial em n , enquanto o número de sequências é exponencial em n , e apenas os tipos próximos de Q ocorrem, para n grande, é possível construir códigos universais que codifiquem eficientemente as sequências produzidas pela fonte, usando aproximadamente $nH(Q)$ bits.

Teorema 33 (Teorema da Codificação de Fonte de Shannon (Teorema da Codificação de Fonte de Shannon)) $\exists$ uma sequência $(2^{nR}, n)$ de códigos universais tais que $P_e^{(n)} \rightarrow 0$ para toda distribuição Q tal que $H(Q) < R$.

Demonstração. Fixe $R > H(Q)$. Defina uma taxa para n que é fixada a um fator:

$$R_n \triangleq R - |\mathcal{X}| \frac{\log(n+1)}{n} < R. \quad (272)$$

Defina um conjunto de sequências que possuem entropia de tipo menor do que esta taxa:

$$A_n \triangleq \{x_{1:n} \in \mathcal{X}^n : H(P_{x_{1:n}}) \leq R_n\} \quad (273a)$$

$$= \left\{ \bigcup_{P \in \mathcal{P}_n} T(P) : H(P) \leq R_n \right\} \quad (273b)$$

A partir desta definição, teremos que

$$|A_n| = \sum_{P \in \mathcal{P}_n : H(P) \leq R_n} |T(P)| \leq \sum_{P \in \mathcal{P}_n : H(P) \leq R_n} 2^{nH(P)} \quad (274a)$$

$$\leq \sum_{P \in \mathcal{P}_n : H(P) \leq R_n} 2^{nR_n} \leq (n+1)^{|\mathcal{X}|} 2^{nR_n} \quad (274b)$$

$$= 2^{n(R_n + |\mathcal{X}| \frac{\log(n+1)}{n})} = 2^{nR}. \quad (274c)$$

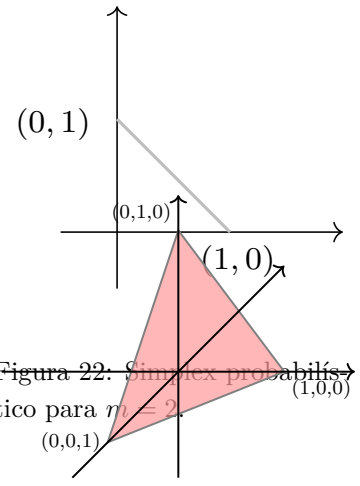


Figura 22: Simplex probabilístico para $m = 2$.

Figura 23: Simplex probabilístico para $m = 3$.

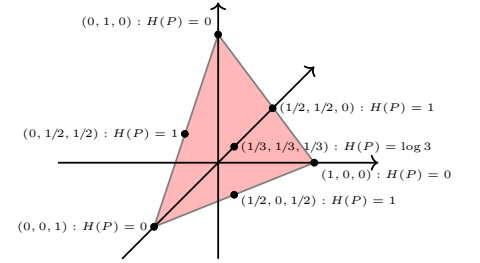


Figura 24: Representação de alguns pontos e suas respectivas entropias.

Como $|A_n| \leq 2^{nR}$, podemos indexar A_n com nR bits.

O codificador será dado por

$$f_n(x_{1:n}) = \begin{cases} \text{índice de } x_{1:n} \text{ em } A_n, & \text{se } x_{1:n} \in A_n \\ 0, & \text{caso contrário.} \end{cases} \quad (275)$$

O codificador associará um índice a $x_{1:n}$ se $H(P_{x_{1:n}}) \leq R_n$ (ou seja, $x_{1:n} \in A_n$); e não associará valor se $H(P_{x_{1:n}}) > R_n$ (ou seja, $x_{1:n} \notin A_n$). Note que $f_n(\cdot)$ não depende da distribuição da fonte, apenas do ordenamento e de $\mathbb{R}^m$. Um erro ocorrerá se $x_{1:n} \notin A_n$. Os tipos podem ser representados por pontos em um simplex probabilístico, conforme ilustrado na ?? . Um erro ocorre quando a sequência não

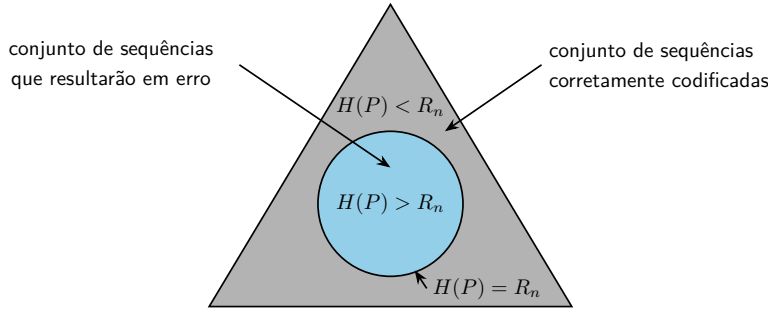


Figura 25: Representação das sequências em um simplex probabilístico. As sequências com $H(P_{x_{1:n}}) \leq R_n$ não acarretarão em erro no processo de codificação.

está em A_n . Desta forma,

$$P_e^{(n)} = Q^n(A_n^c) \quad (276a)$$

$$= \sum_{P: H(P) > R_n} Q^n(T(P)) \quad (276b)$$

$$\leq \sum_{P: H(P) > R_n} \max_{P: H(P) > R_n} Q^n(T(P)) \quad (276c)$$

$$\leq (n+1)^{|\mathcal{X}|} \max_{P: H(P) > R_n} Q^n(T(P)) \quad (276d)$$

$$\leq (n+1)^{|\mathcal{X}|} \max_{P: H(P) > R_n} 2^{-nD(P||Q)} \quad (276e)$$

$$= (n+1)^{|\mathcal{X}|} 2^{-n[\min_{P: H(P) > R_n} D(P||Q)]} \quad (276f)$$

onde utilizamos que $Q^n(T(P)) \leq 2^{-nD(P||Q)}$. Temos que R_n forma uma sequência crescente com n , tal que $R_n < R$ para todo n (veja a Figura 26).

Por hipótese, $H(Q) < R$. Eventualmente, para algum n_0 , teremos que $\forall n > n_0$, $R_n > H(Q)$. Na Equação (276f), escolhemos $P : H(P) > R_n$. Teremos então: $H(P) > R_n > H(Q)$, o que implica em $P \neq Q$. Desta forma, teremos $D(P || Q) > 0$, para P que foi escolhido em 276f. Teremos assim

$$P_e^{(n)} \leq \underbrace{(n+1)^{|\mathcal{X}|}}_{\text{polinomial em } n} \underbrace{2^{-n[\min_{P: H(P) > R_n} D(P||Q)]}}_{\text{exp. decrescente qnd } n \rightarrow \infty} \quad (277)$$

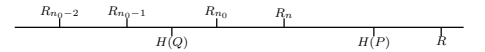


Figura 26: Sequência de taxas R_n .

Logo, $P_e^{(n)} \rightarrow 0$ quando $n \rightarrow \infty$.

□

Por outro lado, se $R < H(Q)$ teremos $P_e^{(n)} \rightarrow 1$. A entropia é então o limite de representação, ou compressão.

Processos Estocásticos

Até o momento, nossa análise considerou variáveis aleatórias independentes, uma simplificação que facilitou a compreensão de conceitos fundamentais como entropia e informação mútua. Ao final, chegamos à conclusão de que $nH(X)$ bits são suficientes, na média, para representar uma sequência de n variáveis aleatórias independentes e identicamente distribuídas. Neste capítulo, removemos essa restrição para abordar processos estocásticos, nos quais as variáveis aleatórias exibem dependências temporais ou estruturais. Introduziremos a entropia de um processo estocástico, que quantifica a incerteza associada a uma sequência de eventos, e a taxa de entropia, uma medida assintótica da informação gerada por unidade de tempo. Esses conceitos são essenciais para modelar sistemas dinâmicos. Em particular, focaremos nas cadeias de Markov, um caso especial de processos estocásticos em que a dependência é limitada ao estado imediatamente anterior, oferecendo um equilíbrio entre simplicidade e aplicabilidade em problemas de codificação e comunicação.

Um *processo estocástico estacionário em sentido estrito* é uma sequência de variáveis aleatórias cujas propriedades estatísticas conjuntas permanecem invariantes sob deslocamentos temporais. Isto implica que, por exemplo, médias, variâncias e outras estatísticas não dependem do instante absoluto, permanecem estáveis ao longo do tempo.

Definição 28 (Processo estocástico estacionário (sentido-estrito)).

Uma sequência de v.a.s. $X_1, X_2, \dots, X_n$ é governada por uma distribuição probabilística é dita *estacionária* em sentido estrito se

$$p(X_{1:n} = x_{1:n}) = p(X_{1+l:n+l} = x_{1:n}), \quad (278)$$

para todo l , todo n e todo $x_{1:n} \in \mathcal{X}^n$.

Um processo de Markov é um processo estocástico em que a probabilidade do próximo estado depende apenas do estado atual, e não de toda a história anterior.

Definição 29 (Processo de Markov de primeira ordem). Um processo

estocástico é um *processo de Markov* de primeira ordem se

$$p(X_{n+1} = x_{n+1} \mid X_{1:n} = x_{1:n}) = p(X_{n+1} = x_{n+1} \mid X_n = x_n) \quad (279)$$

A essência de um processo de Markov reside na ideia de que, dado o estado presente, o futuro e o passado são independentes. Em termos probabilísticos, para o caso de primeira ordem, isto significa que $p(x_{1:n}) = p(x_1)p(x_2 \mid x_1) \dots p(x_n \mid x_{n-1})$. Essa independência condicional reflete a ‘falta de memória’ do processo, onde conhecer o estado atual basta, não sendo necessário avaliar a influência do passado.

De forma geral, podemos estender a ideia anterior para processos de Markov de ordens superiores.

Definição 30 (Processo de Markov de ordem m). Um processo estocástico é um processo de Markov de ordem m se

$$p(X_{n+1} = x_{n+1} \mid X_{1:n} = x_{1:n}) = p(X_{n+1} = x_{n+1} \mid X_n = x_n, X_{n-1} = x_{n-1}, \dots, X_{n-m} = x_{n-m}). \quad (280)$$

Neste caso, isto significa que $p(x_{1:n}) = p(x_{m+1} \mid x_m, x_{m-1}, \dots, p(x_1)) \dots p(x_{n-1} \mid x_{n-2}, x_{n-3}, \dots, x_{n-m-1})p(x_n \mid x_{n-1}, x_{n-2}, \dots, x_{n-m})$.

Uma cadeia de Markov homogênea é um processo de Markov em que as probabilidades de transição entre estados não variam com o tempo.

Definição 31 (Cadeia de Markov homogênea). Uma cadeia de Markov é invariante no tempo (também chamada de *homogênea*) se $p(x_{n+1} \mid x_n)$ não depender do tempo, i.e., se

$$p(X_{n+1} = b \mid X_n = a) = p(X_2 = b \mid X_1 = a) \quad \forall a, b, n. \quad (281)$$

Neste caso, a cadeia de Markov pode ser descrita por uma matriz de transição fixa $\mathbf{P} = [p_{ij}]_{ij}$ em que $p_{ij} = p(X_{n+1} = j \mid X_n = i)$. Podemos representar esta cadeia de Markov como um grafo com setas entre estados cuja probabilidade de transição não é nula. O fato de ser homogênea também facilita o estudo de propriedades assintóticas, como a taxa de entropia, como veremos adiante.

Exemplo 19 (Cadeia de Markov homogênea de primeira ordem).

Vamos considerar neste exemplo uma cadeia de Markov homogênea de primeira ordem com 4 estados. Esta cadeia pode ser representada pela sua matriz de transição:

$$\mathbf{P} = \begin{bmatrix} 0 & p(2 \mid 1) & 0 & p(4 \mid 1) \\ 0 & p(2 \mid 2) & p(3 \mid 2) & 0 \\ 0 & 0 & 0 & p(4 \mid 3) \\ 0 & 0 & 0 & p(4 \mid 4) \end{bmatrix}, \quad (282)$$

ou, alternativamente, pelo grafo ilustrado na Figura 27

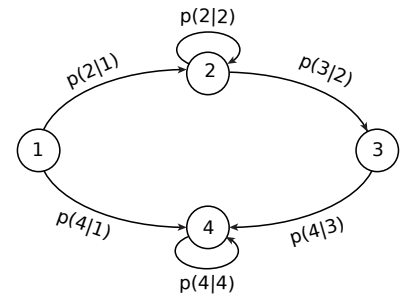


Figura 27: Exemplo de uma cadeia de Markov de primeira ordem com 4 estados.

Para uma cadeia de Markov homogênea de primeira ordem, a probabilidade de um estado no instante $n + 1$, pode ser dada em função dos possíveis estados no instante n e das probabilidades de transição:

$$p(X_{n+1} = i) = \sum_j p(X_n = j)p(i|j), \quad (283)$$

onde $p(i|j)$ é o elemento p_{ij} da matriz de transição $\mathbf{P}$. Esta cadeia de Markov será estacionária se a probabilidade dos estados não mudar ao longo do tempo, ou seja, $p(X_{n+1} = i) = p(X_n = i)$, para todo i . Quando isto ocorrer, haverá uma distribuição estacionária μ tal que $\mu^T \mathbf{P} = \mu^T$, onde $\mathbf{P}$ é a matriz de transição. Isto assegura que a probabilidade de cada estado permaneça constante ao longo do tempo quando o processo parte de, ou alcança, uma distribuição μ .

Definição 32 (Cadeira de Markov irredutível). Uma cadeia de Markov é *irredutível* se $p_{ij}(n) > 0$ para todo i, j e para algum n onde $p_{ij}(n) = p(X_{n+1} = j | X_n = i)$.

Ou seja, qualquer estado é acessível de qualquer outro estado (ao menos em algum instante n), com probabilidade não nula.

Definição 33 (Período de uma cadeia de Markov). Uma cadeia de Markov é *periódica* se $d(i) > 1$ com

$$d(i) = \gcd\{n : p_{ii}(n) > 0\} \quad (284)$$

$d(i)$ é o período do i -ésimo estado.

Note que temos o máximo divisor comum do número de épocas para o qual um retorno ao mesmo estado é possível.

Exemplo 20 (Cadeia de Markov homogênea de primeira ordem com dois estados). Suponha uma cadeia de Markov homogênea de primeira ordem com apenas dois estados. Esta cadeia é caracterizada pela matriz de transição

$$\mathbf{P} = \begin{pmatrix} 1 - \alpha & \alpha \\ \beta & 1 - \beta \end{pmatrix} \quad (285)$$

e pode também ser representada pelo grafo da Figura 28.

Se $\mu = [p_1 \ p_2]^T$ é uma distribuição estacionária, então devemos ter $\mu^T \mathbf{P} = \mu^T$. Neste caso, teremos

$$\mu^T = (p_1 \ p_2) = (p_1 \ p_2) \begin{pmatrix} 1 - \alpha & \alpha \\ \beta & 1 - \beta \end{pmatrix} \quad (286a)$$

$$= ((1 - \alpha)p_1 + \beta p_2 \quad \alpha p_1 + (1 - \beta)p_2) \quad (286b)$$

Obtemos assim:

$$p_1 = (1 - \alpha)p_1 + \beta p_2 \quad (287)$$

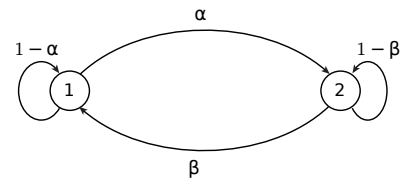


Figura 28: Grafo representando uma cadeia de Markov de primeira ordem com dois estados.

logo, $p_1 = \frac{\beta}{\alpha} p_2$. Sabemos também que devemos ter $p_1 + p_2 = 1$. Por conseguinte, teremos

$$p_1 + p_2 = 1 \quad (288a)$$

$$\frac{\beta}{\alpha} p_2 + p_2 = 1 \quad (288b)$$

$$p_2 = \frac{\alpha}{\alpha + \beta}. \quad (288c)$$

Assim, teremos

$$\mu = \left(\frac{\frac{\beta}{\alpha + \beta}}{\frac{\alpha}{\alpha + \beta}} \right). \quad (289)$$

A Média de Cesàro representada uma técnica matemática que consiste em substituir a soma parcial tradicional de uma série por uma média aritmética de suas somas parciais. Ela pode ser utilizada para obter representações mais precisas de sinais descontínuos; utilizada em análises em processamento de sinais; aplicada a equações diferenciais parciais; e, em processos estocásticos analisar o comportamento assintótico, especialmente ao estudar convergência ou taxas de entropia.

Lema 34 (Média Cesàro) *Sejam a_n números reais, se $a_n \rightarrow a$ quando $n \rightarrow \infty$ e $b_n = \frac{1}{n} \sum_{i=1}^n a_i$, então $b_n \rightarrow a$ quando $n \rightarrow \infty$.*

Demonstração. Como $a_n \xrightarrow{n \rightarrow \infty} a$, para todo $\epsilon > 0$, existe N_ϵ tal que $|a_n - a| < \epsilon$ para todo $n > N_\epsilon$. Para $n > N_\epsilon$ teremos

$$|b_n - a| = \left| \frac{1}{n} \sum_{i=1}^n a_i - a \right| = \left| \frac{1}{n} \sum_{i=1}^n a_i - \frac{1}{n} \sum_{i=1}^n a \right| \quad (290a)$$

$$= \left| \frac{1}{n} \sum_{i=1}^n (a_i - a) \right| \leq \frac{1}{n} \sum_{i=1}^n |a_i - a| \quad (290b)$$

$$= \frac{1}{n} \left(\sum_{i=1}^{N_\epsilon} |a_i - a| + \sum_{i=N_\epsilon+1}^n |a_i - a| \right) \quad (290c)$$

$$< \frac{1}{n} \sum_{i=1}^{N_\epsilon} |a_i - a| + \frac{1}{n} \sum_{i=N_\epsilon+1}^n \epsilon \quad (290d)$$

$$= \frac{1}{n} \sum_{i=1}^{N_\epsilon} |a_i - a| + \underbrace{\frac{n - N_\epsilon}{n}}_{< 1} \epsilon \quad (290e)$$

$$< \underbrace{\frac{1}{n} \sum_{i=1}^{N_\epsilon} |a_i - a|}_{< \epsilon} + \epsilon < 2\epsilon \quad (290f)$$

onde em 290b utilizamos a desigualdade triangular, em 290d utilizamos a convergência de a_n , e em 290f utilizamos o fato de que podemos tomar n grande suficiente de forma que $\frac{1}{n} \sum_{i=1}^{N_\epsilon} |a_i - a| < \epsilon$, pois trata-se de uma soma finita. Então $b_n \rightarrow a$ quando $n \rightarrow \infty$. □

Processos Estocástico possuem taxas de entropia, que intuitivamente representam o quantidade de informação nova, na média, que é fornecida pelo processo estocástico a cada instante.

Definição 34 (Taxa de entropia de um processo estocástico). A *taxa de entropia* de um processo estocástico $\{X_i\}_i$ é definida como

$$H(\mathcal{X}) \triangleq \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, X_2, \dots, X_n), \quad (291)$$

quando existir.

Observe que, na Equação (291), quando as v.a.s são i.i.d. teremos $H(X_1, \dots, X_n) = H(X_1) + \dots + H(X_n) = nH(X)$ e assim $H(\mathcal{X}) = H(X)$, ou seja,

$$H(\mathcal{X}) = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_{1:n}) = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n H(X_i) = H(X_1). \quad (292)$$

A taxa de entropia pode ser vista como a entropia por símbolo para um dado processo estocástico, quando n cresce indefinidamente.

A Taxa de Entropia de um processo estocástico $H(\mathcal{X})$ mede a incerteza média por símbolo em uma sequência longa, enquanto a *taxa de inovação da informação* $H'(\mathcal{X})$, que será definida a seguir, quantifica a incerteza de um novo símbolo dado o passado. Para processos gerais, essas taxas podem diferir devido à dependência entre símbolos; no entanto, em processos estacionários, ambas convergem ao mesmo valor, ou seja, $H(\mathcal{X}) = H'(\mathcal{X})$, refletindo a estabilidade estatística do processo ao longo do tempo.

Definição 35 (Taxa de inovação da informação). Vamos assumir um processo estocástico e definir a taxa da seguinte forma

$$H'(\mathcal{X}) \triangleq \lim_{n \rightarrow \infty} H(X_n \mid X_{n-1}, X_{n-2}, \dots, X_1), \quad (293)$$

se existir.

Veremos a seguir que $H'(\mathcal{X})$ existe para um processo estocástico estacionário.

Teorema 35 (Um processo estocástico estacionário possui taxa de inovação de informação) *Para um processo estocástico estacionário, $H(X_n \mid X_{n-1}, X_{n-2}, \dots, X_1)$ é decrescente com n e possui como limite $H'(\mathcal{X})$.*

Demonstração.

$$H(X_{n+1} \mid X_1, \dots, X_n) \leq H(X_{n+1} \mid X_2, \dots, X_n) \quad (294a)$$

$$= H(X_n \mid X_1, \dots, X_{n-1}) \quad (294b)$$

Onde utilizamos o fato de que condicionar não aumenta (decrece ou não altera) a entropia; e utilizamos o fato de que o processo estocástico é estacionário.

Temos então uma sequência decrescente com limite inferior 0, logo, esta sequência possui um limite: $H'(\mathcal{X})$.

□

Teorema 36 (Em um processo estocástico estacionário taxa de inovação é igual a taxa de entropia) *Para um processo estocástico estacionário temos*

$$\lim_{n \rightarrow \infty} H(X_n | X_{n-1}, X_{n-2}, \dots, X_1) \triangleq H'(\mathcal{X}) \quad (295a)$$

$$= H(\mathcal{X}) \triangleq \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, X_2, \dots, X_n) \quad (295b)$$

Demonstração.

$$b_n = \frac{H(X_1, X_2, \dots, X_n)}{n} = \frac{1}{n} \sum_{i=1}^n \underbrace{H(X_i | X_{i-1}, \dots, X_1)}_{=a_i}, \quad (296)$$

como $a_n \rightarrow H'(\mathcal{X})$, teremos $b_n \rightarrow H'(\mathcal{X})$, mas por definição $b_n \rightarrow H(\mathcal{X})$.

□

Exemplo 21 (Taxa de entropia para cadeia de Markov de primeira ordem). A taxa de entropia para uma cadeia de Markov de primeira ordem estacionária será dada da seguinte forma

$$H(\mathcal{X}) = H'(\mathcal{X}) = \lim_{n \rightarrow \infty} H(X_n | X_{n-1}, \dots, X_1) \quad (297a)$$

$$= \lim_{n \rightarrow \infty} H(X_n | X_{n-1}) \quad (297b)$$

$$= H(X_2 | X_1) \quad (297c)$$

$$= - \sum_{x_2, x_1} p(x_2, x_1) \log p(x_2 | x_1) \quad (297d)$$

$$= \sum_i \mu_i \left[- \sum_j p_{ij} \log p_{ij} \right], \quad (297e)$$

onde μ é a distribuição estacionária, p_{ij} a probabilidade de transição de i para j e, em 297c, utilizamos o fato de termos uma cadeia de Markov de primeira ordem estacionária.

Exemplo 22 (Markov com apenas dois estados). Para a cadeia de Markov simples apresentada no Exemplo 20, teremos

$$H(\mathcal{X}) = H(X_2 | X_1) = \frac{\beta}{\alpha + \beta} H(\alpha) + \frac{\alpha}{\alpha + \beta} H(\beta). \quad (298)$$

Exemplo 23 (Modelos de Ehrenfest). Os modelos de Ehrenfest são cadeias de Markov que descrevem a dinâmica de partículas em um sistema com dois compartimentos. Tais modelos são frequentemente usados para ilustrar conceitos de equilíbrio estatístico e comportamento assintótico. A análise desse processo estocástico revela propriedades como estacionaridade e convergência para uma distribuição de equilíbrio. Paul Ehrenfest propôs um modelo simples para a troca de calor ou moléculas entre dois corpos isolados. Neste modelo da cadeia de Ehrenfest iremos considerar a troca de moléculas de gás entre dois compartimentos, rotulados por 0 e 1, conforme apresentado na Figura 29.

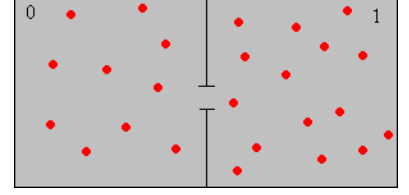


Figura 29: Exemplo de uma cadeia de Markov de primeira ordem com 4 estados.

Partículas movem-se aleatoriamente entre os compartimentos com probabilidades fixas, e o estado do sistema é representado pelo número de partículas em um dos compartimentos. Iremos denotar por X_k o número de partículas no compartimento 1 no instante k . X_k descreve o estado atual do sistema. Temos um processo estocástico gerando $X_{1:n}$, onde $X_k \in \{0, 1, \dots, m\}$.

Supondo que no instante k o compartimento 1 possua i partículas ($X_k = i$). O modelo de Ehrenfest nos fornece:

$$\Pr(X_{k+1} = i + 1 | X_k = i) = \frac{m - i}{m}, \quad (299a)$$

$$\Pr(X_{k+1} = i - 1 | X_k = i) = \frac{i}{m}. \quad (299b)$$

Neste modelo a propriedade de Markov é satisfeita:

$$\Pr(X_{n+1} = i + 1 | X_0 = i_0, X_1 = i_1, \dots, X_{n-1} = i_{n-1}, X_n = i) = \Pr(X_{n+1} = i + 1 | X_n = i), \quad (300a)$$

$$\Pr(X_{n+1} = i - 1 | X_0 = i_0, X_1 = i_1, \dots, X_{n-1} = i_{n-1}, X_n = i) = \Pr(X_{n+1} = i - 1 | X_n = i). \quad (300b)$$

A distribuição condicional não depende de n , desta forma, temos um processo estocástico homogêneo. A cadeia de Markov poderá então ser descrita por uma matriz de transição fixa $P = [p_{ij}]_{ij}$.

Para $m = 4$ teremos a cadeia de Markov representada pelo grafo apresentado na Exemplo 23.

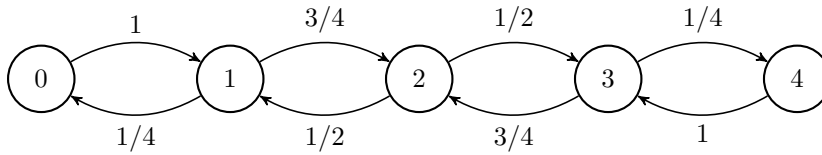


Figura 30: Modelo de Ehrenfest com 4 estados.

Para este modelo, a matriz de transição é

$$\mathbf{P} = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 \\ 0 & 1 & 0 & 0 & 0 \\ 1/4 & 0 & 3/4 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 3/4 & 0 & 1/4 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} \quad (301)$$

A distribuição de estado estacionário é tal que $\mu^T P = \mu^T$, com $\sum_i \mu_i = 1$.

$$\mu^T \mathbf{P} = \mu^T \quad (302a)$$

$$\mu^T (\mathbf{P} - \mathbf{I}) = 0 \quad (302b)$$

$$\mu^T \mathbf{Q} = 0 \quad (302c)$$

Teremos aqui

$$\mathbf{Q} = \begin{pmatrix} -1 & 1 & 0 & 0 & 0 \\ 1/4 & -1 & 3/4 & 0 & 0 \\ 0 & 1/2 & -1 & 1/2 & 0 \\ 0 & 0 & 3/4 & -1 & 1/4 \\ 0 & 0 & 0 & 1 & -1 \end{pmatrix}. \quad (303)$$

Vamos incorporar a condição $\sum_i \mu_i = 1$ fazendo:

$$\tilde{\mathbf{Q}} = \begin{pmatrix} -1 & 1 & 0 & 0 & 1 \\ 1/4 & -1 & 3/4 & 0 & 1 \\ 0 & 1/2 & -1 & 1/2 & 1 \\ 0 & 0 & 3/4 & -1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix}, \quad (304)$$

e o novo sistema de equações será

$$\mu^T \tilde{\mathbf{Q}} = (0, 0, 0, 1). \quad (305)$$

Para solucionar a Equação 305, iremos pós multiplicar ambos os lados pela matriz inversa de $\tilde{\mathbf{Q}}$,

$$\mu^T = (0, 0, 0, 1) \tilde{\mathbf{Q}}^{-1}, \quad (306)$$

obtendo assim

$$\mu^T = \left(\frac{1}{16}, \frac{1}{4}, \frac{6}{16}, \frac{1}{4}, \frac{1}{16} \right) \quad (307)$$

Conforme vimos na Equação (297), a taxa de entropia para um cadeia de Markov de primeira ordem estacionária será dada da seguinte forma

$$H(\mathcal{X}) = \sum_i \mu_i H(\mathbf{p}_i), \quad (308)$$

onde μ é a distribuição estacionária, p_{ij} a probabilidade de transição de i para j (elementos da matriz P) e $\mathbf{p}_i$ a i -ésima linha da matriz $\mathbf{P}$ (as probabilidades de transição à partir do estado i).

Para o exemplo em questão:

$$H(\mathcal{X}) = \mu_1 H(\mathbf{p}_1) + \mu_2 H(\mathbf{p}_2) + \mu_3 H(\mathbf{p}_3) + \mu_4 H(\mathbf{p}_4) + \mu_5 H(\mathbf{p}_5) \quad (309a)$$

$$= \frac{1}{16} H(0, 1, 0, 0, 0) + \frac{1}{4} H\left(\frac{1}{4}, 0, \frac{3}{4}, 0, 0\right) + \frac{6}{16} H\left(0, \frac{1}{2}, 0, \frac{1}{2}, 0\right) + \frac{1}{4} H\left(0, 0, \frac{3}{4}, 0, \frac{1}{4}\right) + \frac{1}{16} H(0, 0, 0, 1, 0) \quad (309b)$$

$$= 0 + 2 \times \frac{1}{4} \left(\frac{1}{4} \log 4 + \frac{3}{4} \log \frac{4}{3} \right) + \frac{6}{16} + 0 \quad (309c)$$

$$= \frac{1}{2} \left(\frac{1}{2} + \frac{3}{2} - \frac{3}{4} \log 3 \right) + \frac{6}{16} \quad (309d)$$

$$= 1 - \frac{3}{8} \log 3 + \frac{6}{16} = \frac{11}{8} - \frac{3}{8} \log 3 = 0.78064 \quad (309e)$$

Codificação de Fonte

A *codificação de fonte* sem perdas é o objetivo central quando buscamos representar os símbolos de uma fonte estocástica de forma eficiente. Ou seja, desejamos utilizar o menor número de bits possível para codificar a informação produzida pela fonte, sem que haja perda de informação, sendo assim possível reconstruir no receptor, exatamente a mesma mensagem enviada pelo emissor.

O Teorema da Codificação de Shannon estabelece que, para uma fonte com entropia $H(X)$, é possível codificar utilizando uma taxa $R > H(X)$ bits por símbolo, desde que se empregue blocos suficientemente grandes. No entanto, a codificação por blocos pode ser impraticável em aplicações reais, devido à latência e à complexidade computacional, o que motiva a busca por alternativas mais práticas que ainda alcancem o limite teórico da entropia.

Neste capítulo, exploraremos duas abordagens principais: códigos de tamanho variável, como a Codificação Huffman, que atribui palavras-código de comprimento variável a símbolos individuais, e a codificação de fluxo (*stream coding*), que opera no fluxo de dados considerando o símbolo atual e sua história, como na Codificação Aritmética e na Codificação Lempel-Ziv — esta última sendo universal, pois não requer conhecimento prévio da distribuição $p(x)$. Assumiremos que a distribuição $p(x)$ da fonte é conhecida ou que uma aproximação $q(x)$ está disponível, e investigaremos o impacto de utilizar $q(x)$ quando a distribuição verdadeira é $p(x)$, sem abordar diretamente o problema de estimação de $p(x)$. Buscaremos analisar métodos práticos que se aproximem do limite de entropia.

Para abordar este assunto, devemos inicialmente fazer algumas definições.

Um código, no contexto de codificação de fonte, é uma função que mapeia os símbolos de uma fonte em sequências de bits (palavras-código) de forma a representá-los. Usualmente, o objetivo em se estabelecer um código é minimizar o número de bits necessário para transmissão ou armazenamento da mensagem produzida pela fonte. Um código pode ser de tamanho fixo ou variável, e, para codificação sem perdas, devemos requerer que cada símbolo seja univocamente

recuperado a partir de sua palavra-código.

Definição 36 (Codificação de fonte). Um código C para a v.a. X é um mapeamento de $\mathcal{X}$ em $\mathcal{D}^*$, ou seja

$$C : \mathcal{X} \rightarrow \mathcal{D}^*, \quad (310)$$

onde $\mathcal{D}^*$ é o conjunto de seqüências (*strings*) finitas em um alfabeto $\mathcal{D}$ -ário. $C(x)$ é a palavra código (*codeword*) correspondente ao símbolo x , e $l(x)$ é o comprimento desta palavra.

De forma geral, $\mathcal{D} = \{0, 1, \dots, D-1\}$, mas usualmente utilizamos $D = 2$, ou seja, codificação binária com alfabeto $\mathcal{D} = \{0, 1\}$. O conjunto $\mathcal{D}^*$ representa a extensão do alfabeto $\mathcal{D}$ (fecho de Kleene²³ do alfabeto), consistindo de todas as seqüências finitas (de qualquer tamanho, incluindo a seqüência vazia) formadas por símbolos de $\mathcal{D}$, excluindo seqüências infinitas, que seriam denotadas por $\mathcal{D}^\infty$.

²³ O fecho de Kleene de um alfabeto $\mathcal{D}$, denotado $\mathcal{D}^*$, é um conceito da teoria de linguagens formais que inclui todas as seqüências finitas que podem ser formadas concatenando zero ou mais símbolos de $\mathcal{D}$, incluindo a seqüência vazia, frequentemente representada por ϵ .

Exemplo 24. Seja $\mathcal{X} = \{\text{azul}, \text{vermelho}\}$. O código pode ser $C(\text{vermelho}) = 00$ e $C(\text{azul}) = 11$, o que seria um código binário para $\mathcal{D} = \{0, 1\}$. Teríamos então $\mathcal{D}^* = \{00, 11\}$. Uma seqüência produzida pela fonte da forma (azul, azul, vermelho) seria codificada como (11, 11, 00), ou melhor, 111100.

O comprimento esperado de um código é uma medida fundamental na codificação de fonte, pois quantifica a eficiência média do código ao representar os símbolos da fonte, permitindo comparar diferentes esquemas de codificação em termos de uso de bits.

Definição 37 (Comprimento esperado). O *comprimento esperado* $L(C)$ de um código C para uma v.a. X com distribuição $p(x)$ é dado por

$$L(C) = \sum_x p(x)l(x). \quad (311)$$

Exemplo 25. Seja $\mathcal{X} = \{1, 2, 3, 4\}$ e $\mathcal{D} = \{0, 1\}$. Podemos definir o código através da Exemplo 25. Neste caso, teremos $H(X) = 1.75$ e

x	p(x)	c(x)	l(x)
1	1/2	0	1
2	1/4	10	2
3	1/8	110	3
4	1/8	111	3

Tabela 2: Código binário de exemplo para quatro símbolos.

$L(C) = El(X) = 1.75$, então este código é muito bom. A decodificação para este código é fácil, por exemplo, dada a seqüência binária 01011011111110100, podemos prontamente identificar a seqüência de símbolos que a produziu: 1,2,3,4,4,3,2,1. Seria como se tivéssemos uma pontuação separando os símbolos: 0,10,110,111,111,110,10,0. Dizemos que o código possui pontuação automática.

Exemplo 26. Vamos considerar $\mathcal{X} = \{1, 2, 3\}$ e $\mathcal{D} = \{0, 1\}$. O código proposto é apresentado na Exemplo 26. Para o código proposto,

x	1	2	3
$p(x)$	1/3	1/3	1/3
$C(X)$	0	10	11

teremos então $El(X) = 1.66 > H$ bits, uma vez que $H = 1.58$ bits. Note ainda como podemos facilmente decodificar uma sequência, como 10110010 = 2, 3, 1, 1, 2.

A eficiência de um código é uma medida que compara o comprimento esperado L do código com a entropia $H(X)$ da fonte. Ela serve como um indicativo do quão próximo o código está do limite teórico de compressão estabelecido pelo Teorema da Codificação de Shannon.

Definição 38 (Eficiência de um código). A *eficiência de um código* é definida da seguinte forma

$$0 \leq \text{eficiência} \triangleq \frac{H_D(X)}{El(X)} \leq 1. \quad (312)$$

Exemplo 27. Para $\mathcal{X} = \{1, 2, 3, 4\}$ e $\mathcal{D} = \{0, 1\}$ (código binário), considere os seguintes códigos dados na Exemplo 27. Neste caso, obte-

x	$p(x)$	C_I	C_{II}	C_{III}	C_{IV}
1	0.5	0	0	0	0
2	0.25	0	1	10	01
3	0.125	1	00	110	011
4	0.125	10	11	111	0111
$H(X)$	1.75	-	-	-	-
$El(X)$	-	1.125	1.25	1.75	1.875
eficiência		1.555	1.4	1	0.933

mos valores impróprios para o comprimento esperado dos códigos C_I e C_{II} , indicando que não podem ser códigos sem perdas, já que violam o teorema de Shannon. O valor esperado do comprimento do código não pode ser menor que a entropia, sem que o código incorra em erro.

Note que o código C_I não é unívoco. A sequência 00001 pode ser decodificada como 1, 2, 1, 2, 3 ou 2, 1, 2, 1, 3 ou 1, 1, 1, 1, 3.

Considere C_{II} . O resultado da decodificação de 0011 poderia ser 1, 1, 2, 2 ou 3, 4 ou 3, 2, 2 ou 1, 1, 4. Novamente, não é possível decodificar sem que a probabilidade de erro seja nula (quando a sequência torna-se mais longa, a probabilidade de erro tende a 1).

O código C_{III} é factível (já que $El \geq H$). O processo de decodificação é simples. Por exemplo: 111110100 = 111, 110, 10, 0 $\rightarrow$ 4, 3, 2, 1.

Utilizando o código C_{IV} é possível decodificar a sequência 00000000111 univocamente, porém os símbolos não são instantaneamente decodificados. É necessário analisar os bits à frente para conseguir decodificar.

Tabela 3: Exemplo de código binário para um alfabeto com 3 símbolos.

Tabela 4: Comparativo de 4 códigos distintos para uma fonte com 4 símbolos e distribuição dada.

Definição 39 (Extensão de um código). Uma *extensão* C^* de um código C é um mapeamento de uma sequência finita de símbolos em $\mathcal{X}$ em uma sequência finita de símbolos em $\mathcal{D}$ através da concatenação dos códigos:

$$C^*(x_1, x_2, \dots, x_n) = C(x_1)C(x_2) \dots C(x_n). \quad (313)$$

Um código não-singular é uma função de codificação que mapeia os símbolos de uma fonte estocástica em palavras-código de forma que cada símbolo tenha uma representação única, sendo essa uma condição necessária para a codificação sem perdas, embora não garanta por si só a decodificação unívoca.

Definição 40 (Código não-singular). Um código é dito *não-singular* se todo elemento de $\mathcal{X}$ é mapeado em sequências (*strings*) diferentes em $\mathcal{D}^*$. I.e.,

$$x_i \neq x_j \Rightarrow C(x_i) \neq C(x_j). \quad (314)$$

Para garantir que cada sequência codificada possa ser recuperada de forma inequívoca, a partir de sua representação em bits, um código precisa ter decodificação unívoca, ou seja, não pode haver ambiguidade na decodificação.

Definição 41 (Código com decodificação unívoca). Um código C com extensão C^* é decodificável *univocamente* se a sua extensão C^* for não-singular.

Em um código de prefixo nenhuma palavra-código é um prefixo de outra, garantindo que a decodificação possa ser realizada de forma unívoca e sequencial sem a necessidade de olhar para além do próximo delimitador.

Definição 42 (Código de prefixo). Um código é chamado *código de prefixo* ou *código instantâneo* se nenhuma palavra é prefixo de qualquer outra palavra.

Quando temos um código de prefixo, sabemos onde está o fim de uma palavra, pois ela não é prefixo de nenhuma outra, ou seja, não existe nenhuma outra palavra que comece com a palavra dada. Podemos dizer que um código de prefixo possui então a propriedade de auto-pontuação.

Dentre o conjunto dos códigos unívocos possíveis, queremos um código com comprimento esperado mínimo possível. Analisando as classes de códigos, intuitivamente pensamos que é mais provável encontrarmos um código com comprimento esperado menor em uma classe maior (ou mais abrangente). A Figura 31 apresenta a estrutura hierárquica dos códigos (todos os códigos $\supset$ não-singulares $\supset$ unicamente decodificáveis $\supset$ prefixo).

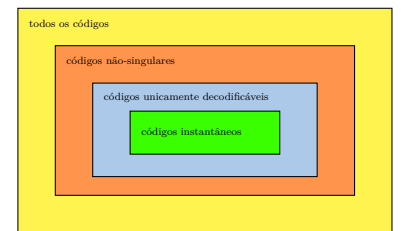


Figura 31: Diagrama de representação dos conjuntos de códigos, evidenciando a estrutura hierárquica existente.

Desigualdade de Kraft

A *desigualdade de Kraft* estabelece uma condição necessária e suficiente para a existência de códigos de prefixo com comprimentos de palavras-código especificados. A demonstração, que será vista, ainda fornecerá um método para construir um código de prefixo.

Teorema 37 (Desigualdade de Kraft) *Para qualquer código instantâneo (código de prefixo) sobre um alfabeto de tamanho D , o comprimento das palavras $l_1, l_2, \dots, l_m$ deve satisfazer*

$$\sum_i D^{-l_i} \leq 1. \quad (315)$$

Por um outro lado, dado um conjunto de comprimentos de código satisfazendo a desigualdade acima, então existe um código de prefixo com estes comprimentos.

Note que, ao dizer que existe um código de prefixo com aqueles comprimentos, não significa que todos códigos cujos comprimentos satisfazem a desigualdade são códigos de prefixo. Se existe um código não-instantâneo com comprimentos l_i satisfazendo a desigualdade de Kraft, então podemos encontrar um outro código, que terá a propriedade de prefixo, e que possuirá os mesmos comprimentos l_i , consequentemente não alterando o comprimento esperado. Logo, será sempre melhor escolher um código de prefixo.

Demonstração (Desigualdade de Kraft). Vamos representar o conjunto de códigos em uma árvore D -ária (não necessariamente balanceada), conforme a Figura 32.

As palavras correspondem às folhas na árvore. O caminho da raiz até a folha determina a palavra código. A condição de prefixo implica que não existe uma palavra a não ser nas folhas (nenhum descendente de uma palavra código será também uma palavra código). Designaremos por $l_{\max} = \max_i(l_i)$ o comprimento da palavra mais longa. Podemos expandir toda a árvore até o comprimento $l_{\max}$. Os nós no nível de $l_{\max}$ são: palavras de código; descendentes de palavras de código; ou apenas nós sem palavra código associada a ele ou seus antecessores. Considere uma palavra i no nível l_i da árvore (então esta palavra possui comprimento l_i). Existem então $D^{l_{\max}-l_i}$ descendentes de i , na árvore, no nível $l_{\max}$. Com a condição de prefixo, podemos afirmar que os descendentes de código i no nível l_i são disjuntos dos descendentes do código j no nível l_j , quando $i \neq j$ (i.e., o conjunto de descendentes para diferentes palavras é disjunto). O número total de nós em um conjunto de todos descendentes é $\leq D^{l_{\max}}$. Utilizando o que vimos anteriormente, temos que a soma do número de descendentes de todos os códigos é menor ou igual ao número de folhas na

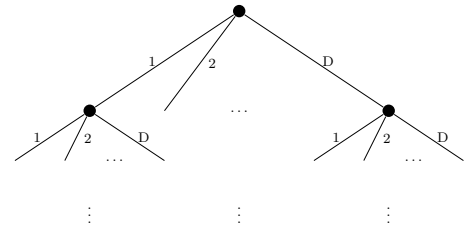


Figura 32: Árvore D -ária.

árvore cheia no nível $l_{\max}$. Podemos então escrever

$$\sum_i D^{l_{\max}-l_i} \leq D^{l_{\max}} \Rightarrow \sum_i D^{-l_i} \leq 1. \quad (316)$$

Por outro lado, dados os comprimentos $l_1, l_2, \dots, l_m$, satisfazendo Kraft, iremos mostrar como construir um código de prefixo utilizando estes comprimentos. Considere uma árvore D -ária cheia de profundidade $l_{\max}$ e $D^{l_{\max}}$ nós terminais. Observe que: no nível 0 existe uma fração 1 de descendentes de cada nó neste nível; no nível 1 existe uma fração $1/D$ de descendentes de cada nó neste nível; no nível 2 existe uma fração $1/D^2$ de descendentes de cada nó neste nível; e assim por diante. De forma geral, em cada nível $i \in [0, l_{\max}]$ da árvore, existe uma fração de D^{-i} nós terminais que são descendentes de uma ramificação de cada um dos D^i nós no nível i . Ordene então os comprimentos $(l_1, l_2, \dots, l_m)$ de forma ascendente $(s_1, s_2, \dots, s_m)$, sendo $s_1 \leq s_2 \leq \dots \leq s_m$. Observe que existem tantos comprimentos quanto existem palavras. Para o comprimento s_1 , escolha um nó no nível s_1 para indicar o código. Para garantir a condição de prefixo, o nó escolhido deve se tornar um nó terminal, eliminando assim uma fração D^{-s_1} de nós terminais no nível $l_{\max}$. Em seguida, escolha um dos nós remanescentes no nível s_2 (neste momento, existem $(D^{s_1} - 1)D^{s_2-s_1}$ possíveis escolhas), eliminando assim uma fração D^{-s_2} de nós terminais no nível $l_{\max}$. A fração total de nós eliminados, até o momento, foi de $D^{-s_1} + D^{-s_2}$. Continuando este processo, iremos eliminar uma fração $\sum_{i=1}^m D^{-s_i}$ de nós, e neste processo estamos garantindo que estamos criando um código instantâneo (uma palavra de código não pode ser prefixo de outra). Como por suposição temos $\sum_{i=1}^m D^{-s_i} \leq 1$, podemos associar palavras de código, com comprimentos dados, a nós na árvore, no nível correspondente, eliminando todos os descendentes e, nesse processo, nunca iremos exaurir a árvore. Criamos assim um código de prefixo com os comprimentos desejados, satisfazendo Kraft.

□

A desigualdade de Kraft é uma condição *sine qua non* (necessária e indispensável) para a existência de um código prefixo. A condição dada na Equação (315) deve ser satisfeita para que um dado código com comprimentos $l_1, l_2, \dots, l_m$ seja um código de prefixo. A demonstração também nos mostra como construir um código de prefixo, dado um conjunto de comprimentos $\{l_1, l_2, \dots, l_m\}$ que satisfaça a Equação (315).

Teorema 38 (Desigualdade de Kraft para códigos infinitos contáveis)
Seja um conjunto infinito contável de palavras de código que formam um código de prefixo sobre um alfabeto D -ário. Então, os comprimen-

tos $\{\ell_i\}_{i=1}^{\infty}$ das palavras satisfazem a desigualdade de Kraft estendida:

$$\sum_{i=1}^{\infty} D^{-\ell_i} \leq 1. \quad (317)$$

Reciprocamente, dada uma sequência de comprimentos $\{\ell_i\}_{i=1}^{\infty}$ satisfazendo a desigualdade acima, existe um código de prefixo com esses comprimentos.

Demonstração. Seja um código de prefixo infinito contável sobre um alfabeto D -ário $\mathcal{D} = \{0, 1, \dots, D-1\}$. Denote a i -ésima palavra de código por $\mathbf{y}_i = y_{i1}y_{i2} \dots y_{il_i}$, onde l_i é o comprimento da palavra. Vamos provar a desigualdade de Kraft e a existência do código.

Prova da desigualdade de Kraft. Associamos cada palavra de código $\mathbf{y}_i = y_{i1}y_{i2} \dots y_{il_i}$ a um número real na forma de uma representação fracionária:

$$0.y_{i1}y_{i2} \dots y_{il_i} = \sum_{j=1}^{l_i} y_{ij} D^{-j}. \quad (318)$$

Este número define o ponto inicial de um intervalo semiaberto da forma $[0.y_{i1}y_{i2} \dots y_{il_i}, 0.y_{i1}y_{i2} \dots y_{il_i} + D^{-l_i})$ no intervalo unitário $[0, 1)$. O comprimento desse intervalo é exatamente D^{-l_i} .

Como o código é de prefixo, nenhuma palavra $\mathbf{y}_i$ é prefixo de outra palavra $\mathbf{y}_k$ ($i \neq k$). Isso implica que os intervalos associados às palavras são disjuntos, pois os números no intervalo de $\mathbf{y}_i$ começam com a sequência $y_{i1}y_{i2} \dots y_{il_i}$, que não é prefixo de nenhuma outra palavra. Como todos os intervalos estão contidos em $[0, 1)$, a soma dos seus comprimentos satisfaz:

$$\sum_{i=1}^{\infty} D^{-l_i} \leq 1. \quad (319)$$

A organização dos intervalos associados ao código de prefixo é exemplificada na Figura 33.

Exemplos ilustrativos. Considere um alfabeto binário ($D = 2$, $\mathcal{D} = \{0, 1\}$). Para a palavra 0.1, temos $0.1 = 1 \times 2^{-1} = \frac{1}{2}$, associada ao intervalo $[\frac{1}{2}, 1)$. Para a palavra 0.01, temos $0.01 = 0 \times 2^{-1} + 1 \times 2^{-2} = \frac{1}{4}$, associada ao intervalo $[\frac{1}{4}, \frac{1}{2})$. Para a palavra 0.001, temos $0.001 = 1 \times 2^{-3} = \frac{1}{8}$, associada ao intervalo $[\frac{1}{8}, \frac{1}{4})$.

Para um alfabeto decimal ($D = 10$, $\mathcal{D} = \{0, 1, \dots, 9\}$). A palavra 0.157 corresponde ao intervalo $[0.157, 0.158)$. A palavra 0.159 corresponde ao intervalo $[0.159, 0.160)$.

Construção do código de prefixo. Para a recíproca, fazemos a construção do código de forma similar ao caso finito. Dada uma sequência

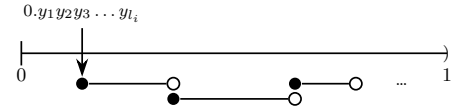


Figura 33: Ilustração dos intervalos disjuntos associados a palavras binárias de um código de prefixo, contidos no intervalo $[0, 1)$.

$\{l_i\}_{i=1}^{\infty}$ tal que $\sum_{i=1}^{\infty} D^{-l_i} \leq 1$, podemos construir um código de prefixo iterativamente. Para simplificar a construção, podemos ordenamos os comprimentos em ordem não decrescente. Comece escolhendo uma palavra de comprimento l_1 . Para cada $i \geq 2$, selecione uma palavra de comprimento l_i que não seja prefixo de nenhuma palavra já escolhida, o que é possível devido à restrição da soma dos comprimentos dos intervalos. Este processo garante a existência de um código de prefixo com os comprimentos especificados.

Portanto, a desigualdade de Kraft é necessária e suficiente para a existência de um código de prefixo infinito contável.

□

Em busca do código ótimo

Desejamos um código que forneça decodificação instantânea e livre de ambiguidade, além disso, queremos um código eficiente, que tenha comprimento esperado mínimo. Desta forma, estamos em busca de um código de prefixo ótimo.

O *comprimento esperado* é dado por

$$L(C) = \sum_i p_i l_i, \quad (320)$$

onde os comprimentos l_i 's devem ser números inteiros. Teremos então o seguinte problema de otimização com restrição:

$$\min_{\{l_{1:m}\} \in \mathbb{Z}_+^m} L(C) = \sum_i p_i l_i, \quad (321)$$

sujeito a

$$\sum_i D^{-l_i} \leq 1. \quad (322)$$

Este é um problema de programação em inteiros que é um problema cuja sua complexidade computacional cresce exponencialmente com o tamanho da entrada. Não seria assim possível resolvê-lo de forma eficiente. Vamos relaxar a condição de que l_i precisa ser inteiro e considerar o Lagrangiano

$$J = \sum_i p_i l_i + \lambda \left(\sum_i D^{-l_i} - 1 \right). \quad (323)$$

Para encontrar os pontos estacionários de J , considerado uma função dos l_i 's e do multiplicador lagrangiano λ , devemos encontrar as condições para que as derivadas se igualem a zero, ou seja, $\partial J / \partial l_i = 0$, $i = 1, \dots, m$, e $\partial J / \partial \lambda = 0$. Teremos assim

$$\frac{\partial J}{\partial l_i} = p_i - \lambda D^{-l_i} \ln D = 0, \quad (324)$$

e assim

$$D^{-l_i} = \frac{p_i}{\lambda \ln D}. \quad (325)$$

Teremos também

$$\frac{\partial J}{\partial \lambda} = \sum_i D^{-l_i} - 1 = 0, \quad (326)$$

que nos fornecerá

$$\lambda = 1/\ln D. \quad (327)$$

Utilizando as Equações (325) e (327) obtemos $D^{-l_i} = p_i$, e assim os comprimentos ótimos são dados por

$$l_i^* = -\log_D p_i. \quad (328)$$

Desta forma, o comprimento esperado do código ótimo será

$$L^* = \sum_i p_i l_i^* = -\sum_i p_i \log_D p_i = H_D(X) = H(X)/\log D. \quad (329)$$

Assumindo que seja possível utilizar comprimentos fracionários, os comprimentos ótimos para as palavras de um código são $l_i^* = -\log_D p_i$, isto implica que os comprimentos ótimos são iguais à informação do evento (o menor comprimento para codificação de um evento é inerentemente a informação sobre o próprio evento — princípio da Navalha de Occam²⁴), e assim o comprimento esperado do código será a entropia.

Apesar de os comprimentos ótimos, dados na Equação (328), não serem, a princípio, inteiros, podemos utilizar a codificação em blocos e assim aproximar do limite ótimo, mesmo quando os comprimentos dos códigos por símbolos são valores fracionários.

²⁴ A Navalha de Occam, princípio atribuído ao frade franciscano Guilherme de Ockham (século XIV), sugere que, entre explicações concorrentes, a mais simples — aquela que requer menos suposições — deve ser preferida, desde que explique os fatos adequadamente.

Teorema 39 (Comprimento esperado mínimo) *Entropia é o comprimento esperado mínimo. O comprimento esperado L de qualquer código D -ário instantâneo (satisfaz Kraft por conseguinte) para uma v.a. X é tal que*

$$L \geq H_D(X), \quad (330)$$

com igualdade sse $D^{-l_i} = p_i$.

Demonstração.

$$L - H_D(X) = \sum_i p_i l_i - \sum_i p_i \log_D 1/p_i \quad (331a)$$

$$= - \sum_i p_i \log_D D^{-l_i} + \sum_i p_i \log_D p_i \quad (331b)$$

$$= - \sum_i p_i \log_D D^{-l_i} + \underbrace{\log_D \left(\sum_i D^{-l_i} \right) - \log_D \left(\sum_i D^{-l_i} \right)}_{=0} + \sum_i p_i \log_D p_i \quad (331c)$$

$$= \sum_i p_i \log_D \frac{p_i}{r_i} - \log_D \left(\sum_i D^{-l_i} \right) \quad (331d)$$

$$= \underbrace{D(p || r)}_{\geq 0} + \underbrace{\log_D(1/c)}_{\geq 0} \quad (331e)$$

$$\geq 0 \quad (331f)$$

onde utilizamos em 331c, para simplificar, a soma de três das quatro parcelas, como é dada na Equação (332); e em 331f utilizamos que $c \leq 1$ pois a desigualdade de Kraft deve ser satisfeita, onde $c = \sum_i D^{-l_i}$. Vejamos a seguir a soma da primeira, segunda e quarta

parcelas da soma em 331c:

$$-\sum_i p_i \log_D D^{-l_i} + \log_D \left(\sum_j D^{-l_j} \right) + \sum_i p_i \log_D p_i =$$

(332a)

$$-\sum_i p_i \log_D D^{-l_i} + \left(\underbrace{\sum_i p_i}_{=1} \right) \log_D \left(\sum_i D^{-l_i} \right) + \sum_i p_i \log_D p_i =$$

(332b)

$$-\sum_i p_i \log_D D^{-l_i} + \sum_i p_i \log_D \left(\sum_j D^{-l_j} \right) + \sum_i p_i \log_D p_i =$$

(332c)

$$\sum_i p_i \log_D \frac{p_i \left(\sum_j D^{-l_j} \right)}{D^{-l_i}} =$$

(332d)

$$\sum_i p_i \log_D \frac{p_i}{\frac{D^{-l_i}}{\left(\sum_j D^{-l_j} \right)}} =$$

(332e)

$$\sum_i p_i \log_D \frac{p_i}{r_i},$$

(332f)

onde definimos

$$r_i = \frac{D^{-l_i}}{\sum_j D^{-l_j}}.$$

(333)

□

Assumindo que Kraft seja satisfeito (existe um código de prefixo com tais comprimentos) teremos comprimentos inteiros de forma que $El \geq H$. Se ainda $l_i = -\log_D p_i$ é inteiro, para todo i , então teremos a igualdade $El = H$. Se $l_i \neq -\log_D p_i$, mas l_i é inteiro, teremos estritamente $El > H$.

Código de Shannon

O *código de Shannon* é um código subótimo que consiste em considerar o comprimento como inteiro mais próximo do valor ótimo, ou seja,

$$l_i = \lceil \log_D 1/p_i \rceil.$$

(334)

Os comprimentos definidos desta forma satisfazem Kraft, uma vez que

$$\sum_i D^{-l_i} = \sum_i D^{-\lceil \log_D 1/p_i \rceil} \leq \sum_i D^{-\log_D 1/p_i} = \sum_i p_i = 1. \quad (335)$$

Pelo Teorema 37, existe então um código de prefixo com os comprimentos dados em 334.

Teorema 40 (Limites para o comprimento esperado do código de Shannon) *Sejam $l_1, l_2, \dots, l_m$ comprimentos inteiros do código de Shannon para uma fonte p e um alfabeto D -ário. L é o comprimento esperado. Então*

$$H_D(X) \leq L \leq H_D(X) + 1. \quad (336)$$

Demonstração. Para o código de Shannon, teremos os seguintes limites:

$$\log_D \frac{1}{p_i} \leq l_i \leq \log_D \frac{1}{p_i} + 1 \quad (337a)$$

$$E_p \left[\log_D \frac{1}{p_i} \right] \leq E_p [l_i] \leq E_p \left[\log_D \frac{1}{p_i} + 1 \right] \quad (337b)$$

$$\sum_i p_i \log_D \frac{1}{p_i} \leq L \leq \sum_i p_i \left(\log_D \frac{1}{p_i} + 1 \right) \quad (337c)$$

$$H_D(X) \leq L \leq H_D(X) + 1. \quad (337d)$$

□

Observe ainda que, $H \leq L^* \leq L$, onde L^* é o comprimento ótimo para códigos de comprimento inteiro. Desta forma, também teremos a seguinte relação para o comprimento esperado de código ótimo:

$$H_D(X) \leq L^* \leq H_D(X) + 1.$$

Podemos melhorar a eficiência do código de Shannon codificando mais de um símbolo por vez (codificação em bloco). Vamos chamar de L_n o comprimento esperado por símbolo de uma sequência de n símbolos $x_{1:n}$. Então,

$$L_n = \frac{1}{n} \sum_{x_{1:n} \in \mathcal{X}^n} p(x_{1:n}) l(x_{1:n}) = \frac{1}{n} El(x_{1:n}). \quad (338)$$

utilizando os comprimentos do código de Shannon, teremos

$$\log 1/p_i \leq l_i \leq \log 1/p_i + 1 \quad (339a)$$

$$\sum_i p_i \log 1/p_i \leq \sum_i p_i l_i \leq \sum_i p_i (\log 1/p_i + 1) \quad (339b)$$

$$H(X_1, \dots, X_n) \leq El(X_{1:n}) \leq H(X_1, \dots, X_n) + 1 \quad (339c)$$

$$H(X) \leq L_n \leq H(X) + \frac{1}{n}, \quad (339d)$$

onde utilizamos que X_i são i.i.d., então $H(X_1, \dots, X_n) = nH(X_i)$. Quando n cresce, a penalidade por símbolo imposta ao código de Shannon diminui e conseguiremos assim aproximar-nos do limite da Entropia (por símbolo), embora teremos que adotar a estratégia de codificação em blocos.

A escolha feita na Equação (334) para definir o comprimento das palavras no código de Shannon, subentende o conhecimento da distribuição p . Entretanto, em geral, a distribuição subjacente não é conhecida. Isso implica que existem erros nos cálculos. Suponha então que o código de Shannon utilize $l(x) = \lceil \log 1/q(x) \rceil$, mas a real distribuição é $p(x) \neq q(x)$. Vamos recalculer o valor esperado do comprimento do código, agora utilizando esta informação. Encontraremos novos limites para o comprimento esperado do código de Shannon.

Teorema 41 (Limites para o código de Shannon com a distribuição aproximada) *O comprimento esperado de um código sob a distribuição $p(x)$ com $l(x) = \lceil \log 1/q(x) \rceil$ satisfaz*

$$H(p) + D(p \parallel q) \leq E_p l(X) \leq H(p) + D(p \parallel q) + 1. \quad (340)$$

Demonstração. Vamos analisar primeiramente o limite superior.

$$El(X) = \sum_x p(x) \lceil \log 1/q(x) \rceil \quad (341a)$$

$$\leq \sum_x p(x) \left(\log \frac{1}{q(x)} + 1 \right) \quad (341b)$$

$$= \sum_x p(x) \left(\log \frac{p(x)}{q(x)} \frac{1}{p(x)} + 1 \right) \quad (341c)$$

$$= \sum_x p(x) \log \frac{p(x)}{q(x)} + \sum_x p(x) \log \frac{1}{p(x)} + 1 \quad (341d)$$

$$= D(p \parallel q) + H(p) + 1. \quad (341e)$$

E agora, para o limite inferior temos

$$El(X) = \sum_x p(x) \lceil \log 1/q(x) \rceil \quad (342a)$$

$$\geq \sum_x p(x) \log 1/q(x) \quad (342b)$$

$$= \sum_x p(x) \left(\log \frac{p(x)}{q(x)} \frac{1}{p(x)} \right) \quad (342c)$$

$$= \sum_x p(x) \log \frac{p(x)}{q(x)} + \sum_x p(x) \log \frac{1}{p(x)} \quad (342d)$$

$$= D(p \parallel q) + H(p). \quad (342e)$$

□

Comparando estes limites com aqueles dados na Equação (337), verificamos que $D(p \parallel q)$ é o custo adicional por símbolo causado por utilizar a distribuição errada.

Um aspecto importante na análise de códigos é entender como seus comprimentos de palavras se comparam a outros códigos univocamente decodificáveis em termos de eficiência. O teorema a seguir aborda essa questão, fornecendo uma cota probabilística para a diferença entre os comprimentos de palavras do código de Shannon e de um código concorrente. Essa cota, expressa em função de uma constante c , destaca a robustez do código de Shannon, mostrando que a probabilidade de seus comprimentos excederem significativamente os de outro código decresce exponencialmente, um resultado que reforça sua relevância.

Teorema 42 (Otimidade competitiva do código de Shannon) *Considere $l(x)$ como o comprimento da palavra associada ao símbolo x no código de Shannon e $l'(x)$ como o comprimento da palavra correspondente em outro código univocamente decodificável, ambos definidos sobre uma variável aleatória X com distribuição p . Então, a probabilidade de que $l(X)$ exceda $l'(X)$ por pelo menos uma constante c é limitada por:*

$$\Pr(l(X) \geq l'(X) + c) \leq \frac{1}{2^{c-1}}. \quad (343)$$

Equivalentemente, em notação mais formal:

$$\Pr(\mathbf{1}_{\{l(X) \geq l'(X) + c\}} = 1) \leq \frac{1}{2^{c-1}}, \quad (344)$$

onde $\mathbf{1}_{\{\cdot\}}$ é a função indicadora.

Demonstração.

$$\Pr(l(X) \geq l'(X) + c) = \Pr(\lceil \log 1/p(X) \rceil \geq l'(X) + c) \quad (345a)$$

$$\leq \Pr(\log 1/p(X) \geq l'(X) + c - 1) \quad (345b)$$

$$= \Pr(p(X) \leq 2^{-l'(X) - c + 1}) \quad (345c)$$

$$= \sum_{x: p(x) \leq 2^{-l'(x) - c + 1}} p(x) \quad (345d)$$

$$\leq \sum_{x: p(x) \leq 2^{-l'(x) - c + 1}} 2^{-l'(x) - c + 1} \quad (345e)$$

$$\leq \sum_x 2^{-l'(x)} 2^{-(c-1)} \leq 2^{-(c-1)}, \quad (345f)$$

onde em 345b utilizamos que o código de Shannon utiliza $l(x) = \lceil \log 1/p(x) \rceil$, e, em 345f, utilizamos a desigualdade de Kraft: $\sum_x 2^{-l'(x)} \leq 1$.

□

A probabilidade do código de Shannon ter comprimento esperado maior do que um outro código unicamente decodificável é exponencialmente decrescente com $c > 1$. Código de Shannon é ótimo para distribuições d -ádicas, já que neste caso teremos que $\log 1/p(x)$ será inteiro.

Em distribuições d -ádicas, é mais provável que os comprimentos das palavra de um código de Shannon sejam menores do que maiores em comparação com um código alternativo. Esse resultado evidencia uma vantagem probabilística do código de Shannon, destacando sua eficiência relativa.

Teorema 43 (Otimidade competitiva do código de Shannon II)
Considere uma distribuição d -ádica p sobre um conjunto de símbolos, onde $p(x) = 1/d^k$ para algum inteiro k , e seja $l(x) = \log_d(1/p(x))$ o comprimento da palavra associada a x no código de Shannon, enquanto $l'(x)$ é o comprimento da palavra correspondente em outro código prefixo, ambos aplicados a uma variável aleatória X com distribuição p . Então:

$$\Pr(l(X) < l'(X)) \geq \Pr(l(X) > l'(X)), \quad (346)$$

com igualdade se, e somente se, $l'(x) = l(x)$ para todo x .

Demonstração. Seja

$$\text{sign}(t) = \begin{cases} 1, & t > 0 \\ 0, & t = 0 \\ -1, & t < 0 \end{cases} \quad (347)$$

É fácil verificar que $\text{sign}(t) \leq 2^t - 1$ para $t = 0, \pm 1, \pm 2, \dots$. Para $t = 0$ teremos $0 = 2^0 - 1 = 0$; para $t \geq 1$ teremos $1 \leq 2^t - 1$, uma vez que $2^t > 1$ para $t \geq 1$; e para $t \leq -1$ teremos $-1 \leq 2^t - 1$, uma vez que $2^t < 1$ para $t \leq -1$.

Teremos então

$$\Pr(l'(X) < l(X)) - \Pr(l'(X) > l(X)) = \sum_{x: l'(x) \leq l(x)} p(x) - \sum_{x: l'(x) > l(x)} p(x) \quad (348a)$$

$$= \sum_x p(x) \text{sign}(l(x) - l'(x)) \quad (348b)$$

$$= E[\text{sign}(l(X) - l'(X))] \quad (348c)$$

$$\leq \sum_x p(x) (2^{l(x) - l'(x)} - 1) \quad (348d)$$

$$= \sum_x 2^{-l(x)} \left(2^{l(x)-l'(x)} - 1 \right) \quad (348e)$$

$$= \sum_x 2^{-l'(x)} - \sum_x 2^{-l(x)} \quad (348f)$$

$$= \sum_x 2^{-l'(x)} - 1 \quad (348g)$$

$$\leq 1 - 1 \quad (348h)$$

$$= 0, \quad (348i)$$

onde em 348b utilizamos que $\{l'(x) \leq l(x)\} \cap \{l'(x) > l(x)\} = \emptyset$, em 348e utilizamos que $p(x)$ é d-ádica, $p(x) = 2^{-l(x)}$, e em 348h utilizamos que $l'(x)$ satisfaz Kraft. Assim, mostramos que $\Pr(l'(X) < l(X)) \leq \Pr(l'(X) > l(X))$, como desejado. $\square$

Desigualdade de Kraft revisitada

Considerando como objetivo principal, encontrar um código unívoco (unicamente decodificável) com comprimento esperado de palavra mínimo, ao observar as classes de códigos, conforme apresentadas na Figura 31, podemos imaginar que a classe mais ampla é a mais provável de se encontrar tal código desejado. Potencialmente, dentre os códigos unicamente decodificáveis (um conjunto maior), podemos encontrar um código com menor comprimento esperado. Entretanto, o teorema a seguir mostra que a desigualdade de Kraft é também condição *sine qua non* para obtermos um código unicamente decodificável. Desta forma, podemos concluir que um código prefixo é a melhor escolha.

Primeiramente, vamos apresentar o seguinte lemma.

Lema 44 (Expansão multinomial) *Seja $\mathcal{X} = \{x_1, x_2, \dots, x_m\}$, $|\mathcal{X}| = m$, temos*

$$\left(\sum_{x \in \mathcal{X}} D^{-l(x)} \right)^k = \sum_{x_1, \dots, x_k \in \mathcal{X}^k} D^{-l(x_1)} D^{-l(x_2)} \dots D^{-l(x_k)}. \quad (349)$$

Demonstração. A demonstração utiliza o *Teorema Multinomial* para expandir a soma, revelando sua equivalência à soma combinatória das

contribuições de cada sequência.

$$\left(\sum_{x \in \mathcal{X}} D^{-l(x)} \right)^k = \left(\sum_{i=1}^m D^{-l(x_i)} \right)^k \quad (350a)$$

$$\begin{aligned} &= \left(D^{-l(x_1)} + D^{-l(x_2)} + \dots + D^{-l(x_m)} \right)^k \\ &= \sum_{\substack{n_1, n_2, \dots, n_m \geq 0 \\ n_1 + n_2 + \dots + n_m = k}} \frac{k!}{n_1! n_2! \dots n_m!} D^{-l(x_1)n_1} \dots D^{-l(x_m)n_m} \end{aligned} \quad (350b)$$

$$= \sum_{x_{1:k} \in \mathcal{X}^k} D^{-l(x_1)} D^{-l(x_2)} \dots D^{-l(x_k)}. \quad (350c)$$

Cada termo $(D^{-l(x_1)})^{n_1} (D^{-l(x_2)})^{n_2} \dots (D^{-l(x_m)})^{n_m}$ pode ser visto como o produto de k fatores $D^{-l(x_i)}$, onde x_i é escolhido n_i vezes. Isso corresponde exatamente a somar $D^{-l(x_1)} D^{-l(x_2)} \dots D^{-l(x_k)}$ sobre todas as sequências possíveis $x_{1:k} = (x_1, x_2, \dots, x_k)$ em $\mathcal{X}^k$:

$$\sum_{x_{1:k} \in \mathcal{X}^k} D^{-l(x_1)} D^{-l(x_2)} \dots D^{-l(x_k)}$$

Para cada sequência $x_{1:k}$, o produto reflete uma combinação específica dos termos $D^{-l(x_i)}$, e a soma cobre todas as m^k possibilidades. □

Teorema 45 (Desigualdade de Kraft para códigos unicamente decodificáveis) *O comprimento de palavras de qualquer código unicamente decodificável (não necessariamente instantâneo) deve satisfazer a desigualdade de Kraft $\sum_i D^{-l_i} \leq 1$. Por outro lado, dado um conjunto de comprimentos que satisfazem Kraft, é possível construir um código unicamente decodificável.*

Demonstração. A proposição inversa já foi previamente mostrada, uma vez que mostramos como construir um código instantâneo utilizando um conjunto de comprimentos satisfazendo Kraft.

Dado um código unicamente decodificável (não necessariamente instantâneo) com comprimentos $l(x)$ e com extensão com comprimentos dados por $l(x_1, \dots, x_k) = \sum_{i=1}^k l(x_i)$, queremos mostrar que $\sum_x D^{-l(x)} \leq 1$. Vamos definir $S = \sum_{x \in \mathcal{X}} D^{-l(x)}$ e então teremos

$$S^k = \left(\sum_{x \in \mathcal{X}} D^{-l(x)} \right)^k \quad (351a)$$

$$= \sum_{x_{1:k} \in \mathcal{X}^k} D^{-l(x_1)} D^{-l(x_2)} \dots D^{-l(x_k)} \quad (351b)$$

$$= \sum_{x_{1:k} \in \mathcal{X}^k} D^{-(\sum_{i=1}^k l(x_i))} \quad (351c)$$

$$= \sum_{x_{1:k} \in \mathcal{X}^k} D^{-l(x_{1:k})} \quad (351d)$$

$$= \sum_{m=1}^{kl_{\max}} a(m) D^{-m} \quad (351e)$$

onde em 351b utilizamos o resultado do Lema 44 e $l_{\max} = \max_x l(x)$, $a(m)$ é o número de sequências $x_{1:k}$ mapeadas em palavras de comprimento m , i.e.,

$$a(m) = |\{x_{1:k} \in \mathcal{X}^k : l(x_{1:k}) = m\}|. \quad (352)$$

Existem D^m palavras de comprimento m , e cada uma delas pode ter no máximo uma sequência da fonte associada, já que o código é unicamente decodificável. Então $a(m) \leq D^m$, e assim

$$\underbrace{S^k}_{\text{exponencial em } k} = \sum_{m=1}^{kl_{\max}} a(m) D^{-m} \quad (353a)$$

$$\leq \sum_{m=1}^{kl_{\max}} D^m D^{-m} = \underbrace{kl_{\max}}_{\text{polinomial em } k} \quad \forall k \quad (353b)$$

Isto só pode ser verdade para todo k se $S \leq 1$. Teremos então

$$S = \sum_{x \in \mathcal{X}} D^{-l(x)} \leq 1. \quad (354)$$

□

Concluimos assim que todo código unicamente decodificável deve satisfazer a desigualdade de Kraft. Sabemos também que é possível construir um código instantâneo com os comprimentos que satisfazem Kraft. Logo, não há razão em se buscar um código unicamente decodificável que não seja um código de prefixo, afinal para um mesmo comprimento esperado podemos construir um código de prefixo.

Código de Huffman

Os códigos de comprimento variável buscam minimizar o comprimento esperado, para tanto, atribuem comprimentos menores a símbolos mais

frequentes em detrimento de códigos maiores para símbolos menos frequentes.

Antes de 1952, as abordagens predominantes para esses códigos eram *top-down*, partindo de uma estrutura inicial e ajustando comprimentos de forma descendente, mas nem sempre resultavam em soluções ótimas. David A. Huffman, em 1952, utilizou uma abordagem inovadora: construir a árvore binária de maneira *bottom-up*, combinando iterativamente os símbolos de menor probabilidade. Esse método garante um código prefixo com comprimento esperado mínimo, respeitando a desigualdade de Kraft e aproximando-se do limite teórico da entropia de Shannon.

Vejamoinicialmente um exemplo de utilização da abordagem gulosa *top-down* para criar um código. A abordagem consiste em: começar pelo topo e realizar divisões com entropia máxima para determinar os símbolos. Em cada etapa dividimos os símbolos em dois grupos, de forma que cada etapa tenha entropia máxima no processo de determinação dos símbolos.

Exemplo 28 (Método Guloso *top-down* para codificação (Bilmes 2013)). Considere uma variável aleatória X com distribuição

	a	b	c	d	e	f	g
p	0.01	0.24	0.05	0.20	0.47	0.01	0.02

Em cada etapa, queremos dividir os símbolos em dois grupos de forma a maximizar a entropia nesta divisão. A pergunta Y_1 para determinação (ou codificação) de X , é aquela que reduz a entropia residual sobre X .

$$H(X|Y_1) = H(X, Y_1) - H(Y_1) \quad (355a)$$

$$= H(X) - H(Y_1) \quad (355b)$$

onde utilizamos o fato de que Y_1 é uma função de X , e assim $H(X, Y_1) = H(X)$. Escolhemos a pergunta Y_1 com maior informação mútua com X :

$$I(Y_1; X) = H(X) - H(X|Y_1) = H(Y_1), \quad (356)$$

e desta forma buscaremos minimizar a incerteza residual sobre X , dado Y_1 .

Considere a seguinte partição $\{a, b, c, d, e, f, g\} = \{a, b, c, d\} \cup \{e, f, g\}$. Com este particionamento, obteremos entropia máxima pois $p(X \in \{a, b, c, d\}) = p(X \in \{e, f, g\}) = 0.5$. Este particionamento corresponde à v.a. $Y_1 = \mathbf{1}_{\{X \in \{e, f, g\}\}}$, e teremos $H(Y_1) = 1$, valor máximo, reduzindo assim $H(X|Y_1)$.

Para o caso $Y_1 = 0$, podemos agora particionar $\{a, b, c, d\} = \{a, b\} \cup \{c, d\}$, uma vez que $p(\{a, b\}) = p(\{c, d\}) = 1/4$. Desta forma, $p(X \in$

$\{a, b\} \mid X \in \{a, b, c, d\} = 1/2$ e $p(X \in \{c, d\} \mid X \in \{a, b, c, d\}) = 1/2$. Este particionamento corresponde a $Y_2 = \mathbf{1}_{\{X \in \{c, d\}\}}$. Obtemos assim $H(Y_2 \mid Y_1 = 0) = 1$, novamente alcançando o máximo.

Para o caso $Y_1 = 1$ precisaremos particionar $\{e, f, g\}$. As opções são listadas a seguir:

	I	II	III
partição	$(\{e\}, \{f, g\})$	$(\{e, f\}, \{g\})$	$(\{e, g\}, \{f\})$
prob	(0.47, 0.03)	(0.48, 0.02)	(0.49, 0.01)
$H(Y_2 \mid Y_1 = 1)$	0.3274	0.2423	0.1414

A partição I é aquela que fornece maior entropia.

Tais escolhas nos levam a maximizar $H(Y_2 \mid Y_1)$ e por conseguinte iremos minimizar a incerteza remanescente sobre X :

$$H(X \mid Y_2, Y_1) = H(X, Y_2 \mid Y_1) - H(Y_2 \mid Y_1) \quad (357a)$$

$$= H(X \mid Y_1) - H(Y_2 \mid Y_1) \quad (357b)$$

$$= H(X) - H(Y_2 \mid Y_1) - H(Y_1). \quad (357c)$$

A cada passo escolhemos o que nos parece melhor, as escolhas futuras terão que lidar com as possibilidades que restaram devido às escolhas passadas. Teremos assim os seguintes particionamentos:

conjunto	partição	probabilidades	entropia condicional
$\{a, b, c, d, e, f, g\}$	$\{a, b, c, d\}, \{e, f, g\}$	(0.5, 0.5)	$H(Y_1) = 1$
$\{a, b, c, d\}$	$\{a, b\}, \{c, d\}$	(0.25, 0.25)	$H(Y_2 \mid Y_1 = 0) = 1$
$\{e, f, g\}$	$\{e\}, \{f, g\}$	(0.47, 0.03)	$H(Y_2 \mid Y_1 = 1) = 0.3274$
$\{a, b\}$	$\{a\}, \{b\}$	(0.01, 0.24)	$H(Y_3 \mid Y_2 = 0, Y_1 = 0) = 0.2423$
$\{c, d\}$	$\{c\}, \{d\}$	(0.05, 0.20)	$H(Y_3 \mid Y_2 = 1, Y_1 = 0) = 0.7219$
$\{e\}$	$\{e\}$	(0.47)	$H(Y_3 \mid Y_2 = 0, Y_1 = 1) = 0$
$\{f, g\}$	$\{f\}, \{g\}$	(0.01, 0.02)	$H(Y_3 \mid Y_2 = 1, Y_1 = 1) = 0.9183$

Ao final, seguindo a abordagem gulosa *top-down* obtemos o código representado na Figura 34. Este código possui comprimento esperado $El = 2.53$ e eficiência $H/El = 0.7638$, uma vez que a entropia é $H = 1.9323$ bits.

A título de comparação, o algoritmo de Huffman produz o código apresentado na Figura 35. O código de Huffman terá comprimento esperado $El_{\text{huffman}} = 1.97$ e eficiência $H/El_{\text{huffman}} = 0.9809$.

Isto demonstra claramente que o código de Huffman é superior e que a estratégia gulosa não foi capaz de nos levar a um código ótimo.

O algoritmo de Huffman pode ser sumarizado através dos seguintes passos:

1. Selecione os dois símbolos menos prováveis no alfabeto.
2. Ambos terão associados as palavras mais longas e se diferirão no último bit.
3. Combine estes dois símbolos em um símbolo auxiliar com probabilidade igual à soma das probabilidades dos dois símbolos. Adicione

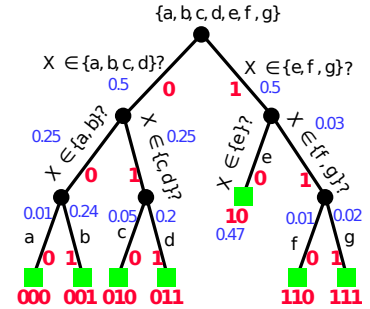


Figura 34: Árvore obtida através do método guloso seguindo a estratégia *top-down* (Bilmes 2013).

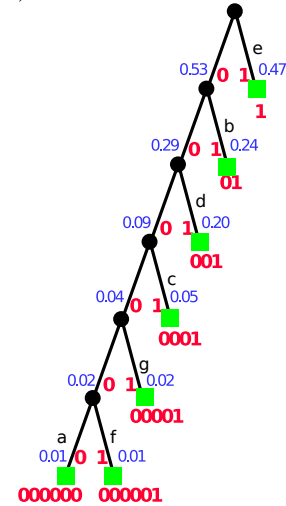


Figura 35: Árvore obtida pelo código de Huffman (Bilmes 2013).

o símbolo auxiliar e remova os dois símbolos previamente selecionados. Repita os passos.

O procedimento é similar para $D > 2$. Neste caso poderá ser necessário utilizar símbolos fictícios no alfabeto.

Exemplo 29 ((Bilmes 2013)). Seja $\mathcal{X} = \{1, 2, 3, 4, 5\}$ com probabilidades $(1/4, 1/4, 1/5, 3/20, 3/20)$. O procedimento e o código de Huffman obtido estão dispostos na Exemplo 29.

$\log \frac{1}{p(x)}$	l	palavra	x	prob	passo1	prob	passo2	prob	passo3	prob	passo4	prob
2.0	2	00	1	0.25	—	0.25	—	0.25	0	0.55	0	1.0
2.0	2	10	2	0.25	—	0.25	0	0.45	—	0.45	1	1.0
2.3	2	11	3	0.2	—	0.2	1	—	—	—	—	—
2.7	3	010	4	0.15	0	0.3	—	0.3	1	—	—	—
2.7	3	011	5	0.15	1	—	—	—	—	—	—	—

Figura 36: Código de Huffman exemplo (Bilmes 2013).

Para este código de Huffman teremos $El = 2.3$ bits e $H = 2.2855$ bits. Note que, algumas palavras são maiores e outras menores que $l^*(x) = I(X) = \log 1/p(x)$. O código de Huffman encontrado pode ser expresso, de maneira compacta, como: $((2, 3), (1, (4, 5)))$, onde cada par de parênteses representa os símbolos combinados em cada passo do algoritmo.

O lema a seguir descreve atributos necessários para códigos de prefixo ótimos, relacionando a frequência dos símbolos à estrutura de suas palavras. Ele assegura que símbolos mais prováveis tenham comprimentos menores ou iguais e que as duas palavras mais longas, ligadas aos símbolos menos frequentes, possuam comprimentos iguais e difiram apenas no último bit. Essas propriedades, intrínsecas a códigos ótimos, refletem a construção *bottom-up* do algoritmo de Huffman.

Lema 46 (Propriedades de códigos instantâneos ótimos) *Para toda distribuição, $\exists$ um código instantâneo ótimo (i.e., com comprimento esperado mínimo) satisfazendo simultaneamente:*

1. Se $p_j > p_k$ então $l_j \leq l_k$ (i.e., o símbolo mais provável não possui palavra com maior comprimento).
2. As duas maiores palavras possuem o mesmo comprimento.
3. As duas maiores palavras diferem apenas no último bit e correspondem aos dois símbolos menos prováveis.

Demonstração. Vejamos primeiramente a propriedade 1.

Suponha que C_m seja um código ótimo (então $L(C_m)$ é mínimo) e escolha j, k de forma tal que $p_j > p_k$. Precisamos mostrar que $\exists$ um código com $l_j \leq l_k$. Considere o código C'_m com a troca das palavras j e k , ou seja,

$$l'_j = l_k \quad \text{e} \quad l'_k = l_j, \quad (358)$$

o que só pode tornar o código maior, então $L(C'_m) \geq L(C_m)$.

Realizando a troca, como $L(C_m)$ é mínimo, teremos

$$0 \leq L(C'_m) - L(C_m) = \sum_i p_i l'_i - \sum_i p_i l_i \quad (359a)$$

$$= p_j l'_j + p_k l'_k - p_j l_j - p_k l_k = p_j l_k + p_k l_j - p_j l_j - p_k l_k \quad (359b)$$

$$= p_j (l_k - l_j) - p_k (l_k - l_j) \quad (359c)$$

$$= \underbrace{(p_j - p_k)}_{>0} (l_k - l_j) \quad (359d)$$

Devemos ter então $(l_k - l_j) \geq 0$, ou seja, $l_k \geq l_j$ quando $p_j > p_k$, satisfazendo assim a propriedade 1. Esta propriedade é verdadeira para todos códigos ótimos.

Vejam agora a propriedade 2: as palavras mais longas possuem o mesmo comprimento. Se as duas palavras maiores não possuem o mesmo comprimento, podemos apagar o último bit da palavra mais longa. Desta forma manteremos a propriedade de prefixo, já que a palavra mais longa é a única com o seu dado comprimento e não existe um prefixo dela que seja uma outra palavra (veja a ??). Desta forma

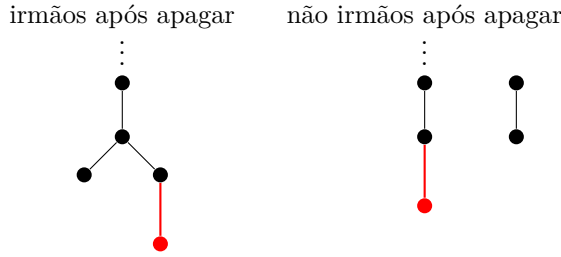


Figura 37: Apagando o último bit da palavra mais longa.

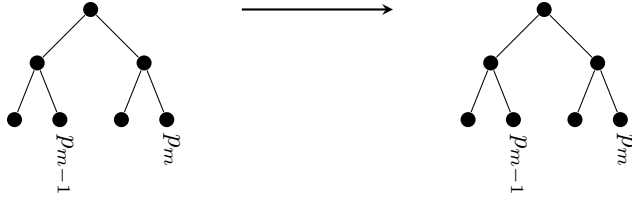
reduzimos o comprimento esperado. Concluimos que um código ótimo deve possuir as duas palavras mais longas com o mesmo comprimento.

Analisaremos agora a propriedade 3: as duas palavras mais longas diferem apenas no último bit e correspondem aos símbolos menos prováveis. Devido à propriedade 1 ($p_k < p_j \Rightarrow l_k \geq l_j$), se p_k é a menor probabilidade, então ela deve possuir associada uma palavra de comprimento que não seja menor do que qualquer outra j com $p_j > p_k$. De forma similar, se p_k é a segunda menor probabilidade, a palavra associada deve possuir comprimento que não seja menor do que qualquer outro símbolo mais provável. As duas palavras mais longas possuem o mesmo comprimento (propriedade 2) e correspondem aos dois símbolos menos prováveis. Se as duas maiores palavras não são irmãs, podemos trocá-las. Isto é, se $p_1 \geq p_2 \geq \dots \geq p_m$, então fazemos a transposição ilustrada na ??.

Isto não altera o comprimento esperado $L = \sum_i p_i l_i$.

□

O Teorema 46 nos fornece que, se $p_1 \geq p_2 \geq \dots \geq p_m$, então existe um código ótimo com $l_1 \leq l_2 \leq \dots \leq l_{m-1} = l_m$ e no qual $C(x_{m-1})$ e



diferem apenas
no último bit

Figura 38: Transposição das
duas palavras mais longas.

$C(x_m)$ diferem apenas no último bit.

Vamos mostrar que Huffman é ótimo através da operação de criar um novo código no qual a otimização é mais simples. Este processo é repetido até que a otimização seja trivial.

Teorema 47 (Otimidade do código de Huffman) *O procedimento de codificação de Huffman cria um código com comprimentos inteiros ótimo.*

Demonstração. Assuma um código C_m (não necessariamente ótimo) que satisfaz as propriedades anteriores. C_m terá as seguintes palavras de código $\{w_i\}_{i=1}^m$. O procedimento de Huffman consiste em transformar C_m em C_{m-1} , com palavras de código $\{w'_i\}_{i=1}^{m-1}$. Para o código C_m teremos as seguintes palavras, comprimento e probabilidades associados:

C_m	comprimento	prob. do símbolo
w_1	l_1	p_1
w_2	l_2	p_2
$\vdots$	$\vdots$	$\vdots$
w_{m-2}	l_{m-2}	p_{m-2}
w_{m-1}	l_{m-1}	p_{m-1}
w_m	l_m	p_m

Para ir de C_m para C_{m-1} , o procedimento de Huffman combina dos dois símbolos de menor probabilidades em C_m , criando um símbolo auxiliar em C_{m-1} com probabilidade dada pela soma $p'_{m-1} = p_{m-1} + p_m$. A transformação de C_m em C_{m-1} é esquematizada a seguir:

prob. símb.	C_{m-1}	comp. $m-1$	rel. código	rel. comp.	prob. símb.
p_1	w'_1	l'_1	$w_1 = w'_1$	$l_1 = l'_1$	p_1
p_2	w'_2	l'_2	$w_2 = w'_2$	$l_2 = l'_2$	p_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
p_{m-2}	w'_{m-2}	l'_{m-2}	$w_{m-2} = w'_{m-2}$	$l_{m-2} = l'_{m-2}$	p_{m-2}
$p_{m-1} + p_m$	w'_{m-1}	l'_{m-1}	$w_{m-1} = w'_{m-1}0$	$l_{m-1} = l'_{m-1} + 1$	p_{m-1}
			$w_m = w'_{m-1}1$	$l_m = l'_{m-1} + 1$	p_m

onde w_i e l_i são as palavras e comprimento das palavras do código C_m e w'_i e l'_i do código C_{m-1} .

As palavras e comprimentos em Huffman são definidos recursivamente. Huffman define uma relação entre palavras (e consequentemente comprimentos) ao dar um passo de um código para outro mais simples.

Obtemos assim a seguinte relação entre o comprimento esperado de C_m e o comprimento esperado de C_{m-1} :

$$L(C_m) = \sum_i p_i l_i \quad (360a)$$

$$= \sum_{i=1}^{m-2} p_i l'_i + p_{m-1} (l'_{m-1} + 1) + p_m (l'_{m-1} + 1) \quad (360b)$$

$$= \sum_{i=1}^{m-2} p_i l'_i + (p_{m-1} + p_m) l'_{m-1} + p_{m-1} + p_m \quad (360c)$$

$$= \sum_{i=1}^{m-1} p'_i l'_i + p_{m-1} + p_m \quad (360d)$$

$$= L(C_{m-1}) + p_{m-1} + p_m \quad (360e)$$

onde o termo $p_{m-1} + p_m$ é constante, não envolve comprimentos de palavras e, consequentemente, não tem nenhuma implicância no processo de otimização.

Desta forma, reduzimos o número de variáveis que iremos otimizar de m para $m - 1$. O procedimento de Huffman implica em

$$\min_{l_{1:m}} L(C_m) = \text{const.} + \min_{l_{1:m-1}} L(C_{m-1}) \quad (361a)$$

$$= \text{const.} + \min_{l_{1:m-2}} L(C_{m-2}) \quad (361b)$$

$$\vdots$$

$$= \text{const.} + \min_{l_{1:2}} L(C_2) \quad (361c)$$

onde em cada passo são preservadas as propriedades.

Ao reduzir para o caso com dois comprimentos (C_2), teremos a solução óbvia, um bit para cada símbolo, e assim podemos refazer o caminho de volta e construir o código. Em cada passo garantimos que estaremos obtendo a solução ótima, e assim o código criado pelo algoritmo de Huffman será ótimo.

□

Codificação Shannon-Fano-Elias

A *codificação Shannon-Fano-Elias* é uma abordagem prática para representar símbolos com base em suas probabilidades cumulativas.

Ela atribui códigos prefixo aproximando os comprimentos ideais por meio de uma representação binária truncada da função de distribuição cumulativa. Historicamente, Shannon-Fano-Elias é um marco na evolução da codificação, sendo ela predecessora da codificação aritmética, que também usa probabilidades cumulativas.

Vejamos algumas definições que serão utilizadas na codificação Shannon-Fano-Elias.

Definição 43 (Distribuição cumulativa). Seja $\mathcal{X} = \{x_1, x_2, \dots, x_m\}$ um conjunto finito de símbolos com uma distribuição p , onde $p(x_i) \geq 0$ e $\sum_{i=1}^m p(x_i) = 1$. A função de *distribuição cumulativa*, denotada por $F(x)$, é definida como:

$$F(x) = \sum_{a \leq x} p(a), \quad (362)$$

em que a soma é tomada sobre todos os símbolos $a \in \mathcal{X}$ tais que $a \leq x$, considerando a ordenação de $\mathcal{X}$: $x_1 < x_2 < \dots < x_m$. Assim, $F(x)$ representa a probabilidade acumulada de todos os símbolos até x , inclusive, e satisfaz as propriedades: $0 \leq F(x) \leq 1$, $F(x_i) \leq F(x_j)$ se $x_i < x_j$ (monotonicidade), e $F(x_m) = 1$.

Definição 44 ($\bar{F}$ -Distribuição cumulativa modificada). Seja $\mathcal{X} = \{x_1, x_2, \dots, x_m\}$ um conjunto finito de símbolos com uma distribuição p , e seja $F(x) = \sum_{a \leq x} p(a)$ a função de distribuição cumulativa associada, considerando uma ordenação fixa $x_1 < x_2 < \dots < x_m$. A função $\bar{F}(x)$, chamada de *distribuição cumulativa modificada*, é definida como:

$$\bar{F}(x) \triangleq \sum_{a < x} p(a) + \frac{1}{2}p(x), \quad (363a)$$

$$= F(x) - \frac{1}{2}p(x), \quad (363b)$$

em que $\sum_{a < x} p(a)$ é a soma das probabilidades dos símbolos estritamente menores que x , e $\frac{1}{2}p(x)$ adiciona metade da probabilidade do símbolo x . Assim, $\bar{F}(x)$ representa um ponto intermediário no intervalo de probabilidade associado a x na distribuição acumulada.

$\bar{F}(x)$ é o ponto entre $F(x-1)$ e $F(x)$, então, como $p(x) > 0$, temos

$$F(x-1) < \bar{F}(x) < F(x) \quad (364)$$

Como $p(x) > 0$, $a \neq b \Rightarrow F(a) \neq F(b) \Leftrightarrow \bar{F}(a) \neq \bar{F}(b)$. Podemos utilizar $\bar{F}(a)$ como um código não singular para a (a expansão binária após a vírgula pode ser utilizada, como visto anteriormente na prova de Kraft para um infinito contável de comprimentos, veja o Teorema 38). Teremos um código unicamente decodificável, porém poderemos ter palavras de tamanho infinito. A solução será truncar $\bar{F}(x)$ para ficar com $l(x)$ bits. A notação para tanto será $\lfloor \bar{F}(x) \rfloor_{l(x)}$.

Exemplo 30 (Truncamento com l bits). Seja $l = 4$ e $\overline{F}(x) = 0.011011001000\dots$, então

$$\lfloor \overline{F}(x) \rfloor_{l(x)} = 0.0110. \quad (365)$$

	0.0110	1100	1000	
–	0.0110	0000	0000	código $\lfloor \overline{F}(x) \rfloor_4$
=	0.0000	1100	1000	
<	0.0001	0000	0000	$= 1/2^{l(x)}$

Precisamos agora determinar qual seria $l(x)$ mínimo para que a decodificação unívoca seja preservada. Sabemos que

$$\overline{F}(x) - \lfloor \overline{F}(x) \rfloor_{l(x)} < \frac{1}{2^{l(x)}}, \quad (366)$$

conforme ilustrado no Exemplo 30. Se considerarmos $l(x) = \lceil \log 1/p(x) \rceil + 1$, então teremos

$$\frac{1}{2^{l(x)}} = \frac{1}{2} 2^{-\lceil \log 1/p(x) \rceil} \leq \frac{1}{2} 2^{-\log 1/p(x)} = \frac{p(x)}{2} \quad (367a)$$

$$= \overline{F}(x) - F(x-1) \quad (367b)$$

Combinando com o limite anterior, teremos

$$\overline{F}(x) - \lfloor \overline{F}(x) \rfloor_{l(x)} < \frac{1}{2^{l(x)}} < \overline{F}(x) - F(x-1), \quad (368)$$

e poderemos concluir que

$$\lfloor \overline{F}(x) \rfloor_{l(x)} > F(x-1). \quad (369)$$

Por fim, temos

$$F(x-1) < \lfloor \overline{F}(x) \rfloor_{l(x)} \leq \overline{F}(x) < F(x). \quad (370)$$

Concluimos que $l(x) = \lceil \log 1/p(x) \rceil + 1$ bits serão suficientes para descrever x de forma não ambígua segundo a representação $\lfloor \overline{F}(x) \rfloor_{l(x)}$, uma vez que para cada x teremos $\lfloor \overline{F}(x) \rfloor_{l(x)}$ em intervalos distintos, conforme a desigualdade anterior.

Precisamos agora verificar que a representação binária do $\lfloor \overline{F}(x) \rfloor_{l(x)}$ nos fornecerá um código de prefixo. Para tanto, considere a palavra $z_1 z_2 \dots z_l$ que corresponde ao intervalo semiaberto:

$$\underbrace{\lfloor \overline{F}(x) \rfloor_{l(x)}}_{[0.z_1 z_2 \dots z_l]}, \quad \underbrace{\lfloor \overline{F}(x) \rfloor_{l(x)}}_{0.z_1 z_2 \dots z_l + 1/2^l} = [0.z_1 z_2 \dots z_l, \quad 0.z_1 z_2 \dots z_l + 0.00\dots 1). \quad (371)$$

que possui comprimento $1/2^l$ (este intervalo contém todos os números binários que se iniciam com $0.z_1 z_2 \dots z_l$). A desigualdade $F(x-1) < \lfloor \overline{F}(x) \rfloor_{l(x)} \leq \overline{F}(x) < F(x)$ e o intervalo de comprimento $1/2^l$ são representados na Figura 39.

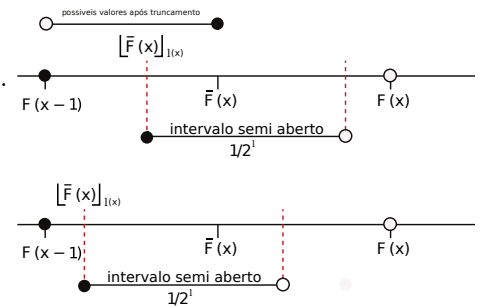


Figura 39: Intervalo de comprimento $1/2^l$ representando $\lfloor \overline{F}(x) \rfloor_{l(x)}$ como uma forma não ambígua para codificação.

Então $\lfloor \bar{F}(x) \rfloor_{l(x)} \in (F(x-1), \bar{F}(x)]$. Como $2^{-l(x)} \leq p(x)/2 = \bar{F}(x) - F(x-1)$ e também $F(x-1) < \lfloor \bar{F}(x) \rfloor_{l(x)} \leq \bar{F}(x) < F(x)$, então teremos que os intervalos abertos são disjuntos, mesmo se $\lfloor \bar{F}(x) \rfloor_{l(x)} = \bar{F}(x)$. A forma de codificação proposta, usando a representação binária de $\lfloor \bar{F}(x) \rfloor_{l(x)}$, nos leva a um código de prefixo (não existirão duas palavras associadas a um mesmo intervalo e os intervalos associados a cada um são disjuntos). Como é suficiente ter $l(x) = \lceil \log 1/p(x) \rceil + 1$, podemos calcular o limite para o comprimento esperado. Assim,

$$L = \sum_x p(x)l(x) = \sum_x p(x)(\lceil \log 1/p(x) \rceil + 1) \leq H(X) + 2. \quad (372)$$

Exemplo 31 (Distribuição d-ádica). Considere a seguinte distribuição d-ádica. Os passos para encontrar o código de Shannon-Fano-Elias são explicitados abaixo.

x	$p(x)$	$F(x)$	$\bar{F}(x)$	$\bar{F}(x)$ (binário)	$l(x)$	palavra código
1	0.25	0.25	0.125	0.001	3	001
2	0.5	0.75	0.5	0.10	2	10
3	0.125	0.875	0.8125	0.1101	4	1101
4	0.125	1.0	0.9375	0.1111	4	1111

Para o código encontrado teremos $El = 2.75$ bits enquanto $H = 1.75$ bits. Comparativamente, para o caso de Huffman teremos a árvore $((((3, 4), 1), 2), 2)$, e assim $El_{\text{huffman}} = 0.25 + 0.25 \times 2 + 0.125 \times 3 + 0.125 \times 3 = 1.75$.

Exemplo 32 (Dízima periódica). Utilizaremos a seguinte notação para dízima periódica: $0.\overline{01} = 0.0101010101 \dots$. Para uma dada distribuição p , abaixo apresentamos os passos para encontrar o código de Shannon-Fano-Elias:

x	$p(x)$	$F(x)$	$\bar{F}(x)$	$\bar{F}(x)$ (binário)	$l(x)$	palavra código
1	0.25	0.25	0.125	0.001	3	001
2	0.25	0.5	0.375	0.011	3	011
3	0.2	0.7	0.6	0.10011	4	1001
4	0.15	0.85	0.775	0.1100011	4	1100
5	0.15	1	0.925	0.1110110	4	1110

Para o código de Shannon-Fano-Elias encontrado, teremos $H = 2.285$ bits, $El = 3.5$ bits, enquanto $El_{\text{huffman}} = 2.3$ bits, sendo a árvore de Huffman $((1, (4, 5)), (3, 2))$.

O Jogo de Shannon

A compreensão da entropia como medida da incerteza em uma fonte de informação é fundamental para o desenvolvimento de códigos eficientes. Em seu artigo de 1951 (Shannon 1951), “*Prediction and Entropy of Printed English*”, Claude Shannon explorou essa ideia por

meio de um experimento simples, conhecido como o *Jogo de Shannon*. Neste jogo, participantes tentam adivinhar a próxima letra em uma sequência de texto em inglês, como completar “TH” em “THE” ou “THEN”, utilizando o contexto das letras anteriores. Shannon observou que, em média, os participantes precisavam de poucas tentativas para acertar, sugerindo que a previsibilidade do idioma reduz significativamente a quantidade de informação por letra.

Para quantificar isso, Shannon modelou o texto como um processo estocástico e estimou a entropia condicional — a incerteza média de uma letra dado o contexto anterior. Usando amostras de até 100 letras e aproximações estatísticas (de monogramas, bigramas e trigramas), ele concluiu que a entropia do inglês impresso está entre 0,6 e 1,3 bits por letra, muito abaixo dos 4,76 bits de um alfabeto de 27 símbolos (26 letras mais espaço) com distribuição uniforme. Esse resultado implica uma redundância de cerca de 75%, evidenciando que a estrutura linguística permite compressão eficiente.

O Jogo de Shannon destaca a dependência entre símbolos em uma sequência. Por exemplo, em português, após “Q”, a próxima letra é quase certamente um “U”, e após “QU”, a próxima letra é quase certamente uma vogal, reduzindo a incerteza. Essa visão sequencial motiva alguns métodos, como a codificação aritmética, que comprime uma mensagem inteira em um único número, aproveitando a entropia de toda a sequência.

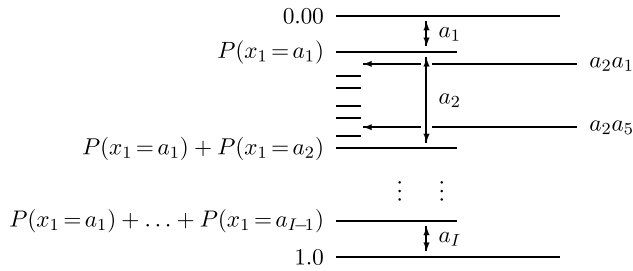
Codificação Aritmética

A *codificação aritmética* supera as limitações dos códigos baseados em símbolos individuais, sendo considerada como um código de fluxo, ou seja, codifica uma sequência inteira em um único número real/binário. Ao mapear a mensagem em um intervalo de probabilidade que se estreita iterativamente com base nas probabilidades cumulativas dos símbolos, a codificação aritmética alcança taxas de compressão que se aproximam da entropia da fonte, mesmo em distribuições complexas com dependências entre símbolos. O algoritmo básico da codificação aritmética foi desenvolvido independentemente por Jorma Rissanen e Richard C. Pasco, na década de 1970. A ideia reflete uma evolução natural das propostas de Shannon, oferecendo uma solução elegante para fontes onde a redundância sequencial desempenha um papel importante.

O princípio da codificação aritmética é a sucessiva divisão do intervalo inicial de probabilidade, $[0, 1)$, com base nas probabilidades dos símbolos de uma sequência, representando uma mensagem, ao final do processo, como um único número nesse intervalo. Para cada novo símbolo, o intervalo atual é particionado proporcionalmente às

probabilidades cumulativas dos símbolos possíveis, e o subintervalo correspondente ao símbolo observado é selecionado. À medida que a sequência avança, esse processo gera intervalos cada vez menores, cuja largura reflete a probabilidade conjunta da sequência inteira. Para transmitir a mensagem, escolhe-se um número binário que caiba dentro do intervalo final — tipicamente, uma sequência binária cuja representação decimal esteja contida nesse intervalo reduzido. Esse número único codifica toda a sequência.

Suponha que o alfabeto da fonte seja $\mathcal{X} = \{a_1, a_2, \dots, a_I\}$. À medida que a fonte produz uma sequência de símbolos $x_1, x_2, \dots, x_n, \dots$, o intervalo real $[0, 1)$ é subseqüentemente subdividido de acordo com as probabilidades dos símbolos em cada instante. A ?? apresenta a subdivisão do intervalo para os dois primeiros símbolos da sequência.



Paralelamente, o codificador deve ir subdividindo o intervalo $[0, 1)$, sempre ao meio em cada passo, para determinar uma sequência binária correspondente a um intervalo que esteja dentro do intervalo selecionado pela sequência de símbolos produzida pela fonte. A Figura 41 apresenta a subdivisão binária dos intervalos.

Exemplo 33 (Codificação (MacKay 2003)). Suponha $x \in \{a, b, \square\}$, onde $\square$ é o símbolo de término. Vamos considerar a codificação da sequência $bbba\square$. A seguir, apresentamos as probabilidades do próximo símbolo, dado o passado:

-	$p(a) = 0.425$	$p(b) = 0.425$	$p(\square) = 0.15$
b	$p(a b) = 0.28$	$p(b b) = 0.57$	$p(\square b) = 0.15$
bb	$p(a bb) = 0.21$	$p(b bb) = 0.64$	$p(\square bb) = 0.15$
bbb	$p(a bbb) = 0.17$	$p(b bbb) = 0.68$	$p(\square bbb) = 0.15$
$bbba$	$p(a bbba) = 0.28$	$p(b bbba) = 0.57$	$p(\square bbba) = 0.15$

Temos subintervalos semiabertos em $[0, 1)$. Os intervalos $[l, u)$ são dados pela probabilidade condicional $p(x_i|x_1, x_2, \dots, x_{i-1})$. Temos a representação de um prefixo $b_1b_2 \dots b_k$ por um intervalo da forma $[0.b_1b_2 \dots b_k, 0.b_1b_2 \dots b_k + 2^{-k})$ para $b_i \in \{0, 1\}$.

A Figura 42 ilustra os intervalos que teremos ao final da codificação da sequência $bbba\square$. Observe que a sequência binária 100111101

Figura 40: Definição dos intervalos probabilísticos para uma sequência de símbolos produzida pela fonte (MacKay 2003).

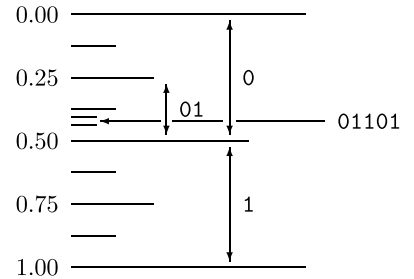


Figura 41: Definição dos intervalos probabilísticos para uma sequência binária na saída do codificador (MacKay 2003).

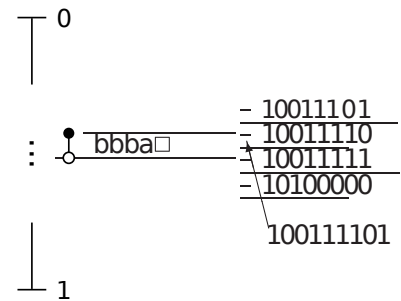


Figura 42: Intervalo final para a sequência de símbolos da fonte e para a sequência binária na saída do codificador (MacKay 2003).

correspondente a um intervalo completamente dentro do intervalo associado à sequência $bbba\Box$, codificando assim a sequência da fonte.

A Exemplo 33 apresenta todo o processo de subdivisão dos intervalos. Note que, o processo de divisão do intervalo $[0, 1)$ proporcionalmente às probabilidades dos símbolos da fonte e a subdivisão binária desse intervalo para identificar uma sequência binária que codifique a mensagem podem ocorrer de forma concomitante. À medida que cada símbolo reduz o intervalo com base em sua probabilidade, o algoritmo pode simultaneamente refinar uma representação binária, ajustando-a para permanecer dentro do intervalo atual, o que otimiza o processo de codificação.

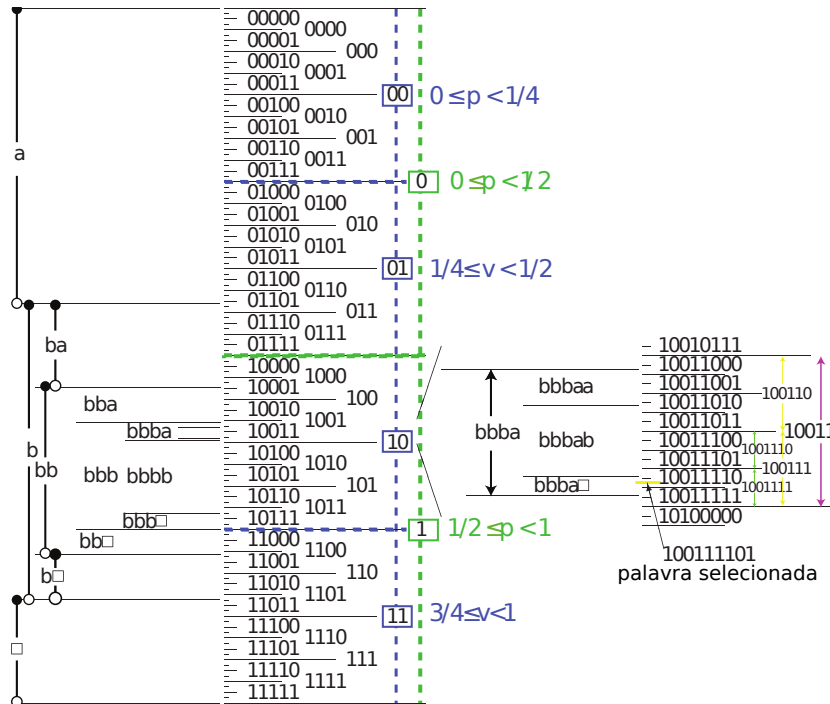


Figura 43: Todo o processo de subdivisão dos intervalos para a sequências de símbolos da fonte e para a sequência binária na saída do codificador (MacKay 2003).

A codificação aritmética comprime uma sequência de símbolos em uma única sequência binária por meio da divisão iterativa de um intervalo inicial $[0, 1)$. Para cada símbolo x_n da sequência, o algoritmo utiliza um modelo probabilístico que fornece a distribuição preditiva $p(x_n = a_i | x_1, \dots, x_{n-1})$, definindo as probabilidades cumulativas inferiores e superiores:

$$Q_n(a_i | x_1, \dots, x_{n-1}) = \sum_{a' < a_i} P(x_n = a' | x_1, \dots, x_{n-1}), \quad (373)$$

$$R_n(a_i | x_1, \dots, x_{n-1}) = \sum_{a' \leq a_i} P(x_n = a' | x_1, \dots, x_{n-1}). \quad (374)$$

O intervalo atual $[u, v]$ é subdividido proporcionalmente a essas probabilidades, e o subintervalo correspondente a x_n , dado por $[u + (v - u)Q_n, u + (v - u)R_n]$, é selecionado. Esse processo estreita o intervalo a cada passo. No fim, uma sequência binária é escolhida dentro do intervalo final, permitindo ao decodificador reconstruir a sequência com o mesmo modelo. Essa técnica maximiza a compressão, aproximando-se da entropia da fonte.

```

1: Entrada: Sequência  $x_1, x_2, \dots, x_N$ ; modelo probabilístico  $P$ 
2: Saída: Intervalo  $[u, v]$ 
3:  $u \leftarrow 0.0$ 
4:  $v \leftarrow 1.0$ 
5: for  $n = 1$  até  $N$  do
6:   Obtenha  $P(x_n = a_i | x_1, \dots, x_{n-1})$  do modelo
7:   Calcule  $Q_n(a_i) = \sum_{a' < x_n} P(x_n = a' | x_1, \dots, x_{n-1})$ 
8:   Calcule  $R_n(a_i) = \sum_{a' \leq x_n} P(x_n = a' | x_1, \dots, x_{n-1})$ 
9:    $v \leftarrow u + (v - u) \cdot R_n(x_n | x_1, \dots, x_{n-1})$ 
10:   $u \leftarrow u + (v - u) \cdot Q_n(x_n | x_1, \dots, x_{n-1})$ 
11: Retorne  $[u, v]$ 

```

Algorithm 2: Codificação Aritmética

Witten, Neal e Cleary (1987) eliminaram a necessidade de cálculos com números reais de precisão arbitrária e operações de ponto flutuante. Eles propuseram uma implementação baseada em aritmética inteira de precisão finita, representando o intervalo de probabilidade ($[low, high]$) com inteiros e escalonando-o dinamicamente para evitar *overflow*, emitindo bits incrementalmente durante o processo. Essa abordagem simplificou os cálculos, reduziu a demanda por recursos de hardware e tornou o algoritmo eficiente o suficiente para aplicações reais de compressão de dados, como em arquivos de texto e imagens, pavimentando o caminho para sua adoção em sistemas práticos de compressão.

Codificação Lempel–Ziv

A *codificação Lempel–Ziv*, desenvolvida por Ziv e Lempel (1977), marca uma transição significativa na compressão de dados ao abandonar abordagens baseadas exclusivamente em probabilidades estatísticas, como Huffman e a codificação aritmética, em favor de uma estratégia que explora padrões repetitivos em sequências. O algoritmo LZ77, o primeiro da família Lempel–Ziv, estabeleceu as bases para métodos de compressão baseados em dicionários dinâmicos, codificando sequências por meio de referências a trechos já observados no texto. Ao utilizar uma janela deslizante para identificar repetições, o LZ77 substitui trechos redundantes por ponteiros (comprimento e distância).

A escolha da correspondência no LZ77 impacta diretamente a eficiência da compressão. Selecionar a última correspondência encontrada simplifica o codificador, pois elimina a necessidade de armazenar todas as correspondências identificadas durante a busca. No entanto, isso tende a gerar valores de *offset* maiores, aumentando o tamanho da saída codificada e reduzindo a eficiência. Por outro lado, escolher a primeira correspondência resulta em *offsets* menores, o que é mais vantajoso para compressão subsequente. Combinando o LZ77 com a codificação Huffman, onde *offsets* menores recebem palavras de código mais curtas, obtém-se uma abordagem otimizada. Esse método, conhecido como LZH, foi proposto por Bernd Herd, integrando a eficiência do dicionário dinâmico do LZ77 com a compressão estatística do Huffman (Salomon, Motta e Bryant 2007).

Ao longo das décadas de 1980 e 1990, o algoritmo LZ77 foi aprimorado de diversas formas para aumentar sua eficiência e versatilidade na compressão de dados. Entre as principais otimizações, destacam-se: a utilização de campos de *offset* e comprimento com tamanhos variáveis nos *tokens*, permitindo maior flexibilidade na codificação; o aumento do tamanho do *search buffer* e do *look-ahead buffer*, expandindo a capacidade de encontrar correspondências mais longas e distantes; a substituição da janela linear por uma janela circular, que elimina a necessidade de mover o texto inteiro após cada correspondência, reduzindo custos computacionais; e a adição de um bit de *flag* nos *tokens* para distinguir entre literais e referências, eliminando o terceiro campo em alguns casos. Além disso, técnicas como o algoritmo Deflate, empregado no formato ZIP, introduziram tabelas de hash para acelerar a busca por correspondências, combinando LZ77 com codificação Huffman para otimizar ainda mais a compressão. Outras melhorias incluíram o uso de árvores binárias ou estruturas de sufixos para buscas mais rápidas, contribuindo para a adoção do LZ77 em padrões como GZIP e PNG.

Codificação LZ78

A codificação LZ78, também conhecida como LZ2, é uma evolução do método LZ77. Diferentemente do LZ77, que utiliza uma janela deslizante com *search buffer* e *look-ahead buffer*, o LZ78 adota um dicionário dinâmico para armazenar strings previamente encontradas, eliminando a dependência de *buffers*. O dicionário, que começa vazio ou com uma string nula na posição zero, cresce à medida que novas strings são processadas, limitado apenas pela memória disponível. O codificador lê os símbolos da entrada, busca correspondências no dicionário e gera *tokens* na forma de pares (ponteiro para o dicionário, símbolo), que representam strings da entrada. Cada *token* adiciona

uma nova string ao dicionário, que nunca é esvaziado, podendo crescer rapidamente. Uma estrutura eficiente para o dicionário é uma árvore, com a string vazia como raiz e strings derivadas como filhos, permitindo buscas rápidas mesmo em alfabetos grandes, como os de 256 símbolos (8 bits). Essa abordagem torna o LZ78 particularmente eficaz para compressão de dados com padrões repetitivos, influenciando formatos como LZW e servindo de base para aplicações modernas de compressão. O pseudo código para o LZ78 é apresentado no Algoritmo 3.

```

1: Entrada: Sequência de símbolos  $S = x_1, x_2, \dots, x_N$ 
2: Saída: Sequência de tokens  $(i, s)$ 
3: Inicialize:
4:    $D \leftarrow$  dicionário vazio com string nula na posição 0
5:    $k \leftarrow 1$  ▷ Próxima posição livre no dicionário
6:    $w \leftarrow$  string vazia ▷ String atual
7:    $T \leftarrow$  lista vazia ▷ Lista de tokens
8: for cada símbolo  $x$  em  $S$  do
9:   if  $w + x$  está em  $D$  then
10:     $w \leftarrow w + x$ 
11:   else
12:     $i \leftarrow$  índice de  $w$  em  $D$ 
13:    Adicione  $(i, x)$  a  $T$ 
14:    Adicione  $w + x$  a  $D$  na posição  $k$ 
15:     $k \leftarrow k + 1$ 
16:     $w \leftarrow$  string vazia
17: if  $w$  não é vazia then
18:    $i \leftarrow$  índice de  $w$  em  $D$ 
19:   Adicione  $(i, \epsilon)$  a  $T$ 
20: Retorne  $T$ 

```

As Figura 46 e ?? apresentam a aplicação do LZ78 à string exemplo `sir sid eastman easily teases sea sick seals`. Na Figura 46 é apresentado o dicionário utilizado pelo algoritmo em cada passo da execução, evidenciando o símbolo de entrada em cada etapa, a *string* de busca e a saída do codificador. A ?? apresenta a árvore em que o dicionário é organizado. Note que as sequências de símbolos são armazenadas como sequências na árvore. Por exemplo, no passo 13 da Figura 46, observamos que a sequência `sil` deverá ser adicionada à árvore. Na ??, podemos observar que 13 (sequência `sil`) foi adicionado abaixo de 5 (sequência `si`), e esta, por sua vez, abaixo de 1 (sequência `s`), de forma que o caminho percorrido desde a raiz até o nó 13 irá então representar a sequência `sil`.

Algorithm 3: Codificação LZ78

`"sir_sid_eastman_easily_teases_sea_sick_seals"`

Dictionary	Token	Dictionary	Token
0	null		
1	"s" (0, "s")	8	"a" (0, "a")
2	"i" (0, "i")	9	"st" (1, "t")
3	"r" (0, "r")	10	"m" (0, "m")
4	"_u" (0, "_u")	11	"an" (8, "n")
5	"si" (1, "i")	12	"_sea" (7, "a")
6	"d" (0, "d")	13	"sil" (5, "l")
7	"_e" (4, "e")	14	"y" (0, "y")

Figura 46: Exemplo (Salomon, Motta e Bryant 2007).

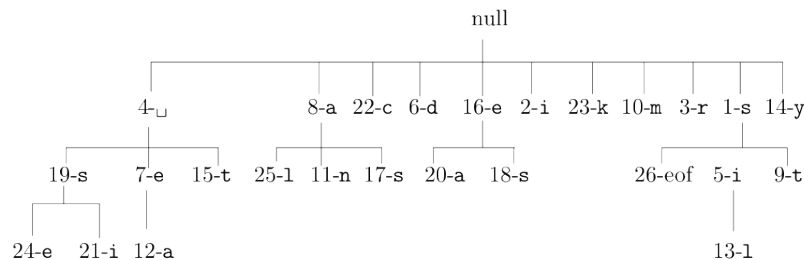


Figura 47: Exemplo (Salomon, Motta e Bryant 2007).

Codificação LZW

A codificação *LZW*, desenvolvida por Welch 1984, como uma variação do LZ78, aprimora a abordagem baseada em dicionário ao simplificar a estrutura dos *tokens* e otimizar o uso do alfabeto inicial. Diferentemente do LZ78, que gera *tokens* com dois campos (ponteiro para o dicionário e símbolo), o LZW emite apenas o ponteiro para uma *string* no dicionário, eliminando o segundo campo e reduzindo a complexidade da saída. O dicionário é inicializado com todas as entradas do alfabeto — por exemplo, 256 símbolos para um alfabeto de 8 bits — permitindo que o codificador comece imediatamente a formar strings sem depender de uma string nula inicial. À medida que processa a entrada, o LZW acumula símbolos em uma string, busca correspondências no dicionário, e adiciona novas strings quando a busca falha, atualizando o dicionário dinamicamente. O LZW é adotado em formatos de compressão como GIF, TIFF e PDF.

O algoritmo LZW, embora eficiente em explorar redundâncias, enfrentou desafios no passado devido ao crescimento do dicionário, que podia esgotar a memória limitada dos computadores da década de 1980. Para contornar isso, implementações práticas frequentemente adotavam limites no tamanho do dicionário ou reinicializações periódicas. No Algoritmo 4 apresentamos o pseudo-código do LZW.

Para o exemplo da codificação da sequência `sir sid eastman easily teases sea sick seals`, a ?? apresenta a saída do codificador e a ?? apresenta o dicionário final.

`sir sid eastman easily teases sea sick seals.`

115 (s), 105 (i), 114 (r), 32 (), 256 (si), 100 (d), 32 (), 101 (e), 97 (a), 115 (s), 116 (t), 109 (m), 97 (a), 110 (n), 262 (se), 264 (as), 105 (i), 108 (l), 121 (y), 32 (), 116 (t), 263 (ea), 115 (s), 101 (e), 115 (s), 259 (se), 263 (ea), 259 (se), 105 (i), 99 (c), 107 (k), 280 (se), 97 (a), 108 (l), 115 (s), eof.

Figura 48: Exemplo de saída do codificador LZW (Salomon, Motta e Bryant 2007).

```

1: Entrada: Sequência de símbolos  $S = x_1, x_2, \dots, x_N$ ; alfabeto  $A$ 
2: Saída: Sequência de ponteiros  $P$ 
3: Inicialize:
4:    $D \leftarrow$  dicionário com todos os símbolos de  $A$  (e.g., 256 entradas
      para 8 bits)
5:    $k \leftarrow |A| + 1$  ▷ Próxima posição livre no dicionário
6:    $I \leftarrow$  string vazia ▷ String atual
7:    $P \leftarrow$  lista vazia ▷ Lista de ponteiros
8: for cada símbolo  $x$  em  $S$  do
9:   if  $I + x$  está em  $D$  then
10:     $I \leftarrow I + x$ 
11:   else
12:     $i \leftarrow$  índice de  $I$  em  $D$ 
13:    Adicione  $i$  a  $P$ 
14:    Adicione  $I + x$  a  $D$  na posição  $k$ 
15:     $k \leftarrow k + 1$ 
16:     $I \leftarrow x$ 
17: if  $I$  não é vazia then
18:    $i \leftarrow$  índice de  $I$  em  $D$ 
19:   Adicione  $i$  a  $P$ 
20: Retorne  $P$ 

```

Algorithm 4: Codificação LZW

0	NULL	110	n	262	␣e	276	te
1	SOH	...		263	ea	277	eas
...		115	s	264	as	278	se
32	SP	116	t	265	st	279	es
...		...		266	tm	280	s␣
97	a	121	y	267	ma	281	␣se
98	b	...		268	an	282	ea␣
99	c	255	255	269	n␣	283	␣si
100	d	256	si	270	␣ea	284	ic
101	e	257	ir	271	asi	285	ck
...		258	r␣	272	il	286	k␣
107	k	259	␣s	273	ly	287	␣sea
108	l	260	sid	274	y␣	288	al
109	m	261	d␣	275	␣t	289	ls

Figura 49: Dicionário final do LZW para o exemplo dado (Salomon, Motta e Bryant 2007).

Canais de Comunicação

Na essência da teoria da informação, um dos problemas centrais é reproduzir, em um ponto, uma mensagem selecionada em outro ponto, seja de maneira exata ou aproximada. Os *canais de comunicação* modelam a transmissão de informação através de um meio físico ou lógico, frequentemente sujeito a imperfeições. O estudo dos canais busca responder a duas questões fundamentais: como garantir a confiabilidade da transmissão? e qual é a taxa máxima de comunicação possível em um dado sistema? Antes do trabalho seminal de Claude Shannon, acreditava-se que, para alcançar uma probabilidade de erro nula, seria necessário reduzir a taxa de transmissão a zero. Shannon, no entanto, mostrou que é possível transmitir informações com uma probabilidade de erro que tende a zero, desde que a taxa de comunicação não exceda um limite fundamental: a *capacidade do canal*.

Canais de comunicação estão presentes em uma ampla variedade de contextos práticos. Exemplos incluem a transmissão de sinais via rádio, satélite ou redes celulares (transmissão entre dois pontos no espaço); a leitura e escrita em discos rígidos ou memórias eletrônicas, onde bits são gravados e recuperados de superfícies físicas (transmissão entre dois instantes de tempo); e até mesmo sistemas biológicos, como a comunicação entre neurônios. Em todos esses casos, o canal é o meio que conecta a fonte ao receptor, e seu desempenho é limitado por fatores como ruído, interferência ou degradação do sinal.

Um modelo típico de canal de comunicação considera a presença de ruído, uma perturbação aleatória que corrompe a mensagem transmitida. As fontes de ruído podem ser diversas: térmicas, como o movimento aleatório de elétrons em circuitos; externas, como interferência de outros sinais; ou intrínsecas ao meio, como dispersão em fibras ópticas. Para estudar esses fenômenos, recorreremos a modelos simplificados de canais. Esses modelos, embora idealizados, capturam características essenciais dos sistemas reais, e nos permitem tratar matematicamente o problema e compreender o problema.

Uma questão fundamental é a separação entre o codificador de fonte e o codificador de canal, um conceito proposto por Claude Shannon. O codificador de fonte comprime os dados de entrada, removendo re-

dundâncias para uma representação eficiente da mensagem, enquanto o codificador de canal introduz redundância controlada para proteger contra erros do canal de comunicação. Shannon demonstrou que esses dois processos podem ser projetados de forma independente sem comprometer a otimalidade, desde que a taxa de compressão da fonte não ultrapasse a capacidade do canal. Essa abordagem simplifica o projeto de sistemas de comunicação.

Código de Repetição: Uma Introdução à Codificação de Canal

A *codificação de canal* é uma técnica essencial para garantir a confiabilidade da transmissão de informação através de canais ruidosos, e um dos exemplos mais simples é o código de repetição. Nesse esquema, cada bit da mensagem é repetido um número fixo mesmo de vezes, formando um código de comprimento maior. Por exemplo, no código de repetição tripla, o bit 0 é codificado como 000 e o bit 1 como 111. No receptor, a decisão é tomada por votação majoritária: se a sequência recebida for 010, o bit decodificado será 0, pois há mais zeros do que uns. Esse método é intuitivo e serve como uma introdução pedagógica aos conceitos de redundância e correção de erros.

O código de repetição é um código muito simples e intuitivo. Ainda assim, ele pode reduzir a probabilidade de erro em canais ruidosos. Por exemplo, em um canal binário simétrico com probabilidade de erro p , repetir um bit três vezes diminui a chance de erro de p para $p^2(1 - p) + p^3$, que é menor que p para qualquer valor de p . Isso ocorre porque o erro só persiste se pelo menos dois dos três bits forem corrompidos, um evento menos provável.

No entanto, o código de repetição também possui desvantagens significativas. A mais evidente é a redução drástica da taxa de transmissão: ao repetir cada bit n vezes, a taxa cai para $1/n$ da taxa original, tornando-o ineficiente. Além disso, sua capacidade de correção é limitada; ele só corrige erros se menos da metade dos bits repetidos forem afetados, o que o torna menos robusto em canais com altas taxas de erro.

Definições Fundamentais sobre Canais de Comunicação

Para compreender o estudo dos canais de comunicação, é essencial estabelecer definições de alguns conceitos fundamentais que descrevem seu comportamento e desempenho.

Definição 45 (Canal Discreto). Um *canal discreto* é caracterizado por um conjunto finito ou contável de símbolos de entrada X , um conjunto finito ou contável de símbolos de saída Y , e uma relação probabilística entre eles, definida por uma matriz de transição condicional $p(y|x)$. Essa matriz especifica a probabilidade de receber o símbolo $y \in Y$ dado que o símbolo $x \in X$ foi transmitido.

Definição 46 (Canal sem memória). Um canal é dito *sem memória* se a saída em um dado instante t depende apenas do símbolo de entrada correspondente naquele instante t , e não de entradas ou saídas anteriores, isto é, $y_t \perp\!\!\!\perp x_{1:t-1} \mid x_t$. Formalmente, para um canal discreto, a probabilidade condicional da sequência de saídas $y_1, y_2, \dots, y_n$ dado um conjunto de entradas $x_1, x_2, \dots, x_n$ é dada por $p(y_1, y_2, \dots, y_n | x_1, x_2, \dots, x_n) = \prod_{i=1}^n p(y_i | x_i)$. Essa propriedade simplifica a análise, pois elimina dependências temporais.

Definição 47 (Taxa de Fluxo de Informação). A *taxa de fluxo de informação* através de um canal mede a quantidade de informação que pode ser transmitida de forma confiável através do canal por uso. Para um canal discreto com entrada X e saída Y , ela é dada pela informação mútua $I(X; Y)$, depende assim da distribuição de probabilidade das entradas $p(x)$ e das características do canal $p(y|x)$.

Definição 48 (Taxa). A *taxa* de um sistema de comunicação é definida como a quantidade média de informação transmitida por uso do canal, geralmente expressa em bits por utilização do canal. Para um código com M mensagens possíveis transmitidas em n usos do canal, a taxa é dada por $R = \log_2 M / n$.

Definição 49 (Capacidade de Canal). A *capacidade de um canal* é o limite superior da taxa na qual a informação pode ser transmitida com uma probabilidade de erro que tende a zero à medida que o comprimento do código aumenta. Para um canal discreto sem memória, a capacidade C é definida como

$$C \triangleq \max_{p(x) \in \Delta} I(X; Y) \quad (375)$$

onde a maximização é feita sobre todas as distribuições de probabilidade possíveis para a entrada X .

Modelos de Canais

Canal binário sem ruído

O *canal binário sem ruído* é o modelo mais simples de canal discreto, onde o alfabeto de entrada e saída é binário, $\mathcal{X} = \mathcal{Y} = \{0, 1\}$, e a transmissão ocorre sem erros. A forma esquemática de representação deste canal é apresentada na Figura 50. A matriz de transição é dada por $p(y|x) = 1$ se $y = x$ e $p(y|x) = 0$ se $y \neq x$. Nesse caso, a saída é uma cópia exata da entrada, e a capacidade do canal é $C = 1$ bit por uso, pois a informação mútua $I(X; Y) = H(X)$ atinge seu máximo quando os símbolos são equiprováveis. Este modelo representa um sistema ideal, útil como referência para comparar canais ruidosos.

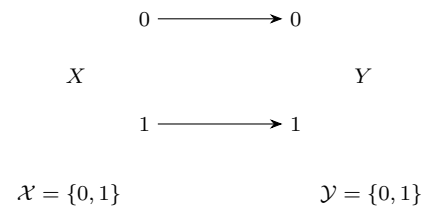


Figura 50: Representação esquemática de um canal binário sem ruído.

Canal com ruído e saídas sem sobreposição

Um canal com ruído e saídas sem sobreposição é caracterizado por um alfabeto de entrada $\mathcal{X}$ e um alfabeto de saída $\mathcal{Y}$, onde cada símbolo de entrada $x \in \mathcal{X}$ produz uma saída $y \in \mathcal{Y}$ pertencente a um subconjunto disjunto $S_x \subseteq \mathcal{Y}$, com probabilidade condicional $p(y|x) > 0$ apenas para $y \in S_x$. Por exemplo, considere um canal com $\mathcal{X} = \{0, 1\}$ e $\mathcal{Y} = \{a, b, c, d\}$, onde 0 pode gerar saídas $\{a, b\}$ e 1 pode gerar $\{c, d\}$, com $S_0 = \{a, b\}$ e $S_1 = \{c, d\}$ disjuntos. Este canal exemplo é apresentado na Figura 51. O ruído está presente na escolha aleatória dentro de S_x , mas a ausência de sobreposição entre os subconjuntos permite identificar a entrada com base na saída. Esse modelo é útil para estudar sistemas onde o ruído não causa confusão entre símbolos distintos.

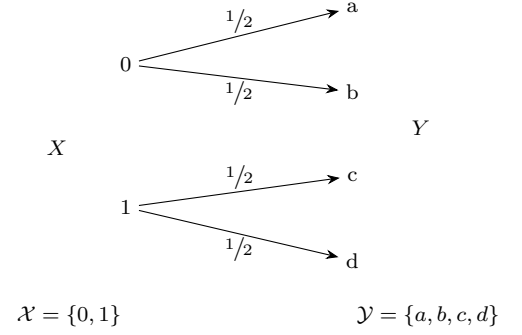


Figura 51: Canal com ruído sem sobreposição na saída.

Canal de permutação

O canal de permutação é um modelo determinístico onde o alfabeto de entrada e saída é o mesmo, $\mathcal{X} = \mathcal{Y}$, e cada entrada x é mapeada para uma saída $y = \pi(x)$, onde π é uma permutação fixa dos símbolos de $\mathcal{X}$. Por exemplo, se $\mathcal{X} = \{0, 1, 2\}$ e $\pi(0) = 1$, $\pi(1) = 2$, $\pi(2) = 0$, a saída é uma reorganização previsível da entrada. A Figura 52 apresenta o esquemático do canal de permutação proposto neste exemplo. Como a permutação é biunívoca, não há perda de informação, e a capacidade do canal é $C = \log |\mathcal{X}|$ bits por uso, uma vez que

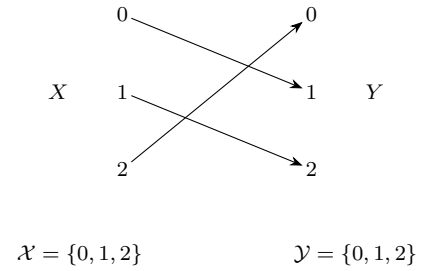


Figura 52: Exemplo de canal de permutação.

$$C = \max_{p(x)} I(X; Y) \quad (376a)$$

$$= \max_{p(x)} H(X) - \underbrace{H(X|Y)}_{=0} \quad (376b)$$

$$= \max_{p(x)} H(X) \quad (376c)$$

$$= \log |\mathcal{X}|, \quad (376d)$$

sendo assim, a distribuição uniforme na entrada é aquela que maximiza a informação mútua e alcança assim a capacidade do canal.

Canal binário simétrico

O canal binário simétrico (CBS) é um dos modelos mais estudados, com alfabeto de entrada e saída $\mathcal{X} = \mathcal{Y} = \{0, 1\}$ e uma probabilidade de erro p , onde $p(y = 1|x = 0) = p(y = 0|x = 1) = p$ e $p(y = x|x) = 1 - p$, para $0 \leq p \leq 1/2$. Este canal é representado na Figura 53. A simetria implica que o erro é igualmente provável em ambas as

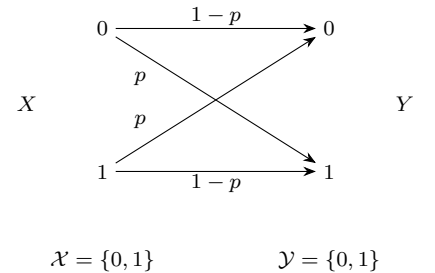


Figura 53: Canal binário simétrico.

direções. A capacidade do CBS é dada por

$$C = \max_{p(x)} I(X; Y) \quad (377a)$$

$$= \max_{p(x)} H(Y) - H(Y|X) \quad (377b)$$

$$= \max_{p(x)} H(Y) - \sum_x p(x) H(Y|X = x) \quad (377c)$$

$$= \max_{p(x)} H(Y) - \sum_x p(x) H(p) \quad (377d)$$

$$= \max_{p(x)} H(Y) - h(p) \quad (377e)$$

$$= 1 - h(p), \quad (377f)$$

onde $H(p) = h(p) = -p \log p - (1 - p) \log(1 - p)$ é a entropia binária, e utilizamos em 377f a simetria do problema, que nos garante que a distribuição uniforme na entrada acarreta a distribuição uniforme na saída, maximizando $H(Y)$. Logo, a distribuição na entrada que alcança a capacidade de canal é a distribuição uniforme.

Canal binário com apagamento

O canal binário com apagamento (CBA) possui um alfabeto de entrada $\mathcal{X} = \{0, 1\}$ e um alfabeto de saída $\mathcal{Y} = \{0, 1, e\}$, onde e representa um apagamento. Nesse modelo, cada símbolo de entrada é transmitido corretamente com probabilidade $1 - \alpha$, ou seja, $p(y = x|x) = 1 - \alpha$, ou é apagado com probabilidade α , ou seja, $p(y = e|x) = \alpha$, para $0 \leq \alpha \leq 1$. Não há erro de inversão ($p(y = 1|x = 0) = p(y = 0|x = 1) = 0$). Este canal é ilustrado na Figura 54. Para obter a capacidade deste canal, prosseguimos da seguinte forma:

$$C = \max_{p(x)} I(X; Y) \quad (378a)$$

$$= \max_{p(x)} (H(Y) - H(Y|X)) \quad (378b)$$

$$= \max_{p(x)} (H(Y) - H(\alpha)) \quad (378c)$$

$$= \max_{p(x)} (H(E) + H(Y|E) - H(\alpha)) \quad (378d)$$

$$= \max_{p(x)} (H(\alpha) + (1 - \alpha)H(\pi) - H(\alpha)) \quad (378e)$$

$$= \max_{p(x)} (1 - \alpha)H(\pi) \quad (378f)$$

$$= 1 - \alpha, \quad (378g)$$

onde em 378d utilizamos a regra da cadeia:

$$H(Y, E) = H(E) + H(Y|E) \quad (379a)$$

$$= H(Y) + \cancel{H(E|Y)} \rightarrow 0 \quad (379b)$$

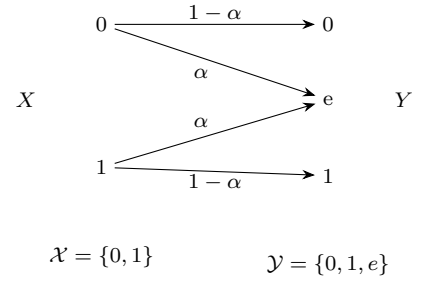


Figura 54: Canal binário com apagamento.

e em 378e utilizamos

$$H(Y|E) = \alpha H(Y|Y = e) + (1 - \alpha) H(Y|Y \neq e) \quad (380a)$$

$$= \alpha \cdot 0 + (1 - \alpha) H(\pi). \quad (380b)$$

Note que também podemos obter a equivalência entre a escolha sob Y e a quebra desta escolha em duas: houve ou não apagamento; e caso não tenha ocorrido apagamento, qual valor observado, 0 ou 1, com probabilidade π e $1 - \pi$. Esta equivalência é ilustrada na Figura 55 e a relação entre as incertezas é expressa a seguir:

$$H(Y) = H \left(\overbrace{(1 - \pi)(1 - \alpha)}^{\text{se } Y=0}, \overbrace{\alpha}^{\text{se } Y=e}, \overbrace{\pi(1 - \alpha)}^{\text{se } Y=1} \right) \quad (381a)$$

$$= H(\alpha) + (1 - \alpha) H(\pi). \quad (381b)$$

A capacidade do canal é $C = 1 - \alpha$ bits por uso, pois apenas a fração $1 - \alpha$ das transmissões contribui para a informação recebida. Este modelo é relevante para sistemas como redes de pacotes, onde dados podem ser perdidos, mas não corrompidos.

Canal de confusão ternário

O *canal de confusão ternário* opera com alfabeto de entrada $\mathcal{X} = \{0, 1, ?\}$ e alfabeto de saída $\mathcal{Y} = \{0, 1\}$. Neste exemplo, o símbolo $?$ representa um símbolo indeterminado na entrada. Supondo que o símbolo indeterminado pode gerar 0 ou 1 na saída com igual probabilidade, como representado na Figura 56, teremos a seguinte capacidade para este canal:

$$C = \max_{p(x)} I(X; Y) \quad (382a)$$

$$= \max_{p(x)} (H(Y) - H(Y|X)) \quad (382b)$$

$$= \max_{p(x)} (H(Y) - \Pr(X = ?)) \quad (382c)$$

$$= 1 - 0 \quad (382d)$$

$$= 1 \quad (382e)$$

onde em 382c utilizamos que

$$H(Y|X) = \underbrace{\Pr(X = 0) H(Y|X = 0)}_{=0} + \underbrace{\Pr(X = ?) H(Y|X = ?)}_{=1} + \quad (383a)$$

$$\underbrace{\Pr(X = 1) H(Y|X = 1)}_{=0} \quad (383b)$$

$$= \Pr(X = ?),$$

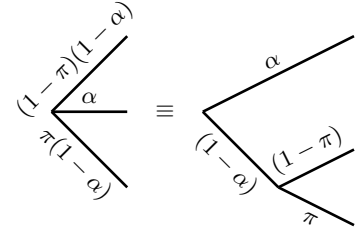
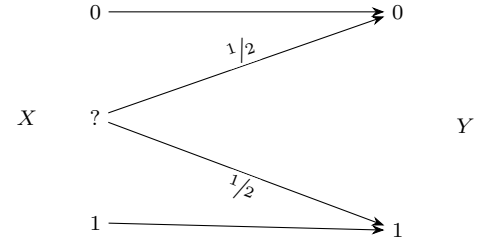


Figura 55: Incertezas equivalentes.



$$\mathcal{X} = \{0, 1, ?\}$$

$$\mathcal{Y} = \{0, 1\}$$

Figura 56: Canal de confusão ternário.

e o fato de que 382c pode ser maximizado ao minimizar o segundo termo, requerendo $p(X = ?) = 0$, e maximizando $H(Y)$ que pode ser obtido ao requerer $p(X = 0) = p(X = 1)$, o que garante uma distribuição uniforme na saída.

Definição 50 (Canal simétrico). Um *canal simétrico* é um canal discreto cuja matriz de transição condicional $p(y|x)$, para $x \in \mathcal{X}$ e $y \in \mathcal{Y}$, possui linhas que são permutações umas das outras e colunas que também são permutações umas das outras. Isso implica que a probabilidade de transição é invariante sob re-rotulação dos símbolos de entrada e saída, simplificando a análise da capacidade. Um canal fracamente simétrico é um canal discreto no qual cada linha da matriz $p(y|x)$ é uma permutação de qualquer outra linha, e a soma $\sum_{x \in \mathcal{X}} p(y|x) = p(y)$ é a mesma para todo $y \in \mathcal{Y}$, ou seja, as saídas têm a mesma probabilidade marginal. Essas propriedades são fundamentais para canais como o binário simétrico, onde a simetria facilita o cálculo da capacidade máxima.

Teorema 48 (Capacidade de canais fracamente simétricos) *Para um canal discreto fracamente simétrico com alfabeto de saída $\mathcal{Y}$ e matriz de transição condicional $p(y|x)$, a capacidade C é dada por*

$$C = \log |\mathcal{Y}| - H(r), \quad (384)$$

onde r representa qualquer linha da matriz $p(y|x)$, e $H(r) = H(Y|X = x) = -\sum_{y \in \mathcal{Y}} p(y|x) \log p(y|x)$ é a entropia condicional da saída dado um símbolo de entrada x . Como as linhas da matriz são permutações umas das outras, $H(Y|X = x)$ é a mesma para todo $x \in \mathcal{X}$. Essa capacidade é atingida com uma distribuição uniforme sobre as entradas.

Demonstração. Seja um canal discreto fracamente simétrico com alfabeto de entrada $\mathcal{X}$, alfabeto de saída $\mathcal{Y}$ e matriz de transição condicional $p(y|x)$. A capacidade é definida como

$$C = \max_{p(x)} I(X; Y), \quad (385)$$

onde $I(X; Y) = H(Y) - H(Y|X)$ é a informação mútua, e a maximização é feita sobre todas as distribuições de probabilidade $p(x)$ das entradas.

Em um canal fracamente simétrico, cada linha da matriz $p(y|x)$ é uma permutação de qualquer outra, o que implica que a entropia condicional $H(Y|X = x) = -\sum_{y \in \mathcal{Y}} p(y|x) \log p(y|x)$ é a mesma para todo $x \in \mathcal{X}$. Denote essa entropia comum por $H(r)$, onde r representa qualquer linha da matriz $p(y|x)$. Assim, a entropia condicional total é

$$H(Y|X) = \sum_{x \in \mathcal{X}} p(x) H(Y|X = x) = \sum_{x \in \mathcal{X}} p(x) H(r) = H(r), \quad (386)$$

independente da escolha de $p(x)$.

Agora, consideremos $H(Y)$, a entropia da saída. A probabilidade marginal $p(y)$ é dada por

$$p(y) = \sum_{x \in \mathcal{X}} p(x)p(y|x). \quad (387)$$

Pela propriedade de simetria fraca, a soma $\sum_{x \in \mathcal{X}} p(y|x)$ é constante para todo $y \in \mathcal{Y}$. Ou seja, a soma das probabilidades $p(y|x)$ na matriz de transição do canal é a mesma para todas as linhas. Seja então $\alpha = \sum_{x \in \mathcal{X}} p(y|x)$ essa constante. Escolher uma distribuição uniforme para as entradas, $p(x) = 1/|\mathcal{X}|$, nos garante uma distribuição uniforme na saída (o que maximiza $H(Y)$). Teremos assim

$$p(y) = \sum_{x \in \mathcal{X}} \frac{1}{|\mathcal{X}|} p(y|x) = \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} p(y|x) = \frac{\alpha}{|\mathcal{X}|}. \quad (388)$$

Como α é igual para todo y , $p(y)$ é uniforme, isto é, $p(y) = 1/|\mathcal{Y}|$.

Assim, $H(Y) = \log |\mathcal{Y}|$.

Substituindo em $I(X; Y)$, obtemos

$$I(X; Y) = H(Y) - H(Y|X) = \log |\mathcal{Y}| - H(r). \quad (389)$$

Assim, a informação mútua é maximizada quando $H(Y) = \log |\mathcal{Y}|$, o que ocorre com a distribuição uniforme de entrada. Portanto, a capacidade é

$$C = \max_{p(x)} I(X; Y) = \log |\mathcal{Y}| - H(r). \quad (390)$$

□

Os canais de comunicação frequentemente estão sujeitos a ruído, corrompendo as mensagens transmitidas. Para garantir a confiabilidade da comunicação, utilizamos códigos, que são esquemas para mapear mensagens em sequências de símbolos (palavras-código) e recuperá-las no receptor, mesmo na presença de erros. Um código é caracterizado pelo número de mensagens que pode representar (M) e pelo comprimento das palavras-código (n), com a taxa de transmissão definida como $R = \log M/n$. A definição a seguir formaliza esses conceitos, estabelecendo a estrutura de um código para um canal discreto.

Definição 51 (Código (M, n)). Um *código* (M, n) para um canal discreto $(\mathcal{X}, p(y|x), \mathcal{Y})$ consiste em:

1. Um conjunto de mensagens $\{1, 2, \dots, M\}$, representando as possíveis entradas da fonte.
2. Uma função de codificação $f : \{1, 2, \dots, M\} \rightarrow \mathcal{X}^n$, que mapeia cada mensagem i a uma palavra-código $f(i) = X^n(i) = (x_1, x_2, \dots, x_n) \in \mathcal{X}^n$ de comprimento n .

3. Uma função de decodificação $g : \mathcal{Y}^n \rightarrow \{1, 2, \dots, M\}$, que estima a mensagem original com base na sequência de n símbolos recebida do canal.

A definição de um código (M, n) estabelece como mensagens são codificadas em palavras-código e decodificadas a partir das saídas do canal. No entanto, devido ao ruído presente em canais de comunicação, a decodificação pode falhar, resultando em uma mensagem estimada diferente da original. Para quantificar a confiabilidade da comunicação, definimos a probabilidade de erro associada a cada mensagem, que mede a chance de uma decodificação incorreta.

Definição 52 (Probabilidade de erro λ_i). Para um código (M, n) definido sobre um canal discreto $(\mathcal{X}, p(y|x), \mathcal{Y})$, com função de codificação $f : \{1, 2, \dots, M\} \rightarrow \mathcal{X}^n$ e função de decodificação $g : \mathcal{Y}^n \rightarrow \{1, 2, \dots, M\}$, a *probabilidade de erro para a mensagem* $i \in \{1, 2, \dots, M\}$, denotada por λ_i , é dada por

$$\lambda_i = \Pr(g(Y^n) \neq i \mid f(i)) = \Pr(g(Y^n) \neq i \mid X^n = x^n(i)), \quad (391)$$

ou equivalentemente,

$$\lambda_i = \sum_{y^n \in \mathcal{Y}^n} p(y^n \mid f(i)) \mathbf{1}_{\{g(y^n) \neq i\}}, \quad (392)$$

onde $f(i) \in \mathcal{X}^n$ é a palavra-código associada à mensagem i , $p(y^n \mid f(i))$ é a probabilidade da sequência de saída y^n dado $f(i)$, e $\mathbf{1}_{\{g(y^n) \neq i\}}$ indica que $g(y^n) \neq i$. Essa probabilidade quantifica a chance da mensagem i ser decodificada incorretamente.

Definição 53 (Probabilidade máxima de erro $\lambda^{(n)}$ para um código (M, n)). Para um código (M, n) com probabilidade de erro λ_i para cada mensagem $i \in \{1, \dots, M\}$, a *probabilidade máxima de erro*, denotada por $\lambda^{(n)}$, é definida como

$$\lambda^{(n)} \triangleq \max_{i \in \{1, 2, \dots, M\}} \lambda_i. \quad (393)$$

Essa quantidade representa a maior probabilidade de erro de decodificação entre todas as mensagens, caracterizando o desempenho do código no pior caso.

Definição 54 (Probabilidade de erro média). Para um código (M, n) com probabilidade de erro λ_i para cada mensagem $i \in \{1, \dots, M\}$, a *probabilidade de erro média*, denotada por $P_e^{(n)}$, é definida como

$$P_e^{(n)} = \frac{1}{M} \sum_{i=1}^M \lambda_i, \quad (394)$$

onde $\lambda_i = \Pr(g(Y^n) \neq i \mid f(i))$, e $f(i)$ é a palavra-código associada à mensagem i . Equivalentemente, se I é uma variável aleatória representando a mensagem escolhida com distribuição uniforme, $\Pr(I = i) = 1/M$, então

$$P_e^{(n)} = \Pr(I \neq g(Y^n)) = E(\mathbf{1}_{\{I \neq g(Y^n)\}}) \quad (395a)$$

$$= \sum_{i=1}^M \Pr(g(Y^n) \neq i \mid X^n = x^n(i))p(i). \quad (395b)$$

$P_e^{(n)}$ representa a probabilidade média de uma mensagem ser decodificada incorretamente.

Observação 3. A probabilidade de erro média $P_e^{(n)}$ é definida assumindo que as mensagens são escolhidas com distribuição uniforme, $\Pr(I = i) = 1/M$. Essa suposição simplifica a análise da confiabilidade do código, permitindo avaliar $P_e^{(n)}$ e $\lambda^{(n)}$ como medidas do desempenho médio e do pior caso, respectivamente, para um dado código (M, n) .

As definições de código (M, n) e probabilidades de erro λ_i , $\lambda^{(n)}$ e $P_e^{(n)}$ nos permitem avaliar a confiabilidade da transmissão em canais ruidosos. Um aspecto importante é determinar quais taxas de comunicação, $R = \log M/n$, permitem transmissão com erro arbitrariamente pequeno à medida que o comprimento do código aumenta. A seguir, definimos formalmente quando uma taxa é alcançável.

Definição 55 (Taxa alcançável). Uma taxa R é dita *alcançável* para um canal discreto $(\mathcal{X}, p(y|x), \mathcal{Y})$ se existe uma família de códigos $(\lceil 2^{nR} \rceil, n)$, com $M = \lceil 2^{nR} \rceil$ mensagens e palavras-código de comprimento n , tal que a probabilidade máxima de erro $\lambda^{(n)}$ satisfaz

$$\lambda^{(n)} \rightarrow 0 \quad \text{à medida que} \quad n \rightarrow \infty. \quad (396)$$

Essa propriedade indica que é possível transmitir mensagens à taxa R com confiabilidade arbitrária para blocos suficientemente longos.

A definição de taxa alcançável estabelece que uma taxa R permite transmissão confiável se a probabilidade máxima de erro $\lambda^{(n)}$ tende a zero à medida que o comprimento do código n aumenta. Um conceito central na teoria da informação é a *capacidade do canal*, que determina a taxa máxima para a qual tal confiabilidade é possível. Para canais discretos sem memória, a capacidade estabelece o limite entre taxas viáveis e não viáveis, conforme estabelecido pelo teorema de codificação de canal de Shannon.

Definição 56 (Capacidade do canal). A *capacidade de um canal discreto sem memória* $(\mathcal{X}, p(y|x), \mathcal{Y})$, denotada por C , é a maior taxa

R para a qual existe uma família de códigos $(\lceil 2^{nR} \rceil, n)$ com probabilidade máxima de erro $\lambda^{(n)}$ satisfazendo a Equação (396). Equivalentemente, C é dada por $C = \max_{p(x)} I(X; Y)$, onde $I(X; Y)$ é a informação mútua entre a entrada X e a saída Y . Taxas $R \leq C$ são alcançáveis, enquanto taxas $R > C$ não permitem que $\lambda^{(n)}$ tenda a zero, caracterizando C como o limite fundamental de transmissão confiável.

A capacidade de um canal discreto sem memória define a taxa máxima para a qual a transmissão confiável é possível, conforme estabelecido pelo teorema de codificação de canal de Shannon. Para provar este resultado, Shannon faz uso do conceito de tipicidade conjunta, que identifica pares de sequências de entrada e saída que são prováveis segundo a distribuição do canal. Essas sequências, chamadas tipicamente conjuntas, desempenham um papel central na construção de códigos eficientes, garantindo que a probabilidade de erro tenda a zero para taxas abaixo da capacidade.

Definição 57 (Tipicidade Conjunta). Seja um canal discreto sem memória $(\mathcal{X}, p(y|x), \mathcal{Y})$ com distribuição conjunta $p(x, y) = p(x)p(y|x)$. Um par de sequências $(x^n, y^n) \in \mathcal{X}^n \times \mathcal{Y}^n$ é dito *típico conjuntamente* com relação a $p(x, y)$ se pertence ao conjunto $A_\epsilon^{(n)}$, definido por

$$A_\epsilon^{(n)} = \left\{ (x^n, y^n) \in \mathcal{X}^n \times \mathcal{Y}^n \mid \begin{array}{l} \text{a) } \left| -\frac{1}{n} \log p(x^n) - H(X) \right| < \epsilon, \\ \text{b) } \left| -\frac{1}{n} \log p(y^n) - H(Y) \right| < \epsilon, \\ \text{c) } \left| -\frac{1}{n} \log p(x^n, y^n) - H(X, Y) \right| < \epsilon \end{array} \right\},$$

onde $p(x^n) = \prod_{i=1}^n p(x_i)$, $p(y^n) = \prod_{i=1}^n p(y_i)$, $p(x^n, y^n) = \prod_{i=1}^n p(x_i, y_i)$, e $\epsilon > 0$ é um parâmetro de tolerância. Essas condições garantem que as sequências têm probabilidades próximas às esperadas pelas entropias marginal e conjunta.

Observação 4. Para um canal discreto sem memória com distribuição $p(x, y)$, o conjunto dos pares de sequências típicas conjuntamente $A_\epsilon^{(n)}$ tem aproximadamente $2^{nH(X, Y)}$ pares de sequências (x^n, y^n) , enquanto os conjuntos típicos marginais para X e Y têm tamanhos aproximados de $2^{nH(X)}$ e $2^{nH(Y)}$, respectivamente. Assim, a razão entre esses tamanhos é $|A_\epsilon^{(n)}| / (2^{nH(X)} 2^{nH(Y)}) \approx 2^{-nI(X; Y)}$, onde $I(X; Y) = H(X) + H(Y) - H(X, Y)$ é a informação mútua. Isso implica que, se x^n e y^n são amostrados independentemente segundo

$p(x)$ e $p(y)$, a probabilidade de $(x^n, y^n) \in A_\epsilon^{(n)}$ decresce exponencialmente como $2^{-nI(X;Y)}$ para $I(X;Y) > 0$. Essa propriedade é crucial na demonstração do Teorema de Codificação de Canal, onde o número de mensagens confiáveis é limitado a aproximadamente 2^{nC} , com $C = \max_{p(x)} I(X;Y)$, garantindo que os pares típicos sejam distinguíveis com alta probabilidade.

Teorema 49 (Propriedade da equipartição assintótica conjunta)

Seja (X^n, Y^n) um par de seqüências aleatórias geradas por um canal discreto sem memória $(\mathcal{X}, p(y|x), \mathcal{Y})$, com distribuição conjunta $p(x^n, y^n) = \prod_{i=1}^n p(x_i, y_i)$. Então, para qualquer $\epsilon > 0$, o conjunto das seqüências conjuntamente típicas $A_\epsilon^{(n)}$ satisfaz as seguintes propriedades:

1. A probabilidade de (X^n, Y^n) ser conjuntamente típico tende a 1:

$$\Pr\left((X^n, Y^n) \in A_\epsilon^{(n)}\right) \rightarrow 1 \quad \text{à medida que } n \rightarrow \infty. \quad (397)$$

2. O tamanho do conjunto $A_\epsilon^{(n)}$ é limitado por:

$$|A_\epsilon^{(n)}| \leq 2^{n(H(X,Y)+\epsilon)}, \quad (398)$$

e,

$$|A_\epsilon^{(n)}| \geq (1 - \epsilon)2^{n(H(X,Y)-\epsilon)}, \quad (399)$$

para n suficientemente grande,

3. Se $(\tilde{X}^n, \tilde{Y}^n) \sim p(x^n)p(y^n)$ são tirados de forma independente, então

$$\Pr\left((\tilde{X}^n, \tilde{Y}^n) \in A_\epsilon^{(n)}\right) \leq 2^{-n(I(X;Y)-3\epsilon)} \quad (400)$$

e,

$$\Pr\left((\tilde{X}^n, \tilde{Y}^n) \in A_\epsilon^{(n)}\right) \geq (1 - \epsilon)2^{-n(I(X;Y)+3\epsilon)} \quad (401)$$

para n suficientemente grande.

Demonstração: $\Pr\left((X^n, Y^n) \in A_\epsilon^{(n)}\right) \rightarrow 1$.

Pela lei fraca dos grandes números temos o seguinte:

$$-\frac{1}{n} \log \Pr(X^n) \rightarrow -E(\log p(X)) = H(X). \quad (402)$$

Então $\forall \epsilon > 0$, $\exists m_1$ tal que para $n > m_1$

$$\Pr\left(\underbrace{\left| -\frac{1}{n} \log \Pr(X^n) - H(X) \right|}_{=S_1} > \epsilon\right) < \epsilon/3, \quad (403)$$

onde definimos S_1 , o evento não típico em X , ou seja, quando a convergência em 402 ocorre.

De forma similar, podemos definir os eventos S_2 e S_3 para expressar o evento não típico em Y e o evento não típico em (X, Y) , respectivamente:

$\exists m_2$ tal que $\forall n > m_2$ teremos

$$\Pr \left(\underbrace{\left| -\frac{1}{n} \log \Pr(Y^n) - H(Y) \right|}_{=S_2} > \epsilon \right) < \epsilon/3 \quad (404)$$

e $\exists m_3$ tal que $\forall n > m_3$ teremos

$$\Pr \left(\underbrace{\left| -\frac{1}{n} \log \Pr(X^n, Y^n) - H(X, Y) \right|}_{=S_3} > \epsilon \right) < \epsilon/3 \quad (405)$$

Para $n > \max(m_1, m_2, m_3)$, temos que $p(S_1 \cup S_2 \cup S_3) \leq \epsilon = 3\epsilon/3$. Não tipicidade possui probabilidade menor do que ϵ , ou seja, $\Pr(A_\epsilon^{(n)c}) \leq \epsilon$, implicando em $\Pr(A_\epsilon^{(n)}) \geq 1 - \epsilon$. A probabilidade de erro é menor do que ϵ .

□

$$|A_\epsilon^{(n)}| \leq 2^{n(H(X,Y)+\epsilon)}.$$

Temos a seguinte relação

$$1 = \sum_{(x^n, y^n)} p(x^n, y^n) \geq \sum_{(x^n, y^n) \in A_\epsilon^{(n)}} p(x^n, y^n) \quad (406a)$$

$$\geq |A_\epsilon^{(n)}| 2^{-n(H(X,Y)+\epsilon)}, \quad (406b)$$

logo, teremos que

$$|A_\epsilon^{(n)}| \leq 2^{n(H(X,Y)+\epsilon)} \quad (407)$$

Como visto anteriormente, $\Pr(A_\epsilon^{(n)}) \geq 1 - \epsilon$, para n grande suficiente, e assim teremos

$$1 - \epsilon \leq \sum_{(x^n, y^n) \in A_\epsilon^{(n)}} p(x^n, y^n) \leq |A_\epsilon^{(n)}| 2^{-n(H(X,Y)-\epsilon)} \quad (408)$$

e assim

$$|A_\epsilon^{(n)}| \geq (1 - \epsilon) 2^{n(H(X,Y)-\epsilon)} \quad (409)$$

□

Duas sequências independentes provavelmente não são conjuntamente típica.

Seja $\tilde{X}^n, \tilde{Y}^n$ independente $\sim p(x^n)p(y^n)$, i.e., as duas sequências são independentes entre si.

Vamos utilizar

$$A, B \perp\!\!\!\perp C \Rightarrow A \perp\!\!\!\perp C \text{ e } B \perp\!\!\!\perp C \quad (410)$$

e desta forma, teremos

$$\tilde{X}_i^n \perp \tilde{Y}_j^n, \quad \forall i, j \quad (411)$$

Teremos o seguinte seguinte limite superior:

$$\Pr\left((\tilde{X}^n, \tilde{Y}^n) \in A_\epsilon^{(n)}\right) = \sum_{(x^n, y^n) \in A_\epsilon^{(n)}} \underbrace{p(x^n)}_{\leq 2^{-n(H(X)-\epsilon)}} \underbrace{p(y^n)}_{\leq 2^{-n(H(Y)-\epsilon)}} \quad (412a)$$

$$\leq \sum_{(x^n, y^n) \in A_\epsilon^{(n)}} 2^{-n(H(X)-\epsilon)} 2^{-n(H(Y)-\epsilon)} \quad (412b)$$

$$\leq 2^{n(H(X,Y)+\epsilon)} 2^{-n(H(X)-\epsilon)} 2^{-n(H(Y)-\epsilon)} \quad (412c)$$

$$= 2^{-n(I(X;Y)-3\epsilon)}. \quad (412d)$$

Podemos também, de forma similar, obter o limite inferior:

$$\Pr\left((\tilde{X}^n, \tilde{Y}^n) \in A_\epsilon^{(n)}\right) = \sum_{(x^n, y^n) \in A_\epsilon^{(n)}} \underbrace{p(x^n)}_{\geq 2^{-n(H(X)+\epsilon)}} \underbrace{p(y^n)}_{\geq 2^{-n(H(Y)+\epsilon)}} \quad (413a)$$

$$\geq \sum_{(x^n, y^n) \in A_\epsilon^{(n)}} 2^{-n(H(X)+\epsilon)} 2^{-n(H(Y)+\epsilon)} \quad (413b)$$

$$\geq (1-\epsilon) 2^{n(H(X,Y)-\epsilon)} 2^{-n(H(X)+\epsilon)} 2^{-n(H(Y)+\epsilon)} \quad (413c)$$

$$= (1-\epsilon) 2^{-n(I(X;Y)+3\epsilon)}.$$

Para mostrar ambos limites, fizemos uso da Equação (214), para obter os limites de probabilidade de sequências típicas, e também utilizamos o Teorema 21 para aplicar os limites de tamanho do conjunto típico. □

A capacidade C de um canal discreto sem memória, definida como $C = \max_{p(x)} I(X; Y)$, representa a taxa máxima teórica para transmissão confiável. A propriedade da equipartição assintótica conjunta e a tipicidade conjunta fornecem as ferramentas para analisar sequências prováveis, limitando o número de mensagens que podem ser transmitidas com erro pequeno. O Teorema de Codificação de Canal de Shannon formaliza esse limite, estabelecendo que taxas abaixo de C são alcançáveis com probabilidade de erro tendendo a zero, enquanto taxas acima de C não permitem confiabilidade.

Na transmissão de informação através de um canal binário simétrico (BSC) com probabilidade de erro p , a decodificação correta de uma sequência recebida $Y^n \in \{0, 1\}^n$ permite ao receptor reconstruir duas informações distintas: a palavra-código transmitida X^n , correspondente a uma mensagem $m \in \{1, \dots, M\}$ com $M = \lceil 2^{nR} \rceil$, equivalente a nR bits de informação; e o padrão de erro $E^n = Y^n \oplus X^n$, onde $\oplus$ denota adição módulo 2 em cada componente. No BSC, essas informações são estatisticamente independentes, pois o erro E^n depende

apenas da probabilidade p do canal. Pela propriedade da equipartição assintótica, os erros típicos E^n têm peso de Hamming próximo de np , com probabilidade aproximada de $2^{-nH(p)}$, onde $H(p) = h(p)$ é a entropia binária. Assim, existem aproximadamente $2^{nH(p)}$ padrões de erro típicos, cada um contribuindo com $nH(p)$ bits de informação. Como a sequência recebida Y^n contém n bits, a soma da informação da mensagem (nR) e do erro ($nH(p)$) não pode exceder n , implicando $R \leq 1 - H(p)$. Essa quantidade, conhecida como capacidade $C = 1 - H(p)$ do BSC, define o limite superior para taxas de transmissão confiáveis, como estabelecido pelo Teorema de Codificação de Canal de Shannon.

Teorema 50 (Teorema de Codificação de Canal de Shannon) *Seja um canal discreto sem memória $(\mathcal{X}, p(y|x), \mathcal{Y})$ com capacidade $C = \max_{p(x)} I(X; Y)$. Então:*

1. *Para qualquer taxa $R < C$, existe uma família de códigos $(\lceil 2^{nR} \rceil, n)$ com probabilidade máxima de erro $\lambda^{(n)}$ satisfazendo*

$$\lambda^{(n)} \rightarrow 0 \quad \text{à medida que} \quad n \rightarrow \infty. \quad (414)$$

2. *Conversamente, para qualquer família de códigos $(\lceil 2^{nR} \rceil, n)$ com $\lambda^{(n)} \rightarrow 0$ à medida que $n \rightarrow \infty$, deve-se ter $R \leq C$. Em particular, taxas $R > C$ não são alcançáveis, pois $\lambda^{(n)}$ não tende a zero.*

Para demonstrar o Teorema de Codificação de Canal de Shannon, consideramos um canal discreto sem memória $(\mathcal{X}, p(y|x), \mathcal{Y})$ com capacidade $C = \max_{p(x)} I(X; Y)$. A prova consiste em duas partes: primeiro, mostramos que, para qualquer taxa $R < C$, existe uma família de códigos $(\lceil 2^{nR} \rceil, n)$ com probabilidade máxima de erro $\lambda^{(n)}$ tendendo a zero à medida que $n \rightarrow \infty$; segundo, provamos que, se $\lambda^{(n)} \rightarrow 0$, a taxa deve satisfazer $R \leq C$, indicando que taxas $R > C$ não permitem transmissão confiável. Utilizamos uma abordagem de codificação aleatória, onde as palavras-código são geradas independentemente segundo uma distribuição p . Analisando a probabilidade média de erro $P_e^{(n)}$ sobre esses códigos, mostramos que ela é pequena para $R < C$, o que implica a existência de pelo menos um código com $\lambda^{(n)}$ pequeno. Para garantir que $\lambda^{(n)}$ seja uniformemente pequeno, descartamos uma fração dos códigos com maior erro, mantendo a taxa desejada. Essa estratégia, apoiada pela tipicidade conjunta e pela propriedade da equipartição assintótica, estabelece C como o limite fundamental para comunicação confiável.

Demonstração Taxas $R < C$ são alcançáveis.

Seja um canal discreto sem memória $(\mathcal{X}, p(y|x), \mathcal{Y})$ com capacidade $C = \max_{p(x)} I(X; Y)$. Para provar que qualquer taxa $R < C$ é alcançável, mostramos que existe uma família de códigos $(\lceil 2^{nR} \rceil, n)$

com probabilidade máxima de erro $\lambda^{(n)} \rightarrow 0$ à medida que $n \rightarrow \infty$, utilizando codificação aleatória e decodificação por tipicidade conjunta.

Construção do código

Escolhemos uma distribuição $p(x)$ tal que $I(X; Y) > R$ (posteriormente, otimizaremos com $p^*(x) = \operatorname{argmax}_{p(x)} I(X; Y)$). Geramos um livro de códigos $\mathcal{C}$ com $M = \lceil 2^{nR} \rceil$ palavras-código $x^n(w) = (x_1(w), \dots, x_n(w))$, para $w \in \{1, \dots, M\}$, onde cada $x_i(w)$ é amostrado independentemente segundo $p(x)$.

O conjunto de palavras-código pode ser representado pela matriz

$$\mathcal{C} = \begin{bmatrix} x_1(1) & x_2(1) & x_3(1) & \dots & x_n(1) \\ x_1(2) & x_2(2) & x_3(2) & \dots & x_n(2) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_1(2^M) & x_2(2^M) & x_3(2^M) & \dots & x_n(2^M) \end{bmatrix} \quad (415)$$

Desta forma, existe 2^M palavras (*codewords*), de comprimento n , geradas por $p(x)$, cada uma em uma linha da matriz.

A probabilidade de uma dada palavra $w \in \{1, \dots, M\}$ é da seguinte forma

$$p(x^n(w)) = \prod_{i=1}^n p(x_i(w)), \quad w \in \{1, \dots, M\}. \quad (416)$$

E a probabilidade do livro de códigos é:

$$p(\mathcal{C}) = \prod_{w=1}^M \prod_{i=1}^n p(x_i(w)). \quad (417)$$

O transmissor e o receptor conhecem $\mathcal{C}$ e $p(y|x)$. As mensagens $W \in \{1, \dots, M\}$ são escolhidas uniformemente, com $p(W = w) = 1/M$. Para enviar $W = w$, o transmissor envia $x^n(w)$, e o canal produz $Y^n \sim p(y^n|x^n(w)) = \prod_{i=1}^n p(y_i|x_i(w))$.

Decodificação por tipicidade conjunta

O receptor decodifica $\hat{w}$ se:

1. $(x^n(\hat{w}), y^n) \in A_\epsilon^{(n)}$, onde $A_\epsilon^{(n)}$ é o conjunto das sequências conjuntamente típicas;
2. $\hat{w}$ é único, ou seja, não existe $w' \neq \hat{w}$ tal que $(x^n(w'), y^n) \in A_\epsilon^{(n)}$.

Se essas condições não forem satisfeitas, declara-se erro. Um erro ocorre se:

1. $\exists w' \neq \hat{w}$ tal que $(x^n(w'), y^n) \in A_\epsilon^{(n)}$, i.e., existe mais do que uma mensagem típica;
2. $\nexists \hat{w}$ tal que $(x^n(\hat{w}), y^n)$ é conjuntamente típico;
3. se $\hat{w} \neq w$, i.e., a palavra errada é conjuntamente típica.

Análise do erro médio

Definimos a probabilidade média de erro para um código $\mathcal{C}$ como:

$$P_e^{(n)}(\mathcal{C}) = \Pr(\hat{w} \neq w \mid \mathcal{C}) = \frac{1}{M} \sum_{w=1}^M \lambda_w(\mathcal{C}), \quad (418)$$

onde $\lambda_w(\mathcal{C}) = \Pr(\hat{W} \neq w \mid W = w, \mathcal{C})$. O erro médio sobre todos os códigos aleatórios é:

$$\Pr(\varepsilon) = \mathbb{E}[P_e^{(n)}(\mathcal{C})] \quad (419a)$$

$$= \sum_{\mathcal{C}} p(\mathcal{C}) \Pr(\hat{W} \neq W \mid \mathcal{C}) \quad (419b)$$

$$= \sum_{\mathcal{C}} p(\mathcal{C}) P_e^{(n)}(\mathcal{C}). \quad (419c)$$

Por simetria, $\mathbb{E}[\lambda_w(\mathcal{C})]$ é igual para todo w . Assim:

$$\Pr(\varepsilon) = \frac{1}{M} \sum_{w=1}^M \mathbb{E}[\lambda_w(\mathcal{C})] = \mathbb{E}[\lambda_1(\mathcal{C})]. \quad (420)$$

Assumindo $W = 1$, definimos os eventos:

$$E_i = \{(x^n(i), Y^n) \in A_\epsilon^{(n)}\}, \quad i = 1, \dots, M. \quad (421)$$

Um erro ocorre se E_1^c (a palavra correta não é típica) ou $\bigcup_{i=2}^M E_i$ (outra palavra é típica). Assim:

$$\Pr(\varepsilon \mid W = 1) = \Pr\left(E_1^c \cup \bigcup_{i=2}^M E_i\right) \leq \Pr(E_1^c) + \sum_{i=2}^M \Pr(E_i). \quad (422)$$

Pela propriedade da equipartição assintótica conjunta, $\Pr(E_1^c) = \Pr((x^n(1), Y^n) \notin A_\epsilon^{(n)}) \rightarrow 0$ à medida que $n \rightarrow \infty$, então $\Pr(E_1^c) \leq \epsilon$ para n grande. Para $i \neq 1$, como $x^n(i)$ é independente de Y^n , temos:

$$\Pr(E_i) = \Pr((x^n(i), Y^n) \in A_\epsilon^{(n)}) \leq 2^{-n(I(X;Y)-3\epsilon)}. \quad (423)$$

Portanto:

$$\Pr(\varepsilon) \leq \epsilon + \sum_{i=2}^M 2^{-n(I(X;Y)-3\epsilon)} \quad (424a)$$

$$= \epsilon + (M - 1)2^{-n(I(X;Y)-3\epsilon)} \quad (424b)$$

$$= \epsilon + (\lceil 2^{nR} \rceil - 1)2^{-n(I(X;Y)-3\epsilon)} \quad (424c)$$

$$\leq \epsilon + (2^{nR} + 1 - 1)2^{-n(I(X;Y)-3\epsilon)} \quad (424d)$$

$$= \epsilon + 2^{3n\epsilon} 2^{-n(I(X;Y)-R)} \quad (424e)$$

$$= \epsilon + 2^{-n((I(X;Y)-3\epsilon)-R)} \quad (424f)$$

$$\leq 2\epsilon, \quad (424g)$$

para n suficientemente grande. Para $R < I(X;Y) - 3\epsilon$, o termo $2^{-n((I(X;Y)-3\epsilon)-R)}$ tende a zero para $n \rightarrow \infty$.

Otimização da taxa

Escolhendo $p^*(x) = \operatorname{argmax}_{p(x)} I(X; Y)$, temos $I(X; Y) = C$. Para qualquer $R < C$, podemos escolher $\epsilon > 0$ pequeno tal que $R < C - 3\epsilon$, garantindo $\Pr(\epsilon) \rightarrow 0$.

Probabilidade máxima de erro

Como $E[P_e^{(n)}(C)] \leq 2\epsilon$, existe um código C^* com:

$$P_e^{(n)}(C^*) \leq 2\epsilon \quad (425)$$

(se a média é menor ou igual a 2ϵ , então devemos ter ao menos um valor menor ou igual a 2ϵ).

Seja $B = \{i : \lambda_i(C^*) \geq 4\epsilon\}$. Suponha $|B| > M/2$. Então:

$$P_e^{(n)}(C^*) \geq \frac{1}{M} \sum_{i \in B} \lambda_i(C^*) \geq \frac{|B|}{M} \cdot 4\epsilon > \frac{1}{2} \cdot 4\epsilon = 2\epsilon, \quad (426)$$

contradizendo $P_e^{(n)}(C^*) \leq 2\epsilon$. Logo, $|B| \leq M/2$, e pelo menos $M/2 = \lceil 2^{nR} \rceil / 2 \geq 2^{n(R-1/n)}$ palavras têm $\lambda_i < 4\epsilon$. Criamos um novo código C' com essas palavras, taxa $R' = R - 1/n$, e $\lambda^{(n)} \leq 4\epsilon$. Como $R' \rightarrow R$ à medida que $n \rightarrow \infty$, e ϵ é arbitrário, existe uma família de códigos com $\lambda^{(n)} \rightarrow 0$ para taxa $R < C$.

□

Demonstração: Taxas $R > C$ não são alcançáveis.

Seja um canal discreto sem memória $(\mathcal{X}, p(y|x), \mathcal{Y})$ com capacidade $C = \max_{p(x)} I(X; Y)$. Queremos mostrar que qualquer família de códigos $(\lceil 2^{nR} \rceil, n)$ com probabilidade máxima de erro $\lambda^{(n)} \rightarrow 0$ à medida que $n \rightarrow \infty$ deve satisfazer $R \leq C$.

Consideremos uma família de códigos $(\lceil 2^{nR} \rceil, n)$, com $M = \lceil 2^{nR} \rceil$ mensagens, onde $W \in \{1, \dots, M\}$ é a mensagem enviada, $X^n = f(W)$ é a palavra-código transmitida, e Y^n é a saída do canal, com $\hat{W} = g(Y^n)$ sendo a mensagem decodificada. Suponhamos que $\lambda^{(n)} \rightarrow 0$, o que implica que a probabilidade média de erro $P_e^{(n)} = \frac{1}{M} \sum_{i=1}^M \lambda_i \rightarrow 0$, pois $\lambda_i \leq \lambda^{(n)}$ para todo i .

A entropia de W satisfaz $H(W) \leq \log M$, aproximando-se da igualdade quando W for uniforme. Pela identidade da entropia:

$$H(W) = H(W | Y^n) + I(W; Y^n). \quad (427)$$

Como $P_e^{(n)} \rightarrow 0$, a incerteza condicional $H(W | Y^n)$ é limitada. Pelo lema de Fano (veja o Teorema 51), temos:

$$H(W | Y^n) \leq h(P_e^{(n)}) + P_e^{(n)} \log(M - 1), \quad (428)$$

onde $h(\cdot)$ é a entropia binária. Como $P_e^{(n)} \rightarrow 0$, temos $h(P_e^{(n)}) \rightarrow 0$.

Como $M = \lceil 2^{nR} \rceil$, segue que $M - 1 \leq 2^{nR}$, então:

$$\log(M - 1) \leq \log(2^{nR}) = nR. \quad (429)$$

Assim, teremos que

$$P_e^{(n)} \log(M-1) \leq P_e^{(n)} \cdot nR \rightarrow 0, \quad (430)$$

pois $P_e^{(n)}$ tende a zero enquanto nR cresce linearmente. Desta forma, a partir da Equação (428), como $h(P_e^{(n)}) \rightarrow 0$ e $P_e^{(n)} \log(M-1) \rightarrow 0$, quando $\delta_n \rightarrow 0$, obtemos:

$$H(W | Y^n) \leq \delta_n, \quad (431)$$

com $\delta_n \rightarrow 0$, à medida que $n \rightarrow \infty$. Portanto, a partir da Equação (427), teremos:

$$H(W) \leq I(W; Y^n) + \delta_n. \quad (432)$$

Pela desigualdade de processamento de dados, já que $W \rightarrow X^n \rightarrow Y^n$ forma uma cadeia de Markov:

$$I(W; Y^n) \leq I(X^n; Y^n). \quad (433)$$

A informação mútua $I(X^n; Y^n)$ é dada por:

$$I(X^n; Y^n) = H(Y^n) - H(Y^n | X^n). \quad (434)$$

Para o canal sem memória, $p(y^n | x^n) = \prod_{i=1}^n p(y_i | x_i)$, então:

$$H(Y^n | X^n) = \sum_{i=1}^n H(Y_i | X_i). \quad (435)$$

Além disso:

$$H(Y^n) \leq \sum_{i=1}^n H(Y_i), \quad (436)$$

pois a entropia conjunta é no máximo a soma das entropias marginais.

Assim:

$$I(X^n; Y^n) \leq \sum_{i=1}^n H(Y_i) - \sum_{i=1}^n H(Y_i | X_i) = \sum_{i=1}^n I(X_i; Y_i). \quad (437)$$

Como $I(X_i; Y_i) \leq \max_{p(x)} I(X; Y) = C$, temos:

$$I(X^n; Y^n) \leq \sum_{i=1}^n I(X_i; Y_i) \leq nC. \quad (438)$$

Portanto, a partir da Equação (432), obtemos:

$$H(W) \leq I(X^n; Y^n) + \delta_n \leq nC + \delta_n. \quad (439)$$

Para n grande suficiente, temos que $\lceil 2^{nR} \rceil \approx 2^{nR}$ e assim $\log \lceil 2^{nR} \rceil \approx \log 2^{nR} = nR$. Para a distribuição uniforme em W teremos $H(W) \approx nR$, segue então que:

$$nR \leq nC + \delta_n. \quad (440)$$

Dividindo por n :

$$R \leq C + \frac{\delta_n}{n}. \quad (441)$$

À medida que $n \rightarrow \infty$, $\delta_n/n \rightarrow 0$, logo $R \leq C$. Assim, se $\lambda^{(n)} \rightarrow 0$, a taxa deve satisfazer $R \leq C$, e taxas $R > C$ não são alcançáveis, pois implicariam $P_e^{(n)}$ não tendendo a zero.

□

O Lema de Fano, também conhecido como desigualdade de Fano, foi utilizado proposição inversa do teorema de codificação de canal de Shannon. Ele fornece um limite superior para a entropia condicional $H(W | Y^n)$ da mensagem W dado o canal de saída Y^n , em termos da probabilidade média de erro $P_e^{(n)}$.

Lema 51 (Desigualdade de Fano) *Seja um canal discreto sem memória $(\mathcal{X}, p(y|x), \mathcal{Y})$ com um código $(\lceil 2^{nR} \rceil, n)$, onde $W \in \{1, \dots, M\}$ é a mensagem enviada, $X^n = f(W)$ é a palavra-código, Y^n é a saída do canal, e $\hat{W} = g(Y^n)$ é a mensagem decodificada, com $M = \lceil 2^{nR} \rceil$. Denote $P_e^{(n)} = \Pr(\hat{W} \neq W)$ a probabilidade média de erro. Então:*

$$H(W | Y^n) \leq h(P_e^{(n)}) + P_e^{(n)} \log(M - 1), \quad (442)$$

onde $h(p) = -p \log p - (1 - p) \log(1 - p)$ é a entropia binária.

Demonstração. Seja um canal discreto sem memória $(\mathcal{X}, p(y|x), \mathcal{Y})$ com um código $(\lceil 2^{nR} \rceil, n)$, onde $W \in \{1, \dots, M\}$ é a mensagem enviada, com $M = \lceil 2^{nR} \rceil$, $X^n = f(W)$ é a palavra-código, Y^n é a saída do canal, e $\hat{W} = g(Y^n)$ é a mensagem decodificada. Definimos a variável indicadora de erro E como:

$$E = \begin{cases} 1, & \text{se } \hat{W} \neq W, \\ 0, & \text{se } \hat{W} = W. \end{cases} \quad (443)$$

A probabilidade de erro é $P_e^{(n)} = \Pr(E = 1) = \Pr(\hat{W} \neq W)$.

Consideremos a entropia conjunta $H(E, W | Y^n)$. Pela regra da cadeia:

$$H(E, W | Y^n) = H(W | Y^n) + H(E | W, Y^n). \quad (444)$$

Como $\hat{W} = g(Y^n)$ é uma função de Y^n , dado W e Y^n , E é completamente determinado (pois $E = 1$ se $\hat{W} \neq W$, e $E = 0$ se $\hat{W} = W$). Assim: $H(E | W, Y^n) = 0$. Portanto:

$$H(E, W | Y^n) = H(W | Y^n). \quad (445)$$

Alternativamente, pela regra da cadeia:

$$H(E, W | Y^n) = H(E | Y^n) + H(W | E, Y^n). \quad (446)$$

Como E é binária ($E \in \{0, 1\}$), sua entropia é limitada, e assim podemos escrever:

$$H(E | Y^n) \leq H(E) \leq h(P_e^{(n)}), \quad (447)$$

onde $h(p) = -p \log p - (1-p) \log(1-p)$ é a entropia binária.

Agora, calculamos $H(W | E, Y^n)$:

$$\begin{aligned} H(W | E, Y^n) &= \Pr(E = 0)H(W | E = 0, Y^n) + \\ &\quad \Pr(E = 1)H(W | E = 1, Y^n). \end{aligned} \quad (448)$$

Para $E = 0$, temos $\hat{W} = W$, ou seja, dado Y^n e $E = 0$, W é conhecido exatamente (pois $\hat{W} = g(Y^n) = W$). Assim: $H(W | E = 0, Y^n) = 0$.

Para $E = 1$, temos $\hat{W} \neq W$, então W pode ser qualquer uma das $M - 1$ mensagens exceto $\hat{W}$. No pior caso, W é uniforme sobre $M - 1$ possibilidades, então: $H(W | E = 1, Y^n) \leq \log(M - 1)$. Como $\Pr(E = 0) = 1 - P_e^{(n)}$ e $\Pr(E = 1) = P_e^{(n)}$, temos:

$$H(W | E, Y^n) \leq (1 - P_e^{(n)}) \cdot 0 + P_e^{(n)} \cdot \log(M - 1) = P_e^{(n)} \log(M - 1). \quad (449)$$

Portanto:

$$H(E, W | Y^n) \leq H(P_e^{(n)}) + P_e^{(n)} \log(M - 1). \quad (450)$$

Igualando as duas expressões para $H(E, W | Y^n)$, Equações (445) e (450), obtemos:

$$H(W | Y^n) = H(E, W | Y^n) \leq H(P_e^{(n)}) + P_e^{(n)} \log(M - 1). \quad (451)$$

□

Na transmissão de informação, a compressão eficiente de uma fonte e a utilização de um canal de comunicação são processos interligados. Pelo Teorema de Codificação de Fonte de Shannon, uma fonte com entropia H pode ser comprimida sem perda a uma taxa $R \geq H$ bits por símbolo, enquanto o Teorema de Codificação de Canal estabelece que a transmissão confiável é possível em um canal discreto sem memória com capacidade C a taxas $R < C$ bits por utilização do canal. Consideremos uma fonte que produz símbolos $V_i \in \mathcal{V}$, formando sequências $V^n = (V_1, \dots, V_n)$, que satisfazem a propriedade da equipartição assintótica, garantindo que sequências típicas dominam a probabilidade. Para transmitir V^n , um codificador mapeia a sequência em uma entrada X^n , que passa pelo canal, gerando uma saída Y^n . O decodificador, a partir de Y^n , estima a sequência original, produzindo $\hat{V}^n$. A confiabilidade da transmissão é medida pela probabilidade de erro $\Pr(V^n \neq \hat{V}^n)$, que deve tender a zero para comunicação eficaz. Intuitivamente, se a taxa de entropia da fonte $H(\mathcal{V})$ for menor que

a capacidade C , é possível combinar compressão e transmissão para alcançar confiabilidade. O Teorema Conjunto de Codificação de Fonte-Canal formaliza essa ideia, mostrando que, para $H(\mathcal{V}) < C$, existe uma estratégia de codificação que assegura erro desprezível, sem exigir separação explícita entre codificação de fonte e canal.

Teorema 52 (Teorema Conjunto de Codificação de Fonte-Canal)

Seja $\{V_i\}$ um processo estocástico com alfabeto finito $\mathcal{V}$, que satisfaz a propriedade da equipartição assintótica, com taxa de entropia $H(\mathcal{V}) = \lim_{n \rightarrow \infty} \frac{1}{n} H(V_1, \dots, V_n)$, e seja $(\mathcal{X}, p(y|x), \mathcal{Y})$ um canal discreto sem memória com capacidade $C = \max_{p(x)} I(X; Y)$. Denote $V^n = (V_1, \dots, V_n)$ a sequência da fonte, codificada como $X^n = f(V^n)$, transmitida pelo canal, gerando Y^n , com a estimativa $\hat{V}^n = g(Y^n)$ produzida pelo decodificador. A probabilidade de erro é definida como:

$$P_e^{(n)} = \Pr(V^n \neq \hat{V}^n) = \sum_{y^n} \sum_{v^n} \Pr(v^n) \Pr(y^n | X^n = f(v^n)) \mathbf{1}_{\{g(y^n) \neq v^n\}}. \quad (452)$$

Se $H(\mathcal{V}) < C$, então existe uma sequência de códigos $(\lceil 2^{nR} \rceil, n)$, com $R > H(\mathcal{V})$, tal que:

$$P_e^{(n)} \rightarrow 0 \quad \text{à medida que } n \rightarrow \infty. \quad (453)$$

Por outro lado, se $H(\mathcal{V}) > C$, então, para qualquer sequência de códigos, $P_e^{(n)}$ é limitada inferiormente por uma constante positiva, e não é possível alcançar $P_e^{(n)} \rightarrow 0$.

Demonstração. Seja $\{V_i\}$ um processo estocástico com alfabeto finito $\mathcal{V}$, que satisfaz a propriedade da equipartição assintótica (AEP), com taxa de entropia $H(\mathcal{V}) = \lim_{n \rightarrow \infty} \frac{1}{n} H(V_1, \dots, V_n)$, e seja $(\mathcal{X}, p(y|x), \mathcal{Y})$ um canal discreto sem memória com capacidade $C = \max_{p(x)} I(X; Y)$. Denote $V^n = (V_1, \dots, V_n)$ a sequência da fonte, codificada como $X^n = f(V^n)$, transmitida pelo canal, gerando Y^n , com a estimativa $\hat{V}^n = g(Y^n)$. A probabilidade de erro é $P_e^{(n)} = \Pr(V^n \neq \hat{V}^n)$.

Parte direta: $H(\mathcal{V}) < C$

Suponha $H(\mathcal{V}) < C$. Escolha $\epsilon > 0$ tal que $H(\mathcal{V}) + \epsilon < C$. Pela AEP, existe um conjunto típico $A_\epsilon^{(n)} \subseteq \mathcal{V}^n$ tal que, para n suficientemente grande:

$$\Pr(V^n \in A_\epsilon^{(n)}) \geq 1 - \epsilon, \quad (454)$$

e o número de sequências típicas satisfaz:

$$|A_\epsilon^{(n)}| \leq 2^{n(H(\mathcal{V}) + \epsilon)}. \quad (455)$$

Construímos um código $(\lceil 2^{nR} \rceil, n)$ com taxa $R = H(\mathcal{V}) + \epsilon$. Indexamos as sequências em $A_\epsilon^{(n)}$ com $M = \lceil 2^{n(H(\mathcal{V}) + \epsilon)} \rceil$ mensagens, ou

seja, $M \leq 2^{n(H(\mathcal{V})+\epsilon)} + 1$. Definimos o codificador $f : \mathcal{V}^n \rightarrow \mathcal{X}^n$ como segue: se $V^n \in A_\epsilon^{(n)}$, mapeie V^n para um índice $m \in \{1, \dots, M\}$, e gere $X^n = f(V^n)$ correspondente a m ; se $V^n \notin A_\epsilon^{(n)}$, declare um erro, contribuindo para $P_e^{(n)}$.

A probabilidade de erro é limitada por:

$$\begin{aligned} P_e^{(n)} &= \Pr(V^n \neq \hat{V}^n) \\ &\leq \Pr(V^n \notin A_\epsilon^{(n)}) + \Pr(\hat{V}^n \neq V^n \mid V^n \in A_\epsilon^{(n)}). \end{aligned} \quad (456)$$

Sabemos que $\Pr(V^n \notin A_\epsilon^{(n)}) \leq \epsilon$ pela AEP. Para a segunda parcela, observe que cada sequência típica $V^n \in A_\epsilon^{(n)}$ é mapeada a um índice m , e transmitimos m usando um código de canal com $M \approx 2^{nR}$ mensagens, onde $R = H(\mathcal{V}) + \epsilon < C$. Pelo Teorema de Codificação de Canal de Shannon, para $R < C$, existe um código $(\lceil 2^{nR} \rceil, n)$ com probabilidade máxima de erro $\lambda^{(n)} \rightarrow 0$. Assim, a probabilidade de decodificar m incorretamente (e, portanto, $\hat{V}^n \neq V^n$) é menor que ϵ para n grande, pois $\lambda^{(n)} < \epsilon$. Logo:

$$P_e^{(n)} \leq \epsilon + \epsilon = 2\epsilon. \quad (457)$$

Como ϵ é arbitrário e n pode ser escolhido suficientemente grande, temos $P_e^{(n)} \rightarrow 0$, provando que a transmissão é confiável para $H(\mathcal{V}) < C$.

Parte inversa: $H(\mathcal{V}) > C$

Suponha $H(\mathcal{V}) > C$. Queremos mostrar que $P_e^{(n)}$ não tende a zero para qualquer sequência de códigos. Considere a entropia da sequência V^n . Pela AEP, $H(V^n) \approx nH(\mathcal{V})$ para n grande. A informação transmitida por V^n deve passar pelo canal, que suporta no máximo nC bits com erro pequeno, segundo o Teorema de Codificação de Canal.

Pela identidade da entropia:

$$H(V^n) = H(V^n \mid Y^n) + I(V^n; Y^n). \quad (458)$$

Como $V^n \rightarrow X^n \rightarrow Y^n$ forma uma cadeia de Markov, temos, pela desigualdade de processamento de dados e pela definição de C , que:

$$I(V^n; Y^n) \leq I(X^n; Y^n) \leq nC, \quad (459)$$

já que $I(X^n; Y^n) \leq \sum_{i=1}^n I(X_i; Y_i) \leq nC$ para um canal sem memória. Assim, a Equação (458) pode ser reescrita como:

$$H(V^n) \leq H(V^n \mid Y^n) + nC. \quad (460)$$

Se $P_e^{(n)} \rightarrow 0$, então $\hat{V}^n = g(Y^n)$ é igual a V^n com alta probabilidade, sugerindo que $H(V^n \mid Y^n)$ é pequeno. Pela desigualdade de Fano (veja o Teorema 51):

$$H(V^n \mid Y^n) \leq h(P_e^{(n)}) + P_e^{(n)} \log(|\mathcal{V}|^n - 1), \quad (461)$$

onde $h(p) = -p \log p - (1-p) \log(1-p)$, a entropia binária. Como $|\mathcal{V}|^n - 1 \leq |\mathcal{V}|^n = 2^{n \log |\mathcal{V}|}$, a Equação (461) pode ser expressa como:

$$H(V^n | Y^n) \leq H(P_e^{(n)}) + P_e^{(n)} n \log |\mathcal{V}|. \quad (462)$$

Se $P_e^{(n)} \rightarrow 0$, então $H(P_e^{(n)}) \rightarrow 0$ e $P_e^{(n)} n \log |\mathcal{V}| \rightarrow 0$, implicando $H(V^n | Y^n) \leq \delta_n$, com $\delta_n \rightarrow 0$. Assim:

$$H(V^n) \leq nC + \delta_n. \quad (463)$$

Mas $H(V^n) \approx nH(\mathcal{V})$ pela AEP, então:

$$nH(\mathcal{V}) \leq nC + \delta_n. \quad (464)$$

Dividindo por n e tomando $n \rightarrow \infty$:

$$H(\mathcal{V}) \leq C + \frac{\delta_n}{n} \rightarrow C. \quad (465)$$

Isso contradiz $H(\mathcal{V}) > C$. Portanto, $P_e^{(n)}$ não pode tender a zero, pois $H(V^n | Y^n)$ permaneceria grande, indicando que o canal não suporta transmitir $H(\mathcal{V}) > C$ bits por símbolo com erro arbitrariamente pequeno. Logo, para $H(\mathcal{V}) > C$, $P_e^{(n)}$ é limitada inferiormente por uma constante positiva.

Portanto, o teorema está provado: $H(\mathcal{V}) < C$ permite $P_e^{(n)} \rightarrow 0$, e $H(\mathcal{V}) > C$ implica que $P_e^{(n)}$ não tende a zero.

□

Códigos de Correção de Erro

Um dos principais resultados visto anteriormente, conhecido como o segundo teorema de Shannon, afirma que, para taxas de transmissão R inferiores à capacidade do canal C , existe uma sequência de códigos capaz de reduzir a probabilidade de erro a zero, desde que o tamanho dos blocos codificados seja suficientemente grande. Em termos matemáticos, se $R < C$, é possível projetar códigos que, com o aumento do comprimento do bloco, garantem uma transmissão praticamente livre de erros. No entanto, esse teorema é um resultado de existência: ele não especifica como construir tais códigos, nem considera as limitações práticas de implementação, como a complexidade computacional ou os recursos físicos disponíveis.

Na prática, a codificação baseada em sequências típicas, que aproxima o limite teórico de Shannon, exige blocos de tamanho exponencialmente grande, o que é inviável para a maioria das aplicações reais. Para superar esse desafio, recorremos aos códigos de correção de erro, que introduzem redundância controlada às mensagens, permitindo não apenas a detecção, mas também a correção de erros causados pelo ruído no canal. Esses códigos são projetados para equilibrar eficiência (taxa de transmissão) e robustez (capacidade de correção), tornando a comunicação confiável mesmo em condições adversas.

Os códigos de correção de erro podem ser classificados em dois grandes grupos. Os códigos de bloco transformam mensagens de k -bits em palavras-código de n -bits, com $n > k$, sem depender de transmissões anteriores. Exemplos incluem o código de Hamming, amplamente utilizado em memórias RAM para proteger dados contra erros, e o código Reed-Solomon, essencial em tecnologias como CDs, DVDs, códigos QR e comunicações espaciais, como as missões Voyager e Galileo. Já os códigos convolucionais operam como máquinas de estado finito, gerando saídas que dependem tanto da entrada atual quanto de estados anteriores. Um exemplo proeminente é o código Turbo, que desempenha um papel crucial em comunicações móveis modernas, como 3G, 4G e LTE.

Além da abordagem baseada em codificação, a confiabilidade da comunicação pode ser melhorada por meios físicos, como o aumento

da potência de transmissão, o uso de componentes eletrônicos com menor tolerância ao ruído ou a redução de interferências ambientais, como ruído térmico ou poeira. Contudo, essas soluções frequentemente elevam os custos ou a complexidade do sistema, tornando os códigos de correção de erro uma alternativa mais elegante e eficiente. Em muitos casos, é a combinação de codificação inteligente e otimizações físicas que garante o sucesso de sistemas de comunicação modernos.

Distância de Hamming

Para projetar códigos de correção de erro eficazes, precisamos de uma métrica que quantifique quão diferentes são as palavras-código de um código. Essa métrica é conhecida como distância de Hamming, um conceito central na teoria dos códigos.

Definição 58 (Distância de Hamming). A *distância de Hamming* entre duas sequências de mesmo comprimento, definidas sobre um alfabeto finito qualquer, é o número de posições em que elas diferem. Formalmente, para duas palavras $x = (x_1, x_2, \dots, x_n)$ e $y = (y_1, y_2, \dots, y_n)$, a distância de Hamming é dada por:

$$d_H(x, y) = |\{i : x_i \neq y_i\}|. \quad (466)$$

Exemplo 34 (Distância de Hamming para sequências binárias). Por exemplo, considere as palavras-código binárias $x = 10110$ e $y = 11011$. Para sequências binárias, podemos calcular a distância de Hamming usando a operação XOR bit a bit, que resulta em 1 nas posições onde os bits diferem, e então contar o número de 1s. Comparando posição por posição:

$$\begin{array}{c|ccccc} x & 1 & 0 & 1 & 1 & 0 \\ y & 1 & 1 & 0 & 1 & 1 \\ \hline x \oplus y & 0 & 1 & 1 & 0 & 1 \end{array}$$

O resultado do XOR é 01101, com 1s nas posições 2, 3 e 5, indicando que essas posições diferem. Assim, $d_H(x, y) = 3$.

Exemplo 35 (Distância de Hamming para alfabeto não binário).

Para um alfabeto não binário, considere as sequências $u = (0, 1, 2, 0)$ e $v = (0, 2, 2, 1)$ sobre o alfabeto $\{0, 1, 2\}$. Comparamos cada posição: $u_1 = v_1 = 0$ (iguais), $u_2 = 1 \neq v_2 = 2$, $u_3 = 2 = v_3 = 2$, $u_4 = 0 \neq v_4 = 1$. As posições 2 e 4 diferem, logo $d_H(u, v) = 2$.

Definição 59 (Distância mínima de um código). Para um código C , composto por um conjunto de palavras-código, a *distância mínima* $d_{\min}$ é definida como a menor distância de Hamming entre quaisquer duas palavras-código distintas:

$$d_{\min} = \min\{d_H(x, y) : x, y \in C, x \neq y\}. \quad (467)$$

A distância mínima é um parâmetro fundamental, pois determina a capacidade do código de detectar e corrigir erros, como estabelecido pelos seguintes teoremas.

Teorema 53 (Detecção de Erros) *Um código C com distância mínima $d_{\min}$ pode detectar até $d_{\min} - 1$ erros em qualquer palavra-código.*

Demonstração. Suponha que uma palavra-código $c \in C$ seja transmitida e receba até k erros, resultando em uma sequência r , tal que $d_H(c, r) \leq k$. Para que o erro seja detectado, r não deve ser uma palavra-código válida, ou seja, $r \notin C$. Como $d_{\min}$ é a menor distância entre quaisquer duas palavras-código distintas, para qualquer $c' \in C$, com $c' \neq c$, temos $d_H(c, c') \geq d_{\min}$. Se $k \leq d_{\min} - 1$, então:

$$d_H(c, r) \leq d_{\min} - 1 < d_{\min}. \quad (468)$$

Assim, r não pode ser igual a nenhuma outra palavra-código c' , pois $d_H(c, r) < d_H(c, c')$. Portanto, $r \notin C$, e o erro é detectado. Se $k = d_{\min}$, r poderia ser igual a outra palavra-código, e o erro não seria detectado. Logo, o código é capaz de detectar até $d_{\min} - 1$ erros. $\square$

Teorema 54 (Correção de Erros) *Um código C com distância mínima $d_{\min}$ pode corrigir até $t = \lfloor (d_{\min} - 1)/2 \rfloor$ erros em qualquer palavra-código.*

Demonstração. Para corrigir erros, o decodificador deve identificar a palavra-código $c \in C$ mais próxima da sequência recebida r , segundo a distância de Hamming (decodificação por máxima verossimilhança). Considere esferas de Hamming de raio t centradas em cada palavra-código (veja a Figura 57), onde a esfera em torno de c contém todas as sequências r tais que $d_H(c, r) \leq t$. Para que o código corrija até t erros, essas esferas não devem se sobrepor, garantindo que r esteja na esfera de uma única palavra-código.

Seja $t = \lfloor (d_{\min} - 1)/2 \rfloor$. Suponha que r está à distância $\leq t$ de c , ou seja, $d_H(c, r) \leq t$, devido a ocorrência de até t erros em r . Para qualquer outra palavra-código $c' \neq c$, temos $d_H(c, c') \geq d_{\min}$. Queremos garantir que r não esteja na esfera de c' , ou seja, $d_H(c', r) > t$. Pela desigualdade triangular:

$$d_H(c', r) \geq d_H(c, c') - d_H(c, r) \geq d_{\min} - t. \quad (469)$$

Substituindo $t = \lfloor (d_{\min} - 1)/2 \rfloor$, temos:

$$d_{\min} - t = d_{\min} - \lfloor (d_{\min} - 1)/2 \rfloor \quad (470a)$$

$$\geq d_{\min} - (d_{\min} - 1)/2 \quad (470b)$$

$$= (d_{\min} + 1)/2 \quad (470c)$$

$$> t, \quad (470d)$$

onde em 470b utilizamos que $\lfloor x \rfloor \leq x$. Assim, $d_H(c', r) > t$, e r está na esfera de apenas uma palavra-código, permitindo corrigir até t erros. Se o número de erros fosse $t + 1$, as esferas poderiam se sobrepor, levando a erros de decodificação. Portanto, o código corrige até t erros.

□

Exemplo 36 (Correção de erros em um código de repetição com $n = 3$). Considere um código simples com palavras-código $\{000, 111\}$ sobre o alfabeto binário. A distância de Hamming entre 000 e 111 é 3, logo $d_{\min} = 3$. Pelo Teorema de Detecção de Erros, esse código detecta até $3 - 1 = 2$ erros. Pelo Teorema de Correção de Erros, ele corrige até $\lfloor (3 - 1)/2 \rfloor = 1$ erro. Se recebermos 001, sabemos que houve erro (pois $001 \notin C$) e corrigimos para 000, que está a distância 1. Note que, para obter a sequência 111 deveríamos considerar a ocorrência de dois erros, neste caso, é menos provável que dois erros tenham ocorrido do que apenas um. A decodificação de máxima verossimilhança escolhe a alternativa mais verossímil, no caso, a sequência 000.

A distância de Hamming está diretamente ligada à redundância do código: adicionar mais símbolos (aumentando n) permite maior separação entre palavras-código, mas reduz a taxa $R = k/n$. Existe então uma relação de compromisso entre aumentar $d_{\min}$ e reduzir R .

Código de Repetição

O código de repetição é o exemplo mais simples de um código de correção de erro. Apesar de sua simplicidade, ele ilustra o papel da redundância e serve como ponto de partida para entender outros códigos.

Definição 60 (Código de repetição). Um *código de repetição* com parâmetro n codifica cada bit da mensagem repetindo-o n vezes. Para uma mensagem de 1 bit:

$$0 \mapsto \underbrace{00 \dots 0}_n, \quad 1 \mapsto \underbrace{11 \dots 1}_n. \quad (471)$$

Um código de repetição terá taxa $R = 1/n$, distância mínima $d_{\min} = n$ e será capaz de corrigir até $t = \lfloor (d_{\min} - 1)/2 \rfloor = \lfloor (n - 1)/2 \rfloor$ bits.

Exemplo 37 (Correção de erros em um código de repetição com $n = 3$). Revisitando o exemplo anterior, um código de repetição com $n = 3$, terá a seguinte tabela de codificação:

mensagem	0	1
palavra-código	000	111

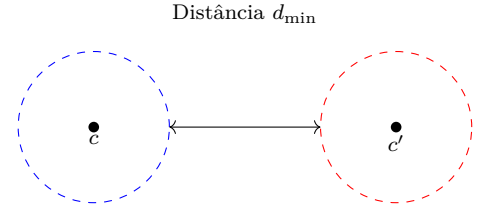


Figura 57: Esferas de Hamming de raio $t = \lfloor (d_{\min} - 1)/2 \rfloor$. As palavras-código c e c' estão separadas por $d_{\min}$, garantindo que erros de até t símbolos sejam corrigidos.

Tabela 5: Código de repetição com $n = 3$.

A mensagem 10, por exemplo, será codificada como $10 \mapsto 111000$. A decodificação é feita por regra/voto de maioria. Para cada bloco de n bits, escolhemos o bit mais frequente. Por exemplo, se $n = 3$ e recebemos 011, contamos dois 1s e um 0, decodificando como 1 (palavra-código 111). Este código terá uma taxa de $R = 1/3$, distância mínima $d_{\min} = 3$ e poderá corrigir apenas 1 bit.

Embora o código de repetição seja fácil de implementar, sua baixa taxa o torna ineficiente para aplicações práticas. Através da análise deste código simples, foi possível verificar a relação entre redundância e distância de Hamming. Aumentar n eleva $d_{\min}$, melhorando a correção de erros, mas reduz drasticamente a taxa.

Código de Verificação de Paridade Simples

O código de verificação de paridade simples consiste em adicionar um único bit de redundância a uma mensagem binária, permitindo detectar certos tipos de erros, embora não seja capaz de corrigi-los. Este código serve como base para muitos esquemas de codificação, como o código de Hamming e os códigos de verificação de paridade de baixa densidade (LDPC).

Definição 61 (Código de verificação de paridade simples). O código de verificação de paridade simples opera com entrada e saída binárias, ou seja, $\mathcal{X} = \mathcal{Y} = \{0, 1\}$. Uma mensagem de $n - 1$ bits, denotada $x_{1:n-1} = (x_1, x_2, \dots, x_{n-1})$, é codificada em uma palavra-código de n bits, onde os primeiros $n - 1$ são iguais àqueles da mensagem original e o n -ésimo bit, chamado bit de paridade, é definido como:

$$x_n = \left(\sum_{i=1}^{n-1} x_i \right) \bmod 2. \quad (472)$$

Assim, $x_n = 0$ se o número de 1s em $x_{1:n-1}$ for par, e $x_n = 1$ se for ímpar, garantindo que a soma total de 1s na palavra-código $(x_1, x_2, \dots, x_n)$ seja par.

Uma palavra-código $(x_1, x_2, \dots, x_n)$ é considerada válida se satisfizer a condição de paridade:

$$\left(\sum_{i=1}^n x_i \right) \bmod 2 = 0. \quad (473)$$

Essa condição implica que o número total de 1s na palavra-código é par. Durante a transmissão, se um número ímpar de bits for alterado (ou seja, se ocorrer um número ímpar de erros), essa soma deixará de ser par, permitindo a detecção do erro. No entanto, um número par de erros manterá a soma par, passando despercebido.

Teorema 55 (Detecção de Erros por Paridade) *O código de verificação de paridade simples detecta qualquer número ímpar de erros em uma palavra-código, mas não detecta um número par de erros.*

Demonstração. Considere uma palavra-código válida $c = (c_1, c_2, \dots, c_n)$, onde $\sum_{i=1}^n c_i \mod 2 = 0$. Suponha que k bits sejam alterados devido a erros, resultando em uma sequência recebida $r = (r_1, r_2, \dots, r_n)$. Cada erro inverte um bit ($0 \rightarrow 1$ ou $1 \rightarrow 0$), alterando a soma dos bits em $\sum_{i=1}^n r_i \mod 2 = k \mod 2$. Dessa forma:

$$\sum_{i=1}^n r_i \mod 2 = \sum_{i=1}^n c_i \mod 2 + k \mod 2. \quad (474)$$

Como $\sum_{i=1}^n c_i \mod 2 = 0$, temos:

$$\sum_{i=1}^n r_i \mod 2 = k \mod 2. \quad (475)$$

Se k é ímpar, $k \mod 2 = 1$, e a soma dos bits de r será ímpar, violando a condição de paridade e indicando um erro. Se k é par, $k \mod 2 = 0$, e a soma permanece par, fazendo r parecer uma palavra-código válida, mesmo que contenha erros. Portanto, o código detecta apenas um número ímpar de erros. $\square$

Exemplo 38 (Verificação de paridade simples). Considere um codificador $E : \{0, 1\}^3 \rightarrow \{0, 1\}^4$, definido por:

$$E(x_1, x_2, x_3) = (x_1, x_2, x_3, \sum_{i=1}^3 x_i \mod 2). \quad (476)$$

As palavras-código são todas as sequências de 4 bits com um número par de 1s:

$$C = \{(0, 0, 0, 0), (0, 0, 1, 1), (0, 1, 0, 1), (0, 1, 1, 0), (1, 0, 0, 1), (1, 0, 1, 0), (1, 1, 0, 0), (1, 1, 1, 1)\}.$$

A tabela 38 lista as mensagens e suas respectivas palavras-código.

Mensagem (x_1, x_2, x_3)	Palavra-código (x_1, x_2, x_3, x_4)
(0,0,0)	(0,0,0,0)
(0,0,1)	(0,0,1,1)
(0,1,0)	(0,1,0,1)
(0,1,1)	(0,1,1,0)
(1,0,0)	(1,0,0,1)
(1,0,1)	(1,0,1,0)
(1,1,0)	(1,1,0,0)
(1,1,1)	(1,1,1,1)

Tabela 6: Código de verificação de paridade simples com $n = 4$.

Suponha que a palavra-código $(0, 1, 0, 1)$ seja transmitida. Vamos analisar dois cenários: apagamento e erro. Para o caso de apagamento,

suponha que a sequência recebida seja $r = (0, x, 0, 1)$, onde o segundo bit foi apagado. Calculamos a soma dos bits conhecidos: $0 + 0 + 1 = 1 \pmod{2}$. Para que r seja válida, a soma total deve ser par, logo o bit apagado deve ser 1 (pois $1 + 1 = 0 \pmod{2}$). Assim, reconstruímos $r = (0, 1, 0, 1)$, recuperando a palavra-código correta. Agora, paa o caso de erro, suponha que a sequência recebida seja $r = (0, 0, 0, 1)$. Calculamos a soma: $0 + 0 + 0 + 1 = 1 \pmod{2}$, que é ímpar. Como a soma viola a condição de paridade, detectamos que houve um erro (neste caso, um erro no segundo bit, pois $(0, 1, 0, 1) \rightarrow (0, 0, 0, 1)$). Se a sequência recebida fosse $r = (0, 0, 0, 0)$ (decorrente de dois erros, nos bits 2 e 4), a soma seria $0 + 0 + 0 + 0 = 0 \pmod{2}$, indicando paridade válida. No entanto, como $(0, 0, 0, 0) \neq (0, 1, 0, 1)$, houve dois erros que não foram detectados, confirmando que o código falha para um número par de erros.

A distância mínima do código de paridade é $d_{\min} = 2$. Duas palavras-código diferem ao menos em 1 bit dentre os primeiros $n - 1$ bits, por serem palavras diferentes. Além disso, neste caso, elas devem também diferir no bit paridade, fazendo com que $d_{\min} = 2$. Note que, o bit de paridade de duas palavras distintas pode ser o mesmo, desde que as duas palavras tenham ao menos dois bits distintos entre os primeiros $n - 1$ bits. De qualquer forma, teremos $d_{\min} = 2$.

Pelo teorema de detecção de erros, um código com $d_{\min} = 2$ detecta até 1 erro, o que é consistente com nossa análise: um erro (ímpar) viola a paridade, mas dois erros (par) não são detectados. Para correção, usando o teorema de correção de erros, temos $t = \lfloor (d_{\min} - 1)/2 \rfloor = 0$, indicando que o código não corrige erros, apenas os detecta.

O código de verificação de paridade simples é limitado, pois não corrige erros e falha em detectar erros pares. No entanto, sua simplicidade o torna útil em aplicações como memórias de computadores e comunicações básicas, onde a detecção de erros é suficiente. Ele ainda é a base para outros códigos, como por exemplo, o código de Hamming.

Definição de Códigos $[n, k, d]$

Na teoria dos códigos de correção de erros, é comum descrever um código por meio de três parâmetros que resumem suas propriedades essenciais. Essa notação, conhecida como *códigos $[n, k, d]$* , fornece uma maneira concisa de caracterizar a estrutura e a capacidade de um código, permitindo comparar diferentes esquemas de codificação.

Definição 62 (Códigos $[n, k, d]$). Um código $[n, k, d]$ é um código de correção de erros linear com as seguintes características: possui comprimento de palavra de código n (incluindo os bits de dados e os bits de redundância); possui k bits de dados (tamanho da mensagem

antes da codificação); e com distância mínima de Hamming d entre quaisquer duas palavras-código distintas do código.

Como discutido anteriormente, a distância mínima $d_{\min}$ determina diretamente a capacidade de um código de lidar com erros. Utilizando resultados já estabelecidos anteriormente, podemos concluir que um código $[n, k, d]$ pode detectar até $d - 1$ erros, pois qualquer alteração de até $d - 1$ símbolos em uma palavra-código não a transforma em outra palavra-código válida; corrigir até $\lfloor (d - 1)/2 \rfloor$ erros, pois esferas de Hamming de raio $\lfloor (d - 1)/2 \rfloor$ centradas nas palavras-código não se sobrepõem, garantindo que a palavra-código correta seja identificada.

A taxa do código, definida como $R = k/n$, mede a eficiência do código: quanto maior k em relação a n , menos redundância e maior a taxa, mas isso pode reduzir d , comprometendo a capacidade de correção. O desafio no projeto de códigos é maximizar d e k para um dado n .

Data a Definição 62, podemos agora caracterizar os códigos vistos anteriormente como: código de repetição com $n = 3$ (mensagem de 1 bit repetida três vezes) é um código $[3, 1, 3]$; código de paridade simples com $n = 4$ é um código $[4, 3, 2]$.

Definição 63 (Código Linear). Um *código linear* $\mathcal{C}$ de comprimento n sobre um corpo $\mathbb{F} = \text{GF}(q)$ (onde q é geralmente uma potência de um primo) é um subespaço vetorial de $\mathbb{F}^n$ sobre $\mathbb{F}$. Em outras palavras, $\mathcal{C} \subseteq \mathcal{F}^n$ é um código linear se, para quaisquer palavras-código $\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}$, e quaisquer escalares $\alpha, \beta \in \mathbb{F}$, temos: $\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 \in \mathcal{C}$, onde as operações de soma e multiplicação por escalar são definidas componente a componente no corpo finito $\mathbb{F}_q$. Para um código binário ($q = 2$), $\mathbb{F} = \{0, 1\}$, e a soma é feita módulo 2.

Um código linear $\mathcal{C}$ com k dimensões (ou seja, $|\mathcal{C}| = q^k$) pode ser descrito por uma matriz geradora $\mathbf{G}$ de tamanho $k \times n$, onde cada palavra-código é da forma $\mathbf{v}\mathbf{G}$, com $\mathbf{v} \in \mathbb{F}^k$. Alternativamente, $\mathcal{C}$ é o espaço nulo de uma matriz de verificação de paridade $\mathbf{H}$ de tamanho $(n - k) \times n$, tal que $\mathbf{H}\mathbf{c} = \mathbf{0}$ para toda palavra $\mathbf{c} \in \mathcal{C}$.

Se $\mathcal{C}$ é um código linear (n, M, d) sobre $\mathbb{F}$ e a dimensionalidade de $\mathcal{C}$ é k , então dizemos que $\mathcal{C}$ é um código linear $[n, k, d]$ sobre $\mathbb{F}$. A diferença $n - k$ é a redundância do código. Toda base de um código linear $[n, k, d]$ $\mathcal{C}$ sobre $\mathbb{F} = \text{GF}(q)$ contém k palavras distintas que geram todo o código $\mathcal{C}$ por combinações lineares delas. Assim, $|\mathcal{C}| = M = q^k$ e a taxa do código é $R = \log_q M/n = k/n$. As linhas da matriz geradora $\mathbf{G}$ serão a base do código (podendo não ser única).

Definição 64 (Peso de Hamming). O *peso de Hamming* de uma palavra $\mathbf{c} \in \mathcal{C}$ é o número de posições em que $\mathbf{c}$ tem elementos não

nulos, ou seja:

$$w(\mathbf{c}) = |\{i \in \{1, 2, \dots, n\} : c_i \neq 0\}|, \quad (477)$$

onde $\mathbf{c} = (c_1, c_2, \dots, c_n)$. Para um código binário ($\mathcal{X} = \{0, 1\}$), o peso de Hamming é simplesmente o número de 1s na palavra.

Exemplo 39. Considere $\mathbf{c} = (1, 0, 1, 1, 0)$ em um código binário ($q = 2, n = 5$). O peso de Hamming é: $w(\mathbf{c}) = |\{1, 3, 4\}| = 3$, pois $\mathbf{c}$ tem 1s nas posições 1, 3 e 4.

Definição 65 (Peso mínimo de um código). O *peso mínimo de um código* $\mathcal{C} \subseteq \mathbb{F}^n$, onde $\mathcal{X} = \{0, 1, \dots, q-1\}$ é um alfabeto q -ário, é o menor peso de Hamming entre todas as palavras-código não nulas de $\mathcal{C}$, ou seja:

$$w_{\min}(\mathcal{C}) = \min\{w(\mathbf{c}) : \mathbf{c} \in \mathcal{C}, \mathbf{c} \neq \mathbf{0}\}, \quad (478)$$

onde $w(\mathbf{c})$ é o peso de Hamming de $\mathbf{c}$, conforme definido anteriormente.

Proposição 16 (Distância e peso mínimo) *Seja $\mathcal{C}$ um código linear $[n, k, d]$ sobre $\mathbb{F}$, então*

$$d = \min_{\mathbf{c} \in \mathcal{C} \setminus \{\mathbf{0}\}} w(\mathbf{c}). \quad (479)$$

Demonstração. Como $\mathcal{C}$ é linear, dadas duas palavras $\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C}$, teremos que $\mathbf{c}_1 - \mathbf{c}_2 \in \mathcal{C}$. Temos que $d(\mathbf{c}_1, \mathbf{c}_2) = w(\mathbf{c}_1 - \mathbf{c}_2)$ e assim

$$d = \min_{\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C} : \mathbf{c}_1 \neq \mathbf{c}_2} d(\mathbf{c}_1, \mathbf{c}_2) = \min_{\mathbf{c}_1, \mathbf{c}_2 \in \mathcal{C} : \mathbf{c}_1 \neq \mathbf{c}_2} w(\mathbf{c}_1 - \mathbf{c}_2) = \min_{\mathbf{c} \in \mathcal{C} \setminus \{\mathbf{0}\}} w(\mathbf{c}) \quad (480)$$

□

Definição 66 (Codificador de um código linear). Seja $\mathcal{C}$ um código linear $[n, k, d]$ sobre um corpo finito $\mathbb{F}_q$, com uma matriz geradora $\mathbf{G}$ de tamanho $k \times n$. Um codificador para $\mathcal{C}$ é um mapeamento bijetivo $\phi : \mathbb{F}_q^k \rightarrow \mathcal{C}$, definido por:

$$\phi(\mathbf{u}) = \mathbf{uG}, \quad (481)$$

onde $\mathbf{u} \in \mathbb{F}_q^k$ é a mensagem a ser codificada, e $\mathbf{uG} \in \mathcal{C}$ é a palavra-código correspondente. Como $\mathbf{G}$ tem posto k (suas k linhas são linearmente independentes), o mapeamento ϕ é uma bijeção, garantindo uma correspondência 1-a-1 entre mensagens e palavras-código.

A matriz geradora $\mathbf{G}$ está na forma sistemática se pode ser escrita como $\mathbf{G} = (\mathbf{I}_k \mid \mathbf{A})$, onde $\mathbf{I}_k$ é a matriz identidade $k \times k$, e $\mathbf{A}$ é uma matriz $k \times (n - k)$. Nesse caso, a codificação de uma mensagem $\mathbf{u}$ resulta em:

$$\mathbf{uG} = (\mathbf{u} \mid \mathbf{uA}), \quad (482)$$

onde a primeira parte $\mathbf{u}$ (os primeiros k bits) contém a mensagem original, e a segunda parte $\mathbf{uA}$ (os $n - k$ bits restantes) contém os bits de

paridade. Se $\mathbf{G}$ não estiver na forma sistemática, é possível permutar suas colunas para obter uma matriz equivalente $\mathbf{G}'$ na forma $(\mathbf{I}_k \mid \mathbf{A}')$, gerando o mesmo código $\mathcal{C}$, mas com uma ordenação diferente das posições dos bits.

Definição 67 (Matriz verificadora de paridade). Seja $\mathcal{C}$ um código linear $[n, k, d]$ sobre um corpo finito $\mathbb{F}_q$. A *matriz verificadora de paridade* de $\mathcal{C}$ é uma matriz $\mathbf{H}$ de tamanho $(n - k) \times n$ sobre $\mathbb{F}_q$, tal que:

$$\mathbf{H}\mathbf{c}^T = \mathbf{0}, \quad (483)$$

para todo $\mathbf{c} \in \mathcal{C}$, onde $\mathbf{c}^T$ é o transposto de $\mathbf{c}$, e $\mathbf{0}$ é o vetor nulo em $\mathbb{F}_q^{n-k}$. Em outras palavras, $\mathcal{C}$ é o kernel (ou espaço nulo) de $\mathbf{H}$, ou seja:

$$\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_q^n : \mathbf{H}\mathbf{c}^T = \mathbf{0}\}. \quad (484)$$

Pelo teorema do posto-nulidade²⁵, o posto de $\mathbf{H}$ mais a dimensão do seu kernel é igual ao número de colunas de $\mathbf{H}$. Como $\mathbf{H}$ tem n colunas e $\mathcal{C}$ tem dimensão k , temos:

$$\text{posto}(\mathbf{H}) + \dim(\mathcal{C}) = n, \quad (485)$$

e assim $\text{posto}(\mathbf{H}) = n - k$. Desta forma, as $n - k$ linhas de $\mathbf{H}$ são linearmente independentes, garantindo que $\mathbf{H}$ tenha posto máximo $n - k$.

Seja $\mathbf{G}$ uma matriz geradora $k \times n$ de $\mathcal{C}$. Como $\mathbf{c} = \mathbf{u}\mathbf{G} \in \mathcal{C}$ para algum $\mathbf{u} \in \mathbb{F}_q^k$, temos $\mathbf{H}(\mathbf{u}\mathbf{G})^T = \mathbf{0}$, ou seja:

$$\mathbf{H}\mathbf{G}^T\mathbf{u}^T = \mathbf{0}. \quad (486)$$

Para que isso seja verdadeiro para todo $\mathbf{u}$, deve-se ter:

$$\mathbf{H}\mathbf{G}^T = \mathbf{0}, \quad (487)$$

onde $\mathbf{0}$ é a matriz nula $(n - k) \times k$. Equivalentemente, $\mathbf{G}\mathbf{H}^T = \mathbf{0}$, pois $(\mathbf{H}\mathbf{G}^T)^T = \mathbf{G}\mathbf{H}^T$. Isso implica que as linhas de $\mathbf{G}$ estão no kernel de $\mathbf{H}$, e como $\mathbf{G}$ gera $\mathcal{C}$, as linhas de $\mathbf{G}$ formam uma base para o kernel de $\mathbf{H}$. Da mesma forma, as linhas de $\mathbf{H}$ estão no kernel de $\mathbf{G}$, ou seja, $\mathbf{H}$ gera o espaço ortogonal a $\mathcal{C}$, chamado de código dual $\mathcal{C}^\perp$, que tem dimensão $n - k$.

Quando $\mathbf{G}$ está na forma sistemática, ou seja, $\mathbf{G} = (\mathbf{I}_k \mid \mathbf{A})$, onde $\mathbf{I}_k$ é a identidade $k \times k$ e $\mathbf{A}$ é $k \times (n - k)$, a matriz $\mathbf{H}$ pode ser escrita como:

$$\mathbf{H} = (-\mathbf{A}^T \mid \mathbf{I}_{n-k}), \quad (488)$$

onde $\mathbf{I}_{n-k}$ é a identidade $(n - k) \times (n - k)$, e $\mathbf{A}^T$ é a transposta de $\mathbf{A}$. Isso ocorre porque $\mathbf{H}\mathbf{G}^T = (-\mathbf{A}^T \mid \mathbf{I}_{n-k})(\mathbf{I}_k \mid \mathbf{A})^T = -\mathbf{A}^T\mathbf{I}_k + \mathbf{I}_{n-k}\mathbf{A}^T = \mathbf{0}$, satisfazendo a condição de ortogonalidade. A forma sistemática de $\mathbf{H}$ facilita a verificação de paridade, pois os últimos $n - k$ bits da palavra-código são diretamente relacionados aos bits de paridade.

²⁵ O teorema de posto-nulidade, em álgebra linear, afirma que, para uma transformação linear $T : V \rightarrow W$ entre espaços vetoriais de dimensão finita, a soma da dimensão do núcleo (nulidade) de T e a dimensão da imagem (posto) de T é igual à dimensão do espaço de origem V , isto é, $\dim(\ker(T)) + \dim(\text{im}(T)) = \dim(V)$.

Código de Hamming (7,4,3)

O *código de Hamming* (Hamming 1950) é um exemplo clássico de código linear que combina detecção e correção de erros. Especificamente, o código de Hamming [7,4,3], capaz de corrigir um erro por palavra-código, sendo amplamente utilizado em aplicações como memórias RAM e sistemas de armazenamento RAID 2. Ele generaliza o conceito de paridade simples, usando múltiplos bits de verificação para localizar e corrigir erros.

Definição 68 (Código de Hamming (7,4,3)). O código de Hamming (7,4,3) é um código linear com os seguintes características: 7 bits de comprimento ($n = 7$); 4 bits de informação ($k = 4$); distância mínima 3; e 3 bits de verificação de paridade ($n - k = 3$). O alfabeto é binário, $\mathcal{X} = \mathcal{Y} = \{0, 1\}$, e a taxa do código é $R = k/n = 4/7$.

No Código de Hamming (7,4,3), uma mensagem de 4 bits, denotada (x_0, x_1, x_2, x_3) , é codificada em uma palavra-código de 7 bits, $(x_0, x_1, x_2, x_3, x_4, x_5, x_6)$, onde x_4, x_5, x_6 são bits de paridade. Esses bits são determinados pelas seguintes equações:

$$x_4 = \text{mod}(x_1 + x_2 + x_3, 2), \quad (489a)$$

$$x_5 = \text{mod}(x_0 + x_2 + x_3, 2), \quad (489b)$$

$$x_6 = \text{mod}(x_0 + x_1 + x_3, 2). \quad (489c)$$

Cada bit de paridade verifica a paridade (soma par de 1s) de um subconjunto específico de bits de dados, garantindo que a palavra-código satisfaça condições de validade.

Exemplo 40. Considere a mensagem $(x_0, x_1, x_2, x_3) = (0, 1, 1, 0)$. Calculamos os bits de paridade:

$$x_4 = (1 + 1 + 0) \text{ mod } 2 = 2 \text{ mod } 2 = 0, \quad (490a)$$

$$x_5 = (0 + 1 + 0) \text{ mod } 2 = 1 \text{ mod } 2 = 1, \quad (490b)$$

$$x_6 = (0 + 1 + 0) \text{ mod } 2 = 1 \text{ mod } 2 = 1. \quad (490c)$$

Assim, a palavra-código é $(0, 1, 1, 0, 0, 1, 1)$.

O formato como foi aqui proposto para o código é sistemático, separando os bits de dados dos bits de paridade em regiões distintas da palavra-código. Os bits de dados (x_0, x_1, x_2, x_3) aparecem nos primeiros 4 bits da palavra-código, seguidos pelos bits de paridade (x_4, x_5, x_6) nos 3 bits subsequentes. A codificação pode então ser representada matricialmente como:

$$\mathbf{x} = \mathbf{G}\mathbf{v}, \quad (491)$$

onde $\mathbf{v} = (x_0, x_1, x_2, x_3)$ é o vetor de dados, $\mathbf{x} = (x_0, x_1, x_2, x_3, x_4, x_5, x_6)$ é a palavra-código, e $\mathbf{G}$ é a matriz geradora:

$$\mathbf{G} = \left(\begin{array}{cccc|cccc} 1 & 0 & 0 & 0 & & & & \\ 0 & 1 & 0 & 0 & & & & \\ 0 & 0 & 1 & 0 & & & & \\ 0 & 0 & 0 & 1 & & & & \\ \hline 0 & 1 & 1 & 1 & & & & \\ 1 & 0 & 1 & 1 & & & & \\ 1 & 1 & 0 & 1 & & & & \end{array} \right) = \left(\begin{array}{c} \mathbf{I}_4 \\ \mathbf{P} \end{array} \right), \quad (492)$$

onde $\mathbf{I}_4$ é a matriz identidade 4x4, e $\mathbf{P}$ define as paridades. As colunas de $\mathbf{G}$ geram o espaço vetorial do código, e as $2^4 = 16$ palavras-código são combinações lineares dessas colunas, ponderadas por $\mathbf{v}$.

As palavras-código também podem ser definidas como o espaço nulo da matriz de verificação de paridade $\mathbf{H}$:

$$\mathbf{H} = \left(\begin{array}{cccccc|ccc} 0 & 1 & 1 & 1 & 1 & 0 & 0 & & \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 & & \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & & \end{array} \right) = \left(\mathbf{P} \quad \mathbf{I}_3 \right). \quad (493)$$

Uma palavra $\mathbf{x}$ é válida se $\mathbf{H}\mathbf{x} = \mathbf{0}$, onde $\mathbf{x}^T = (x_0, x_1, \dots, x_6)$. Isso equivale às condições:

$$x_1 + x_2 + x_3 + x_4 = 0 \pmod{2}, \quad (494a)$$

$$x_0 + x_2 + x_3 + x_5 = 0 \pmod{2}, \quad (494b)$$

$$x_0 + x_1 + x_3 + x_6 = 0 \pmod{2}. \quad (494c)$$

As colunas de $\mathbf{H}$ são as 7 permutações não nulas de vetores binários de 3 bits, garantindo que cada posição da palavra-código tenha uma “assinatura” única para identificação de erros.

Para os códigos lineares, como é o caso do código de Hamming, a matriz geradora é uma forma compacta de representar tal código. Já a matriz de verificação de paridade funciona como uma ferramenta para decodificação e análise do código.

As $2^4 = 16$ palavras-código são os vetores no espaço nulo de $\mathbf{H}$, $C = \{\mathbf{x} : \mathbf{H}\mathbf{x} = \mathbf{0}\}$, sendo eles elencados a seguir:

$$\begin{array}{cccc} 0000000 & 0100101 & 1000011 & 1100110 \\ 0001111 & 0101010 & 1001100 & 1101001 \\ 0010110 & 0110011 & 1010101 & 1110000 \\ 0011001 & 0111100 & 1011010 & 1111111 \end{array} \quad (495)$$

Note que o código é fechado em relação à soma e à subtração. Se $v_1, v_2 \in C$ então $\mathbf{H}(v_1 + v_2) = \mathbf{H}v_1 + \mathbf{H}v_2 = \mathbf{0}$, desta forma, $v_1 + v_2 \in C$. Da mesma forma também temos $v_1 - v_2 \in C$, uma vez que soma e subtração módulo 2 são equivalentes.

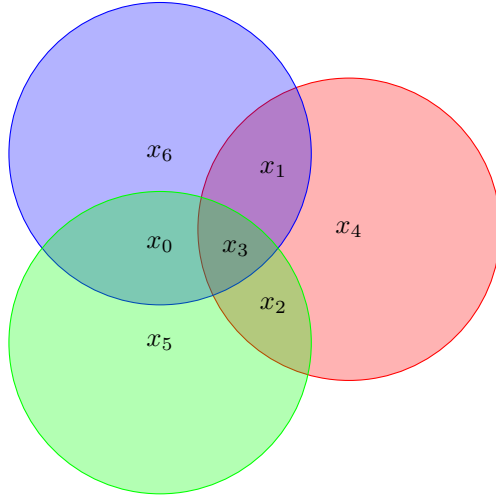


Figura 58: Diagrama de Venn para o código de Hamming (7,4,3). Cada círculo representa uma verificação de paridade: x_4 cobre x_1, x_2, x_3 , x_5 cobre x_0, x_2, x_3 , e x_6 cobre x_0, x_1, x_3 . As interseções mostram os bits compartilhados entre as verificações.

A distância mínima do código é $d_{\min} = 3$, igual ao peso mínimo, que é o menor número de 1s em uma palavra-código não nula. Para verificar, note que não existem palavras-código com peso 1 ou 2. Primeiramente, para peso 1, veja que uma palavra com um único 1 na posição i não satisfaz $\mathbf{H}\mathbf{x} = \mathbf{0}$, pois não existe coluna nula em $\mathbf{H}$. Para obter $\mathbf{H}\mathbf{x} = \mathbf{0}$, sendo $\mathbf{x}$ não nulo apenas na posição i , deveríamos ter a i -ésima coluna de $\mathbf{H}$ nula, o que não ocorre. Vejamos agora o próximo caso, peso 2. Note que a soma de duas colunas de $\mathbf{H}$ (módulo 2) nunca é zero, pois todas as colunas são distintas e não nulas. Por fim, podemos verificar que existem palavras com peso 3, como $(0, 0, 1, 0, 0, 1, 1)$, que satisfazem $\mathbf{H}\mathbf{x} = \mathbf{0}$. Como a distância de Hamming entre duas palavras $\mathbf{v}_1, \mathbf{v}_2 \in C$ é o peso de $\mathbf{v}_1 + \mathbf{v}_2$ (outra palavra-código), a distância mínima é igual ao peso mínimo, $d_{\min} = 3$.

Pelo teorema de detecção de erros, o código detecta até $d - 1 = 2$ erros. Pelo teorema de correção de erros, ele corrige até $\lfloor (d - 1)/2 \rfloor = \lfloor 2/2 \rfloor = 1$ erro por palavra-código. Isso torna o código de Hamming (7,4,3) ideal para canais onde erros únicos são mais prováveis, como em um canal binário simétrico (BSC) com baixa probabilidade de erro.

Decodificação por Síndrome

Para corrigir erros, usamos a *decodificação por síndrome*. Seja $\mathbf{x}$ a palavra-código transmitida e $\mathbf{y} = \mathbf{x} + \mathbf{z}$ a palavra recebida, onde $\mathbf{z}$ é o vetor de erro (com 1s nas posições erradas). A síndrome é calculada como:

$$\mathbf{s} = \mathbf{H}\mathbf{y} = \mathbf{H}(\mathbf{x} + \mathbf{z}) = \mathbf{H}\mathbf{x} + \mathbf{H}\mathbf{z} = \mathbf{H}\mathbf{z}, \quad (496)$$

pois $\mathbf{H}\mathbf{x} = \mathbf{0}$ para $\mathbf{x} \in C$. A síndrome $\mathbf{s}$ depende apenas do erro $\mathbf{z}$. Se $\mathbf{s} = \mathbf{0}$, assumimos que não houve erro (ou houve um erro indetectável, com probabilidade baixa). Se $\mathbf{s} \neq \mathbf{0}$ e assumindo um erro de 1 bit na

posição i , teremos $\mathbf{s} = \mathbf{h}_i$, a i -ésima coluna de $\mathbf{H}$. Como as colunas de $\mathbf{H}$ são todas distintas e não nulas, podemos identificar i , a posição do bit errôneo e corrigi-lo.

Exemplo 41. Suponha que a palavra recebida seja $\mathbf{y}^T = (0, 1, 1, 1, 0, 0, 1)$, que não é uma palavra-código válida. Calculamos a síndrome:

$$\mathbf{H}\mathbf{y} = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1+1+1 \\ 1+1 \\ 1+1+1 \end{pmatrix} \mod 2 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}. \quad (497)$$

A síndrome $\mathbf{s} = (1, 0, 1)^T$ corresponde à coluna 2 de $\mathbf{H}$ (indexada de 0 a 6, coluna $\mathbf{h}_1$). Isso indica um erro na posição 2 (bit y_1). Corrigimos definindo $\mathbf{z} = (0, 1, 0, 0, 0, 0, 0)$, e calculamos:

$$\mathbf{x} = \mathbf{y} + \mathbf{z} = (0, 1, 1, 1, 0, 0, 1) + (0, 1, 0, 0, 0, 0, 0) = (0, 0, 1, 1, 0, 0, 1). \quad (498)$$

Verificamos que $\mathbf{x} = (0, 0, 1, 1, 0, 0, 1)$ é uma palavra-código válida, pois:

$$\mathbf{H}\mathbf{x} = \begin{pmatrix} 0+1+1 \\ 0+1+1 \\ 0+1+1 \end{pmatrix} \mod 2 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}. \quad (499)$$

Os bits de dados são $(x_0, x_1, x_2, x_3) = (0, 0, 1, 1)$.

O procedimento de decodificação por síndrome é esquematizado da seguinte forma:

1. Calcule a síndrome $\mathbf{s} = \mathbf{H}\mathbf{y}$.
2. Se $\mathbf{s} = \mathbf{0}$, assumo $\mathbf{z} = \mathbf{0}$ e vá ao passo 4.
3. Caso contrário, identifique a coluna $\mathbf{h}_i$ de $\mathbf{H}$ igual a $\mathbf{s}$, e defina $\mathbf{z}$ com 1 na posição i e 0s nas demais.
4. Calcule $\mathbf{x} = \mathbf{y} + \mathbf{z}$.
5. Retorne os bits de dados (x_0, x_1, x_2, x_3) .

Esse procedimento corrige exatamente um erro por palavra-código, mas falha se houver dois ou mais erros, pois a síndrome pode apontar para uma correção incorreta.

Propriedades e Aplicações

O código de Hamming (7,4,3) tem $2^4 = 16$ palavras-código, correspondendo às combinações lineares das colunas de $\mathbf{G}$. A matriz $\mathbf{H}$ tem posto 3, e seu espaço nulo tem dimensão $7 - 3 = 4$, confirmando as 2^4 palavras. Isso segue do Teorema do Posto-nulidade, que afirma que,

para uma matriz $m \times n$, o posto somado à dimensão do espaço nulo (nulidade) é igual ao número de colunas: posto + nulidade = n^{26} .

Assim, com $n = 7$ e posto 3, a nulidade é 4. A probabilidade de erro indetectável em um canal binário simétrico (BSC) com probabilidade de erro p é limitada por:

$$P_u(E) \leq 2^{-(n-k)}[1 - (1-p)^n] = 2^{-3}[1 - (1-p)^7], \quad (500)$$

o que indica que a adição de 3 bits de paridade reduz significativamente os erros indetectáveis (Costello e Lin 2004).

Para entender por que o código de Hamming (7,4,3) é eficiente, precisamos introduzir o conceito de esfera de Hamming e o limite de Hamming, que determina o número máximo de palavras-código para um dado nível de correção de erros.

Definição 69 (Esfera de Hamming). Uma *esfera de Hamming* de raio t centrada em uma sequência $\mathbf{x}$ de n símbolos em um alfabeto q -ário é o conjunto de todas as sequências $\mathbf{y}$ tais que a distância de Hamming $d_H(\mathbf{x}, \mathbf{y}) \leq t$. Formalmente:

$$B(\mathbf{x}, t) = \{\mathbf{y} \in \{0, 1, \dots, q-1\}^n : d_H(\mathbf{x}, \mathbf{y}) \leq t\}. \quad (501)$$

O volume ou número de sequências em uma esfera de raio t é dado por:

$$|B(\mathbf{x}, t)| = \sum_{i=0}^t \binom{n}{i} (q-1)^i, \quad (502)$$

onde $\binom{n}{i}$ é o número de maneiras de escolher i posições para alterar, e $(q-1)^i$ é o número de escolhas para cada uma das i posições (podemos escolher $q-1$ símbolos diferentes do original em cada posição).

Para um código binário ($q = 2$), temos $(q-1)^i = 1$, e para $t = 1$, o tamanho da esfera é:

$$\sum_{i=0}^1 \binom{n}{i} = \binom{n}{0} + \binom{n}{1} = 1 + n. \quad (503)$$

Por exemplo, com $n = 7$, uma esfera de raio 1 contém $1 + 7 = 8$ sequências: a palavra-código e as 7 sequências que diferem dela em exatamente 1 bit.

Teorema 56 (Limite de Hamming) *Um código q -ário de comprimento n que corrige até t erros pode ter no máximo $q^n / |B(\mathbf{x}, t)|$ palavras-código, ou seja:*

$$|B(\mathbf{x}, t)| \cdot |C| \leq q^n, \quad (504)$$

onde $|C|$ é o número de palavras-código, e $|B(\mathbf{x}, t)| = \sum_{i=0}^t \binom{n}{i} (q-1)^i$ é o tamanho de uma esfera de Hamming de raio t . Um código que atinge a igualdade é chamado de código perfeito²⁷.

²⁶ O posto de uma matriz é a dimensão do espaço vetorial gerado por suas colunas (ou linhas), representando o número de linhas ou colunas linearmente independentes. A nulidade é a dimensão do espaço nulo, o conjunto de vetores $\mathbf{x}$ tais que $\mathbf{Ax} = \mathbf{0}$, ou seja, o número de soluções independentes para o sistema homogêneo.

²⁷ Um código perfeito é um código que atinge o limite de Hamming com igualdade, ou seja, as esferas de Hamming de raio t centradas nas palavras-código são disjuntas e cobrem exatamente todas as q^n sequências possíveis. Isso maximiza o número de palavras-código para um dado nível de correção de erros, sem deixar lacunas ou sobreposições no espaço de sequências.

Demonstração. Para corrigir até t erros, cada sequência recebida deve ser decodificada para uma única palavra-código. Isso implica que as esferas de Hamming de raio t centradas nas palavras-código devem ser disjuntas: se $\mathbf{c}_1$ e $\mathbf{c}_2$ são palavras-código distintas, então $B(\mathbf{c}_1, t) \cap B(\mathbf{c}_2, t) = \emptyset$. Caso contrário, uma sequência $\mathbf{y} \in B(\mathbf{c}_1, t) \cap B(\mathbf{c}_2, t)$ estaria a distância $\leq t$ de duas palavras-código, impossibilitando a correção única.

Cada esfera $B(\mathbf{c}_i, t)$ contém $\sum_{i=0}^t \binom{n}{i} (q-1)^i$ sequências. Se o código tem $|\mathcal{C}|$ palavras-código, o número total de sequências cobertas pelas esferas é:

$$|\mathcal{C}| \cdot |B(\mathbf{c}_i, t)| = |\mathcal{C}| \cdot \sum_{i=0}^t \binom{n}{i} (q-1)^i. \quad (505)$$

O número total de sequências possíveis de n símbolos é q^n . Como as esferas são disjuntas e cada sequência pertence a no máximo uma esfera, temos:

$$|\mathcal{C}| \cdot \sum_{i=0}^t \binom{n}{i} (q-1)^i \leq q^n. \quad (506)$$

Se a igualdade for atingida, as esferas cobrem exatamente todas as q^n sequências, e o código é perfeito.

□

Apliquemos o teorema ao código de Hamming (7,4,3). Ele é um código binário $q = 2$ com $n = 7$, $k = 4$, e $d = 3$, que corrige $t = \lfloor (d-1)/2 \rfloor = 1$ erro. O número de palavras-código é $|\mathcal{C}| = 2^k = 2^4 = 16$, e o tamanho de uma esfera de raio $t = 1$ é:

$$\sum_{i=0}^1 \binom{7}{i} (2-1)^i = \binom{7}{0} + \binom{7}{1} = 1 + 7 = 8. \quad (507)$$

Substituímos no limite de Hamming:

$$|B(\mathbf{x}, 1)| \cdot |\mathcal{C}| = 8 \cdot 16 = 128 \leq 2^7 = 128. \quad (508)$$

A igualdade indica que o código de Hamming (7,4,3) é perfeito: as esferas de Hamming de raio 1 centradas nas 16 palavras-código cobrem exatamente todas as $2^7 = 128$ sequências possíveis de 7 bits, otimizando a correção de erros sem deixar lacunas ou sobreposições.

Códigos da família de Hamming, como o (7,4,3) e o (31,26,3), são amplamente utilizados em sistemas onde a confiabilidade é crítica, incluindo memórias de computadores (e.g., ECC RAM) e comunicações digitais. O código (7,4,3), proposto por Richard Hamming, corrige 1 erro e detecta 2 erros com uma sobrecarga de aproximadamente 43% de bits de paridade, sendo um marco teórico e histórico, especialmente nos primeiros sistemas computacionais, como o IBM 701. Já o código (31,26,3), com sobrecarga de cerca de 16%, é mais comum em memórias RAM modernas, protegendo blocos de 32 bits com eficiência,

ideal para cenários onde erros de bit único predominam, como em chips DRAM. Sua estrutura linear e decodificação eficiente, baseada na abordagem por síndrome, estabeleceram os códigos Hamming como uma base para o desenvolvimento de códigos mais avançados, como os códigos BCH e Reed-Muller, que oferecem maior capacidade de correção, porém com maior complexidade e overhead.

Construção dos Códigos de Hamming

Os códigos de Hamming formam uma família de códigos lineares que podem ser gerados de maneira sistemática com base na representação binária das posições dos bits. Essa construção permite determinar os bits de paridade e os bits de dados de forma algorítmica, garantindo uma distância mínima $d_{\min} = 3$, que corrige 1 erro e detecta 2 erros. Abaixo, descrevemos o processo de construção.

A ideia central é associar cada posição de bit a uma representação binária e usar os bits de paridade para verificar subconjuntos específicos de posições, determinados pelos bits 1 na representação binária. As posições são numeradas de 1 a n , e os bits de paridade são colocados em posições que são potências de 2 (1, 2, 4, 8, etc.), enquanto os bits de dados ocupam as demais posições.

	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111	10000	10001	10010	10011	10100
posição do bit	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
bits de dado codificados	p_1	p_2	p_3	p_4	p_5	p_6	p_7	p_8	p_9	p_{10}	p_{11}	p_{12}	p_{13}	p_{14}	p_{15}	p_{16}	p_{17}	p_{18}	p_{19}	p_{20}
Cobertura dos bits de paridade	p_1	X		X		X			X		X		X		X					
	p_2		X	X			X	X		X	X			X	X			X	X	
	p_4				X	X	X	X				X	X	X	X					X
	p_8							X	X	X	X	X	X	X	X					
	p_{16}															X	X	X	X	X

O algoritmo de construção segue estas regras:

- As posições dos bits de paridade são $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, etc., e são denotadas por $p_1, p_2, p_4, p_8, \dots$
- Cada bit de paridade p_i (onde $i = 2^j$ para algum j) cobre as posições cuja representação binária tem um 1 na posição $j+1$ (contando de 1 a partir do bit menos significativo). Por exemplo:
 - p_1 (posição 1): cobre posições com 1 no primeiro bit, como 1 (1), 3 (11), 5 (101), 7 (111), etc.
 - p_2 (posição 2): cobre posições com 1 no segundo bit, como 2 (10), 3 (11), 6 (110), 7 (111), etc.
 - p_4 (posição 4): cobre posições com 1 no terceiro bit, como 4 (100), 5 (101), 6 (110), 7 (111), etc.
 - E assim por diante...
- Os bits de paridade são calculados para garantir que a soma (mó-

Tabela 7: Tabela de construção de um código de Hamming com até 20 bits. As posições marcadas com p_i são bits de paridade, e as demais são bits de dados. A cobertura de cada bit de paridade é determinada pela presença de 1 na posição correspondente na representação binária.

dulo 2) dos bits que eles cobrem seja zero, satisfazendo as equações de verificação.

- As posições restantes são ocupadas pelos bits de dados.

Se utilizarmos m bits de paridade, as posições totais cobertas vão de 1 a $2^m - 1$, pois essas são todas as combinações possíveis com m bits. Subtraindo os m bits de paridade, o número de bits de dados é $2^m - 1 - m$. A distância mínima do código resultante é $d = 3$, permitindo correção de 1 erro e detecção de 2 erros. Variando m , obtemos a família de códigos de Hamming, como mostrado na tabela a seguir:

bits de paridade	total de bits	bits de dados	nome	taxa
2	3	1	Hamming(3,1)	$1/3 \approx 0.333$
3	7	4	Hamming(7,4)	$4/7 \approx 0.571$
4	15	11	Hamming(15,11)	$11/15 \approx 0.733$
5	31	26	Hamming(31,26)	$26/31 \approx 0.839$

Por exemplo, para $m = 3$, temos $n = 2^3 - 1 = 7$ bits totais, com 3 bits de paridade (posições 1, 2, 4) e 4 bits de dados (posições 3, 5, 6, 7), resultando no código Hamming(7,4). Para $m = 5$, $n = 2^5 - 1 = 31$, com 5 bits de paridade e 26 bits de dados, formando o Hamming(31,26).

Essa construção sistemática permite gerar códigos de Hamming para diferentes tamanhos de dados, equilibrando redundância e capacidade de correção, e serve como base para entender códigos lineares mais complexos.

Tabela 8: Variações da família de códigos de Hamming para diferentes valores de m , com $n = 2^m - 1$ e $k = 2^m - 1 - m$. A taxa $R = k/n$ indica a eficiência do código.

Índice

- AES, 13
- aleatoriaidade, 7, 8
- cadeia de Markov, 46, 54, 71, 96
 - homogênea, 96
 - irredutível, 97
 - periódica, 97
 - taxa de entropia, 100
- canal, 140
 - binário com apagamento, 144
 - binário sem ruído, 142
 - binário simétrico, 143
 - capacidade, 140, 142, 149
 - fracamente simétrico, 146
 - codificação, 141
 - com ruído e sem sobreposição, 143
 - confusão ternário, 145
 - discreto, 141
 - ruidoso, 53
 - sem memória, 142
 - simétrico, 146
 - taxa, 142
 - taxa de fluxo de informação, 142
- canal de comunicação, *ver* canal
- capacidade de canal, 15
- codebook, 79
- codeword
 - comprimento, 79
 - codificador, 73
 - taxa, 73
 - comunicação, 15
 - conjunto típico, 75, 90
 - codificação, 78
 - probabilidade, 90
 - convexidade, 39
 - código
 - (M,n) , 147
 - $[n,k,d]$, 170
 - Braille, 4
 - codificação aritmética, 131
 - codificação de fonte, 105
 - comprimento esperado, 105, 111
 - comprimento esperado mínimo, 112
 - de bloco
 - com taxa fixa, 91
 - universal, 91
 - de Hamming, 174, 180
 - de Huffman, 121
 - otimalidade, 126
 - de prefixo, 107
 - desigualdade de Kraft, 108
 - de repetição, 167
 - de Shannon, 114
 - limites para o comprimento esperado, 115
 - otimalidade competitiva, 117, 118
 - de verificação de paridade,

- 168
 - decodificação por síndrome, 176
 - eficiência, 106
 - Escrita Noturna, 4
 - esfera de Hamming, 178
 - extensão, 107
 - instantâneo, *ver* de prefixo
 - instantâneo ótimo
 - propriedade, 124
 - Lempel–Ziv, 134
 - LZ77, 135, 136
 - LZW, 138
 - limite de Hamming, 178
 - linear, 171
 - codificador, 172
 - matriz verificadora de
 - paridade, 173
 - Morse, 4
 - não-singular, 107
 - peso da palavra, 171
 - peso mínimo, 172
 - Shannon-Fano-Elias, 127
 - síndrome, 176
 - taxa, 142
 - taxa alcançável, 149
 - unívoco, 107
 - ótimo, 111
-
- desigualdade da soma dos
 - logaritmos, 42
 - desigualdade de Fano, 54
 - desigualdade de Jensen, 39, 40
 - não negatividade, 41
 - desigualdade de Kraft, 108, 119, 120
 - infinitos contáveis, 109
 - desigualdade de processamento
 - de dados, 46, 47
 - distribuição cumulativa, 128
 - modificada, 128
 - distância
 - mínima, 165
 - distância de Hamming, 165
 - divergência de
 - Kullback-Leibler, 30
 - convexidade, 43, 44
 - minimização, 60
 - não negatividade, 32, 43
 - relação com entropia
 - cruzada, 68
 - relação com informação
 - mútua, 30
 - entropia, 15, 18
 - bases, 23
 - binária, 20
 - concavidade, 45
 - condicional, 25, 27
 - condicionar não aumenta, 50
 - conjunta, 25
 - cruzada, 67
 - cruzada, relação com
 - divergência, 68
 - de colisão, 53
 - de Rényi, 53
 - distribuição uniforme, 20
 - estimador *plug-in*, 64, 65
 - estimador de Miller, 66
 - estimação, 60
 - limite de independência, 51
 - máximo, 24, 34
 - propriedades, 22
 - regra da cadeia, 27
 - regra da cadeia condicional, 37
 - regra da cadeia condicional
 - generalizada, 38
 - regra da cadeia generalizada, 37
 - Zipf, 21
 - entropia condicional
 - nula, 49
 - estatística
 - amostral, 70
 - estimação de distribuição, 60
 - histograma empírico, 72, 82
 - suficiente, 71

estimativa de máxima
verossimilhança, 60
expansão multinomial, 119

histograma empírico, *ver*
estatística, *ver* tipo

incerteza, 29, 60
nula, 49

independência, 50

informação, 4, 5, 18
auto-informação, 36
diagrama, 36
Hartley, 16

informação mútua, 29

concavidade, 45
condicional, 36
convexidade, 45
monotonicidade, 51
não negatividade, 33
nula, 50
regra da cadeia, 38

informação mútua condicional
não nula, 50

jogo de Shannon, 131

medida

campo, 57
com sinal, 58
correspondências com as
medidas de Shannon, 59
de informação, 36, 57
de informação de Shannon,
59
átomo, 57

média Cesàro, 98

método bola-traço, 84

método de tipos, 82

palavra

comprimento, 79

peso de Hamming, 154

probabilidade de erro, 54

máxima, 148

média, 148

para a mensagem, 148

processo estocástico

estacionário em sentido
estrito, 95

Markov, *ver* cadeia de
Markov

taxa de entropia, 99

taxa de inovação da
informação, 99

taxa de inovação de
informação, 99

teorema taxa de inovação e
taxa de entropia, 100

propriedade da equipartição
assintótica, 70, 75, 82, 154,
161

pseudoaleatório, 7

redundância, 17, 167

RNG

autômatos celulares, 12

caos, 12

CSPRNG, 13

funções criptográficas, 12

gerador de Lehmer, 8, 9, 12

gerador de números

aleatórios do Linux, 8

LCG, 9, 10, 12

LFG, 12

Mersenne Twister, 10

sequência típica, 70, 74, 80

tipicidade conjunta, 150

simplex probabilístico, *ver* tipo

suavização, 61, 65

regra de Dirichlet, 63

regra de sucessão de Laplace,
62

teorema

conjunto de codificação de
fonte-canal, 161

- correção de erros, 166
 - da codificação de fonte, 80, 92
 - da desigualdade de Fano, 56
 - da fatoração de
 - Fisher-Neyman, 72
 - de Bayes, 27, 46
 - de codificação de canal, 151, 160, 162
 - de codificação de fonte, 160
 - de correção de erros, 176
 - de detecção de erros, 176
 - detecção de erros, 166
 - detecção de erros por
 - paridade, 169
 - do valor médio, 23
 - multinomial, 72, 119
 - posto-nulidade, 173, 177
 - primeiro teorema de
 - Shannon, 80, 92
 - tipo, 82
 - classe de tipo, 83
 - limites da probabilidade, 89
 - limites para o tamanho, 88
 - classe de tipo com maior
 - probabilidade, 86
 - conjunto de tipos, 83
 - limite no número de tipos, 85
 - número de tipos, 84
 - probabilidade, 85
 - simplex probabilístico, 91
 - tamanho da classe de tipo, 87
- variável aleatória, 16
 - discreta, 16
 - verossimilhança, 60

Bibliografia

- Abel, Niels Henrik (1826). “Mémoire sur les équations algébriques où l’on démontre l’impossibilité de la résolution de l’équation générale du cinquième degré”. Em.
- Antos, András e Ioannis Kontoyiannis (2001). “Convergence Properties of Functional Estimates for Discrete Distributions”. Em: *Random Structures and Algorithms* 19.3-4, pp. 163–193.
- Araújo, Leonardo Carneiro, Thaïs Cristófar-Silva e Hani Camille Yehia (2013). “Entropy of a Zipfian Distributed Lexicon”. Em: *Glottometrics* 26, pp. 38–49.
- Artin, Michael (1991). *Algebra*. Prentice Hall.
- Aruz, Joan, Kim Benzel e Jean M. Evans, eds. (2008). *Beyond Babylon: Art, trade, and diplomacy in the second millennium B.C.* New York: Metropolitan Museum of Art. ISBN: 0300141432.
- Bansal, Ayush, Pramod Subramanyan e Satyadev Nandakumar (2023). *Analysis of Linux-PRNG (Pseudo Random Number Generator)*. DOI: 10.48550/arxiv.2312.03369. URL: <https://arxiv.org/abs/2312.03369>.
- Berger, Roger L. e George Casella (2002). *Statistical Inference*. Duxbury Press.
- Bilmes, Jeffrey A. (2013). *Lecture notes on Information Theory*.
- Chao, Anne e Tsung-Jen Shen (2003). Em: 10.4, pp. 429–443. ISSN: 1352-8505. DOI: 10.1023/a:1026096204727. URL: <http://dx.doi.org/10.1023/A:1026096204727>.
- Chen, I-Te (2013). “Random Numbers Generated from Audio and Video Sources”. Em: *Mathematical Problems in Engineering* 2013, pp. 1–7. ISSN: 1563-5147. DOI: 10.1155/2013/285373. URL: <http://dx.doi.org/10.1155/2013/285373>.
- Cloudflare (2025). *How do lava lamps help with Internet encryption?* Accessed: 2025-04-24. URL: <https://www.cloudflare.com/learning/ssl/lava-lamp-encryption/>.
- Costello, Daniel e Shu Lin (mai. de 2004). *Error Control Coding*. 2ª ed. Philadelphia, PA: Pearson Education.

- Cover, Thomas M. e Joy A. Thomas (2006). *Elements of Information Theory*. 2ª ed. Hoboken, NJ: Wiley-Interscience. ISBN: 978-0-471-24195-9.
- Delfs, Hans e Helmut Knebl (2015). *Introduction to Cryptography: Principles and Applications*. Springer Berlin Heidelberg. ISBN: 9783662479742. DOI: 10.1007/978-3-662-47974-2.
- Edgar, Tom (2018). “Staircase Series”. Em: *Mathematics Magazine* 91.2, pp. 92–95. ISSN: 0025-570x. DOI: 10.1080/0025570x.2017.1415584.
- Fano, Robert M. (1961). *Transmission of Information: A Statistical Theory of Communication*. Cambridge, MA: The MIT Press.
- Ferrer i Cancho, Ramon e Ricard V. Solé (dez. de 2001). “Two Regimes in the Frequency of Words and the Origins of Complex Lexicons: Zipf’s Law Revisited*”. Em: *Journal of Quantitative Linguistics* 8.3, pp. 165–173. ISSN: 1744-5035. DOI: 10.1076/jqul.8.3.165.4101. URL: <http://dx.doi.org/10.1076/jqul.8.3.165.4101>.
- Gallager, Robert Gray (jan. de 1962). “Low-density parity-check codes”. Em: *IEEE Transactions on Information Theory* 8.1, pp. 21–28. ISSN: 0018-9448. DOI: 10.1109/tit.1962.1057683.
- Galois, Évariste (1832). “Mémoire sur les conditions de résolubilité des équations par radicaux”. Em.
- Gauss, Carl Friedrich (1801). *Disquisitiones Arithmeticae*. Gerhard Fleischer.
- Golay, Marcel JE (1949). “Notes on digital coding”. Em: *Proc. IEEE* 37, p. 657.
- Good, Irving John (1953). “The population frequencies of species and the estimation of population parameters”. Em: *Biometrika* 40(3-4), pp. 237–264.
- Grassberger, P. (2008). *Entropy Estimates from Insufficient Samplings*. arXiv: physics/0307138 [physics.data-an]. URL: <https://arxiv.org/abs/physics/0307138>.
- Guterman, Zvi, Benny Pinkas e Tzachy Reinman (2006). “Analysis of the linux random number generator”. Em: *2006 IEEE Symposium on Security and Privacy (S&P’06)*. Ieee. Ieee, pp. 371–385. DOI: 10.1109/sp.2006.5. URL: <http://dx.doi.org/10.1109/SP.2006.5>.
- Hamming, Richard Wesley (abr. de 1950). “Error Detecting and Error Correcting Codes”. Em: *Bell System Technical Journal* 29.2, pp. 147–160. ISSN: 0005-8580. DOI: 10.1002/j.1538-7305.1950.tb00463.x.
- Harris, Bernard (1975). *The statistical estimation of Entropy in the non-parametric case*. Rel. téc. 1605. University of Wisconsin-Madison.

- Hartley, Ralph Vinton Lyon (jul. de 1928). “Transmission of Information”. Em: *Bell System Technical Journal* 7.3, pp. 535–563. ISSN: 0005-8580. DOI: 10.1002/j.1538-7305.1928.tb01236.x.
- Klein, Felix (1926). *Vorlesungen über die Entwicklung der Mathematik im 19. Jahrhundert*. Vol. 1. Springer.
- Kneusel, Ronald T. (2018). *Random Numbers and Computers*. Cham: Springer. ISBN: 978-3-319-77696-5. DOI: 10.1007/978-3-319-77697-2.
- Knuth, Donald E. (1997). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3^a ed. Reading, MA: Addison-Wesley. ISBN: 978-0-201-89684-8.
- Kocarev, Ljupco e Shiguo Lian (2011). *Chaos-Based Cryptography: Theory, Algorithms and Applications*. Berlin, Heidelberg: Springer. ISBN: 978-3-642-20541-5. DOI: 10.1007/978-3-642-20542-2.
- Krhovják, Jan, Petr Švenda, Václav Matyáš et al. (2007). “The sources of randomness in mobile devices”. Em: *Proceedings of the Nordsec*.
- Laplace, Pierre-Simon (1840). *Essai philosophique sur les probabilités*. Paris: Paris Bachelier.
- MacKay, David John Cameron (2003). *Information theory, inference, and learning algorithms*. Cambridge, UK ; New York: Cambridge University Press. ISBN: 9780521642989.
- Matsumoto, Makoto e Takuji Nishimura (1998). “Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudo-Random Number Generator”. Em: *ACM Transactions on Modeling and Computer Simulation* 8.1, pp. 3–30. DOI: 10.1145/272991.272995.
- Mazet, Edmond (2003). “La théorie des séries de Nicole Oresme dans sa perspective aristotélicienne. ‘Questions 1 et 2 sur la Géométrie d’Euclide’”. Em: *Revue d’histoire des mathématiques* 9.1, pp. 33–80. URL: http://www.numdam.org/item/RHM_2003__9_1_33_0/.
- Miller, George (1955). “Note on the bias of information estimates”. Em: *Information theory in psychology: Problems and methods*.
- Noether, Emmy (1921). “Idealtheorie in Ringbereichen”. Em: *Mathematische Annalen* 83.1, pp. 24–66.
- Pinchas, Assaf, Irad Ben-Gal e Amichai Painsky (abr. de 2024). “A Comparative Analysis of Discrete Entropy Estimators for Large-Alphabet Problems”. Em: *Entropy* 26.5, p. 369. ISSN: 1099-4300. DOI: 10.3390/e26050369. URL: <http://dx.doi.org/10.3390/e26050369>.
- Quenouille, M. H. (dez. de 1956). “Notes on Bias in Estimation”. Em: *Biometrika* 43.3/4, p. 353. ISSN: 0006-3444. DOI: 10.2307/2332914. URL: <http://dx.doi.org/10.2307/2332914>.
- Salomon, David, Giovanni Motta e David Bryant (2007). *Data Compression: The Complete Reference*. Springer London. ISBN: 9781846286032.

- Shannon, Claude E. (1948). “A mathematical theory of communication.” Em: *Bell Syst. Tech. J.* 27.3, pp. 379–423.
- (jan. de 1951). “Prediction and Entropy of Printed English”. Em: *Bell System Technical Journal* 30.1, pp. 50–64. DOI: 10.1002/j.1538-7305.1951.tb01366.x.
- Shirley, John W. (nov. de 1951). “Binary Numeration before Leibniz”. Em: *American Journal of Physics* 19.8, pp. 452–454. ISSN: 1943-2909. DOI: 10.1119/1.1933042. URL: <http://dx.doi.org/10.1119/1.1933042>.
- Tan, Ying (2016). “GPU-Based Random Number Generators”. Em: *Gpu-Based Parallel Implementation of Swarm Intelligence Algorithms*. Elsevier, pp. 147–165. ISBN: 9780128093627. DOI: 10.1016/b978-0-12-809362-7.50010-8. URL: <http://dx.doi.org/10.1016/B978-0-12-809362-7.50010-8>.
- Van der Waerden, Bartel Leendert (1930). *Moderne Algebra*. Springer.
- Welch, Terry Archer (jun. de 1984). “A Technique for High-Performance Data Compression”. Em: *Computer* 17.6, pp. 8–19. DOI: 10.1109/mc.1984.1659158.
- Wikipédia (2025). *Geometric series*. Acessado em 4 de junho de 2025. URL: https://en.wikipedia.org/wiki/Geometric_series.
- Witten, Ian H., Radford M. Neal e John G. Cleary (jun. de 1987). “Arithmetic Coding for Data Compression”. Em: *Communications of the ACM* 30.6, pp. 520–540. DOI: 10.1145/214762.214771.
- Wolfram, Stephen (2002). *A New Kind of Science*. Champaign, IL: Wolfram Media. ISBN: 978-1-57955-008-0.
- Yeung, Raymond W. (2002). *A First Course in Information Theory*. Springer US. ISBN: 9780306467912.
- Zahl, Samuel (jul. de 1977). “Jackknifing An Index of Diversity”. Em: *Ecology* 58.4, pp. 907–913. ISSN: 1939-9170. DOI: 10.2307/1936227. URL: <http://dx.doi.org/10.2307/1936227>.
- Zandonade, Tarcisio (dez. de 2024). “Leibniz e a Criação da Numeração Binária”. Em: *Brazilian Journal of Information Science: research trends* 18, e024039. ISSN: 1981-1640. DOI: 10.36311/1981-1640.2024.v18.e024039. URL: <http://dx.doi.org/10.36311/1981-1640.2024.v18.e024039>.
- Zhang, Zhiyi e Xing Zhang (2012). “A Normal Law for the Plug-in Estimator of Entropy”. Em: *IEEE Trans. Information Theory* 58.5, pp. 2745–2747.
- Zipf, George Kingsley (1935). *An Introduction to Dynamic Philology*. Cambridge, MA: The MIT Press.
- (1949). *Human Behavior and the Principle of Least Effort*. Cambridge, MA: Addison-Wesley.
- Ziv, Jacob e Abraham Lempel (mai. de 1977). “A universal algorithm for sequential data compression”. Em: *IEEE Transactions on In-*

formation Theory 23.3, pp. 337–343. DOI: 10.1109/tit.1977.1055714.